

SECOND EDITION

Microservices Patterns

Designing applications for fast flow
and high reliability

Chris Richardson





MEAP Edition
Manning Early Access Program

Microservices Patterns, Second Edition

**Designing applications for fast flow
and high reliability**

Version 5

Copyright 2026 Manning Publications

For more information on this and other Manning titles go to
manning.com.

Welcome

Thank you for purchasing the MEAP for the second edition of *Microservices Patterns*. This book focuses on the architecture and development of microservice-based applications. It's written for anyone responsible for developing and delivering software—whether you're a developer, architect, CTO, or VP of engineering.

My goal is to make this material accessible regardless of your technology stack. You'll get the most from this book if you're already familiar with the basics of enterprise application architecture and design—concepts like three-tier architecture, web application design, relational databases, inter-process communication using REST or messaging, and basic application security.

Since the first edition of *Microservices Patterns* was published in late 2018, I've had the privilege of working with countless organizations on their microservice journeys. I've delivered many talks and had countless conversations with teams about how to think about microservices, how to avoid common pitfalls, and how to solve the real-world challenges that arise when implementing these architectures.

This second edition is a substantial update that reflects what I've learned over the past several years and includes new insights, and techniques for applying microservices effectively in today's complex software environments.

The second edition also has a sharper focus on **fast flow** — the continuous delivery of a stream of small changes and rapid feedback on each one - as the primary motivation for

adopting microservices is to enable fast flow. In the first edition, I touched on this idea, but it wasn't explicitly named. Since then, my thinking has become much more focused on the **fast flow success triangle**, which describes the three essential elements necessary to achieve fast flow:

- **Team Topologies** - designing your organization to support independent teams
- **DevOps** - principles and practices for delivering software rapidly, frequently, and reliably
- **Architecture** - a loosely design-time coupled, highly testable, and deployable architecture that enables Team Topologies and DevOps practices.

For some applications, a monolithic architecture is a good choice. But for larger applications developed by many teams, a microservice architecture is often the better option. Understanding the elements of the success triangle and the role of architecture is key to high-performance software delivery.

The second edition also emphasizes the importance of architectural fundamentals. A solid grasp of architectural fundamentals is essential for understanding what the microservice architecture is, when to use it, and how to apply it effectively. Most notably, the book explains several important software architecture concepts, including the multi-dimensional nature of software architecture; different flavors of coupling and why loose coupling matters; and the concept of software quality attributes and how they can enable fast flow.

Finally, I have significantly expanded the coverage of architectural decision-making techniques. You'll learn how to decide between a monolithic and microservice

architecture based on your specific context. I introduce the ten dark energy and dark matter forces that serve as essential criteria for evaluating and comparing candidate solutions to the many design problems you will encounter when defining a microservice architecture. You will also learn a step-by-step process for designing and evolving a microservice architecture.

It's a big book about an important topic, and I hope you find it as valuable to read as I did to write. Please don't hesitate to share your questions, comments, or suggestions in the [liveBook discussion forum](#). Your feedback is essential to making this the best book it can be.

Thank you again for your interest and support.

— Chris Richardson

Brief contents

PART 1: INTRODUCTION

[1 *Microservices: an architecture for fast flow*](#)

[2 *FTGO: A case study in moving from one kind of development hell to another*](#)

PART 2: ARCHITECTURE FUNDAMENTALS

[3 *Software architecture: what is it, why does it matter and who does it?*](#)

[4 *The importance of loose coupling.*](#)

[5 *Architectural requirements for fast flow*](#)

[6 *The monolithic architecture*](#)

[7 *The Microservice architecture: enabling fast sustainable flow*](#)

[8 *Interprocess communication in a microservice architecture*](#)

PART 3: SERVICE COLLABORATION

[9 *Implementing system commands with sagas*](#)

10 Implementing commands with sagas

11 Implementing queries in a microservice architecture

12 External API patterns

PART 4: SERVICE INTERNALS

13 Service design

14 Developing business logic with event sourcing

PART 5: TESTING MICROSERVICES

15 Testing microservices: Part 1

16 Testing microservices: Part 2

PART 6: MICROSERVICES IN PRODUCTION

17 Developing production-ready services

18 Deploying microservices

19 Deploying services on Kubernetes

PART 7: ADOPTING MICROSERVICES

20 Assemblage: a process for designing a microservice architecture

21 Refactoring a monolith to microservices

1 Microservices: an architecture for fast flow

This chapter covers

- The primary motivation for using the microservice architecture
- The key characteristics of microservice architecture
- The obstacles to overcome when using the microservice architecture
- When and when not to use the microservice architecture

A lot has been said and written about the microservice architecture over the years. There are conflicting claims about when to use it. Some in the community say it's the future of software development. There are others who even argue it's an anti-pattern. The reality, as always, is **it depends**. But upon what? When should you use the microservice architecture? What problems does it solve and how?

The main motivation for using the microservice is to solve a problem that you are most likely facing: the need to deliver software faster, more reliably and with less risk so that your business can thrive in today's world. While you might be able to accelerate development by, as a manager once put it, buckling down (or is it up?) and burning the midnight oil, any improvements are likely to be short-lived, and you will most likely burn out. For lasting improvements, you need to change the way you develop software, and, in some cases, the architecture of the software itself.

To accelerate software delivery, you must adopt a style of software delivery known as fast flow and an architecture that enables it. As you might expect, for some but not all applications, the architecture that enables fast flow is the microservice architecture. Later in this chapter, I'll define the microservice architecture, explain its relationship with fast flow and describe its drawbacks and challenges. But let's start by looking at fast flow and why it's needed.

1.1 The need for fast flow

Fast flow is a style of software delivery where IT continuously delivers a stream of small changes and rapidly receives feedback from users and the production environment on each one. The need for fast flow is driven by two trends. The first is that to succeed in today's world, businesses need to be much more nimble. They must innovate quickly, rapidly respond to changes in the market, and continually experiment to learn what works and what doesn't. The second trend is that modern businesses are powered by software, which is, of course, built by IT. In order to support the business's need for agility and experimentation, IT can no longer release software infrequently. Instead, IT must release smaller changes more frequently.

In this section, I'll explain why fast flow is needed. You will learn about its benefits, which, perhaps surprisingly, include more reliable releases. I describe to achieve fast flow using a combination of the right kind of development process, organization structure, and architecture. To understand why it's needed, let's first look at the feedback loop that is at the heart of software development.

1.1.1 The software delivery feedback loop

At the heart of software development is the feedback loop shown in figure [1.1](#). During development, we determine (or quite often, guess!) what features the users need, implement them making many design decisions along the way and then release those features into production. And then we get feedback from both users and the production environment. From the users, we learn how well the newly released features meet their needs. And, from production, we learn whether the myriad of design decisions that we made during development are correct. We then do more development, release features, get more feedback, and so on.

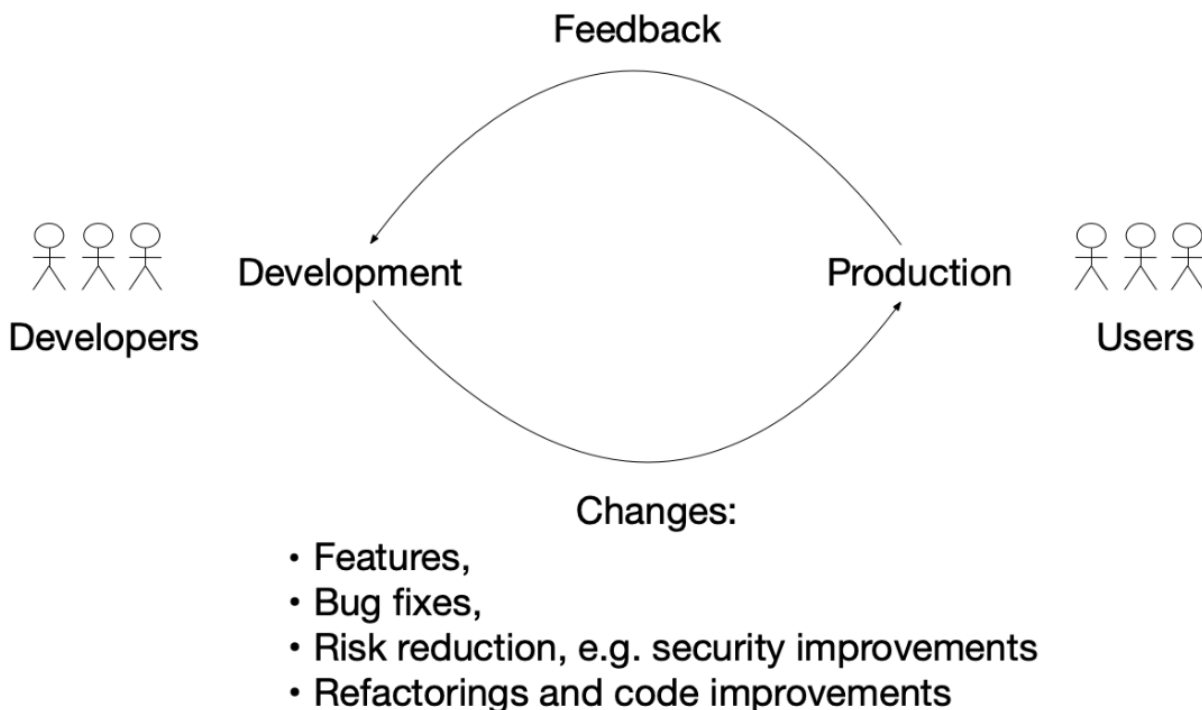


Figure 1.1 The feedback loop at the heart of software development. Changes flow from developers into the production and the hands of the users. Feedback that guides future development flows back from the users and production back to developers.

The speed of this feedback loop determines how often you can learn from users and have your technical decisions validated. The faster the feedback loop, the more quickly you can learn what works and what doesn't. Unfortunately,

there's a large gap between the feedback loop that's needed and the feedback loop that most organizations have.

TRADITIONAL SOFTWARE DELIVERY HAS SLOW FEEDBACK LOOPS

The problem with how many organizations develop software is that the interval between releases is far too long. I have, for example, encountered application modernization projects that release the first version after a year or more of development. More capable organizations release a new version of an application every month or so. In fact, the State of DevOps 2024 report found that around 80% of respondents released changes less frequently than once a day. While some domains, such as safety-critical domains, often require a long release cycle, for a typical enterprise application, slow feedback loops are far too long.

MODERN SOFTWARE DEVELOPMENT REQUIRES FAST FEEDBACK

The problem with such slow feedback, especially when days or weeks, is that it's at odds with the needs of modern software development. Modern development requires feedback early and often to avoid building the wrong application the wrong way. There are two reasons why this is the case. The first reason is that today's world - including the market within which a business competes - is volatile, uncertain, complex and ambiguous (VUCA), or to put it simply, crazy and unpredictable. It's often unclear what features users actually need. For example, a study by Microsoft (**Online Experimentation at Microsoft**, <http://ai.stanford.edu/~ronnyk/ExPThinkWeek2009Public.pdf>) found that only 1/3rd of features improved the product, 1/3rd had no improvement, and 1/3rd made the product

worse. To handle this unpredictability, businesses must continually experiment in order to learn what works and what doesn't.

The second reason that fast feedback is essential is that the technology landscape is also constantly evolving. We often build complex applications using new and unfamiliar technologies. For example, an application modernization project could very well be the first time that an organization is using a new technology, such as the microservice architecture. If the organization is not releasing early and frequently into production, then they are not validating the design decisions that they have made. They risk repeating the same mistake throughout their architecture, or even worse, constructing a house of cards where multiple bad design decisions are built on top of each other.

1.1.2 Fast flow: continuous change and feedback

In order to deliver software successfully today, we need what is known as fast flow. An organization that practices fast flow delivers changes very frequently, ideally many times a day. As a result, there's a continuous stream of feedback flowing in the opposite direction. With fast flow, an organization is constantly learning about both the user needs and the technologies that they are using. What's more, the organization is able to run frequent experiments to determine user needs.

Later I'll discuss the three elements - process, organization, and architecture — that are necessary to achieve fast flow. But first, I'll briefly describe the DORA (DevOps Research and Assessment) metrics that measure software delivery performance. I'll also discuss the evidence showing that fast

flow is essential for business success. Let's start with the DORA metrics.

QUANTIFYING FAST FLOW WITH THE DORA METRICS

On the surface, the definition of fast flow software delivery might appear to be fuzzy. But it's not, thanks to work described in *Accelerate* by Nicole Forsgren, Jez Humble, Gene Kim (IT Revolution Press, 2018). This book defines four key DORA (DevOps Research and Assessment) metrics that measure software delivery performance:

- Lead time - the time it takes to go from code committed to code running in production
- Deployment frequency - how often you deploy changes to production
- Change failure rate - the percentage of changes that fail in production
- Failed deployment recovery time - how long it takes to restore service after a failure

Figure [1.2](#) is reproduced from the State of DevOps 2024 report. It shows the metrics for each of the four clusters of software delivery performance along with the approximate percentage of respondents in each cluster.

Performance level	Change lead time	Deployment frequency	Change fail rate	Failed deployment recovery time	Percentage of respondents*
Elite	Less than one day	On demand (multiple deploys per day)	5%	Less than one hour	19% (18-20%)
High	Between one day and one week	Between once per day and once per week	20%	Less than one day	22% (21-23%)
Medium	Between one week and one month	Between once per week and once per month	10%	Less than one day	35% (33-36%)
Low	Between one month and six months	Between once per month and once every six months	40%	Between one week and one month	25% (23-26%)

Figure 1.2 The four clusters of software delivery performance (reproduced from the State of DevOps 2024) and the approximate percentage of respondents in each cluster. Elite performers deliver software much faster and more reliably than low performers.

As you can see, elite performers deliver software much faster than others. For instance, elite performers perform 182 times as many deployments per year at low performers. They are able to learn and improve their products and services at a much faster rate. And not only that, they also deliver software much more reliably.

MOVE QUICKLY AND DON'T BREAK THINGS

You might find the idea that software delivery can be fast **and** reliable surprising. After all, the traditional approach to 'quality' is to proceed slowly and carefully. But, in fact, as the State of DevOps report shows, there's ample evidence showing that the opposite is true. Delivering smaller changes

more frequently improves reliability. Elite performers have a change failure rate that's 8 times lower than low performers. It's about moving fast and not breaking things!

FAST FLOW MEANS HAPPY DEVELOPERS

What's more, the State of DevOps report describes how developers in fast flow organizations are generally happier and more satisfied with their work. They find deployments far less painful since they occur more frequently. There are far fewer stressful deployments at midnight. Developers spend much less time firefighting and much more time building new features. The report also describes how they experience far less burnout.

FAST FLOW FOR BUSINESS SUCCESS

The Accelerate authors have found that if you develop software this way, your business will actually be much more successful in terms of profitability, market share and productivity. Their research consistently shows that high software delivery performance is a strong predictor of organizational performance. McKinsey have also described similar findings in their report **Developer Velocity: How software excellence fuels business performance** (<https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/developer-velocity-how-software-excellence-fuels-business-performance>). Let's now look at how to achieve fast flow: the fast flow success triangle.

1.1.3 The fast flow success triangle = DevOps + Team Topologies + Architecture

As I've previously mentioned, to achieve fast flow, an organization needs a combination of three things: development process, organization and architecture. Specifically, it requires what I call the fast flow success triangle. The fast flow success triangle is a simple yet effective graphic that I developed to describe the three elements and the relationships between them. As figure [1.3](#) shows, those three elements are DevOps as the development process, an organization structure based on the Team topologies patterns, and an architecture with the characteristics that enable DevOps and Team Topologies.

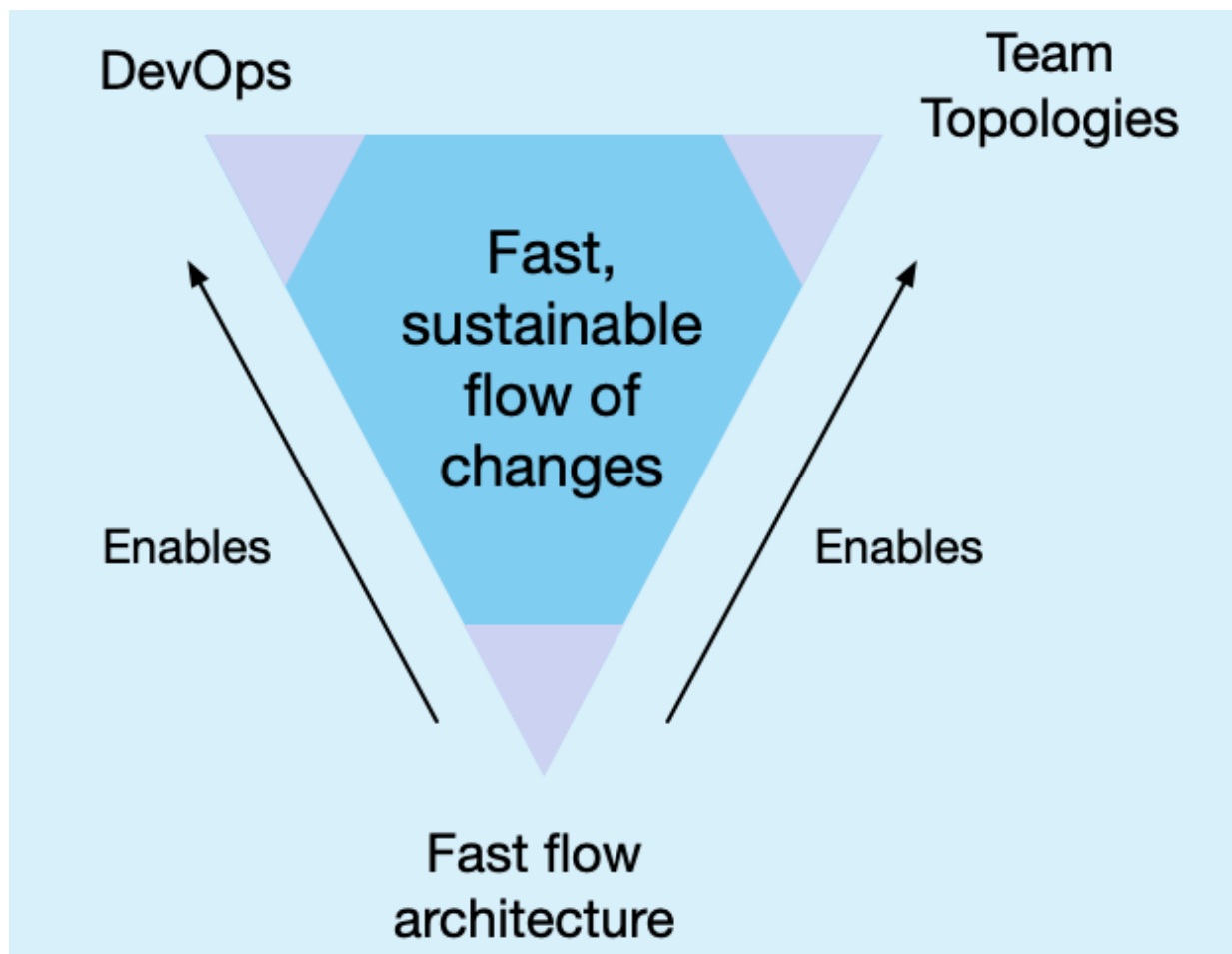


Figure 1.3 Fast flow requires a combination of three elements: DevOps as the development process, an organization structured using the Team Topologies patterns, and an architecture that enables DevOps and Team Topologies.

Let's look at each element of the success triangle in more detail starting with DevOps.

SUCCESS TRIANGLE PROCESS: DEVOPS

The first element of the success triangle is DevOps. DevOps, as defined by DevOps handbook, Second edition by Gene Kim, Jez Humble, Patrick Debois, John Willis, and Nicole Forsgren (IT revolution press, 2021), which is the go-to resource for understanding DevOps, is a set of principles and practices for delivering software quickly and reliably. It's not, as in the case of many organizations, a team name or a job title given to engineers who work on primarily infrastructure automation and deployments. DevOps is a way of working that the entire engineering organization must adopt.

DevOps has three groups of foundational principles, which are known as the three ways:

- First way - fast flow of changes from development into production.
- Second way - the fast flow of feedback from production back into development.
- Third way - a culture of continual experimentation and learning.

The first two ways embody the fast flow concept that I introduced earlier. The third way is also an essential ingredient of high performance software delivery.

The three ways describe numerous principles. There are too many to list here, but I will mention a couple of them here that are particularly relevant to fast flow. One notable first way principle is **reduce batch sizes**. Software development consists of a series of steps, such as requirements gathering, design, implementation, testing, and deployment.

The batch size refers to the size of the work item that flows through the steps. Traditional software development has large batch sizes and infrequent releases. The key idea of DevOps is that developers must practice continuous delivery. They must frequently commit and push small changes, ideally at least once a day. Each change is tested and then deployed into production.

Another notable first way principle is **reduce the number of hand-offs between teams**. The key idea is that hand-offs between teams introduce delays. They can also cause knowledge to be lost. We can reduce the number of handoffs through a combination of automation, platforms and tooling and adopting the ideas of Team Topologies, which I describe next.

SUCCESS TRIANGLE ORGANIZATION: TEAM TOPOLOGIES

The second element of the success triangle is Team Topologies, which overlaps with and complements DevOps. The go-to resource to understand Team Topologies is the book **Team Topologies: Organizing Business and Technology Teams for Fast Flow** By Matthew Skelton, and Manuel Pais (IT Revolution Press, 2019). Team Topologies is a set of principles and patterns for structuring an organization for fast flow. Most notably, it emphasizes small (5-9 people), long-lived teams, to reduce communication overhead and increase trust and psychological safety within each team.

As figure [1.4](#) shows, Team Topologies defines four types of teams and three ways that they may interact. The four team types are:

- stream-aligned team - responsible an end-to-end stream of work that turns requirements into code running in production
- platform team - provides a platform, which is an artifact, such as a tool, infrastructure, or library, that makes it easier for stream-aligned teams to get work done
- enabling teams - consults with stream-aligned teams to enable them to learn new capabilities and overcome obstacles
- complicated subsystem team - helps stream-aligned teams by developing software modules that require deep mathematical or technical expertise

Most of the work is done by stream-aligned teams, which embody the DevOps first way principle of minimizing hand-offs between teams. All work is done within the team, so there are no hand-offs required. Stream-aligned teams are assisted by the three other team types.

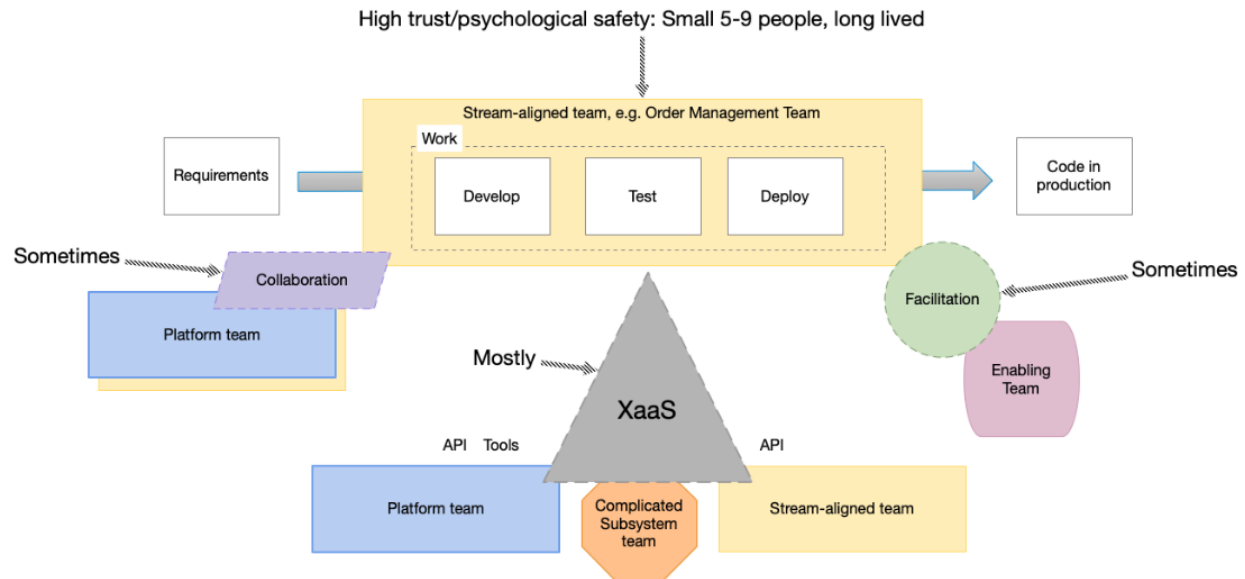


Figure 1.4 The four types of Team Topologies teams and the three ways that they interact. Stream aligned teams are responsible for turning requirements into code running in production. They are supported by platform, enabling, and complicated subsystem teams. Teams primarily interact via the X-aaS interaction style: consuming the work of another team "as a service". But sometimes they work together to discover new things or help each other improve their ways of working.

There are three interaction styles:

- X-aaS interaction style - one team consumes the work, such as an API or a tool, of another team "as a service".
- Collaboration - two teams working together to discover new things
- Facilitating - one team helping another team to improve their ways of working

The teams mostly work independently of each other using the X-aaS interaction style. Sometimes, however, teams might temporarily use one of the other two interaction styles. A stream aligned team might, for example, collaborate with a platform team to develop a new platform feature. Similarly, an enabling team might facilitate a stream-aligned team's learning of the new capability by

providing expertise, structured guidance, and practical support.

Now that we have looked at the DevOps and Team Topologies elements of the success triangle, let's look at the third element, which enables them: the architecture, which, as the focus of this book, has its own section in this chapter.

1.2 Architectural requirements for fast flow

The purpose of architecture - at least as far as fast flow is concerned - is to enable DevOps and Team Topologies. As you will see below, enabling the other two elements of the success triangle is the main reason to adopt microservices, and that you are unlikely to accelerate software delivery without them. In the same way that real world architects design spaces that meet the needs of people, software architects must create application architectures that enable the organization, which practices DevOps and is structured using Team Topologies, to deliver software quickly and reliably.

In this section, we will explore the two main architectural properties that fast flow requires. You will also learn how for some applications the monolithic architecture can enable fast flow, whereas for others, it's an obstacle. Let's begin by looking at the architectural requirements for fast flow.

1.2.1 Architectural requirements for fast flow

When we think of architecture, there's a tendency to focus on those aspects of architecture that satisfy end-user requirements, such as scalability, availability, and security. The reality, however, is that the architecture must also

satisfy the needs of the engineering organization that develops the software. A unique property of software is that its creators also live within it. The decisions that they make today influence the ease of development tomorrow. In particular, an application's architecture plays a critical role in enabling or preventing fast flow software delivery using Team Topologies and DevOps.

In order for a team to be able to quickly and easily implement a work item (feature, defect, etc.) and deploy the changes to production without needing to coordinate with other teams requires an architecture that has certain features. I describe these features in more detail in chapter 5, but for now, I'll provide a brief overview of two key features: loose design-time coupling and a fast, automated deployment pipeline. Let's first look at each of these features in more detail, starting with loose design-time coupling.

FAST FLOW REQUIRES A LOOSELY DESIGN-TIME COUPLED ARCHITECTURE

A key aspect of DevOps and Team Topologies is that teams are autonomous. Each team works mostly independently of other teams. It interacts with other teams primarily via the Team Topologies X-aaS interaction style. The team rarely needs to coordinate with other teams or wait for them to complete a work item. Hand-offs between teams are minimized in accordance with the DevOps first way principle.

In fact, the need for team autonomy is one of the key findings by the authors of Accelerate who researched the characteristics of high performance teams. To achieve team autonomy, you must design teams that are aligned to a stream of work, the so-called stream-aligned team concept from team topologies. But this is insufficient. You also need a loosely design-time coupled architecture.

Loose design-time coupling, which I describe in detail in chapter 4, is an incredibly important property of architecture in particular and software in general. If an application's architecture is loosely design-time coupled, then each team can change their part of the application without regularly needing other teams to change their code. Consider, for example, the architecture shown in figure 1.5. The `Orders` team owns the `Order Management` module, which uses the `Consumer Management` module that's owned by `Consumers` team. If the architecture is loosely design-time coupled, then the `Orders` team can change their `Order Management` module without regularly needing to coordinate with the `Consumers` team. In other words, the `Orders` team can be autonomous.

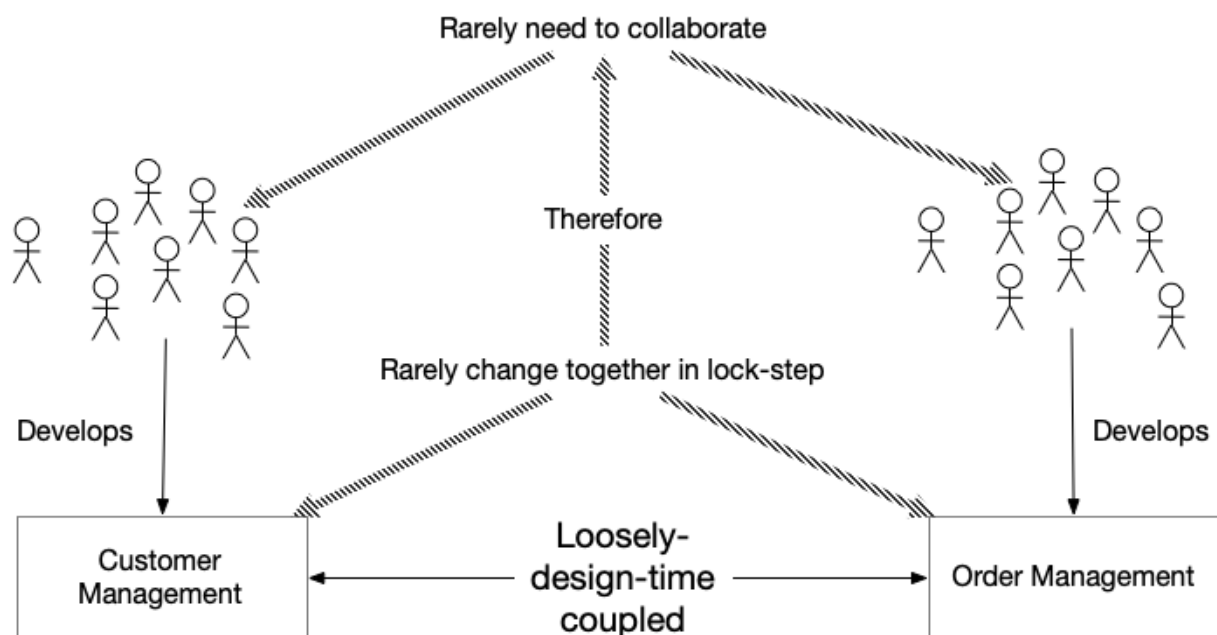


Figure 1.5 In order for teams to be autonomous, the software elements that they develop must be loosely design-time coupled

If, on the other hand, the architecture were tightly design-time coupled, then a change to one module would frequently require the other to be updated. The two teams would constantly need to coordinate with each other. The endless

meetings along with needing to wait for the other team to finish their changes would reduce both teams' productivity.

FAST FLOW REQUIRES A FAST DEPLOYMENT PIPELINE

The first architectural requirement for fast flow — loose design-time coupling — allows a developer to easily modify the codebase. The second requirement, however, focusses on what happens once that change is pushed or merged into the repository. To shorten the feedback loop, the push or merge of the change must trigger a deployment pipeline that rapidly compiles, tests, and deploys the update.

An automated deployment pipeline is a key part of the DevOps first way. It's the path that changes take from a developer's laptop to production. The deployment pipeline is triggered when a change is pushed to or merged into a code repository branch. It compiles, tests, and deploys the change.

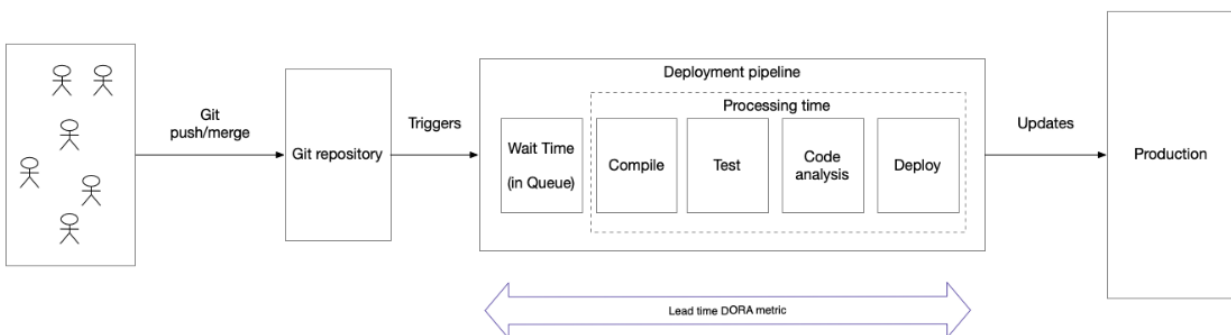


Figure 1.6 Fast flow requires a fast deployment that can handle the stream of changes from the development teams

The lead time DORA metric is the time that it takes for a change to flow through the deployment pipeline. A low lead time is what is meant by fast deployment pipeline. The key architectural challenge is keeping the lead time low as the application and its organization grows. The lead time is the sum of two times. First, there is the wait time, which is the

time that a change is queued until it can be processed by the deployment pipeline. Second, there is the processing time, which is the time that it takes the deployment pipeline to compile, test, analyze and deploy the change.

For most Java applications, the compilation time is negligible. Most of the time is spent running the tests. The deployment time varies depending on the application and the deployment technology. The wait time is proportional to the frequency of changes and the processing time of the deployment pipeline. For large applications developed by large teams, there's the double risk of long wait times and long processing times.

1.2.2 The Monolithic architecture can enable fast flow

A monolithic application consists of a single codebase that's deployed as a single deployable unit (e.g., executable JAR or WAR file) by a single deployment pipeline. Contrary to certain rumors, the monolithic architecture is not an architectural anti-pattern. Nor, is it inevitably a big ball of mud (Big Ball of Mud by Brian Foote and Joseph Yoder, <http://www.laputan.org/mud/>), a tangled mess of spaghetti code. In fact, the monolithic architecture is a good choice for many applications and is fully capable of enabling fast flow. In chapter 6, I describe the Modular Monolith pattern, which is a fantastic improvement to the traditional three-layer monolith because it can improve team autonomy and reduce the deployment pipeline's execution time.

The challenge, however, is that an application and its organization can still outgrow a modular monolith. If they continue to grow, then sooner or later team autonomy declines and the deployment pipeline becomes a bottleneck.

Let's look at each of these two problems in more detail, starting with the decline in team autonomy.

TEAM AUTONOMY DECLINES AS THE APPLICATION AND ORGANIZATION GROW

The first problem with a large monolith is that some changes can be very difficult to make because they impact numerous teams. Consider, for example, the application shown in figure 1.7. Most modules are loosely design-time coupled and can be changed easily. But there are, however, some changes that can have widespread impact, such as breaking changes to APIs of widely used modules; upgrades of widely used 3rd party libraries that have breaking API changes; and upgrades of the JVM.

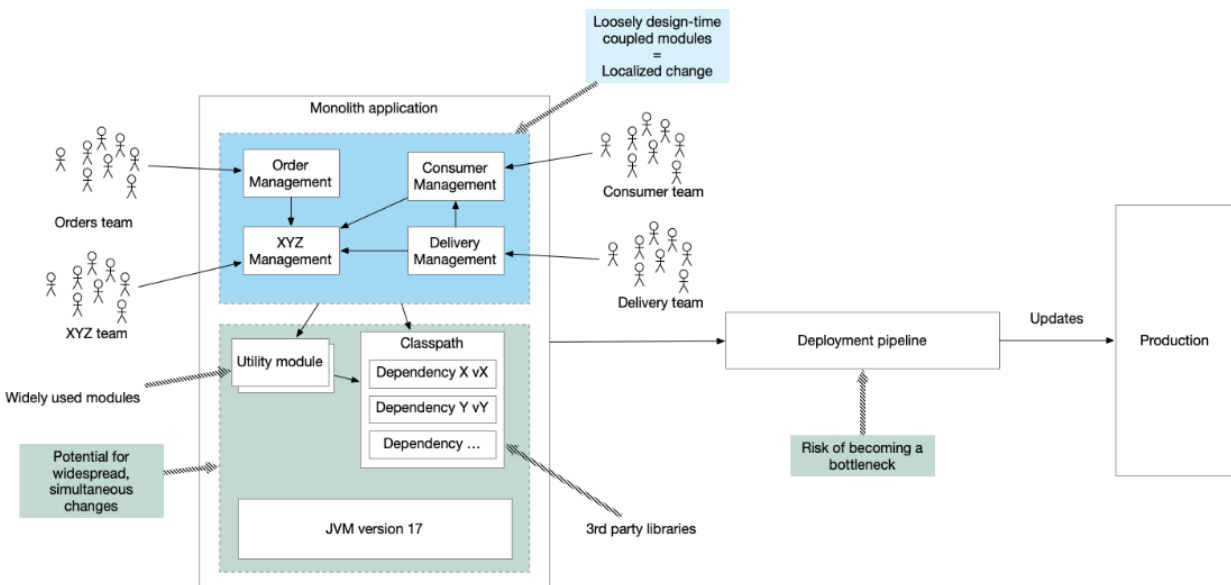


Figure 1.7 Even though most modules in this monolithic application are loosely design-time coupled, since there is a single code base, some changes can be difficult to make because they impact numerous teams. A change to a utility module might require simultaneous changes to many other modules. Upgrading 3rd party libraries or the language runtime might also require widespread changes. There's also a risk that the deployment pipeline becomes a bottleneck due to the large codebase and large number of developers.

If while implementing a new feature, the `Orders` team, for example, needs to make such a change then they would have the unenviable task of coordinating with the other teams. Since each team has their own priorities and schedules, coordinating these types of changes can be difficult. In some cases, it can be virtually impossible. The `Orders` team might need to find a workaround to avoid making a breaking change.

THE DEPLOYMENT PIPELINE BECOMES A BOTTLENECK

The second problem is that the monolith's single deployment pipeline can become a bottleneck. As the application grows, it has more tests, which take longer to execute and so the deployment pipeline's processing time increases. And to make matters worse, as the number of developers grows, the frequency of changes increases, which can overwhelm the deployment pipeline and result in long wait times. The deployment pipeline could batch changes to reduce wait times, but this violates the DevOps principle of **reduce batch sizes** and complicates troubleshooting build failures and production outages.

Also, a monolith might even be so large that it's not practical to test it thoroughly on a developer's laptop. It might simply be too large and have too many dependencies to run locally. Or, even if the tests can be run locally, they might take too long. Developers might end up relying on the deployment pipeline to test their changes or have to ship them to a dedicated testing environment with sufficient resources to run them quickly. As a result, developers no longer receive fast local feedback about their changes.

For large applications developed by large organizations, the solution to the lack of team autonomy and the slow deployment pipeline is to use the microservice architecture.

1.3 A brief introduction to the microservice architecture

While the monolithic architecture has various benefits, such as simplicity, it's often an obstacle to fast flow when the application and the organization are large. Teams spend too much time coordinating changes. The deployment pipeline is a bottleneck. In such cases, the microservice architecture is often a better choice.

But what is the microservice architecture? Does it actually mean lots of little services, each 100 lines of code (LOC) or two weeks worth of work? How does it enable fast flow? What exactly are its drawbacks? When should you use it? In the remainder of this chapter, I will answer these and other questions. Let's start by looking at a definition of the microservice architecture.

1.3.1 The microservice architecture: a definition

The microservice architecture, which I describe in more detail in chapter 7, is defined as follows:

The microservice architecture is an architectural style that structures an application as a collection of two or more loosely design-time coupled, independently deployable services.

On the surface, this is a very simple definition. But there is, however, a lot of depth to it, since it's a bundle of four key concepts. Let's break down the definition one concept at a time.

THE MICROSERVICE ARCHITECTURE IS AN ARCHITECTURAL STYLE

The first important concept is an architectural style. In the same way that there are real world architectural styles, such as Victorian and Art Deco, which define a building's overall appearance, arrangement of elements, materials, etc., there are software architectural styles, which dictate how an application's architecture is structured. Examples of common software architecture styles, which are discussed later in this book, include a layered architecture and a hexagonal architecture. The monolithic architecture is also an architectural style.

THE APPLICATION CONSISTS OF TWO OR MORE SERVICES

The second important concept is that an application consists of two or more services. A service is a deployable unit, such as a Java executable JAR or WAR file, that implements one or more business capabilities (also known as subdomains), which are slices of business functionality. Figure [1.8](#) shows a simple example of a microservice architecture. The application consists of three services - `Customer Service`, `Order Service`, and `Delivery Service` - each of which implements a slice of business functionality. This architecture also has an `API Gateway`, which routes external traffic to the services and is covered in more detail in chapter 12.

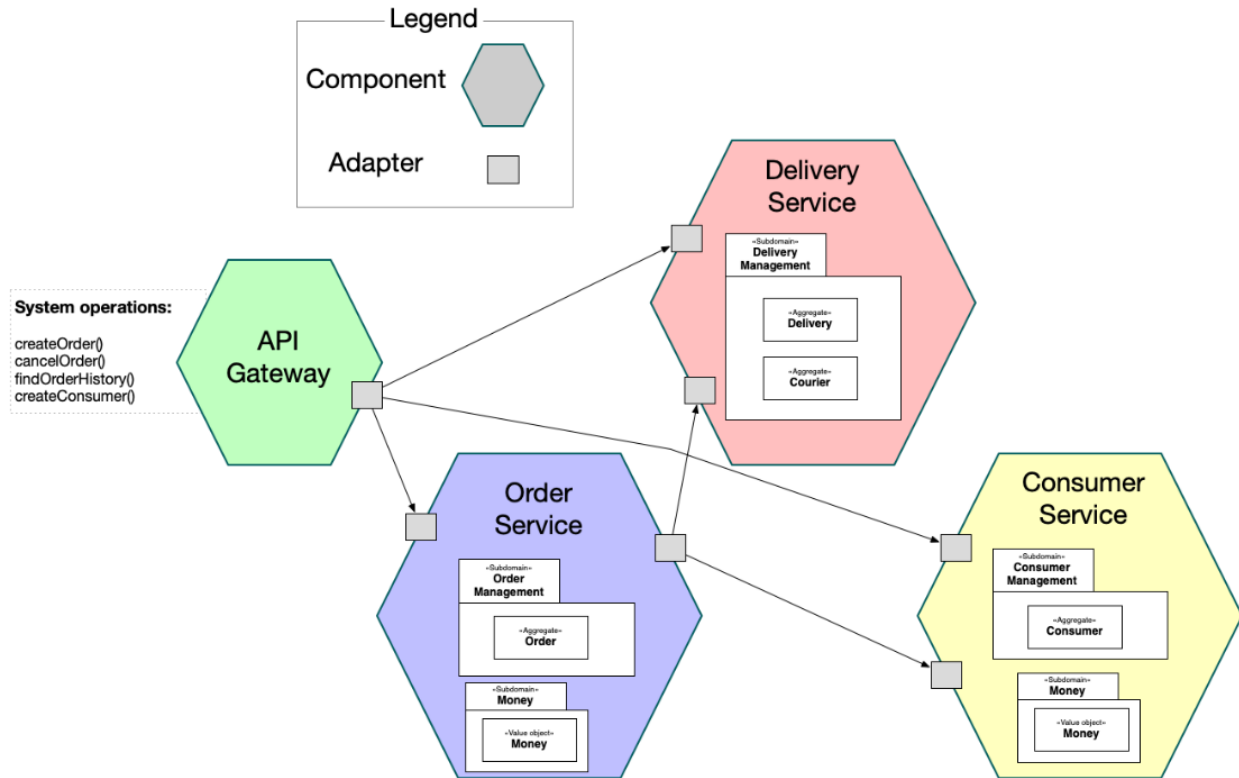


Figure 1.8 A microservice architecture consists of two or more services, most of which implement slices of business functionality, such as Order Service and Delivery Service

In essence, what would otherwise be a large monolithic application consists of multiple, smaller applications. It also means that the application has a distributed architecture where some, but not all, requests will span multiple services. For example, a `POST /consumers` request, which creates a consumer, is handled solely by the `Customer Service` whereas the `POST /orders` request, which creates an order, is handled initially by the `Order Service`, which then collaborates with the other two services. As you will see, the distributed nature of the microservice architecture is a double-edged sword. Not only is it responsible for many of the architecture's benefits, but it's also the cause of many of the architecture's downsides.

MICROSERVICE ARCHITECTURE != LOTS OF LITTLE SERVICES

A common misconception is that a microservice architecture must consist of lots of little services. There are several reasons for the confusion: The first is that the name "MICROservice architecture" implies small and numerous. The second reason for the confusion is that early definitions of service size included 100 LOC, or 2 weeks worth of work. There were also numerous high-profile examples of microservice architectures that consisted of hundreds or even thousands of services, some of which were described as having a death star architecture since their service dependency graph resembled the Star Wars Death Star.

While some of those organizations might have had legitimate reasons for having so many services, that's not a pattern to blindly emulate. You risk being overwhelmed by excessive complexity. I am always suspicious of architectures that have more services than there are teams. A good rule of thumb is that a service should exist to solve a tangible problem. You will often find that a simple architecture consisting of fewer, larger services, such as the one shown in figure [1.8](#) is a better choice.

THE SERVICES ARE LOOSELY DESIGN-TIME COUPLED

The third concept is that the services are loosely design-time coupled. This means that a change to one service rarely requires a change to another service. Loosely design-time coupled services enable the teams that own those services to mostly interact through X-aaS interaction style. They only need to collaborate occasionally to evolve service APIs.

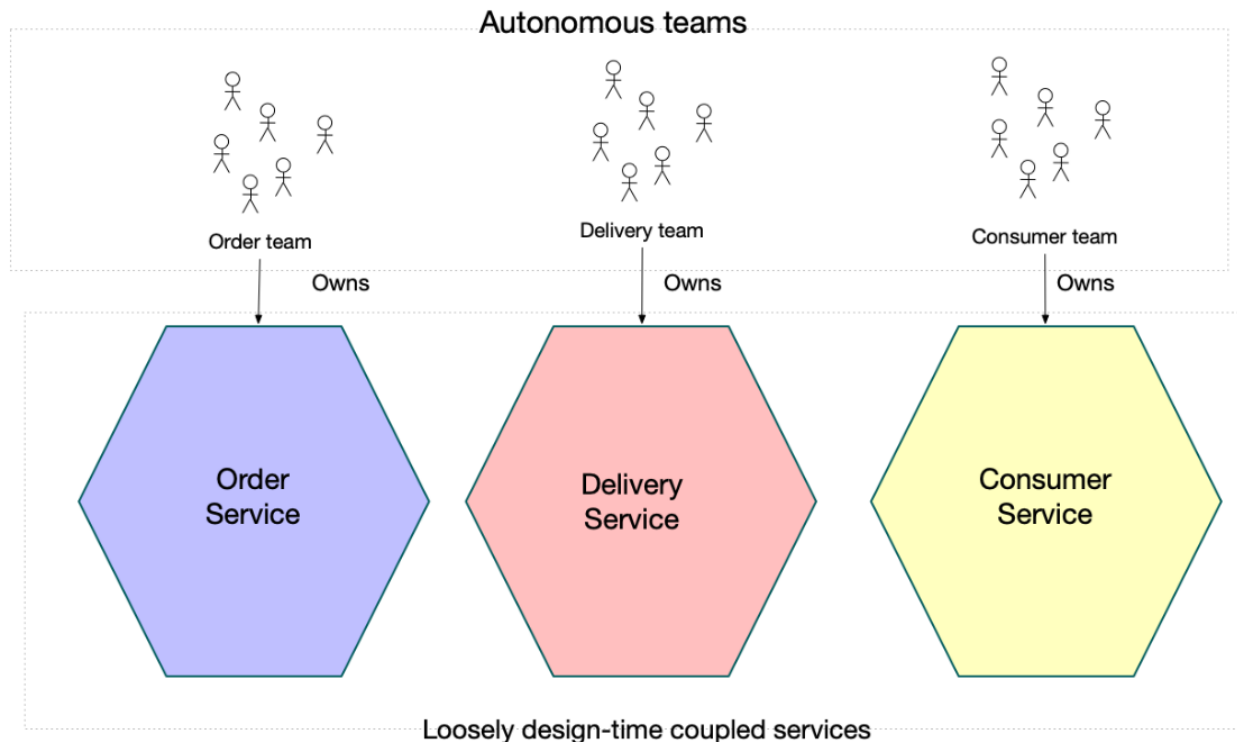


Figure 1.9 The services in a microservice architecture are loosely design-time coupled. A change to one service rarely requires a change to another service. Loose design-time coupled services enables the teams that own those services to be autonomous.

What's more, even though the application is large and complex, a service developer can focus on their relatively small, simpler business domain. Their day-to-day experience is much like that of a developer working on a small application.

THE SERVICES ARE INDEPENDENTLY DEPLOYABLE

The fourth concept is that the services are independently deployable. While this might appear to simply restate that a service is a deployable unit, it doesn't. Instead, as figure [1.10](#) shows, the concept of independent deployability it has a far more profound meaning: a service has its own deployment pipeline, which builds and tests the service in isolation from other services, and then deploys it to

production. There are no slow, brittle, and complex-to-write end-to-end tests that involve other services.

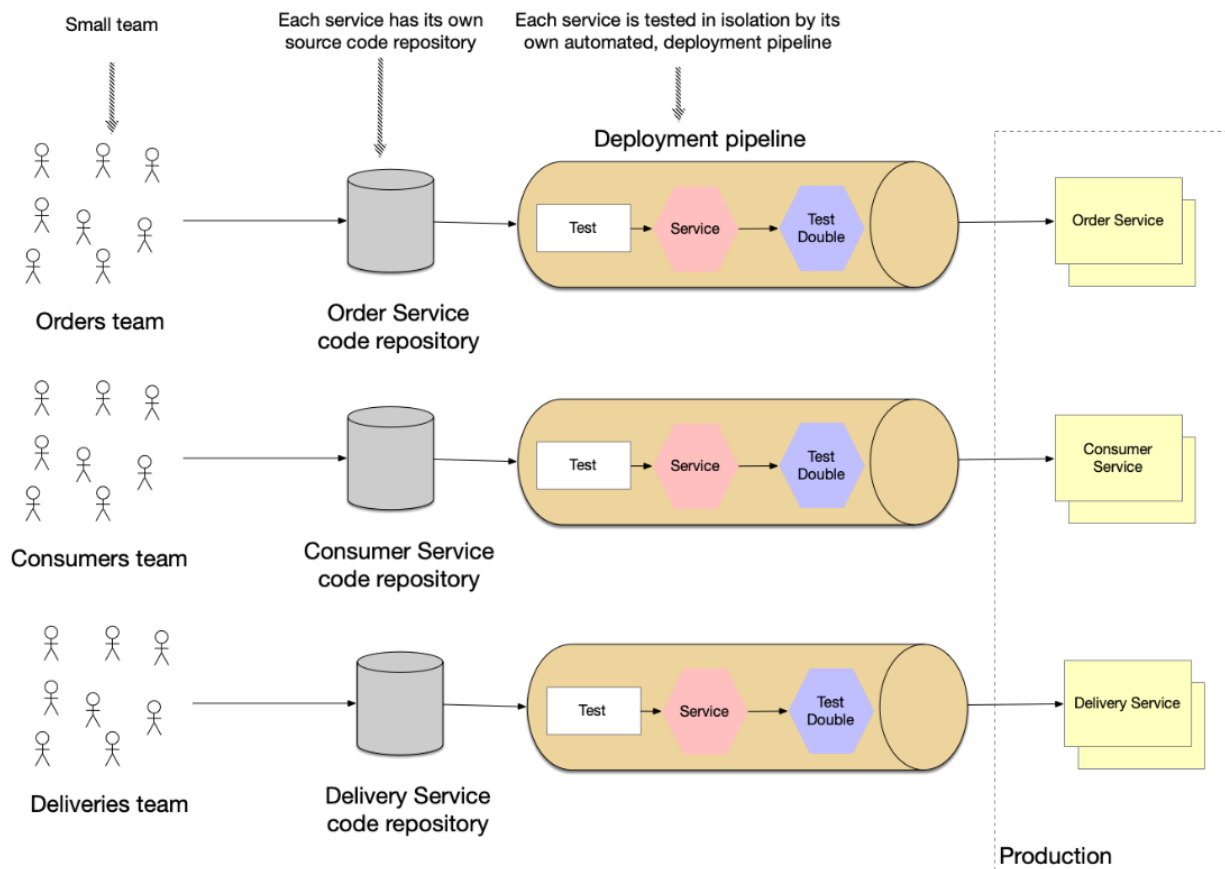


Figure 1.10 A service must be independently deployable: packaged as a deployable or executable unit, has its own deployment pipeline and is production-ready after it has been tested in isolation.

Also, since a service is developed by a small team, the frequency of changes that flow through the service's deployment pipeline is relatively low and delays are rare. As a result, a service developer can quickly and easily make a change and deploy it to production.

1.3.2 The microservice architecture is a fast flow architecture

The primary benefit of the Microservice Architecture and the biggest motivation for its adoption is that it enables fast flow

for large complex applications developed by a large organization. There are two reasons why it enables fast flow. The first is that it promotes a high degree of team autonomy. The second reason is that it has fast deployment pipelines. Let's look at each of these benefits in more detail, starting with team autonomy.

THE MICROSERVICE ARCHITECTURE PROMOTES TEAM AUTONOMY

Since a microservice architecture consists of multiple, loosely design-coupled services, it provides a high degree of team autonomy that's essential for fast flow. Because the services are loosely design-time coupled, a team can change their service without regularly needing to coordinate with the 10s or 100s of other teams. What's more, since each service has its own technology stack, a service team can change it freely. A team can, for example, much more easily upgrade to a new version of a library that requires code changes.

Also, because each team works on a separate code base, there's less opportunity for accidental interference. If, for example, the `Orders` team breaks their build, that's their problem. The other teams, including the `Delivery` team, can still deploy their changes.

THE MICROSERVICE ARCHITECTURE HAS FAST DEPLOYMENT PIPELINES

The other reason why the microservice architecture enables fast flow is that it has fast deployment pipelines and, hence, a low lead time. The lead time is low because, as figure [1.11](#) shows, a service has a dedicated deployment pipeline that is handling relatively infrequent changes to a code base that is relatively small and faster to test. A service's deployment pipeline simply has much less work to do than a monolith's

deployment pipeline. It's unlikely that it will be a bottleneck. Developers spend less waiting for builds to complete and are more productive.

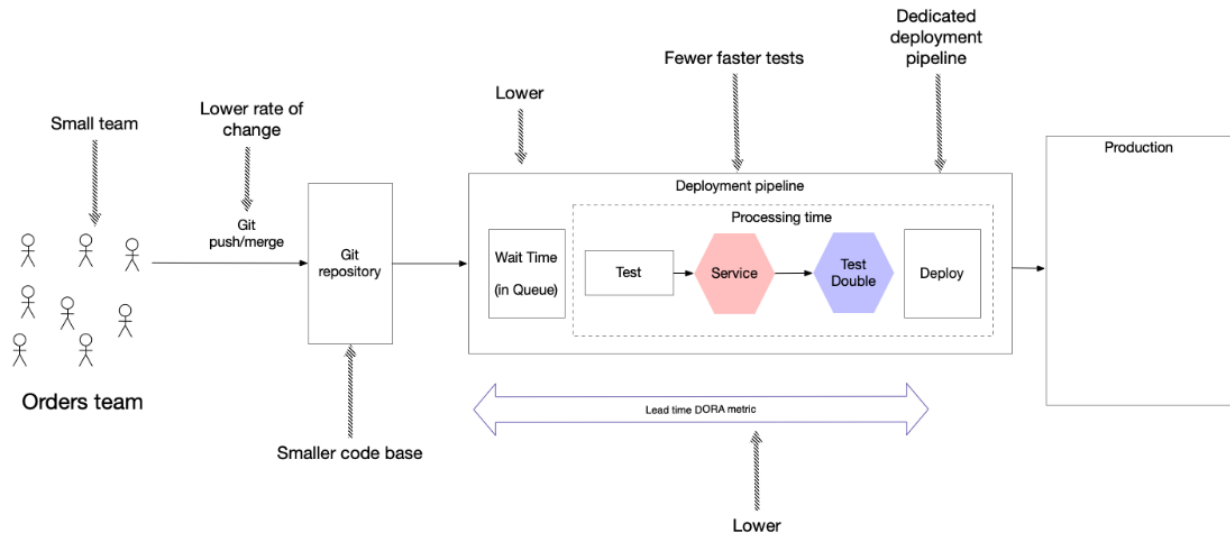


Figure 1.11 A change to a service that's part of a microservice architecture can be quickly deployed to production because every service has its own deployment pipeline, which is handling relatively infrequent changes to a code base that is relatively small and fast to test, especially since services are tested in isolation

A service is just one part of an application, and so it has a much smaller code base. There are fewer tests, and because it's tested in isolation, there are no slow end-to-end tests. Consequently, the deployment pipeline's processing time is lower.

Also, because a service is developed by a small team, the frequency of changes to the service is lower than for the entire application. Consequently, the deployment pipeline's utilization is lower. It's unlikely that changes will be queued for long periods of time.

Furthermore, a service is usually small enough to be quickly tested on a developer's laptop. A developer can get feedback quickly and easily without needing to wait for the deployment pipeline to complete, or worse, having to ship

their code to a dedicated testing environment before running any tests.

1.3.3 Other benefits of the microservice architecture

In addition to enabling fast flow development of large complex applications by large organizations, the microservice architecture has other important benefits. It supports multiple technology stacks. It also enables application modules with different characteristics to be packaged as separate services, which can improve various runtime and development-time architectural properties of the application. Let's look at each of these benefits in more detail, starting with support for multiple technology stacks.

SUPPORT FOR MULTIPLE TECHNOLOGY STACKS

An important benefit of the microservice architecture is that it supports multiple technology stacks. For example, figure [1.12](#) shows an architecture containing a mixture of technology stacks. There are services using different languages runtimes as well as different versions of the same technology stack.

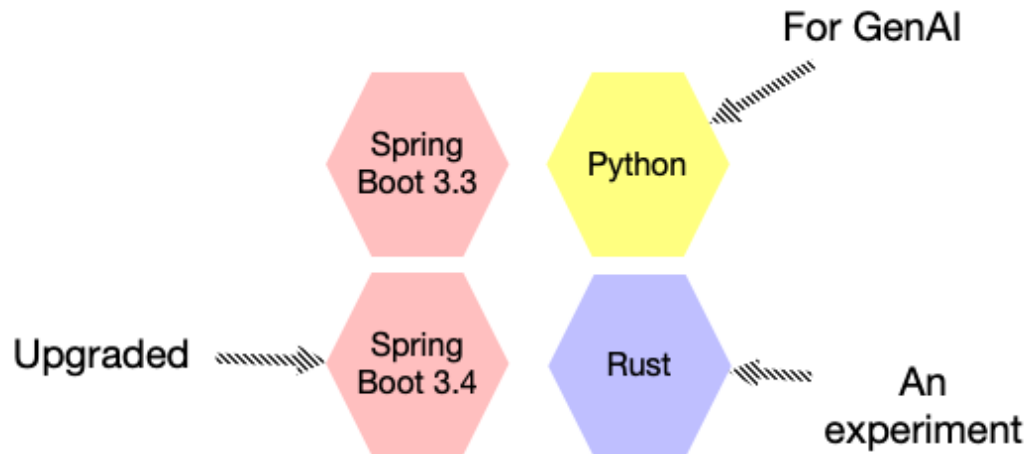


Figure 1.12 Different services can use different technology stacks. This enables incremental upgrades, experimentation with new technologies, and the use of the best technology for the job.

The ability for different services to use different technology stacks is important because it enables teams to:

- Pick the best technology for the job, such as Java for most services and Python for AI-related services
- Incrementally upgrade an application's technology stack(s) one service at a time, which is less risky and easier to schedule
- Conduct low cost and meaningful experiments with new technologies stacks

As a result, an organization can much more easily keep an application's technology stack up to date throughout the years or decades of its lifetime.

SEPARATE DEPLOYMENT OF APPLICATION MODULES WITH DIFFERENT CHARACTERISTICS

Another benefit of the microservice architecture is that application modules with different characteristics can be packaged as separate services in order to improve architectural properties, such as scalability, availability,

security and ease of development. You could, for instance, improve the availability of critical functionality by putting its implementation in its own service that runs on its own high availability infrastructure, which isolates it from changes to the rest of the application. I describe how other architectural properties can be improved by segregating modules by characteristics in chapter 7.

1.3.4 No such thing as a silver bullet: the drawbacks of microservices

As Fred Brooks said back in 1986 (No Silver Bullet Essence and Accidents of Software Engineering by Fred Brooks) "There is no silver bullet". Like every other technology, the microservice architecture has both benefits and drawbacks. The microservice architecture has five key drawbacks:

- Distributed operations can be more complicated to design
- There is the risk of creating a distributed monolith
- Designing a good microservice architecture requires a level of rigor and discipline that many organizations are unaccustomed too
- The microservice architecture requires the rest of the fast flow success triangle
- Embracing independent deployability is a significant change

Let's look at each of these drawbacks in more detail, starting with how distributed operations can be more complicated to design.

DISTRIBUTED OPERATIONS CAN BE COMPLICATED TO DESIGN

Some operations in a microservice architecture will span multiple services. Figure 1.13 shows a simple example of a `createOrder()` operation that spans the `Order Service` and the `Customer Service`. Distributed operations, such as `createOrder()`, are more complicated to design because services are process and transaction boundaries. You must solve problems that simply do not occur in a monolithic architecture.

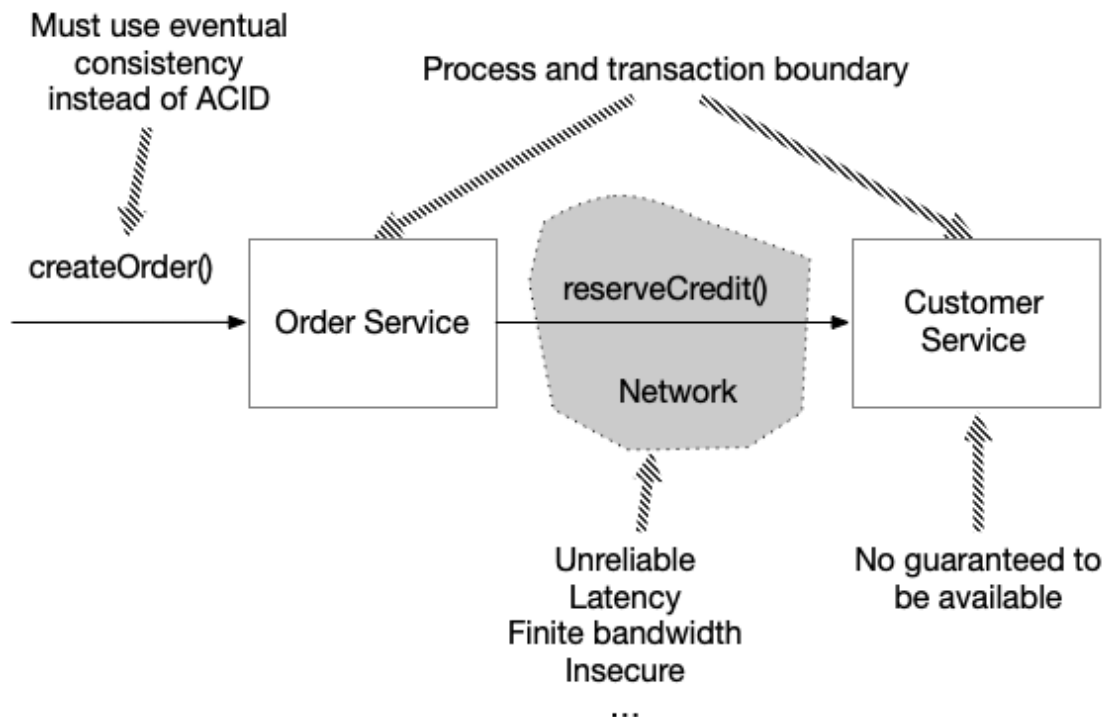


Figure 1.13 Some, but not all, operations in a microservice architecture will span multiple services. Distributed operations are more complicated to design because services are process and transaction boundaries. This creates numerous design challenges since the network is not free, it's unreliable, and not all services are guaranteed to be simultaneously available. You must also use an unfamiliar transaction model known as eventual consistency.

The first problem is that since services are process boundaries, they must communicate over the network. You often need to handle failure modes that don't occur within a monolith. For example, the `Order Service` must handle the scenario where the `Customer Service` is unavailable.

Also, as stated by the Fallacies of Distributed Computing (https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing), the network isn't free. You must ensure that distributed operations are efficient. You must also avoid introducing runtime coupling, which occurs when services call each other synchronously. Runtime coupling is a problem because it reduces availability and increases latency. Careful design is required to avoid creating a Christmas tree light architecture (<https://akfpartners.com/growth-blog/microservice-anti-pattern-calls-in-series-the-xmas-tree-light-anti-pattern>), where a failure in one service cascades through the system and causes a production outage.

The second problem is that for reasons I'll discuss in chapter 7, operations, such as `createOrder()`, that span multiple services cannot be implemented as a single database transaction. Instead, you must implement it using an unfamiliar and more complex transaction model known as eventual consistency. An eventually consistent operation consists of a series of transactions in each of the services. In this model, you must sometimes write undo logic and handle concurrency explicitly, rather than simply relying on the database to manage it for you.

THERE IS THE RISK OF CREATING A DISTRIBUTED MONOLITH

There's also the risk of creating a distributed monolith, which consists of tightly design-time coupled services that change in lockstep. It is an anti-pattern that combines the worst aspects of the monolithic and microservice architectures. The problem is that you cannot simply make a breaking change to a service's API because that will cause a production outage. Instead, you must carefully and incrementally evolve an API in ways that don't break the

service's clients. It can be a lot of work and require a lot of coordination between the teams that own the affected services.

DESIGNING A MICROSERVICE ARCHITECTURE REQUIRES RIGOR AND DISCIPLINE

Fortunately, we can avoid many of the problems of a distributed architecture by carefully defining the service boundaries. We need to be vigilant to ensure that the services are loosely design-time coupled. We can also, for example, design the services so that every performance critical operation is implemented by a single service. This might mean that the services are larger than you would normally want, but quite often it's a worthwhile trade-off.

The challenge, however, is designing a good microservice architecture requires rigor and discipline that many organizations are unaccustomed to. There's a tendency to not think through design decisions, especially when deadlines are looming. After all, in a monolithic application, you can simply add a new class anywhere in the code base and the application will work. Unfortunately, such a haphazard approach to software design doesn't work in a microservice architecture.

For example, one common problem is the **More the Merrier anti-pattern** where an organization takes the 'micro' in microservices far too literally and blindly adds services. Unfortunately, this style of development results in an overly complex, fine-grained architecture that's difficult to understand and maintain. The architecture is also inefficient because of the overhead of the network calls. And it's brittle because of the runtime coupling between the services.

Design decisions about which service should implement a feature and whether to add a new service are extremely consequential. In particular, as I mentioned earlier in [Section 1.3.1](#), a service should exist only if it solves an actual problem. You need to consider numerous trade-offs when designing a microservice architecture. In chapter 20, I describe Assemblage, which is a design process for doing exactly that. And, throughout this book, I describe various patterns from the Microservice Pattern Language that can help you design a good microservice architecture.

THE CONSEQUENCES OF LOTS OF LITTLE SERVICES

I once met with a team that was having problems testing their services. I asked my usual questions: "How many services do you have?", "How many developers do you have?". The answer to both questions was five. My advice was to simplify their architecture by merging the services into a single service. As an added benefit, it solved their testing problem.

THE MICROSERVICE ARCHITECTURE REQUIRES THE REST OF THE FAST FLOW SUCCESS TRIANGLE

The third challenge is that in order to benefit from the microservice architecture, an organization must adopt the other elements of the fast flow success triangle: DevOps and Team Topologies. For some organizations, changing how the work is done is extremely difficult. Just look, for example, at how organizations have spent the past 20+ years struggling and often failing to adopt the principles of the Manifesto for Agile Software Development (<https://agilemanifesto.org/principles.html>). The trouble is that without these changes, the microservice architecture is

unlikely to deliver the benefits that are promised. It's like buying a Ferrari and pulling it around with a horse.

EMBRACING INDEPENDENT DEPLOYABILITY IS A SIGNIFICANT CHANGE

The fourth challenge is that organizations often struggle to embrace the concept of independently deployable services. A key benefit of microservice architecture is that each service has its own deployment pipeline. Yet some organizations struggle to adopt this concept. They insist on running end-to-end tests of the entire application before deploying an updated service.

The problem is that end-to-end tests involve a large number of moving parts which makes them slow and prone to random failures. Manual review is required to determine whether a change is releasable. End-to-end tests prevent the organization from achieving the goal of deploying changes quickly and easily. The solution is to rethink the approach to testing and move end-to-end tests out of the deployment pipeline.

1.4 When to use the microservice architecture

If you are contemplating adopting the microservice architecture, the first thing to remember is that the monolithic architecture is a perfectly good choice for many applications. As described earlier, it's simple, easy to understand, and easy to develop - at least when the application is small. Consequently, you should not automatically choose the microservice architecture for your application. Instead, you should choose the architecture that best fits your application's requirements. Or to put it another

way, you should pick the architecture whose benefits outweigh its drawbacks for your application.

There are two scenarios when you need to decide between the monolithic architecture and the microservice architecture. The first scenario is when building a brand-new application. The second is when you have a monolithic application that you feel has outgrown the monolithic architecture. In both scenarios, careful analysis is required to determine whether the microservice architecture is the right choice. Let's look at each of these scenarios in more detail, starting with the greenfield scenario when you build an application from scratch.

1.4.1 Building an application from scratch: typically a monolith-first approach

Let's imagine that you are starting the development of a brand-new application using a list of the latest, shiny new technologies. You might be tempted to include the microservice architecture in that list of modern technologies. The reality is, however, that it's unlikely that you have the fast flow-related problems that the microservice architecture addresses. You haven't written a line of code, so your application won't be large and complex for some time, maybe never. You don't have a large organization, so you don't need to worry about team autonomy. You can, therefore, achieve fast flow with a monolithic architecture.

While fast flow is not normally a motivation for starting with the microservice architecture, it's possible that some of the benefits outlined in [Section 1.3.3](#) might be. If your application needs to be written using a mixture of technologies, then a simple microservice architecture consisting of a few services makes sense. Or perhaps, you can address significant scalability, availability and security

problems by segregating modules that have different characteristics in their own services.

Even if the microservice architecture is a good fit for your application, you risk creating a distributed monolith. You need stable APIs in order for services to be loosely design-time coupled. But when you are building a brand-new application, it's unlikely that you understand the problem domain well enough to design stable APIs. You will often need to iterate on boundaries and APIs before they become stable. It's far easier to do that in a monolithic architecture than in a microservice architecture. Of course, there are exceptions, such when you must use multiple technologies or there are compelling reasons to segregate modules, but I strongly recommend a monolith-first approach.

1.4.2 Refactoring a monolith to the microservice architecture

The second scenario for adopting the microservice architecture is when your application has outgrown its monolithic architecture. Perhaps, the pace of software development has slowed down. Deployments have become less frequent. Or, teams are spending increasing amounts of time in meetings coordinating changes.

On the surface, these are all signs that to achieve fast flow you need to adopt the microservice architecture. But before you make that decision, you must be sure that the monolithic architecture is the root cause of the slow software development. And, if it is, you must migrate to microservices in a way that minimizes risk and maximizes the delivery of value. Let's first look at how to understand the root cause of slow software development.

UNDERSTAND THE ROOT CAUSE OF SLOW SOFTWARE DEVELOPMENT

The challenge that you face in this scenario, however, is being sure that the monolithic architecture is preventing fast flow. Perhaps you simply need to modularize the monolith so that teams own loosely design-time coupled modules. Or perhaps, the problem is that you have not properly implemented the other elements of the fast flow success triangle: DevOps and Team Topologies. You need to adopt DevOps as your way of working and Team Topologies as your organizational structure. You also need an automated deployment pipeline. If and only if these changes are insufficient to achieve fast flow should you consider migrating to the microservice architecture.

INCREMENTALLY MIGRATE TO THE MICROSERVICE ARCHITECTURE

If you decide to migrate your application to the microservice architecture, the next decision you must make is how and where to start. What not to do is big bang rewrite, where you write a brand new microservices-based application from scratch. While it's always tempting to start from scratch since you can ignore the existing mess, it's also risky. You don't deliver anything of value until the new application is complete, which is likely to far in the future. Also, you don't get any feedback on any of the design decisions you made until the distant future. As Martin Fowler is rumored to have said, "A big bang rewrite often results in a big bang".

As I describe in chapter 21, a much better approach is to use the Strangler Fig pattern, which incrementally migrates functionality from the monolith to the microservice architecture. Not only are you delivering value early and often, but you are also getting constant feedback on your

design decisions. You can also implement new features that the business needed yesterday directly as services. It's not always easy to do because you are breaking apart a legacy code base, but it's almost always better than the big bang rewrite.

1.5 Summary

- In order to thrive in today's world, businesses need fast flow, which is the continuous delivery of small, valuable changes to customers and continuous stream of feedback on each change to back to development.
- Fast flow requires the success triangle: a combination of DevOps, Team Topologies and an architecture that enables them by promoting team autonomy and fast deployment pipelines.
- DevOps is a set of principles and practices for delivering software quickly and reliably.
- Team Topologies is a set of principles and practices for designing organizations to enable fast flow.
- A monolithic architecture consists of a single codebase that's deployed as a single deployable unit, such as an executable JAR or WAR file, by a single deployment pipeline.
- While the monolithic architecture is not an anti-pattern, some applications, especially those that are large and are developed by large teams, must typically use the microservice architecture in order to achieve fast flow.
- A microservice architecture consists of two or more loosely design-time coupled and independently deployable services, which are characteristics that enable teams to be autonomous and the deployment pipelines to be fast.

- The microservice architecture has several drawbacks, including the potential complexities of a distributed architecture and need for rigorous design practices, which many organizations are unaccustomed to.
- An organization should typically start with a monolith-first approach and migrate to microservices if and only if the application outgrows its architecture.

2 FTGO: A case study in moving from one kind of development hell to another

This chapter covers

- How an organization can outgrow a monolithic architecture
- How migrating a monolith to microservices can go terribly wrong

Many organizations have enthusiastically adopted the microservice architecture, only to then struggle with it. In this chapter, I'm going to tell the story of a fictitious, online food delivery business called Food to Go (FTGO) that did exactly that. Due to their success, the FTGO organization and their application had outgrown their monolithic architecture. As a result, the microservice architecture was an excellent fit for the scale and complexity of their organization and main application. But sadly, as the story describes, their enthusiasm for the microservice architecture combined with the lack of experience led them to make several critical mistakes - mistakes that I've seen numerous real organizations make. FTGO ended up replacing one kind of architectural hell with another.

This chapter consists of two parts. The first part describes how FTGO outgrew their monolithic architecture and the problems that resulted. The second part of the chapter describes their disastrous attempt to fix those problems by

adopting the microservice architecture. You will notice that many of the key ideas in chapter 1 are absent. FTGO focussed on the microservice architecture and ignored important concepts, such as the fast flow success triangle. As a result, they were doomed to fail.

Throughout this chapter, you will most likely see problems that you have experienced or are currently experiencing. Perhaps your application has outgrown its monolithic architecture or your migration to microservices has not gone as planned. But don't worry. In later chapters, you will learn how to avoid the problems that FTGO experienced. Let's start with exploring how FTGO outgrew their monolithic architecture.

2.1 FTGO architecture V1: The eventual descent into monolithic hell

Like many other applications that were developed in the early 2000s, the first version of the FTGO application was a monolith. It consisted of a single Java Web Application Archive (WAR) file. Initially, the monolithic architecture worked well. It enabled FTGO to grow by leaps and bounds and become one of the leading online food delivery companies in the United States.

But as the business grew, so did the application and the engineering organization. The engineering organization grew from a single team to over twenty teams. The application became large and complex. Despite the best efforts of the FTGO engineering organization, it became an example of the Big Ball of Mud pattern (<http://www.laputan.org/mud/>). To quote Foote and Yoder, the authors of that pattern, it's a "haphazardly structured, sprawling, sloppy, duct-tape and bailing wire, spaghetti code jungle." As a result,

development slowed down to the point that it even jeopardized FTGO's plans to expand overseas.

This section starts by describing the original version of the FTGO architecture. Then it talks about why the monolithic architecture worked well initially. After that, we'll get into how the FTGO application outgrew its architecture and how development became hell.

2.1.1 The architecture of the FTGO application

At its core, the FTGO application is quite simple. Consumers use the FTGO website or mobile application to place food orders at local restaurants. FTGO coordinates a network of couriers who deliver the orders. It's also responsible for paying couriers and restaurants. Restaurants use the FTGO website to edit their menus and manage orders. The application uses various web services, including Stripe for payments, Twilio for sending messages, such as order status updates, and Amazon Simple Email Service (SES) sending email messages, such as order confirmations and receipts.

FTGO is a typical enterprise Java application. Figure [2.1](#) shows its architecture. The FTGO application has a hexagonal architecture, which is an architectural style described in more detail in chapter 13. In a hexagonal architecture, the heart of the application is the business logic. Surrounding the business logic are various adapters that implement UIs and integrate with external systems.

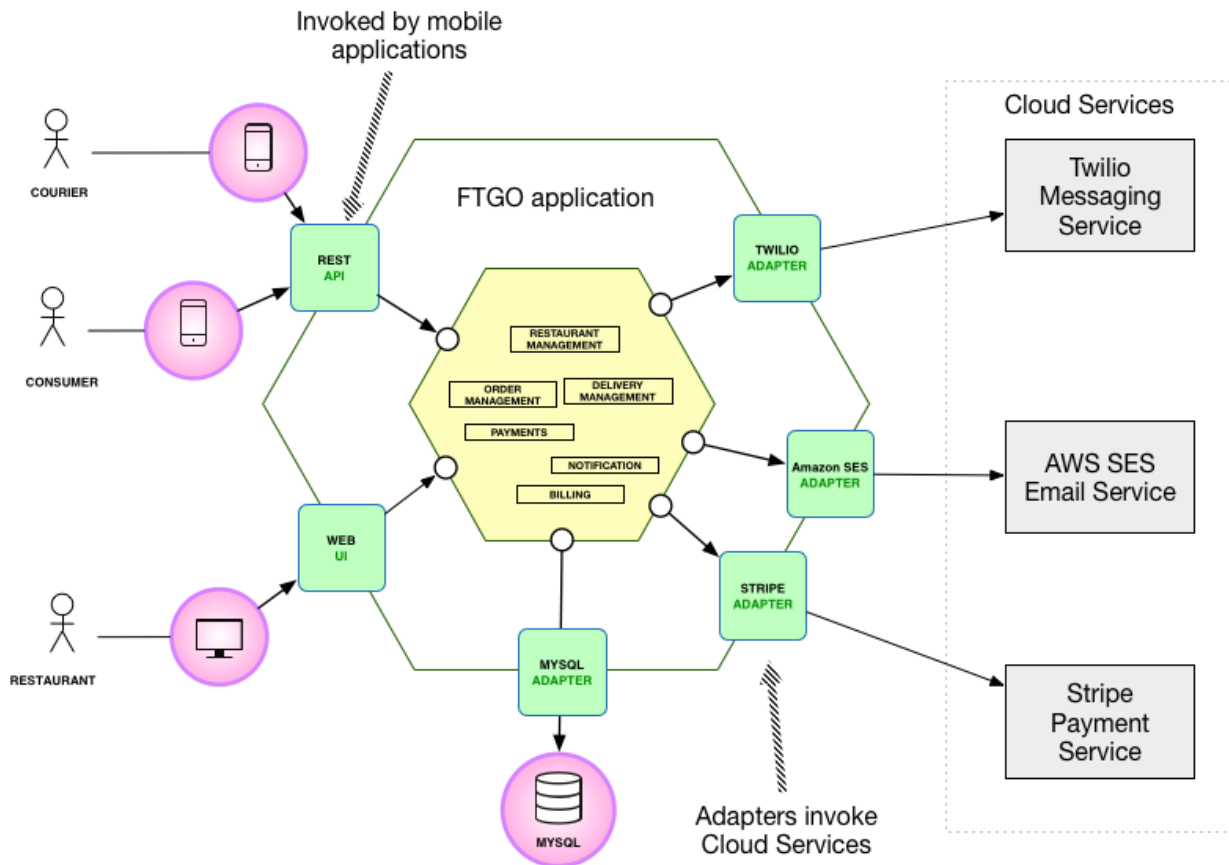


Figure 2.1 The FTGO application has a hexagonal architecture. It consists of business logic surrounded by adapters that implement UIs and interface with external systems such as mobile applications and cloud services for payments, messaging, and email.

The business logic consists of modules, each of which is a collection of domain objects. Examples of the modules include `Order Management`, `Delivery Management`, `Billing`, and `Payments`. There are several adapters that interface with the external systems. Some are *inbound* adapters, which handle requests by invoking the business logic, including the `REST API` and `Web UI` adapters. Others are *outbound* adapters, which enable the business logic to access the MySQL database and invoke cloud services such as Twilio and Stripe.

Despite having a logically modular architecture, the FTGO application is packaged as a single WAR file. The application

is an example of the widely used *monolithic* style of software architecture. The monolithic architecture isn't inherently bad. The FTGO developers made a good decision when they picked monolithic architecture for their application. In the early days of FTGO, when the application was relatively small, development was easy and fast. This was partly due to the application's monolithic architecture.

2.1.2 Why the monolithic architecture worked well initially

The FTGO developers enjoyed several benefits of the monolithic architecture:

- *Simple to develop* — IDEs and other developer tools are focused on building a single application.
- *Easy to make radical changes to the application* — You can change the code and the database schema, build, and deploy.
- *Straightforward to test* — The developers wrote end-to-end tests that launched the application, invoked the REST API, and tested the UI with Selenium.
- *Straightforward to deploy* — All a developer had to do was copy the WAR file to a server that had Tomcat installed.
- *Easy to scale* — FTGO ran multiple instances of the application behind a load balancer.

Over time, though, development, testing, deployment, and scaling became increasingly more difficult. And by 2016, it became a painful death march. Let's look at why.

2.1.3 Living in what they thought was monolithic hell

Like many successful applications, the FTGO application just kept growing. Each sprint, the FTGO engineering organization implemented a few more stories, which made the code base larger. Moreover, as the company became more successful, the size of the engineering organization steadily grew. Not only did this increase the growth rate of the code base, it also increased the management overhead.

As figure [2.2](#) shows, the once small, simple FTGO application has grown over the years into a large, complex application. Similarly, the small development team has now become multiple teams, each of which works on a particular functional area. As a result, development is slow and painful. Agile development and deployment are impossible.

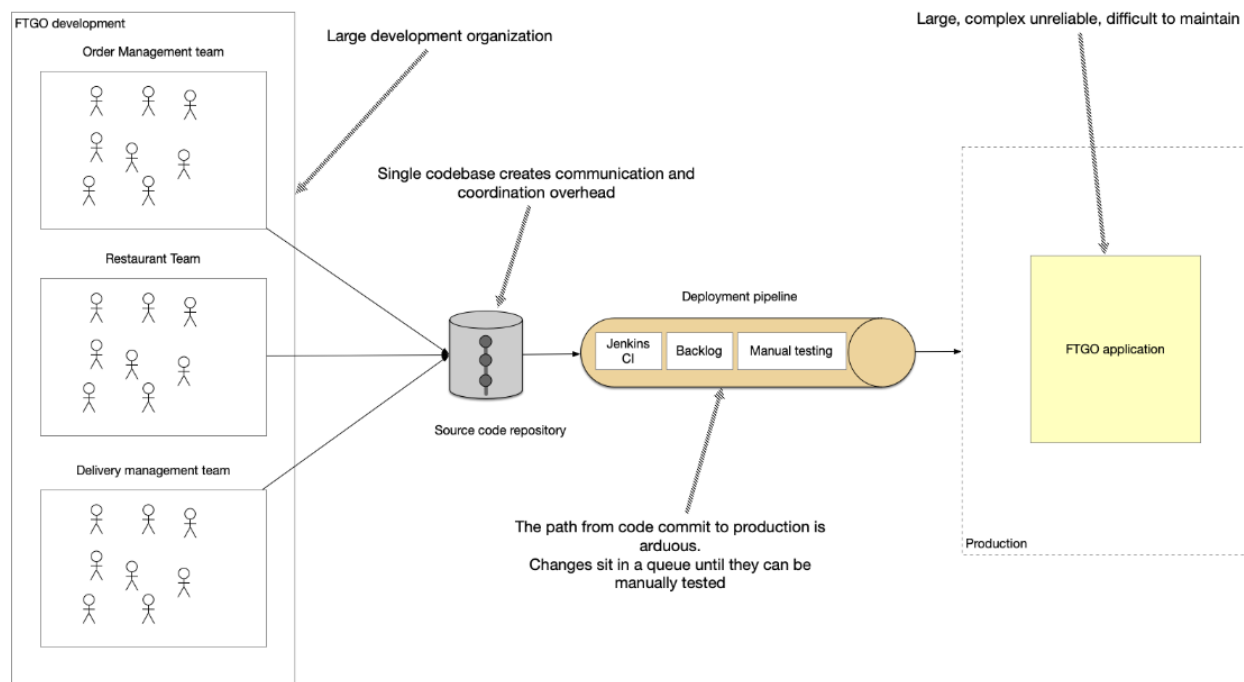


Figure 2.2 A case of outgrowing a monolithic architecture. A large number of FTGO developers commit their changes to a single source code repository. The path from code commit to production is long and arduous and involves manual testing. The FTGO application is large, complex, unreliable, and difficult to maintain.

Let's look at why this happened.

OVERWHELMING COMPLEXITY

At first glance, the hexagonal architecture of the FTGO application looks clean and modular. But the reality is different, mostly because the FTGO developers and their leaders paid insufficient attention to the quality of the design and code. In particular, the domain was not very modular. It was a tangled web of classes, some of which are extremely complex. The `Order` class, for example, had grown to thousands of lines of code. It was responsible for the entire order lifecycle, from creation to delivery to payment.

Consequently, the FTGO application was simply too complex. It was too large for any developer to fully understand. Fixing bugs and correctly implementing new features become difficult and time-consuming. Deadlines were missed.

To make matters worse, this overwhelming complexity became a downward spiral. If the code base is difficult to understand, a developer will struggle to make changes correctly. Each incorrect change makes the code base incrementally more complex and harder to understand. The clean, modular architecture shown earlier in figure [2.1](#) doesn't reflect reality. FTGO gradually became a monstrous, incomprehensible, big ball of mud.

DEVELOPMENT IS SLOW

As well having to fight overwhelming complexity, FTGO developers found day-to-day development tasks slow. The large application overloaded and slowed down a developer's IDE. Building the FTGO application took a long time. Moreover, because it was so large, the application took a long time to start up. As a result, the edit-build-run-test loop took a long time, which badly impacted productivity.

PATH FROM COMMIT TO DEPLOYMENT IS LONG AND ARDUOUS

Another problem with the FTGO application was that deploying changes into production was a long and painful process. The team typically deployed changes to production once a month, usually late on a Friday or Saturday night. The FTGO developers kept reading that the state-of-the-art for Software-as-a-Service (SaaS) applications is *continuous deployment*: deploying changes to production many times a day during business hours. Apparently, as of 2015, Amazon performed 130,000 deploys per day—approximately every 0.66 seconds—without ever impacting the user! For the FTGO developers, updating production more than once a month seemed like a distant dream. And adopting continuous deployment seemed next to impossible.

FTGO had partially adopted agile. The engineering team was divided into squads and uses two-week sprints. Unfortunately, the journey from code complete to running in production was still long and arduous. One problem with so many developers committing to the same code base was that the build was frequently in an unreleasable state. When the FTGO developers tried to solve this problem by using feature branches, their attempt resulted in lengthy, painful merges. Consequently, once a team completed its sprint, a long period of testing and code stabilization followed.

Another reason it took so long to get changes into production was that testing takes a long time. Because the code base was so complex and the impact of a change wasn't well understood, developers and the Continuous Integration (CI) server needed to run the entire test suite. Some parts of the system even required manual testing. It also took a while to diagnose and fix the cause of a test

failure. As a result, it took a couple of days to complete a testing cycle.

DELIVERING A RELIABLE MONOLITH WAS CHALLENGING

Another problem with the FTGO application was the lack of reliability. Testing the application thoroughly was difficult due to its large size. This lack of testability means bugs made their way into production. As a result, there were frequent production outages.

What's more, the application lacked *fault isolation* because all modules were running within the same process. Every so often, a bug in one module — for example, a memory leak — crashed all instances of the application, one by one. To make matters worse, since production was updated infrequently, each deployment consisted of a large number of changes. Sometimes identifying the root cause was like looking for a needle in a haystack and took a long time.

The FTGO developers didn't enjoy being paged in the middle of the night because of a production outage. The business people liked the loss of revenue and trust even less. And, some customers took their business elsewhere—sometimes permanently—when they couldn't order their midnight munchies.

SCALING WAS CHALLENGING

The FTGO team also had problems scaling its application. That's because different application modules had conflicting resource requirements. The restaurant data, for example, is stored in a large in-memory database, which is ideally deployed on servers with lots of memory. In contrast, the image processing module is CPU intensive and best deployed

on servers with lots of CPU. Because these modules are part of the same application, FTGO needed to compromise on the server configuration.

EVOLVING THE TECHNOLOGY STACK WAS DIFFICULT

The final serious problem experienced by the FTGO team was that keeping the application's technology stack current was extremely challenging. That's because upgrading the JVM and the application's dependencies—frameworks and libraries—often impacted the entire application. Some upgrades required the focus of the entire engineering organization. The business was reluctant to approve such upgrades because they delayed the delivery of new features, sometimes for unexpectedly long periods of time. As a result, the application's technology stack was often several versions behind. And, to make matters worse, there were periodically major panics when security vulnerabilities required urgent upgrades.

Another technology stack related problem was that the application's monolithic architecture meant that the organization was locked into the JVM. FTGO developers could not conduct realistic experiments with non-JVM languages such as GoLang or Rust to determine whether they were worth adopting. Fortunately, the JVM ecosystem has continued to thrive, so this is not as big a problem as it could have otherwise been.

Let's now look at the next version of the FTGO application, which is based on the microservice architecture.

2.2 FTGO architecture V2: Fine-grained microservices everywhere - a different kind of hell

After attending a software architecture conference in early 2016, Mary - the CTO of Food To Go - came to two important conclusions. First, she concluded that FTGO was suffering from a classic case of monolithic hell. The application and engineering organization had outgrown the monolithic architecture. Second, she decided that the microservice architecture was the solution. FTGO need to migrate their application from the monolithic architecture to the microservice architecture.

Mary and her team scoured the web for information about the microservice architecture. They watched numerous talks, read many blog posts, and bought several books. Mary also talked to members of her network about their experiences adopting microservices. Armed with what they thought was a good understanding of the microservice architecture, Mary and the FTGO engineering organization enthusiastically adopted the microservice architecture.

This section begins by describing version 2 of the FTGO architecture, which is based on microservices. It then explains where the FTGO engineers went wrong—including taking the definition of MICROservices too literally—and the numerous problems that resulted.

2.2.1 How FTGO adopted the microservice architecture

The FTGO engineers adopted microservices in two different ways. First, they started projects to rewrite their existing applications from scratch using the microservice architecture. Those applications consist of the main FTGO application and also applications that were developed by companies that FTGO acquired. Second, FTGO also used a microservices-first approach when developing new applications.

Not surprisingly, the rewrites took a few years, especially for the huge, main FTGO application. But eventually they succeeded. Let's first look at the new V2 architecture of the main FTGO application. After that, I'll describe what worked and what didn't.

2.2.2 The V2 architecture of the main FTGO application

The FTGO architecture V2 consists of many services. Figure [2.3](#) shows some of the key services and their teams. The most notable aspect of the architecture was that different teams had different ideas about what a service should be. Some services were oriented around business domains. Others were oriented around technical capabilities. Some services were quite large while others were very small.

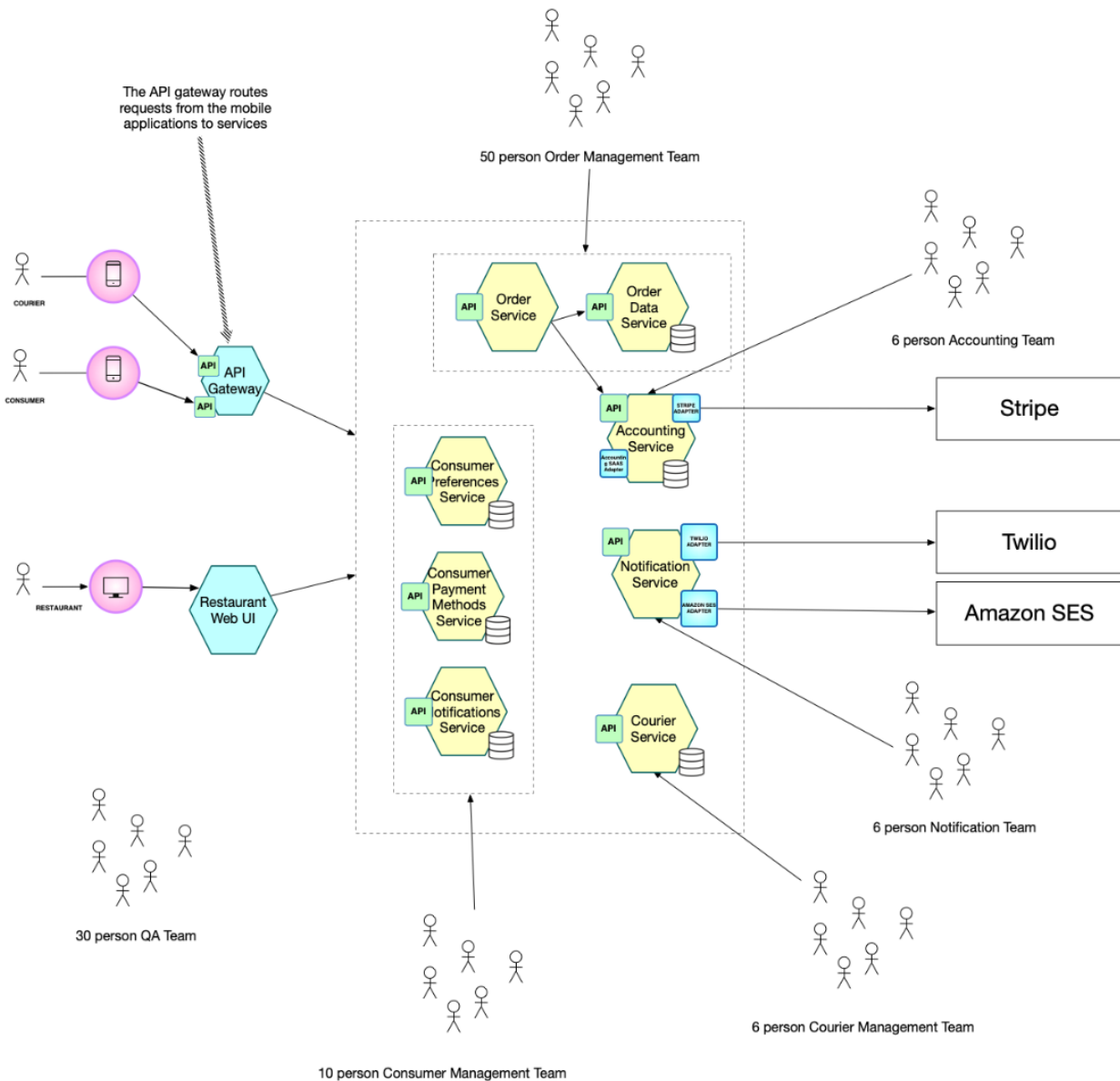


Figure 2.3 Part of the V2 microservices-based FTGO architecture. There are many services, some of which are domain oriented and others are technical capabilities oriented. Some services are quite large while others are tiny. Similarly, the teams that own the services vary in size. There’s also a separate QA team responsible for testing.

One key service is the `Order Service`. It’s a complex service that implements most of the order flow: order taking, preparation of the order by the kitchen, and delivery management. It also maintains the database of restaurants, their opening times and their menus. The service owned by a rather large, 50 person `Order Management` team. One notable

aspect of this service is that it does not have its own database. Instead, there's a separate `Order Data Service`.

The `Order Management` team is also responsible for the `Courier Service`. It knows about each `Courier`'s basic information such as their name, address, and banking details. The service also tracks their availability, current location and their assigned route. Like the `Order Service`, its data is stored in a separate `Courier Data Service`.

The `Consumer Management` team had a different idea about how to design microservices. Despite `Consumer Management` being a relatively simple domain, it's implemented by a collection of tiny services, each of which stores a different aspect of user data. For example, `Consumer Profile Service` stores the consumer's profile including their name, email address, and delivery addresses; the `Consumer Payment Methods Service` stores their payment methods; and the `Consumer Notification Service` stores their notification preferences. Unlike the `Order Service` and the `Courier Service` each of these services have their own database.

In addition to these three services, there are various other services, such as the `Accounting Service`, which integrates with Stripe and SaaS accounting system, and a `Notification Service`, which sends out notifications to users, including couriers and consumers.

2.2.3 What worked well with the V2 architecture

The new architecture certainly had several benefits:

- *Simpler components* — the individual services were much smaller and simpler than the monolith, and so were easier to understand, change and troubleshoot

- *Improved team autonomy* - each team could develop and deploy their service or services independently of other teams
- *Ability to use multiple technology stacks* - different services could use different technology stacks and technology stack upgrades could be done one service at a time
- *Ability to segregate modules by characteristics* - modules with distinctive requirements, such as resource, availability or security, etc., could be deployed as separate services

But sadly, the architectural also had several severe problems. So much so that the team felt that they had swapped one kind of hell for another.

2.2.4 What went wrong with their microservice architecture

FTGO neglected many of the important concepts that I described in chapter 1, such as the fast flow success triangle. They also neglected the defining characteristics of the microservice architecture: loose design-time coupling and independent deployability. As a result, the V2 architecture of the main FTGO application had many problems:

- Slow flow of changes into production
- Tight design-time coupling between services
- Bugs caused by inconsistent data
- Poor performance
- Tight runtime coupling between services
- Technology stack upgrades were still too difficult

In addition, FTGO discovered that migrating all applications to microservices delivered little value and that a microservices-first approach was a bad idea.

Let's look at each one of these issues in turn.

SLOW FLOW OF CHANGES INTO PRODUCTION

The most serious problem with the new architecture was that it failed to deliver on the promise of a fast deployment pipeline. The FTGO engineers had hoped that the microservice architecture would enable them to deploy a continuous stream of small changes into production. But sadly, the deployments were still infrequent. And, when they did occur, the changes were large and complex. To make matters worse, production outages were often difficult to diagnose and fix.

There were three obstacles to frequent deployments. The first obstacle was that developers still needed to hand off the code to a QA team for testing. They had not applied the DevOps concepts and Team Topologies patterns, which I described earlier in chapter 1. As a result, there were delays and misunderstandings between the developers and the QA team.

The second obstacle was that FTGO had ignored the concept of independent deployability, which is, as I mentioned in chapter 1, a defining characteristic of the microservice architecture. The QA team insisted on testing the entire application rather than just the service that had changed. They used running slow and brittle end-to-end tests of the entire application rather than just the needed to test the entire application.

The third obstacle to frequent deployments was the need for a significant amount of manual testing. This manual testing was time-consuming and prone to human error, leading to delays and inconsistencies. As a result, deployments occurred once every week or two.

Because the deployments occurred infrequently, each production update consisted of a large number of changes spread across many services. Quite often, debugging a production outage was like looking for a needle in a haystack. It took time to identify the root cause and fix it.

The problem was made even more difficult because diagnosis required understanding the interactions between many services. Multiple teams had to jump on a conference—sometimes late at night—to diagnose and fix the problem. And since a lot of time had passed since the change was made and when it was deployed, developers often struggled to remember the details.

TIGHT DESIGN-TIME COUPLING BETWEEN SERVICES

Another key problem with the architecture was tight design-time coupling between services. The FTGO engineers had hoped that the microservice architecture would enable them to make changes to a service without needing to coordinate with other teams. After all, that is, as I described in chapter 1, one of the defining characteristics of the microservice architecture. Yet in some cases, FTGO had intentionally designed services to be tightly coupled. And, in other cases, they had failed to pay attention to the design-time coupling between services.

One common source of design-time coupling was the intentional use of data services, such as `Order Data Service`. The split between the `Order Service` and the `Order Data Service`

meant that changes to the `Order Service` often required changes to the `Order Data Service`. For example, when writing business logic in the `Order service` that needed a new attribute in the `Order Data Service`, the developers had to change both services. Fortunately, the changes didn't require coordination with other teams since both services were owned by the same team.

However, if the new business logic required a breaking change to the `Order Data Service`'s API, the single team still needed to carefully manage the roll-out of changes. Typically, they needed to:

1. Add new versions of the affected `Order Data Service` REST API endpoints alongside the original endpoints
2. Update the `X Service` to use the new endpoints
3. Remove the now unused endpoints.

This is lots of work to make what should be a simple change.

The second and even more serious source of design-time coupling was the unintentional coupling between services. For example, as the application's requirements evolved, FTGO discovered that the `Order Service` and `Restaurant Service` were tightly coupled. Initially, the `Order Service` an `Order` had one or more `OrderLineItems`. Each `OrderLineItem` consisted of a quantity and a reference to a `MenuItem` in `Restaurant Service`. This simple model worked well in the beginning. But as FTGO onboarded an increasing number of restaurants, they discovered that the simple model was no longer sufficient since some menu items, such as Burritos, could be customized.

In order to evolve the model, the teams that owned the two services constantly needed to collaborate. Each enhancement to a `MenuItem` to support a different kind of

customization required a corresponding enhancement to the `OrderLineItem`. The teams had to contend with the similar type of API evolution issues that they had with the `X Service` and `X Data Service`.

Because of tight design-time coupling between services, changes involved much more work. They took far longer to implement than was planned. The developers were frustrated with what should be simple changes were quite complex. The business was very unhappy with the slow rate of change because it impacted FTGO's competitiveness.

BUGS CAUSED BY INCONSISTENT DATA

The fine-grained nature of the architecture meant that many requests spanned multiple services. As a result, some requests were quite complex which meant they were difficult to understand and implement correctly. A serious consequence of this complexity was data inconsistency bugs. To maintain consistency, requests that update data in multiple services must use the Saga pattern, which I describe in chapter 10. However, in the FTGO application, some of the sagas were excessively complex due to the fine-grained architecture.

This complexity sometimes obscured subtle bugs that occasionally manifested as data inconsistencies. For example, occasionally an order would not be delivered because the application failed to assign a courier. And, if a customer immediately cancelled an order after placing it, the application would sometimes fail to refund the customer. These bugs were difficult to diagnose and fix because they were caused by complex interactions between services.

POOR PERFORMANCE

Another consequence of the fine-grained architecture was poor performance. Some requests involved lots of trips over the network between services. A few requests even transferred large amounts of data between services. In some cases, this resulted in a poor performance and high latency. Customers would often be frustrated that the application was sometimes slow to respond.

TIGHT RUNTIME COUPLING

Another consequence of the fine-grained architecture was frequent failures caused by tight runtime coupling between services. As I describe in more detail in chapter 4, Service A is tightly coupled to Service B when it cannot respond to a request without Service B being available. In the FTGO architecture, some requests fanned out into long chains of REST calls between many services. These services behaved like traditional Christmas tree lights: a single service failure could cause the entire request to fail. As a result, customers would frequently be frustrated by error pages when they tried to place an order.

TECHNOLOGY STACK UPGRADES WERE STILL TOO DIFFICULT

Another source of disappointment was that it was still too difficult to upgrade the technology stack. Services could certainly use different technology stacks and be upgraded one at a time. The challenge was, however, the lack of consistency of each service's plumbing: the code that implements infrastructure concerns such as logging, security, monitoring, and distributed tracing. This was because a team would write the plumbing for a new service from scratch. Or they would copy and paste the plumbing from another service.

The gratuitous differences meant that how to upgrade the plumbing varied between services. As a result, upgrades often took longer than expected. In some cases, this even discouraged teams from making application-wide infrastructure improvements. In chapter 17, I describe how to solve these problems using the Service Template and Microservice Chassis patterns.

HOW NOT TO MIGRATE: THE BIG BANG REWRITE OFTEN GOES BANG

The problems with the V2 architecture were amplified by how FTGO migrated to microservices. They should have used the Strangler Fig pattern, which I describe in chapter 21. But, instead, the FTGO engineers did a big bang rewrite that created a brand-new application from scratch. While this meant that they could ignore the legacy code, it created several problems.

The first problem is that no value was delivered until the new application was complete. The second problem was that the rewrite took longer than planned, partly because they were chasing a moving target. The third, and perhaps the most severe problem, was that none of their design decisions were validated until they tried to deploy the new application into production. At that point, they discovered many long-standing design errors repeated throughout the code base. One prime example of a problem that was discovered far too late was the excessively fine-grained architecture, which was the root cause of numerous other problems. Lots of rework was required before the application was ready for production.

MIGRATING ALL APPLICATIONS TO MICROSERVICES DELIVERED LITTLE VALUE

Another mistake was that while the main FTGO application and its organization warranted a migration to microservices, many of the other applications did not. For example, most were quite small and were developed by small teams. Except in a few cases, there was little value in migrating these applications to microservices. And, in some cases, development became slower.

MICROSERVICES-FIRST DEVELOPMENT WAS MOSTLY A FAILURE

There were similar problems with the microservices-first approach to development. Many of the new applications were also quite small and were developed by small teams. The microservice architecture did not accelerate delivery. And to make matters worse, because the applications were new, it was very difficult to define stable service APIs. The teams spent a lot of time changing service APIs, which slowed development to a crawl.

2.3 Summary

- Without adopting the DevOps and Team Topologies elements of the fast flow success triangle, the benefits of microservices are not realized.
- Taking MICROservices literally leads to a fine-grained architecture that has many problems including tight design-time coupling, tight runtime coupling, and poor performance.
- Incremental migration to microservices is better than a big bang rewrite because it provides continual feedback on design decisions, without which you are even more likely to create a flawed architecture.
- A monolith should only be migrated to the microservice architecture, if and only if, there is a tangible benefit to

doing so.

- A microservices first approach to development is rarely a good idea because a brand-new application is unlikely to suffer from the problems that microservices solve, and it is difficult to define stable service APIs for a brand-new application.

3 Software architecture: what is it, why does it matter and who does it?

This chapter covers

- The definition of software architecture
- Why architecture is important
- The concept of architectural styles
- Defining an architecture and who does it

Software architecture is a crucial aspect of software development. A lot of time and attention is devoted to architecture. Organizations are filled with architects and developers aspiring to become architects. There are conferences that dedicate entire tracks to must-know architectures. Yet, for all this focus, surprisingly little attention is given to what software architecture truly is and the discipline that underpins it.

This gap in understanding becomes particularly important when using the microservice architecture. Before we can effectively apply the microservice architecture — when and how to use it — we need a solid foundation in architecture fundamentals. That means stepping back to examine core concepts — after all, without this understanding, you risk repeating the mistakes of the FTGO organization, as described chapter 2.

This chapter explores some fundamental software architecture concepts. You will learn what software

architecture is and how it's multidimensional in nature. You'll see that architecture is important because it enables an application to satisfy certain development-time and runtime requirements. After that, you will learn about the concept of an architectural style, which like real world architectural styles, shapes and constrains an application's architecture - both the monolithic and microservice architecture are examples of architectural styles. Finally, you will learn who is responsible for making architectural decisions. Let's start by defining what is meant by software architecture and why it matters.

3.1 What is software architecture and why does it matter?

This book is about the microservice architecture. It makes sense, therefore, to begin by defining the term 'architecture'. We also need to explore why architecture receives so much attention. In this section, I start by defining the term *software architecture*. I then describe that software architecture is important because it enables an application to satisfy certain requirements. But before diving into the details of software architecture, let's first consider what an application looks like from the outside.

3.1.1 An application as a black box

An application exists to implement its externally visible behavior. A simple black box view of an application is that it implements one or more system operations and publishes events. Each system operation represents an invokable slice of behavior. An event represents something notable, typically something that has occurred as the result of executing an operation. A user of the application would

invoke the operations via a user interface, and receive events as notifications or emails.

For example, the FTGO application, which is shown in figure [3.1](#), implements operations, such as `createOrder()`, `cancelOrder()`, and `findOrderHistory()`. Examples of events published by the FTGO application include `Order Created`, `Order Cancelled`, and `Order Delivered`.

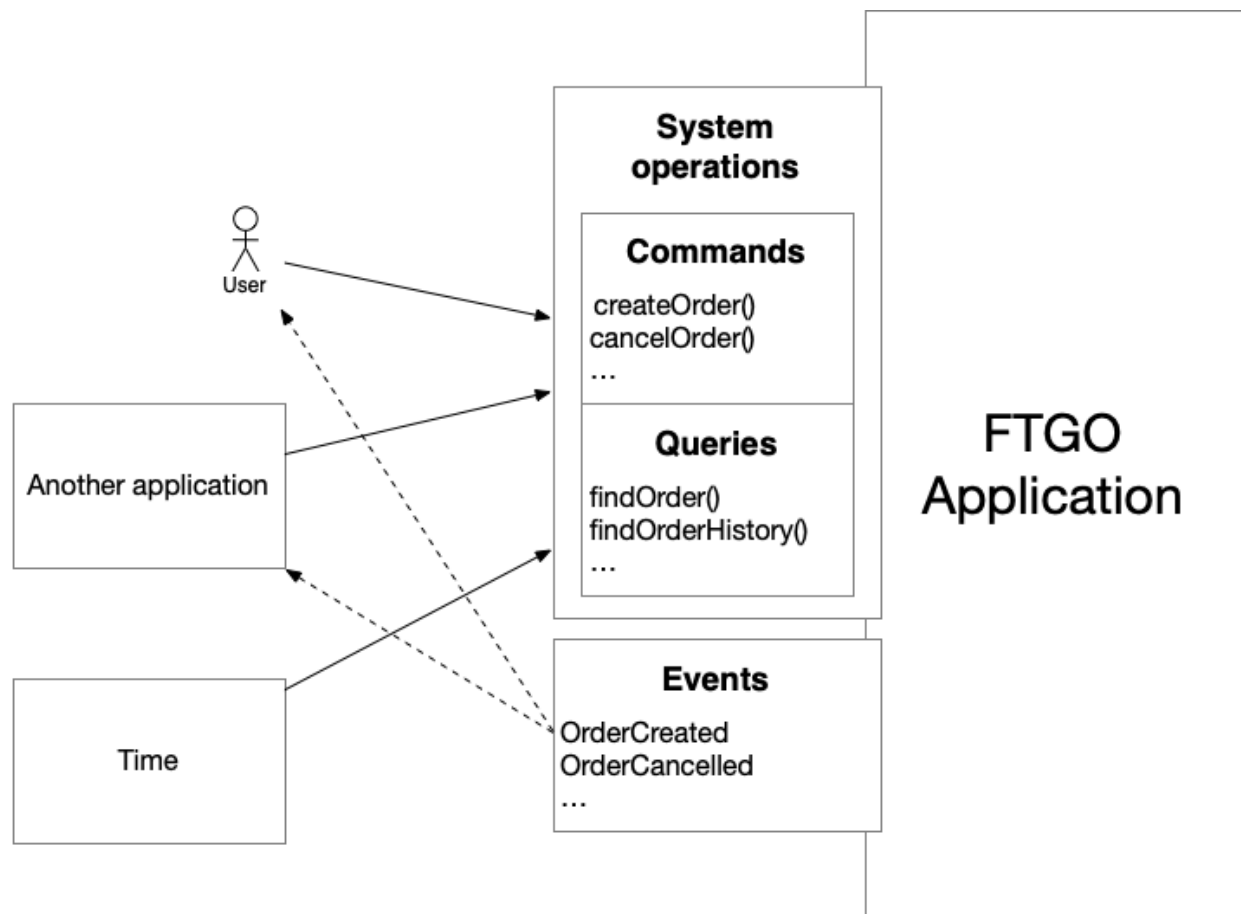


Figure 3.1 An application, such as FTGO, implements system operations - invoked by users, other applications, and the passing of time - and publishes events, which are the result of executing operations, and are subscribed to by other applications and users

A consumer would, via a browser-based or mobile application, invoke the FTGO application's operations, such as a `createOrder()` and `findOrderHistory()`. Events, such as a

`OrderCreated` would be delivered to the consumer either via email and/or mobile notifications.

Let's have a closer look at operations and events starting with operations.

ABOUT SYSTEM OPERATIONS

You can think of a system operation as a method of the "Application-object". There are two types of operations:

- commands - operations that mutate data, such as `createOrder()` and `cancelOrder()`
- queries - operations that retrieve data, such as `findOrderHistory()`

An operation may be invoked in one of the following ways:

- By a human user - the user, for example, clicks on links and buttons in a browser-based UI, which then invokes system operations
- By another application - another application invokes a system operation via an API
- In response to an event that occurs in another application - the other application publishes an event, which the application consumes and then invokes a system operation
- In response to the passing of time - for example, periodic batch processing

System operations are technology neutral. Ultimately, however, they will, of course, be implemented using a particular technology. A human (via a UI) or another application could invoke an operations using a REST API. A messaging technology can be used to invoke operations

asynchronously and deliver events from other applications. An application can use a variety of technologies to invoke an operation according to a schedule.

As you will learn throughout this book, system operations play a central role in defining an architecture. They shape the application's architecture by resisting decomposition into services. That's because, as I described in chapter 1, distributed operations, which span multiple services and require eventual consistency and inter-service communication, are far more complicated than operations that are local to a service. As you will learn in chapter 20, when designing a microservice architecture, you need to make numerous trade-offs between smaller services that promote fast flow and simpler, local operations. In fact, the design process described in that chapter incrementally defines a service architecture one operation at a time.

System operations also play a role in documenting an architecture. Later in [Section 3.2.5](#), I discuss scenarios, which describe an architecture's dynamic behavior. A scenario describes how the elements of an architecture collaborate to implement one or more system operations. Let's now look at events, which are published by system commands.

ABOUT SYSTEM EVENTS

Events are published when something of interest occurs within the application. An event usually represents the state change of a business entity that's caused by the execution of a command. The `createOrder()` command, which creates an `Order`, might publish, for example, an `OrderCreated` event. The failed execution of a command can also trigger events, such as `CustomerCreditLimitExceeded`, if there's insufficient credit to place an order.

An application's events can be delivered to interested parties using a variety of different technologies. Applications within organizations often exchange events using a message broker. Web hooks are often used to deliver events to applications in other organizations. For example, Stripe uses web hooks to notify an application when various events occur, including successful and failed payments. An application might use mobile notifications to deliver event notifications to users.

Let's now look what's inside the black box: the application's architecture.

3.1.2 Software architecture: a way to reason about your application

There are numerous definitions of software architecture. For example,

Architecture is about the important stuff—whatever that is

— Martin Fowler quoting Ralph Johnson

and

Architecture represents the significant design decisions that shape a system, where significant is measured by the cost of change

— Grady Booch

See https://en.wikiquote.org/wiki/Software_architecture to read even more definitions.

My favorite definition comes from Len Bass and colleagues at the Software Engineering Institute

(<https://www.sei.cmu.edu>), who played a key role in establishing software architecture as a discipline. They define software architecture as follows:

The software architecture of a computing system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.

— Documenting Software Architectures by Bass et al

That's obviously a quite abstract definition. But its essence is that an application's architecture is its decomposition into parts (the elements) and the relationships (the relations) between those parts.

Decomposition is important for a couple of reasons:

- Facilitates the division of labor and knowledge - an application's architecture determines how well people organized into one or more teams, with possibly specialized knowledge, can work together on the application.
- Defines how the software elements interact - as you will see very shortly, it's the interactions between software elements that determine the application's runtime behavior.

3.1.3 Why architecture matters

Architecture is important because almost every non-trivial application has certain requirements that can only be satisfied by its architecture. As is shown in figure [3.2](#), there's a few different perspectives on what those requirements are.

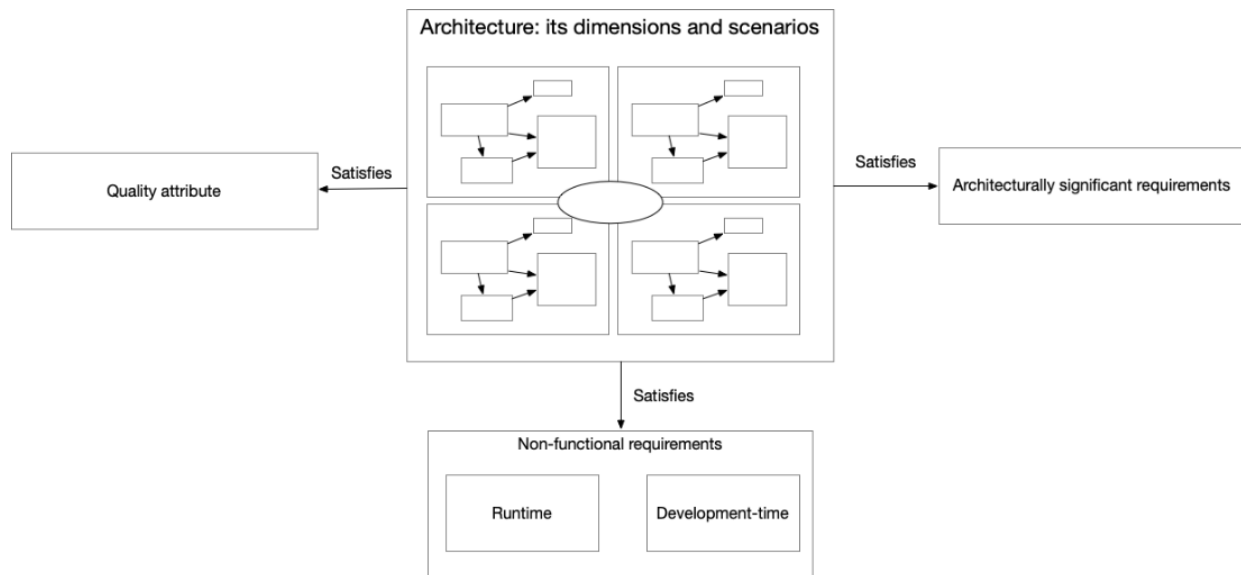


Figure 3.2 Three different perspectives on the purpose of an architecture's architecture, which is multidimensional: traditional non-functional requirements, modern architecturally significant requirements, and quality attributes

The traditional perspective is that those requirements are what are known as non-functional requirements, which describe how well the application must do something. There's also a more modern perspective that architectural solves the application's architecturally significant requirements. Finally, there's a third perspective that entirely avoids the debate over how to categorize requirements and instead argues that architecture satisfies the application's quality attributes. Let's briefly look at each of these perspectives.

ARCHITECTURE SATISFIES AN APPLICATION'S NON-FUNCTIONAL REQUIREMENTS

The traditional view of requirements defines two types of requirements:

- Functional requirements - defines the features and capabilities that the application must provide to the user

- Non-functional requirements - defines how well it performs the functional requirements (runtime characteristics) and how easily it can be developed and maintained (development-time characteristics)

Functional requirements are typically expressed as use cases or user stories. For example, the FTGO application's functional requirements include user stories for placing and canceling an order. These requirements have very little to do with the application's architecture. In fact, you can implement most, if not all, functional requirements with almost any architecture, even a big ball of mud.

Non-functional requirements, on the other hand, are closely tied to the application's architecture. They define two key aspects of the application:

- Runtime behavior - these requirements include the application's scalability, performance, etc. Examples for the FTGO application include handling 233 create order requests per second with an average response time of 356 milliseconds.
- Development-time requirements - these requirements describe how easily the application can be changed and deployed. For instance, a developer-time requirement for the FTGO application might state that a developer can make a simple change to the application and deploy it to production in less than 40 minutes.

Development-time requirements have become increasingly important because of today's need to deliver new features quickly and reliably. In other words, an application must satisfy the needs of both its end-users and its developers. In chapter 5, I discuss how an application's architecture influences the ease and speed of development, directly impacting the ability of teams to achieve fast flow.

ARCHITECTURE SATISFIES ARCHITECTURALLY SIGNIFICANT REQUIREMENTS

The idea that the purpose of architecture is to satisfy non-functional requirements is appealing. However, recent research of real world requirements, such as *Are "Non-functional" Requirements really Non-functional?* by Eckhardt et al (<https://ieeexplore.ieee.org/document/7886960>), and *Characterizing Architecturally Significant Requirements* by Chen et al (<https://ieeexplore.ieee.org/document/6365165>), has shown that this simple model is not entirely accurate. First, not all so-called 'non-functional requirements' influence the architecture. Second, some functional requirements do influence the architecture. Finally, some non-functional requirements are actually functional requirements in disguise.

These findings have led to the concept of *architecturally significant requirements*. Architecturally significant requirements are those requirements that have a significant influence on the architecture. The concept is somewhat vague. It also leads to a circular description of the purpose of architecture: "architecture is important because it enables an application to satisfy those of its requirements that influence the architecture". Some authors have suggested that an architecturally significant requirement has characteristics such as wide-ranging impact; high risk; high business value. There's even an architectural significance test (<https://medium.com/olzzio/architectural-significance-test-9ff17a9b4490>), which consists of seven criteria for determining whether a requirement is architecturally significant.

ARCHITECTURE SATISFIES QUALITY ATTRIBUTES

The first two perspectives on the purpose of architecture are useful, yet they both suffer from the same problem: subjectivity. The first perspective is based on the distinction between functional and non-functional requirements. But, as I mentioned earlier, this distinction is blurry. The second perspective relies on the concept of architecturally significant requirements. Its flaw is that determining whether a requirement is architecturally significant is subjective.

Fortunately, there's a third perspective on the purpose of architecture, which I think is better because it's more precise. This perspective comes from Len Bass and colleagues at SEI who write:

A system's ability to meet its desired (or required) quality attributes is substantially determined by its architecture. If you remember nothing else from this book, remember that.

— Software Architecture in Practice by Bass et al

In other words, the goal of an architecture is to satisfy the application's quality attributes.

They define a quality attribute as follows:

A quality attribute (QA) is a measurable or testable property of a system that is used to indicate how well the system satisfies the needs of its stakeholders beyond the basic function of the system.

— Software Architecture in Practice by Bass et al

In a sense, the phrase "beyond the basic function" hints at quality attributes being similar to non-functional requirements. However, they also write:

After more than 30 years of writing about and discussing the distinction between functional requirements and quality requirements, the definition of functional requirements still eludes me. Quality attribute requirements are well-defined: Performance has to do with the system's timing behavior, modifiability has to do with the system's ability to support changes in its behavior or other qualities after initial deployment, availability has to do with the system's ability to survive failures, and so forth.

— Software Architecture in Practice by Bass et al

There are two main categories of quality attributes: development-time and runtime. Development-time quality attributes include modifiability, testability, and deployability. Runtime quality attributes include availability, performance and security.

Quality attributes requirements are specified in the form of scenarios. A quality attribute scenario consists of six parts:

- stimulus - event that arrives at the application during runtime or at the engineering organization during development time
- stimulus source - where the event comes from
- response - activity that occurs as a result of the stimulus, which is performed by either the application at runtime or the engineering organization at development time
- response measure - a measure of the activity, such as, latency, throughput, or time to make change, which is used to determine whether the architecture satisfies the quality attribute
- environment - the conditions under which the scenario occurs

- artifact - part of the system that is affected by the scenario

Figure 3.3 is an example of an availability scenario. The scenario specifies that when a service instance fails, the application continues to handle requests and that the component instance is automatically restarted.

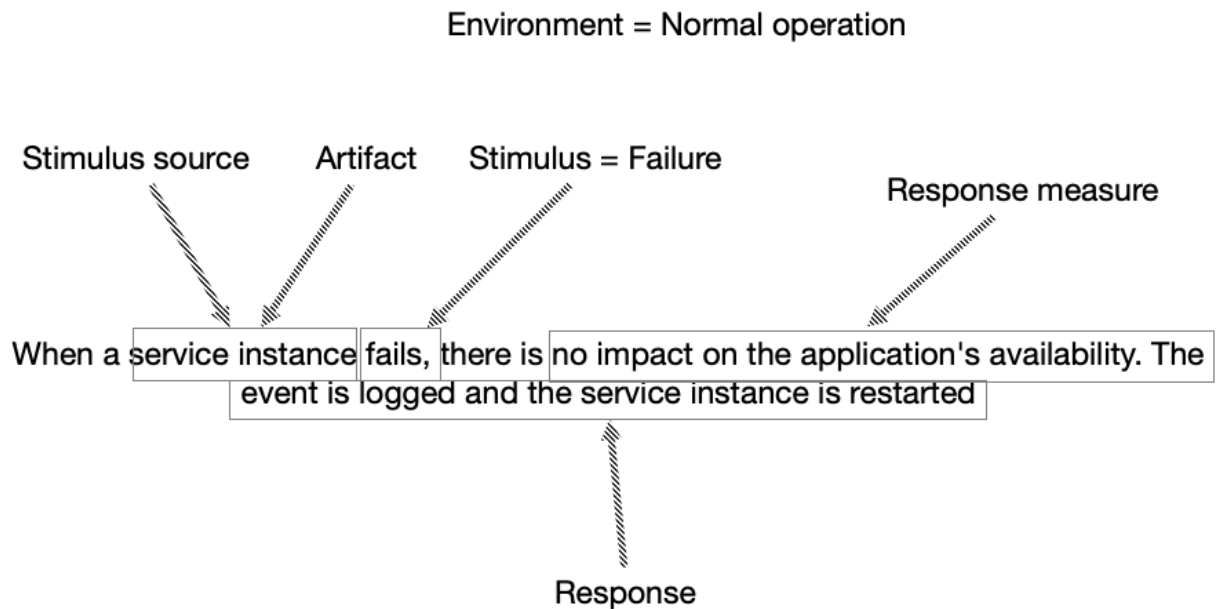


Figure 3.3 An example availability scenario, which specifies that the application continues to handle requests when a service instance, such as the FTGO application's Order Service, fails

This scenario is a runtime quality attribute. It captures various desired aspects of the application's architecture including the ability to have multiple instances of each service, load balancing across services instances, observability, and the ability to automatically restart service instances.

Figure 3.4 is an example of modifiability scenario. The scenario specifies how long it should take a developer to change part of the application and deploy it.

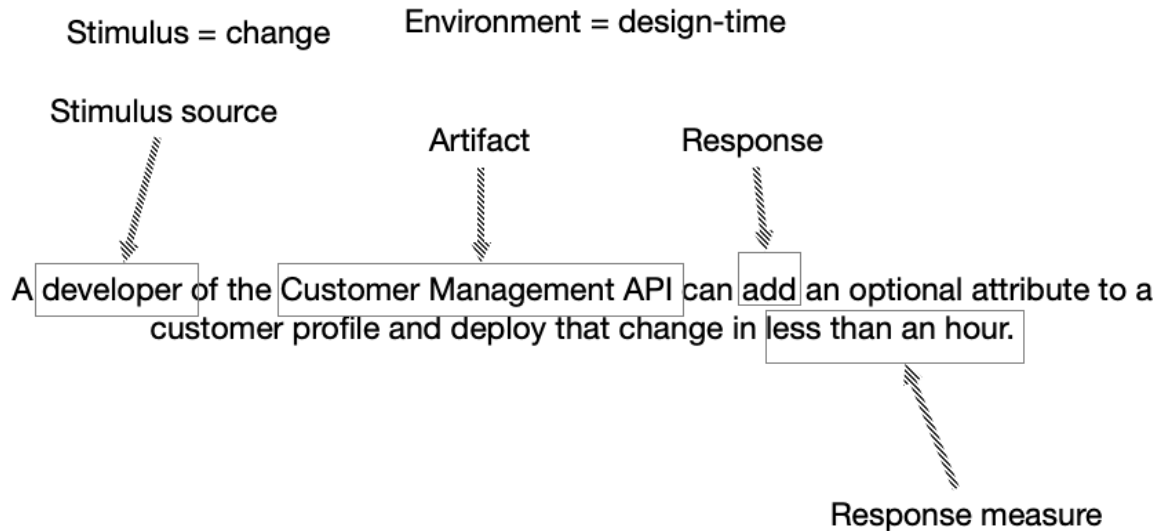


Figure 3.4 An example modifiability scenario, which describes how long it should take for a developer to make a change and deploy it

This scenario is a development-time quality attribute. It captures both the desired ease of change and deployment pipeline execution time.

A valuable feature of quality attribute scenarios is that they are testable. We could, for example, simply measure how long it actually takes a developer to make a change and deploy it. Similarly, we can terminate a service instance and verify that the application responds as expected.

As you have just seen, there are several different perspectives on the nature of the requirements that an application's architecture satisfies. Architecture is, however, important regardless of whether you view it as satisfying non-functional requirements, architecturally significant requirements, or quality attributes. It defines many of an application's development-time and runtime properties. Later in chapter 5, I'll describe the system quality attributes needed for fast flow. But let's now look at who defines an application architecture and when.

3.2 Software architecture is multidimensional

In the same way that a building's architecture can be viewed from multiple perspectives - structural, plumbing, electrical, etc. - so can an application's architecture. An important concept in the field of software architecture is an $N+1$ view model. A view model is an excellent way to describe an application's architecture. It consists of N views, each of which describes an important static aspect - or dimension - of the architecture. A static view consists of elements and relations. In addition to the multiple elements and relations views, the view model also has a '+1' scenario view. The scenario view consists of scenarios that describe the application's dynamic behavior by showing how the static views' elements interact.

THE ORIGINAL 4+1 VIEW MODEL

Phillip Krutchen proposed the concept of a view model in his classic paper "Architectural Blueprints—The '4+1' View Model of Software Architecture"

(<https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf>). The paper describes the 4+1 view model, which consists of the following four views:

- logical - the object model consisting of classes and their relationships that are created by developers
- implementation - the application's code compiled/packaged into modules (e.g., JAR files) and components (e.g., executable JARs or WAR files)
- process - the application's components as a set of running processes and the communication between them
- physical/deployment - the physical/virtual machines, the networks that connected them and the processes (from the process view) that run on them

The fifth view (the +1) consists of the scenarios that animate each of the four views and describe the architecture's dynamic behavior.

Figure [3.5](#) shows my preferred way of describing an architecture. This view model consists of 4+1 views. I chose the four static views to emphasize the important aspects of a microservice architecture.

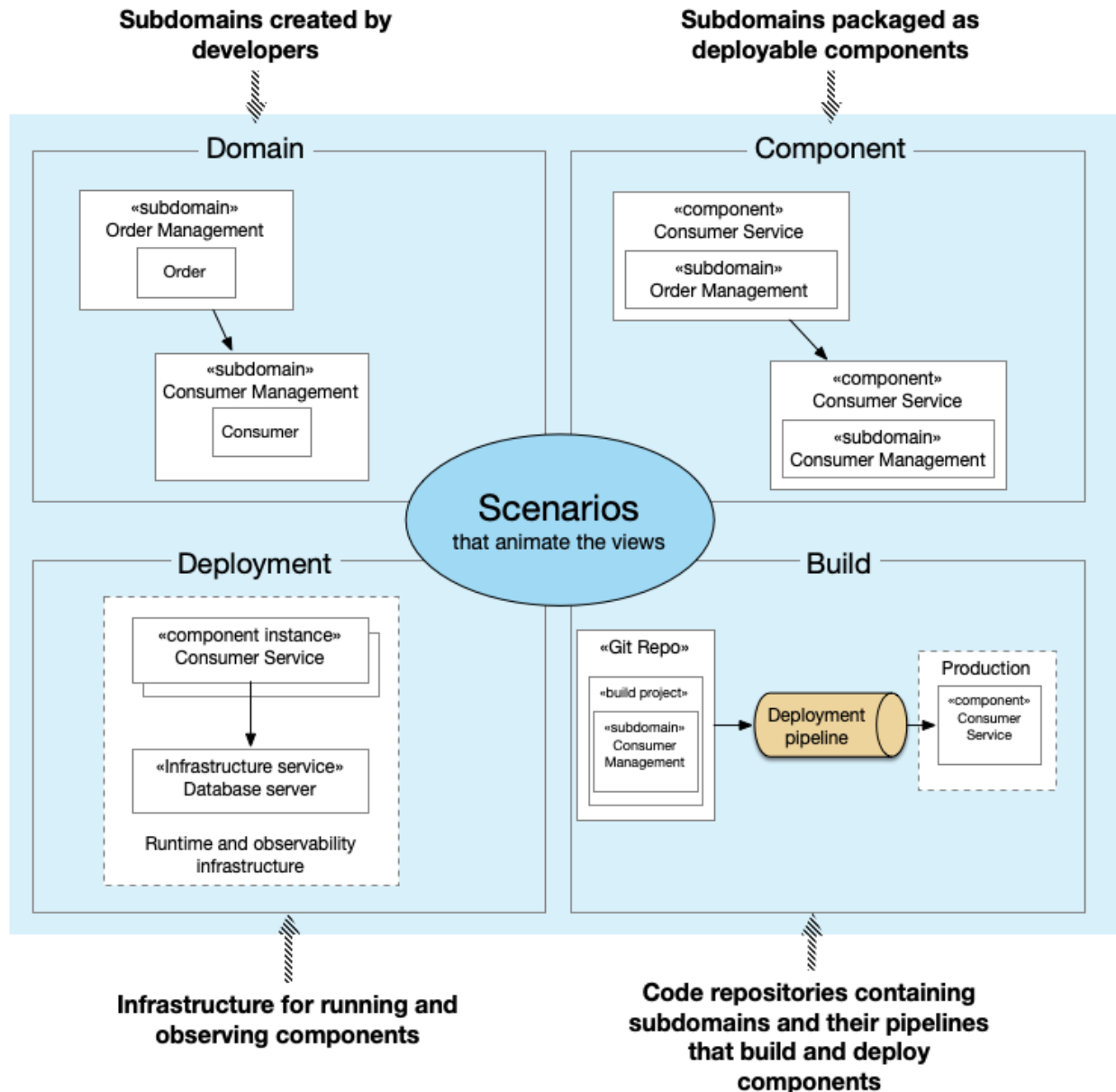


Figure 3.5 My 4+1 view model, which uses four views (+ a scenario view) to describe an application's architecture.

The four static views are:

- Domain - the organization of programming language elements - classes, packages, etc - into subdomains, which are slices of business functionality
- Component - the organization of the subdomains into components, which are deployment units, e.g., monolith

vs. microservices

- Deployment - the technologies, such as Kubernetes, that are used to run and observe the components
- Build - the organization of the source code repositories, and build projects and the path that changes take from development to production, which includes the deployment pipelines that build, and deploy the components

The architecture's dynamic behavior is described by the +1 fifth view, which consists of scenarios. Let's look at each of these views in more detail starting with the domain view.

3.2.1 Domain view

As the name intentionally suggests, the primary purpose of the domain view is to implement the application's business domains. The domain view consists of the programming language elements that developers write and the relationships between them. It organizes those programming language elements into subdomains. A subdomain is a slice of business functionality and is owned by a small team. In an object-oriented language, such as Java, a subdomain consists of classes, which are organized into a hierarchy of packages.

The domain view's elements have many types of relationships between them. These relationships include association and inheritance relationships between classes. There are also Whole-Part relationships between classes, packages, and subdomains: a package contains classes and sub-packages; and a subdomain contains packages. There is also the uses relationship between subdomains since classes in one subdomain might invoke classes in another.

A system operation is implemented by one or more subdomains. The operation is first handled by one subdomain, which then might collaborate with other subdomains. For example, the `createOrder()` operation is first handled by the `Order Management` subdomain. It then uses the `Consumer Management` subdomain to verify that the consumer exists and can place orders and the `Delivery Management` subdomain to schedule a delivery.

Figure [3.6](#) shows the domain view of the FTGO application. It consists of numerous subdomains including `Consumer Management`, `Order Management`, and `Delivery Management`. The `Consumer Management` subdomain is responsible for managing consumer information. It contains entities such as `Consumer`. The `Order Management` subdomain is responsible for managing orders and contains entities such as `Order`. The `Delivery Management` subdomain is responsible for managing deliveries and contains entities such as `Delivery` and `Courier`.

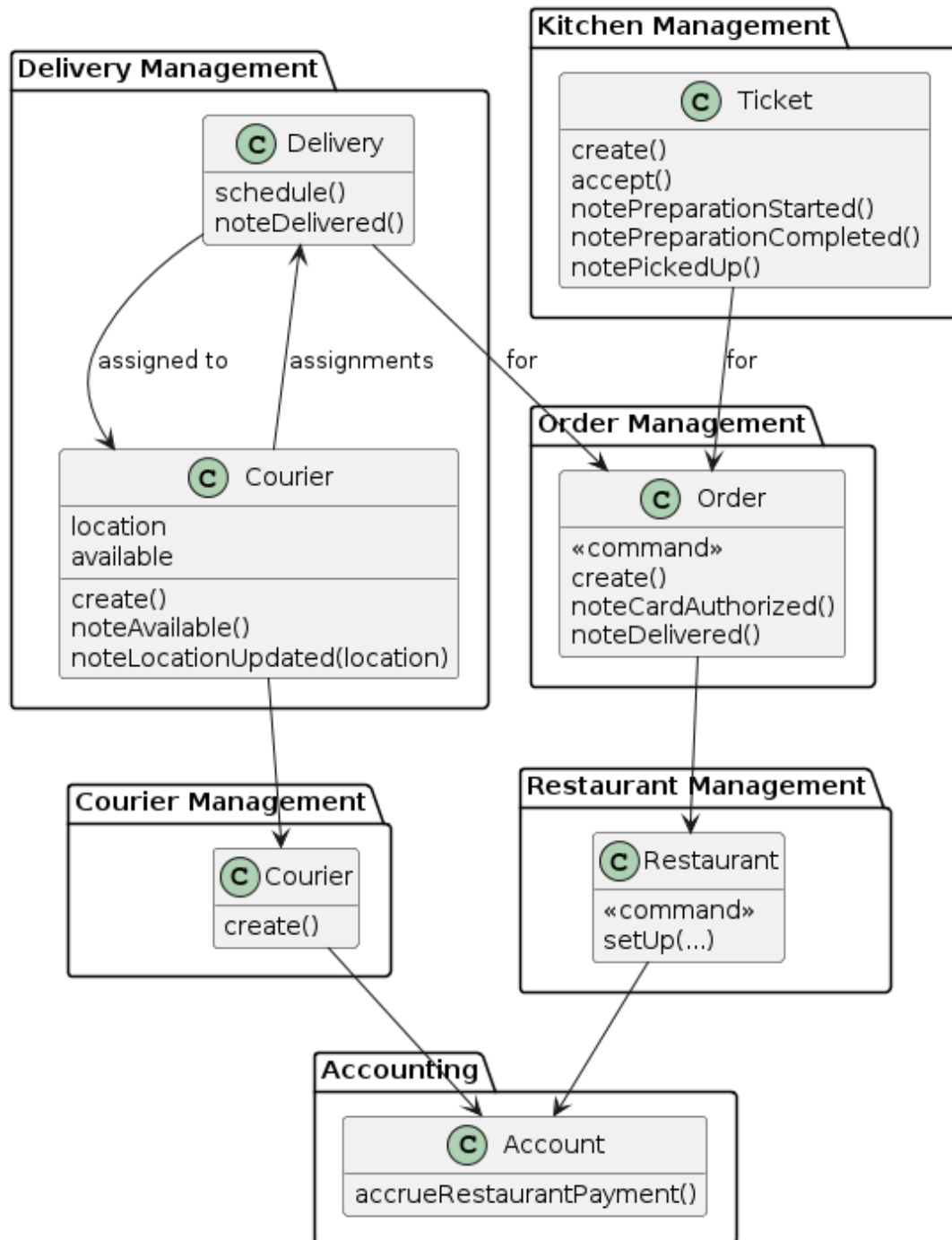


Figure 3.6 The domain view of the FTGO application, which consists of subdomains that contain the business entities that are acted up on by the application's system operations

There are various relationships between the subdomains. For example, the `Order Management` subdomain uses the `Consumer Management` subdomain to verify that a `Consumer` exists and can

place orders. The `Order Management` subdomain also uses the `Delivery Management` subdomain to schedule a delivery by a `Courier`.

While a subdomain's focus is business logic, it typically also includes supporting presentation logic, such as web controllers, which handle HTTP requests, and infrastructure logic, such as database access classes. In fact, a good way to structure a subdomain is the hexagonal architecture, which as I describe later in [Section 3.3.2](#) is an improvement over the more traditional layered architecture.

At the core of each hexagon is the subdomain's business logic. The business logic is surrounded by adapters that enable the subdomain to interact with other subdomains, infrastructure services, and external applications. As figure [3.7](#) shows, an application's domain view can be viewed as a stack of hexagons, one per subdomain.

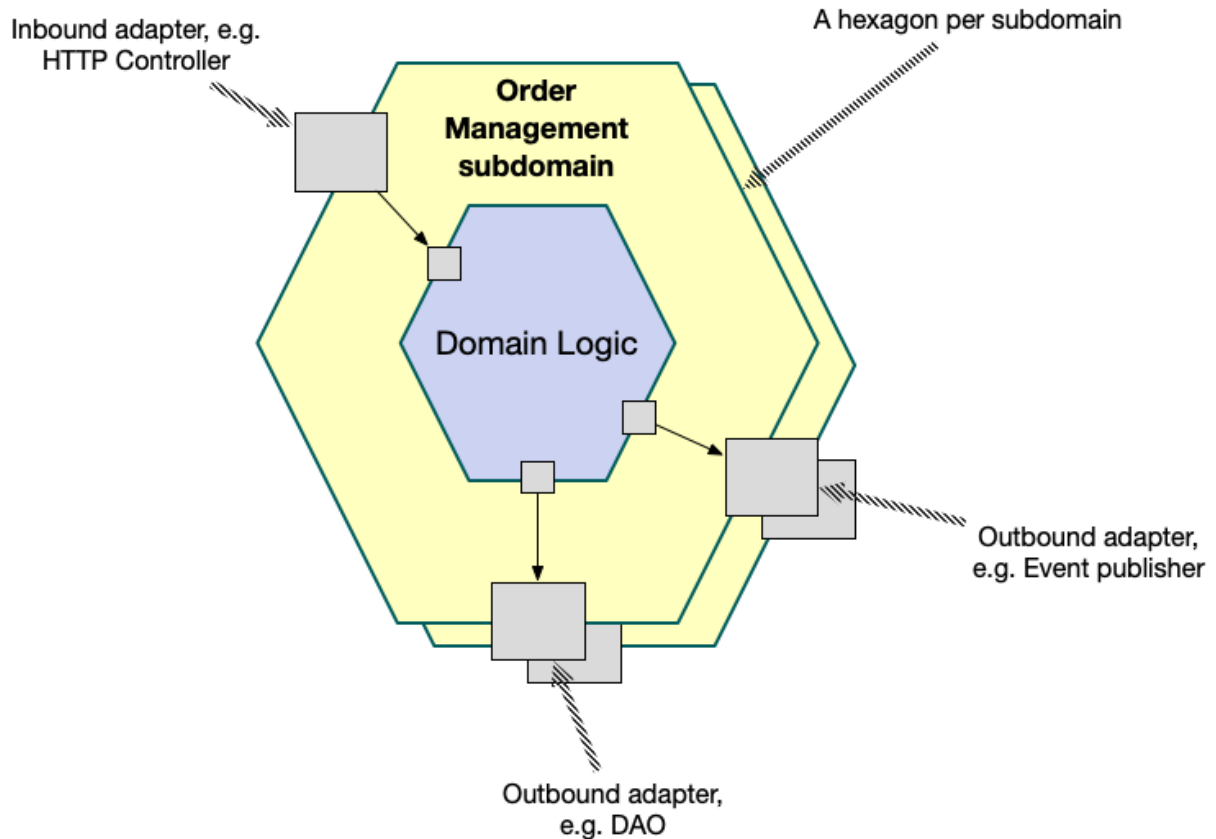


Figure 3.7 An application's domain view can be viewed as a stack of hexagons, one per subdomain. The core of each subdomain is its business logic, which is surrounded by adapters that enable the subdomain to interact with other subdomains, infrastructure services, and external applications.

There are two types of adapters:

- inbound adapters - handle requests from the outside by invoking the business logic, e.g., web controller and message handlers
- outbound adapters - make requests to the outside on behalf of the business logic, e.g., database access objects and REST API clients

In order to run in production, the elements of the domain view must be packaged as components. The organization of these components is described in the component view. Let's now look at this view.

3.2.2 Component view

The component view describes how the domain view's subdomains are organized into one or more components. A component is a language and technology specific executable or deployable unit. For example, a Java component is typically an executable JAR or WAR file. For some other language, a component could be an executable or just a collection of source files. A component could even be a container image, although I prefer to think of a container image as a deployment artifact.

The key decision to make when designing the component architecture is how many components to have. Let's imagine, for example, that the FTGO application consists of just three subdomains: `Consumer Management`, `Order Management`, and `Delivery Management`. One option is to package all three subdomains into a single `FTGO` component. Alternatively, you can organize the subdomains into two or more components.

There are many multi-component architectures to choose from. For example:

- an `Order Service` that contains the `Order Management`, and `Delivery Management` subdomains, and `Consumer Service` that contains the ``Consumer Management`` subdomain
- a separate component for each subdomain: `Order Service`, `Delivery Service`, and `Consumer Service`
- an `Order Service` containing `Order Management` and `Consumer Management`, and a `Delivery Service` containing `Delivery Management`.

Figure [3.8](#) shows two possible component architectures for this application. Later in chapter 20, I describe how to decide first between a single component and multiple components, and then, if you choose multiple components,

how to decide which subdomains to group into each component.

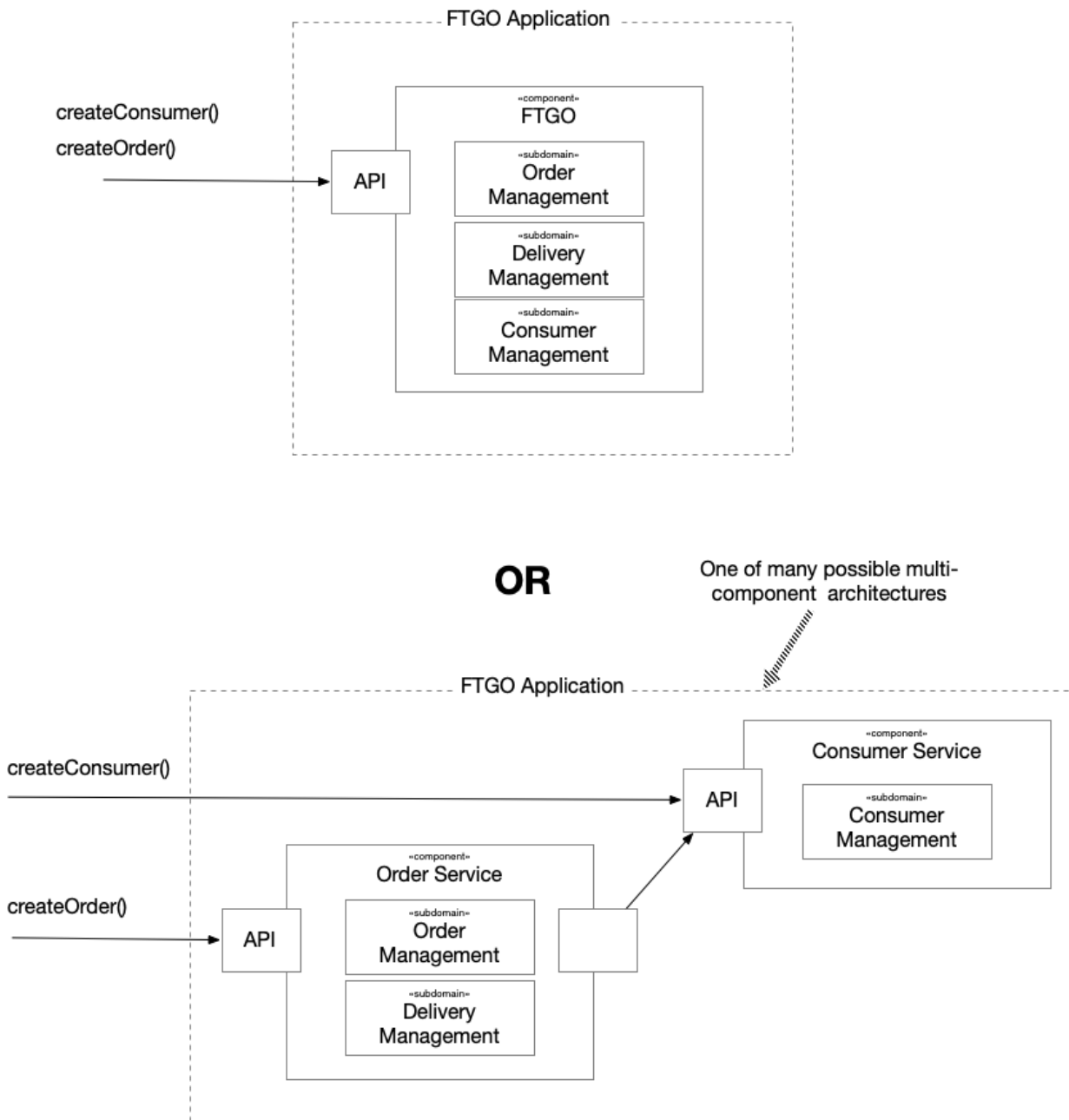


Figure 3.8 Two ways of organizing an application's subdomains into components: grouping all subdomains into a single component (e.g., the monolithic architecture) or partitioning the subdomains into multiple components (e.g. the microservice architecture).

System operations that span multiple subdomains in different components require the components to collaborate. Since a running component is typically a process, collaboration typically happens over the network using technologies, such as REST or messaging. To enable such collaboration, each component has an API, which is implemented by its subdomains. The API can define one or more operations that other components can invoke. It can also publish one or more events that other components can consume.

There are two types of relationships between a component view's elements. One key relationship is the Whole-Part relationship between a component and the subdomains it contains. The other key relationship is the uses relationship between components: one component can use another component's API.

Components are executable/deployable units that need to run on infrastructure. Let's look at the deployment view, which defines that infrastructure.

3.2.3 Deployment view

The deployment view describes the infrastructure that the components run on. A deployment view's element types and relations depend upon the deployment technology. Each component from the application's component view maps to one or more component instances running on infrastructure elements. There are other infrastructure elements that run or provide the infrastructure services. There are also networking resources that connect the components and infrastructure elements.

As I describe in chapter 18, there's a wide range of deployment technologies. In the distant past, an

application's infrastructure consisted of physical servers, which ran component instances, connected by physical networks. These days, the physical infrastructure hasn't gone away but is typically managed by the cloud vendor. Modern applications run on highly abstracted or virtualized infrastructure. Components might run on (or as) virtual elements, such as virtual machines, containers, and serverless functions.

Traditionally, the deployment architecture was configured and maintained manually. System administrators would either ssh into machines or use a GUI console to configure the infrastructure. These days, the deployment architecture is typically defined using code, such as Terraform or CloudFormation to define cloud resources, and Kubernetes YAML manifests to define the resources that run the application's components. This infrastructure code, just like application code, is versioned and stored in a source code repository. The tooling is responsible for ensuring that the infrastructure is in the desired state. I'll talk more about Infrastructure-as-code in chapter 18.

An application's deployment architecture is determined by various considerations, most notably the application's system quality attributes. Figure [3.9](#) shows an example of a simple AWS-based deployment architecture for the monolithic version of the application. It consists of an Elastic Load Balancer that routes traffic to multiple EC2 instances (Virtual Machines), which are managed by an autoscaling group and run the application's component; a Postgres Relational Database Service (RDS) instance; and a Virtual Private Cloud (VPC), which defines the virtualized network resources.

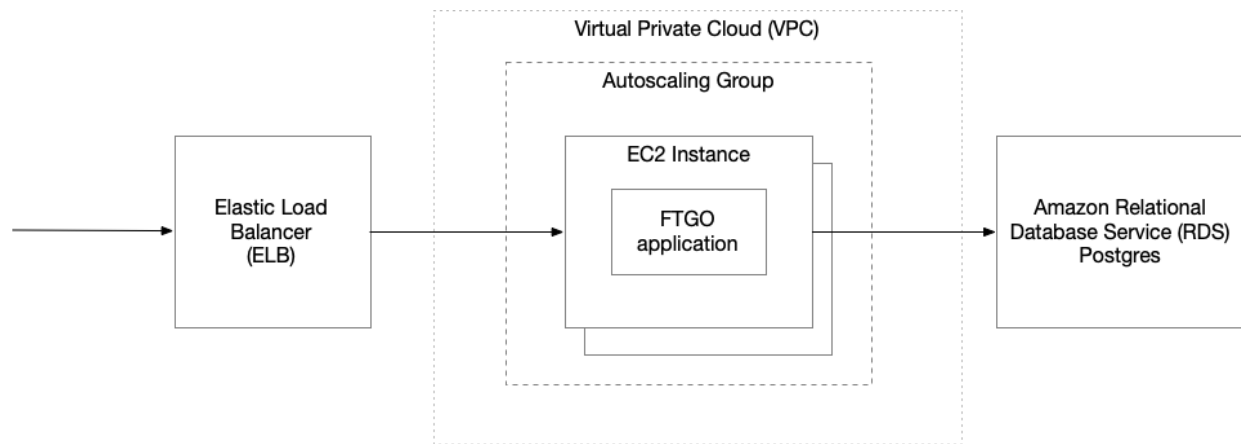


Figure 3.9 A simple way of deploying the monolithic (single component) FTGO application on AWS Cloud. An Elastic Load Balancer that routes traffic to multiple EC2 instances, which are managed by an autoscaling group and run the application's component; a Postgres RDS instance; and a VPC, which defines the virtualized network resources

While such an architecture might satisfy the needs of many applications, it's lacking in terms of availability. The single database instance might also limit scalability.

A more elaborate architecture might consist of a Kubernetes cluster running a set of services, each of which has its own database server. In such an architecture, each component is packaged as a container image and deployed as a Kubernetes Deployment resource, which runs a set of Kubernetes pods.

An even more sophisticated architecture might consist of multiple Amazon EKS-based Kubernetes clusters that are geographically distributed to provide higher availability as well as low latency access to users in different regions. The application might use multiple Aurora RDS database clusters for higher availability and scalability. It's important to remember, however, that you are not Google and to pick the simplest architecture that meets your needs.

The deployment architecture runs the application's components. But how do the components get from the

developer's laptop to production? That's the concern of the build view.

3.2.4 Build view

The Build view describes the path that code takes from the developer's laptop to production. It consists of the following elements:

- Git repositories - contain the code for subdomains (the domain view) and the infrastructure configuration (the deployment view).
- Build projects - within each code repository that build and test components.
- Deployment pipelines - build and test components using a build tool and deploy them into production.

A deployment pipeline is usually triggered by a Git push (or equivalent), but it might also be initiated by the successful completion of a different pipeline. It invokes the build tool one or more times to build and test the component or components. Once the tests for the components pass, the deployment pipeline deploys the updated components to production. As I described in chapter 1, a deployment pipeline needs to be fast in order to provide developers with fast feedback that's required by fast flow.

One way to accelerate the build is to use incremental builds. A build tool that supports incremental builds reuses the results of previous executions of the deployment pipeline in order to build and test only those software elements affected by a change. As a result, the deployment pipeline is much faster than if it were to build and test everything scratch.

ABOUT INCREMENTAL BUILDS

Incremental builds are a key technique for speeding up the deployment pipeline. It's not a new idea and is a key feature of the Unix `make` tool, which was written in 1978. Modern build tools, such as [Gradle](#), have generalized this feature. A Gradle build consists of a graph of inter-dependent tasks. When it executes a build, Gradle attempts to determine which tasks are up to date and skips them.

One challenge with using incremental builds is that cloud-based deployment pipelines, such as those implemented by GitHub Actions, run in ephemeral environments. You need to use an additional mechanism, such as a build cache

(https://docs.gradle.org/current/userguide/build_cache.html), to preserve the results of previous builds.

In addition to incremental builds, there are several other techniques for accelerating the deployment pipeline including:

- Test suite design - in chapters 15 and 16 I describe how to design fast yet effective test suites using the concept of a test pyramid, which describes the proportions of the different types of tests that should be in a test suite.
- Architect for fast builds - in chapter 6 I describe how to structure the code base of a monolithic application to minimize what's known as build-time coupling in order to lessen the impact of a change
- Build parallelization and clustering technology - execute the tests using multiple threads and/or multiple machines

We can also accelerate deployment by taking advantage of the fact that there are numerous ways to structure repositories, build projects, and pipelines. Instead of a single deployment pipeline, an application can have multiple independent deployment pipelines, each responsible for a different component. In fact, a key difference between the monolithic and microservice architectures is how they structure the build view. As described in chapter 1, the monolithic architecture has a single code repository and a single deployment pipeline. Figure [3.10](#) shows the build view for a monolithic architecture.

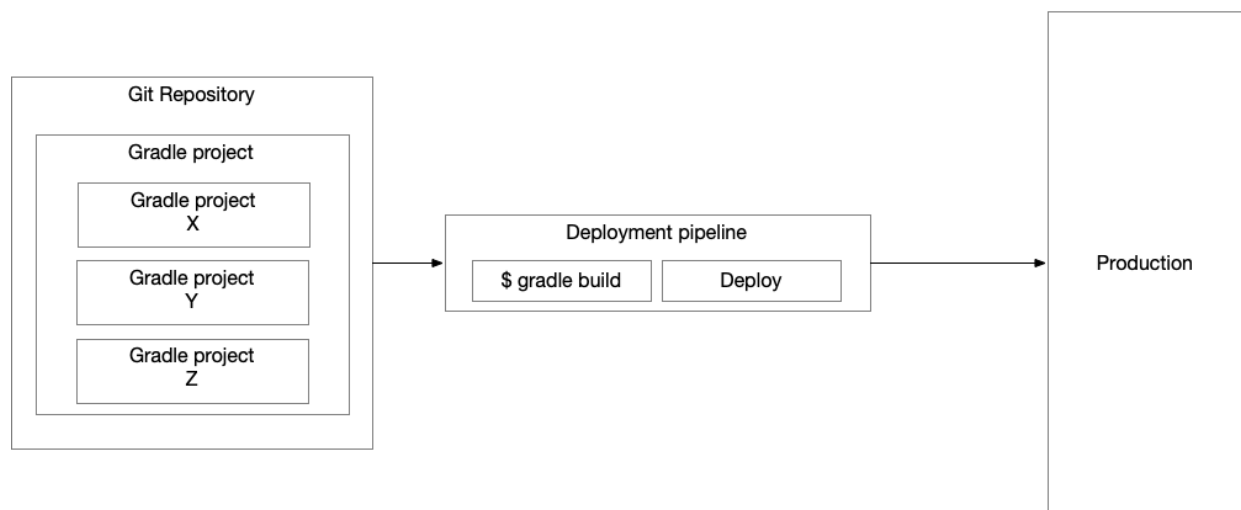


Figure 3.10 One of many ways of organizing code repositories, build projects and deployment pipelines is a single repository, build project and pipeline, as seen in monolithic architectures

This simple build structure works well for many architectures. However, as the number of developers and codebase grow, it can become a bottleneck, slowing down the feedback loop.

To avoid this problem, a defining characteristic of the microservice architecture is a separate deployment pipeline. A service might also have its own repository, although a single repository - a so-called monorepo - is sometimes used. Figure [3.11](#) illustrates this approach.

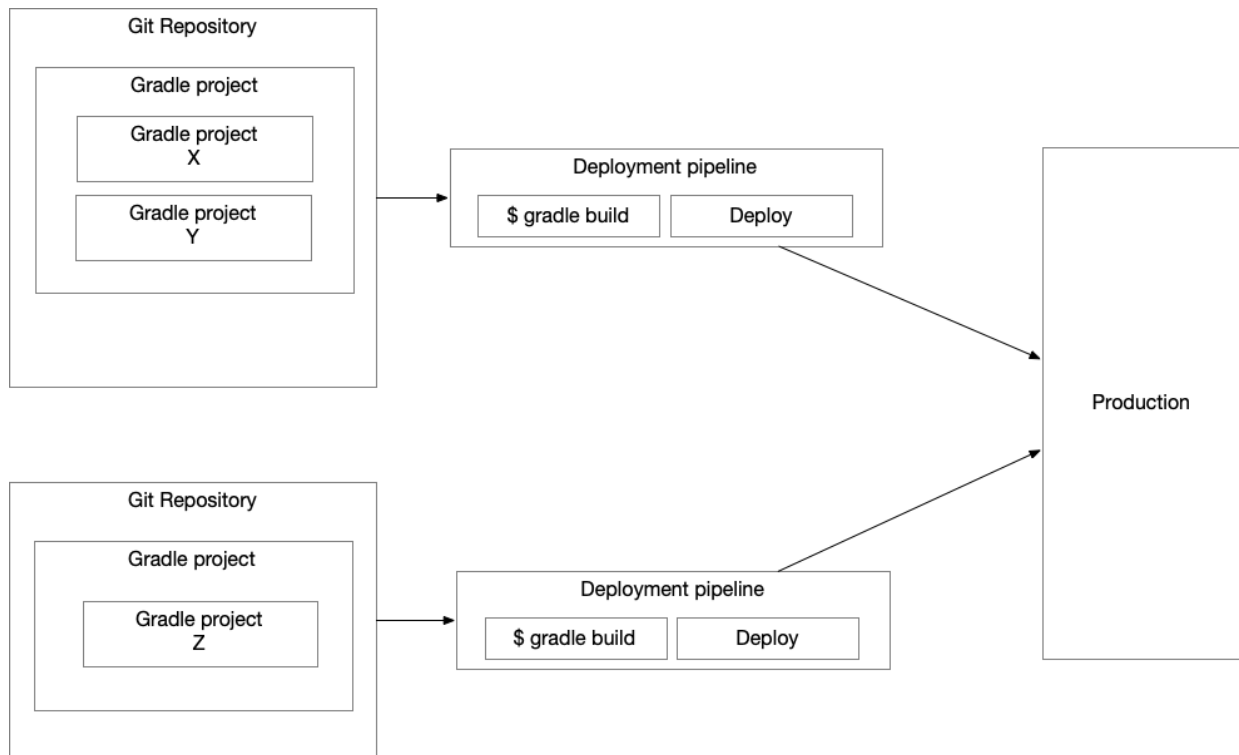


Figure 3.11 Another way of organizing code repositories, build projects and deployment pipelines is multiple repositories, and one build project and one pipeline per repository, as seen in microservice architectures.

As described, a pipeline per service results in shorter lead times and faster feedback, a key benefit of the microservice architecture.

The build view is the fourth architectural view. Let's now look at the fifth view: the scenarios, which animate each of the other views.

3.2.5 +1 Scenarios

The four views that we have looked at so far describe the structure of the architecture, each a collection of elements and relations. However, architecture is more than just a static structure. It must also define how the elements interact to fulfill the application's requirements.

The fifth and final view consists of scenarios that describe how architectural elements collaborate. Scenarios are essential since without them the architecture is a lifeless collection of boxes of lines disconnected from actual requirements. They bring the architecture to life by showing how the application behaves. The scenarios for the build view are different than those for the other three views. The domain, component, and deployment view scenarios describe how the elements collaborate to implement system operations or user story scenarios. In contrast, the scenarios for the build view describe how changes flow from a developer's laptop to production. Let's first look at the scenarios for the domain, component, and deployment views.

DOMAIN, COMPONENT, AND DEPLOYMENT VIEW SCENARIOS

A scenario for the domain, component, and deployment views shows how the architectural elements collaborate to implement either a system operation or user story scenario. A scenario describes the sequence of interactions between architecture elements. The first element might handle an HTTP request, and publish an event, which is handled by a second element, and so on.

User story scenarios, which typically consist of a sequence of system operations, are especially valuable since they directly connect architectural elements to requirements. In fact, I find it useful to identify architecturally significant scenarios when defining and documenting an architecture. These are scenarios that either capture the essence of the application or shape the architecture because of their technical requirements. For the Food to Go application, the architecturally significant scenarios are those scenarios concerned with the ordering, preparation and delivery flow

since that's the essence of the application. For different applications, the architecturally significant scenarios could be those that either process large amounts of data or have stringent performance requirements.

The types of elements that collaborate in a scenario depend upon the view. For the domain view, they are classes or subdomains; for the component view, they are components; and for the deployment view, they are infrastructure elements. Sometimes, however, it can be useful for a scenario to span multiple views. I've found it particularly useful to have services (components) and their key business entities (subdomains) in the same scenario.

There are a few different ways to visualize a scenario including sequence diagrams, collaboration diagrams and service blueprints. Three common approaches are:

- UML sequence diagrams
- UML collaboration diagrams
- Service blueprints

The first way to visualize a scenario is a UML sequence diagram. A sequence diagram shows the participants of a scenario and the sequence of interactions between them. Figure [3.12](#) is a sequence diagram that shows how the elements of the FTGO application - API Gateway, Order Service, Consumer Service, and Kitchen Service - collaborate to implement the `createOrder()` operation. It also shows some of the key classes within each service, such as `Order`, `Consumer` and `Restaurant`.

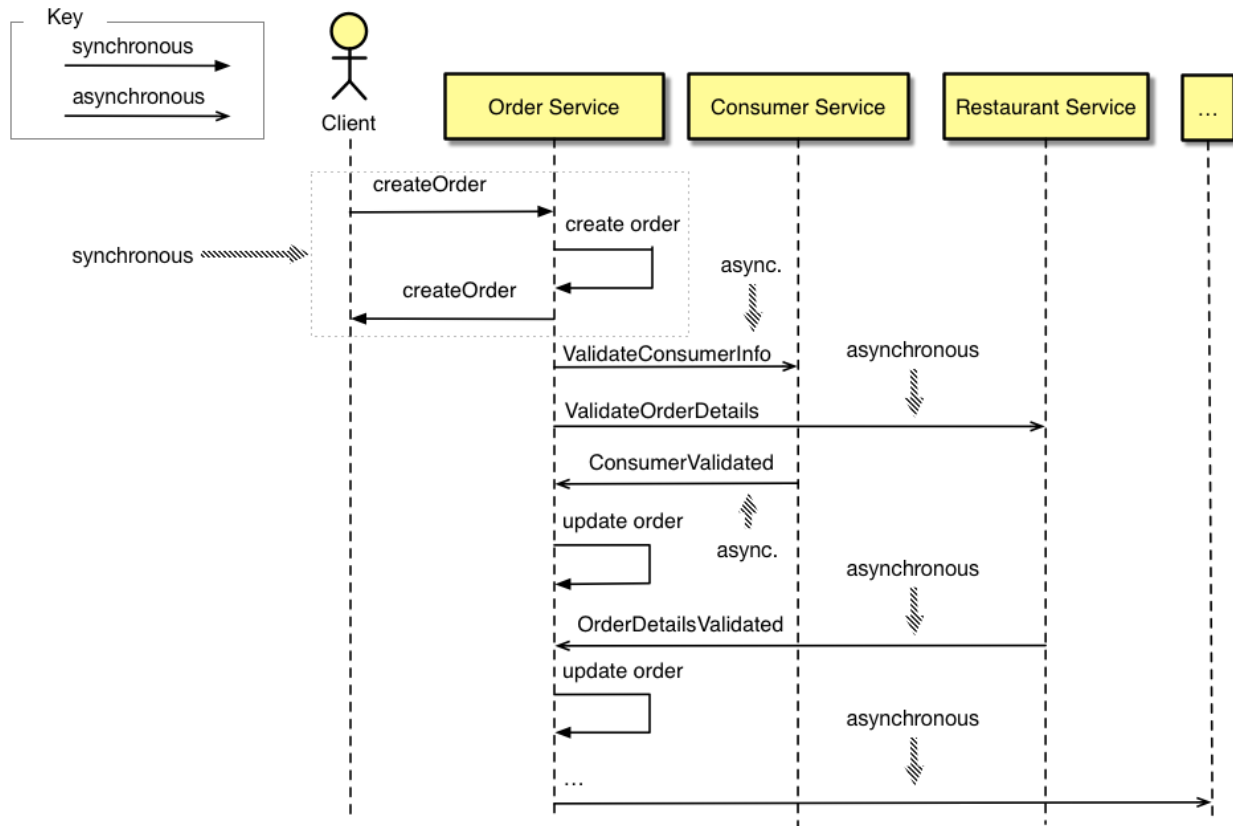


Figure 3.12 A sequence diagram for the `createOrder()` operation that shows how the API Gateway, Order Service, Consumer Service, and Kitchen Service collaborate to implement the operation

A notable feature of this diagram is that not only does it show services, but it also includes the key entities of some services. In practice, I often find that including such child elements, even those from a different view, makes sequence diagrams more informative.

The second way to visualize a scenario is a collaboration diagram, which is also known as a communication diagram. Unlike a sequence diagram, a collaboration diagram has a freeform arrangement of the participants. Figure [3.13](#) is a collaboration diagram that shows the interactions between services during the execution of the `createOrder()` system operation. The numbered arrows show the sequence of interactions.

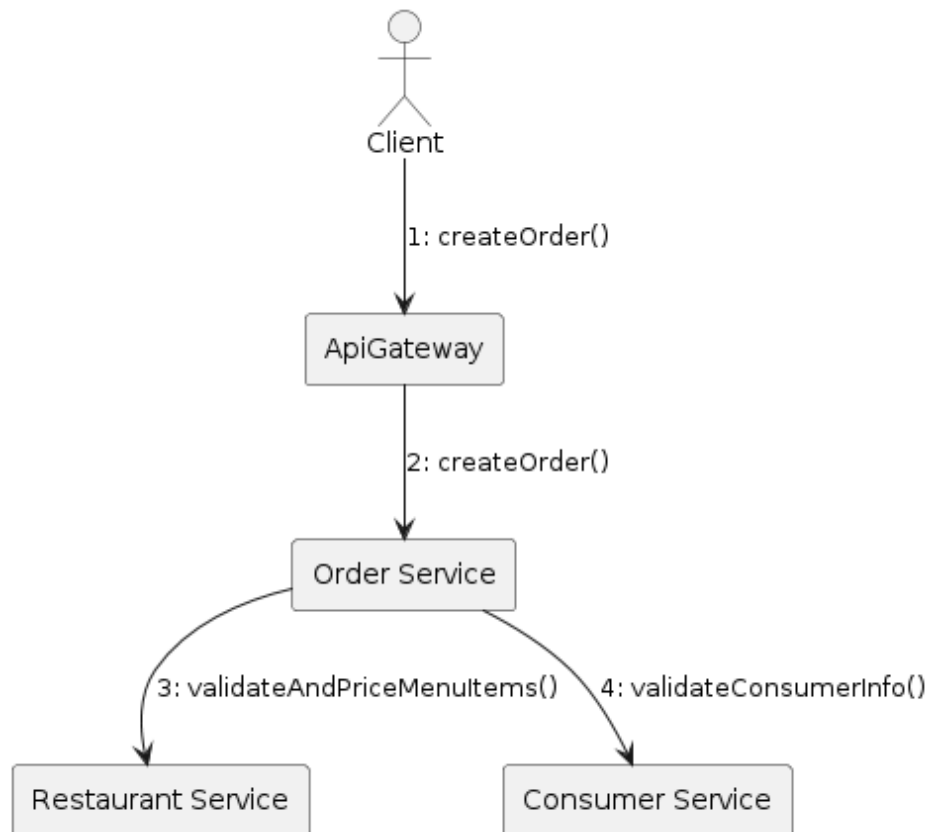


Figure 3.13 A collaboration diagram for the `createOrder()` operation that shows how the API Gateway, Order Service, Consumer Service, and Kitchen Service collaborate to implement the operation

Collaboration diagrams are functionally equivalent to sequence diagrams. The principle difference is that collaboration diagrams focus on the participants. Sequence diagrams, however, clearly show the sequence of the interactions.

The third way to visualize a scenario, which is more accessible to a broader audience, is a service blueprint. A service blueprint is a diagram that shows a customer journey, and the supporting architectural elements. Figure [3.14](#) shows an example service blueprint that shows the customer journey for placing an order and the system operations and business entities that implement it. In the first step of the journey, the user enters the delivery address and desired delivery time. This action is implemented by the

`updateDeliveryAddress()` and `findAvailableRestaurants()` system operations. The `updateDeliveryAddress` system command updates the `ShoppingCart` entity with the delivery address and time. The `findAvailableRestaurants()` system operation finds the restaurants that can deliver to the delivery address at the specified time. The results of this query are displayed to the user in the `Restaurant Selection` step.

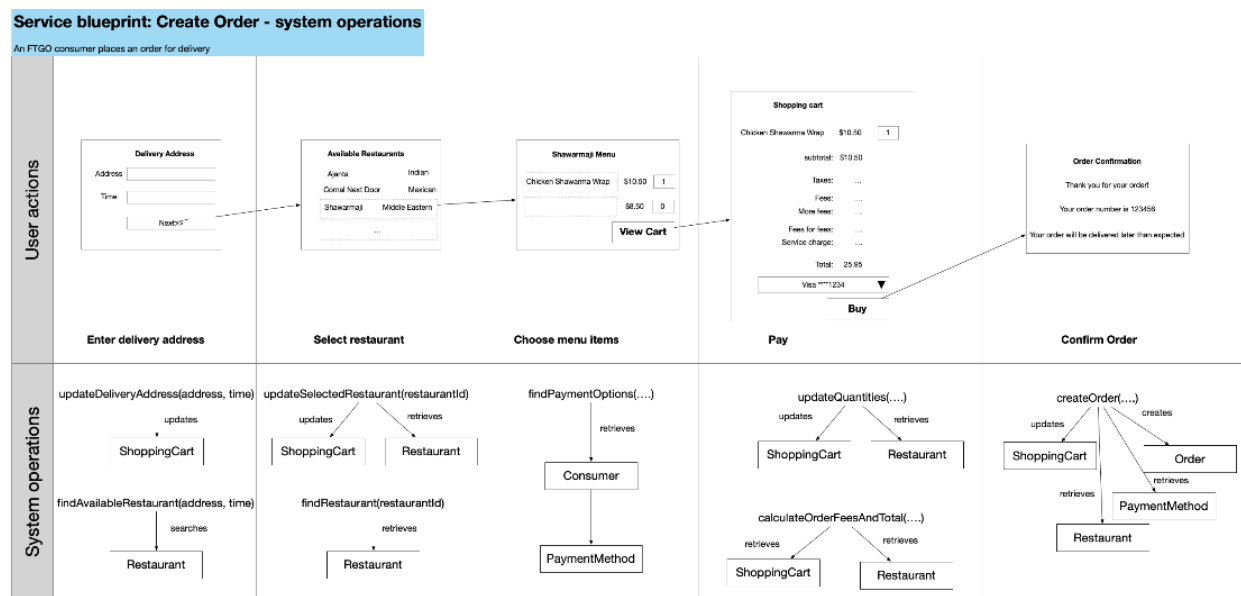


Figure 3.14 A service blueprint diagram the Create Order customer journey that shows the system operations and business entities that implement it

This particular service blueprint consists of elements from the domain view. Other service blueprints would show elements from the other views.

A great feature of service blueprints that they directly connect architectural elements to the user experience. As a result, they are accessible to a broader audience than more technical sequence diagrams. Service blueprints enable both technical and non-technical stakeholders to understand how the architecture supports the user experience. What's more, sequence diagrams and collaboration diagrams can be used

with service blueprints to provide a more detailed picture of the architecture.

REPOSITORY AND DEPLOYMENT PIPELINE SCENARIOS

The repository and deployment pipeline scenarios are much simpler than those for other views. They only need to show how changes flow from a developer's laptop to production. For a fully automated pipeline, the scenarios are typically very simple. The developer pushes a commit or merges a pull request, the deployment pipeline is triggered, and the component is deployed. However, pipelines that include manual steps, such as approvals, can be more complex.

An activity diagram is a good way to visualize a complex repository and deployment pipeline scenario. It shows the participants of a workflow and the steps that they perform. Figure [3.15](#) is an example of an activity diagram that shows the steps in a deployment pipeline that's a mix of automated and manual steps. The deployment pipeline starts by the developer creating a pull request. It's then reviewed by a second developer who merges it into the `dev` branch. CircleCI then builds and tests the change and, if the tests pass, deploys the component to the `dev` environment.

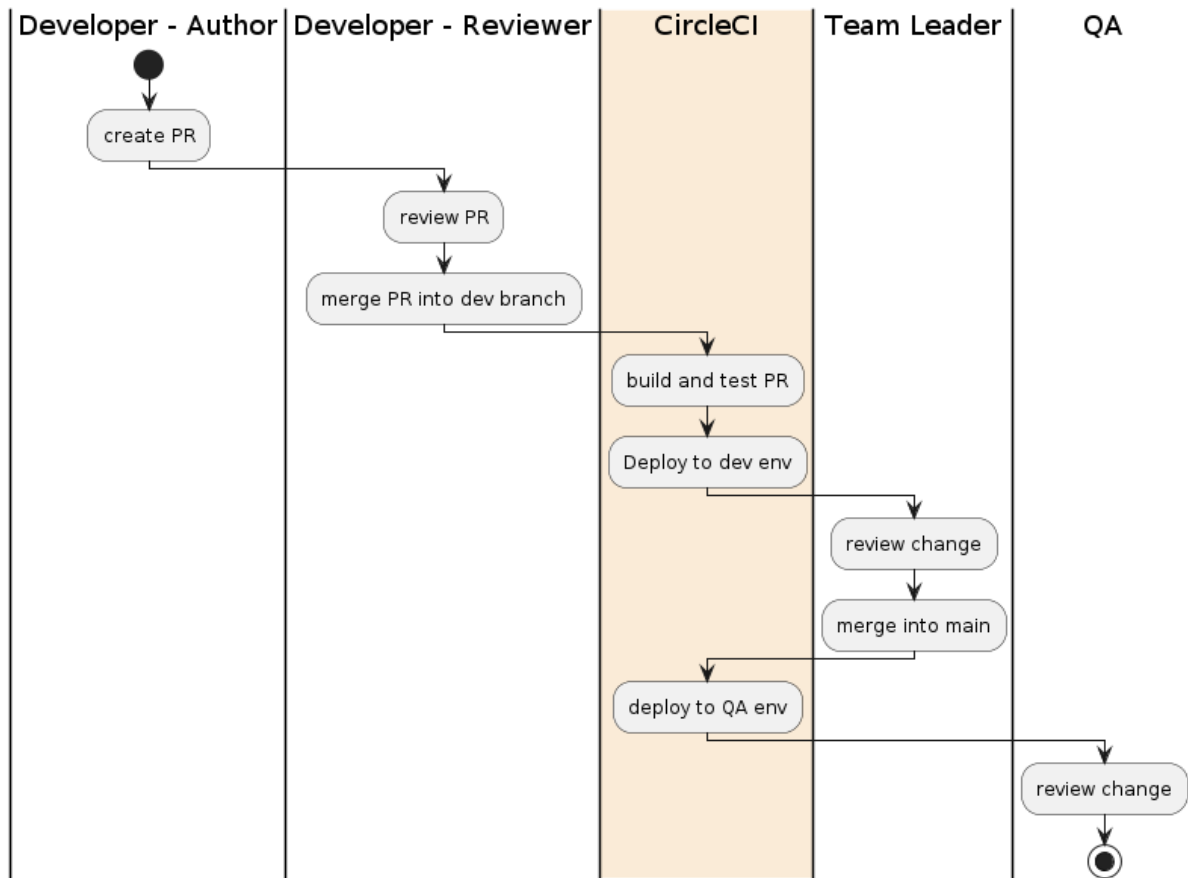


Figure 3.15 An activity diagram that shows the steps in a deployment pipeline that’s a mix of automated and manual steps

Constructing an activity diagram is a good first step when optimizing a deployment pipeline. It clearly visualizes the manual steps, which are candidates for automation. The QA review step could, for example, be replaced by automated tests. And, perhaps the pull request review could be eliminated by pair programming or only used for the riskiest changes.

Now that we have explored the multidimensional nature of software architecture and the importance of scenarios, let’s look at another important architectural concept: architectural styles.

3.3 Overview of architectural styles

In the physical world, a building's architecture often follows a particular style, such as Victorian, American Craftsman, or Art Deco. Each style is a package of design decisions that constrains a building's features and building materials. The concept of architectural style also applies to software. David Garlan and Mary Shaw, pioneers in the discipline of software architecture, define an architectural style as follows:

An architectural style, then, defines a family of such systems in terms of a pattern of structural organization. More specifically, an architectural style determines the vocabulary of components and connectors that can be used in instances of that style, together with a set of constraints on how they can be combined.

— David Garlan and Mary Shaw
An Introduction to Software Architecture

A particular architectural style provides a limited palette of elements (components) and relations (connectors) from which you can define a view of your application's architecture. An application typically uses a combination of architectural styles.

3.3.1 Layered architecture

The classic example of an architectural style is the layered architecture. A *layered architecture* organizes software elements into layers. Each layer has a well-defined set of responsibilities. A layered architecture also constrains the dependencies between the layers. A layer can only depend on either the layer immediately below it (if strict layering) or any of the layers below it.

You can apply the layered architecture to any of the four views discussed earlier. The popular three-tier architecture is the layered architecture applied to the domain view. It

organizes the application's classes into the following tiers or layers:

- *Presentation layer* — Contains code that implements the user interface or external APIs
- *Business logic layer* — Contains the business logic
- *Persistence layer* — Implements the logic of interacting with the database

The layered architecture is a great example of an architectural style, but it does have some significant drawbacks:

- *Single presentation layer* — It doesn't represent the fact that an application is likely to be invoked by more than just a single system.
- *Single persistence layer* — It doesn't represent the fact that an application is likely to interact with more than just a single database.
- *Defines the business logic layer as depending on the persistence layer* — In theory, this dependency prevents you from testing the business logic without the database.

Also, the layered architecture misrepresents the dependencies in a well-designed application. The business logic typically defines an interface or a repository of interfaces that define data access methods. The persistence tier defines DAO classes that implement the repository interfaces. In other words, the dependencies are the reverse of what's depicted by a layered architecture.

Let's look at an alternative architecture that overcomes these drawbacks: the hexagonal architecture.

3.3.2 About the hexagonal architecture style

Hexagonal architecture is an alternative to the layered architectural style. As figure 3.16 shows, the hexagonal architecture style organizes the logical view in a way that places the business logic at the center. Instead of the presentation layer, the application has one or more *inbound adapters* that handle requests from the outside by invoking the business logic. Similarly, instead of a data persistence tier, the application has one or more *outbound adapters* that are invoked by the business logic and invoke external applications. A key characteristic and benefit of this architecture is that the business logic doesn't depend on the adapters. Instead, the adapters depend upon it.

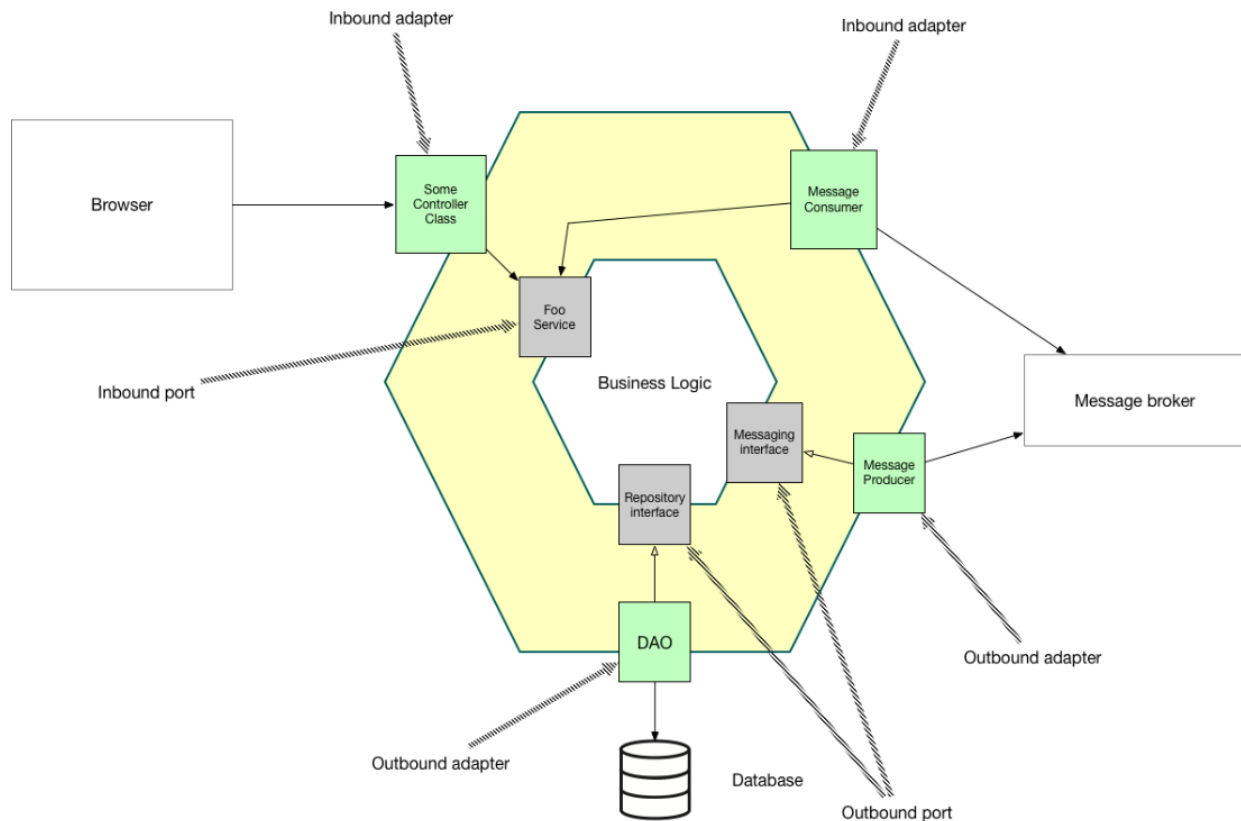


Figure 3.16 An example of a hexagonal architecture, which consists of the business logic and one or more adapters that communicate with external systems. The business logic has one or more ports. Inbound adapters, which handles requests from external systems, invoke an inbound port. An outbound adapter implements an outbound port and invokes an external system.

The business logic has one or more ports. A *port* defines a set of operations and is how the business logic interacts with what's outside of it. In Java, for example, a port is often a Java interface. There are two kinds of ports: inbound and outbound ports. An inbound port is an API exposed by the business logic, which enables it to be invoked by external applications. An example of an inbound port is a service interface, which defines a service's public methods. An outbound port is how the business logic invokes external systems. An example of an output port is a repository interface, which defines a collection of data access operations.

Surrounding the business logic are adapters. As with ports, there are two types of adapters: inbound and outbound. An inbound adapter handles requests from the outside world by invoking an inbound port. An example of an inbound adapter is a Spring MVC Controller that implements either a set of REST endpoints or a set of web pages. Another example is a message broker client that subscribes to messages. Multiple inbound adapters can invoke the same inbound port.

An outbound adapter implements an outbound port and handles requests from the business logic by invoking an external application or service. An example of an outbound adapter is a *data access object* (DAO) class that implements operations for accessing a database. Another example would be a proxy class that invokes a remote service. Outbound adapters can also publish events.

An important benefit of the hexagonal architectural style is that it decouples the business logic from the presentation and data access logic in the adapters. The business logic doesn't depend on either the presentation logic or the data access logic. Because of this decoupling, it's much easier to test the business logic in isolation. For example, tests can

mock outbound adapters and invoke the business logic in the same way an inbound adapter would.

Another benefit is that it more accurately reflects the architecture of a modern application. The business logic can be invoked via multiple adapters, each of which implements a particular API or UI. The business logic can also invoke multiple adapters, each one of which invokes a different external system. Hexagonal architecture is a great way to describe the architecture of each service in a microservice architecture.

The layered and hexagonal architectures are both examples of architectural styles. Each defines the building blocks of an architecture and imposes constraints on the relationships between them. The hexagonal architecture and the layered architecture, in the form of a three-tier architecture, organize the logical view.

The layered architecture and the hexagonal architecture are two examples of many common architectural styles. Later in this chapter, I describe how the monolithic architecture and the microservice architecture are also architectural styles. But let's next look at why architecture is important.

3.4 Architecting: what is it, who does it and when?

An application's architecture is an artifact created by the process of architecting. In this section, I'll describe what architecting is, who does it, and when it's done. Let's start with what architecting is.

3.4.1 Architecture as a series of design decisions

An interesting way to think about architecture and architecting is described by the paper Software Architecture as a Set of Architectural Design Decisions* by Anton Jason and Jan Bosch

(https://www.researchgate.net/publication/220864796_Software_Architecture_as_a_Set_of_Architectural_Design_Decisions). The authors view, as the title suggests, "architecture as a composition of a set of explicit design decisions". By implication, architecting is the process of making those design decisions.

As figure [3.17](#) shows, a design decision consists of the following elements:

- Problem - the problem to solve
- Motivation - why the problem is a problem
- Cause - what causes the problem
- Candidate solutions - possible solutions to the problem and their trade-offs
- Decision - the solution chosen
- Rationale - why the solution was chosen, e.g., an architectural decision record

The chosen solution modifies the architecture. It adds, deletes and modifies elements and relations. A solution can also add, delete and modify design rules and constraints, which influence future design decisions. The design rules and constraints restrict how elements can be combined and interact.

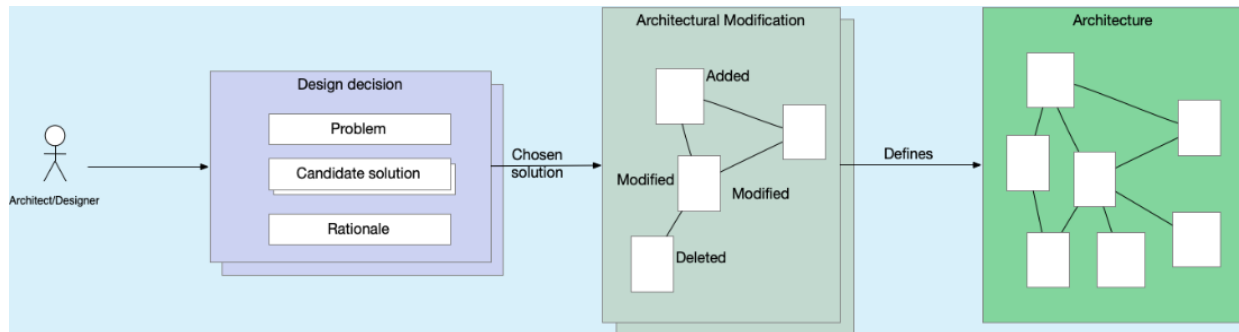


Figure 3.17 An architecture is the composition of design decisions, each of which consists of the problem, motivation, cause, candidate solutions, chosen solution, and rationale. The chosen solution modifies the architecture by adding, deleting and modifying elements, relations, rules, and constraints.

Let's imagine, for example, that a design decision chooses the layered architecture, which I described in [Section 3.3.1](#). The solution consists of the layers (elements), such as `web/api`, `domain`, and `infrastructure`, along with rules/constraints that specify that all elements must belong to one of the layers and that a layer can only depend on the layer immediately below it.

Initially, the architecture is a blank sheet. The first design decision adds the initial elements, relations, rules and constraints. Each subsequent design decision that's then made modifies the architecture. The architecture is a composition of all of those design decisions.

This design decision oriented view of software architecture has several benefits. The first is that each architecture element can be traced back to a design decision. In other words, it should exist if and only if it solves a problem. Keeping this principle in mind can help avoid over-engineering.

Another benefit, which is one of the main motivations for the paper, is that it preserves the rationale behind the architecture. Normally, the rationale is lost as the architecture evolves - a phenomenon that the authors refer

to as **knowledge vaporization**. This approach encourages the preservation of the rationale behind each design decision by explicitly documenting it.

Now that we have seen how architecting is a process of making architectural design decisions, let's look at who does it.

3.4.2 Who defines an application's architecture?

People, of course, make architectural decisions. But the key question is: which people? Are decisions made by developers, or are they responsibility of architects? And, if architects are involved, are they embedded in the teams or, do they operate as a separate group?

When deciding who is responsible for architecture, there are several factors you need to consider:

- Quality - architectural decisions must satisfy the needs of everyone impacted by those decisions
- Speed - in the era of fast flow, architectural decisions must be made quickly
- Buy-in - the people impacted by the decisions must buy into them

Gregor Hohpe, a well-known software architect, has a good article **Would you like architects with your architecture?**

(<https://architectelevators.com/architecture/organizing-architecture/>) on the topic of who does architecture. The article describes four different options.

1. Benevolent dictator - a person or team that's separate from the development team who/which makes all the architectural decisions, which the teams then implement

2. Architect in the team - the architects are team members who make architecture decisions
3. Architecture done by team members - one or more members of each team make architectural decisions
4. Implicit architecture - nobody consciously thinks about architecture

Each of the four options, which are shown in figure 3.18, have different trade-offs. The first and fourth options are generally a bad idea, albeit for different reasons. The second and third options are much better for reasons I'll explain below.

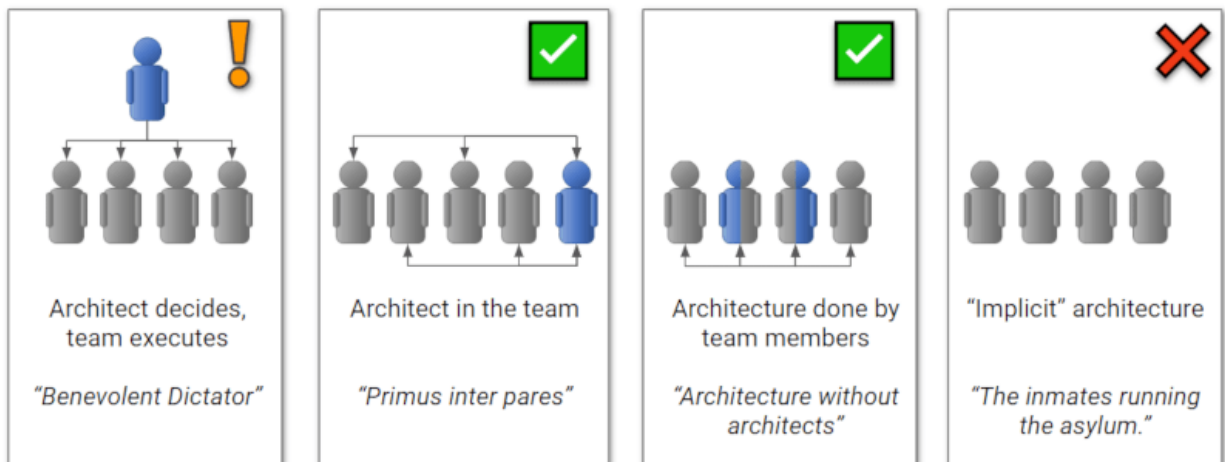


Figure 3.18 The four options for who does architecture: benevolent dictator, architect in the team, architecture done by team members, and implicit architecture - image from <https://architectelelevator.com/architecture/organizing-architecture/>

Let's first look at the two bad options before considering the two good options.

HOW NOT TO ARCHITECT: FULLY CENTRALIZED DECISION-MAKING

The first option - benevolent dictator - where architecture decision-making is centralized, might be useful in some

situations, such as when the teams are paralyzed by indecision. There are, however, three problems with this option:

- poor quality decisions - architects can be out of touch with the needs of developers and make bad decisions
- slow decision-making - development teams need to wait for the centralized team to make decisions
- Lack of buy-in - if architects impose decisions on the development teams, there can be a lack of buy-in.

A much better approach is to decentralize decision-making and move it to the teams. But before discussing how to do that, let's consider the other bad option.

NO EXPLICIT ARCHITECTING IS EVEN WORSE

The fourth option - implicit architecture - where no thought is given to architecture is likely to be a disaster, especially for a non-trivial application. That's because every application has an architecture even if it's not explicitly designed. Typically, the application will end up being a big ball of mud and development will be slow.

DECENTRALIZED ARCHITECTING IS BETTER

The second option - embedding architects in the teams - and third option - having architecture done by team members - are much better ways of organizing architectural decision-making. By moving architecting to the teams, you enable the teams to quickly make architectural decisions by avoiding the bottleneck of centralization. What's more, when teams design the architecture, it ensures that architectural decisions are more practical and grounded in the reality of the code. And since the teams participate in the decision-

making process, they are more likely to buy into the decisions.

However, decentralizing architecture decision-making also introduces a challenge: how do teams ensure architectural decisions are well-informed and aligned across the organization? This is where the architecture advice process is invaluable.

ARCHITECTURE ADVICE PROCESS

The book *Facilitating Software Architecture* by Andrew Harmel-Law (O'Reilly Media, 2024) describes the architecture advice process: anyone - development team members or architects outside the teams - can make and communicate an architectural decision if and only if they seek advice from everyone impacted by the decision and experts in the decision's area. In particular, the people impacted and the experts provide their input on the candidate solutions and their trade-offs. Not only does the architecture advice process ensure that architects understand the impact of their decisions on the development teams, but it also ensures that development teams understand the broader context. It's a practice that every organization should follow.

ARCHITECTING A MICROSERVICE ARCHITECTURE

A distributed architecture, such as a microservice architecture, needs architectural decision-making at two levels. First, there is the architecture of the individual services. This is typically done by the team that owns the service. Second, there's also a need for architects to think about the application as a whole. This includes the cross-cutting concerns, such as infrastructure, that affect all services. For a smaller application, the teams might be

collectively responsible for the overall architecture. But for a larger application, it typically makes sense to have a separate architecture team.

From a Team Topologies perspective, this architecture team is a combination of a platform team and an enabling team. Its goal is to reduce the cognitive load of the development teams by providing them with a platform that embodies the architecture. Yet at the same time, it might play the role of an enabling team and provide architectural consulting to the development teams.

3.4.3 When to architect?

Gall's Law is a rule of thumb for systems design from his book *Systemantics: How Systems Really Work and How They Fail*. It states:

A complex system that works is invariably found to have evolved from a simple system that worked. A complex system designed from scratch never works and cannot be patched up to make it work. You have to start over with a working simple system.

The implication of Gall's Law is that you start with the simplest architecture that works. You then evolve the architecture as the application grows.

Similarly, from the perspective of the *Architecture as a Set of Architectural Design Decisions* paper I mentioned earlier, this means that initially you only make design decisions to solve the problems that truly need to be solved. And then, as your application grows, and you encounter new problems that need to be solved, you make design decisions that solve those problems.

Solving only the problems you actually have with its inherent focus on simplicity is generally a good idea. It avoids over-engineering before you even have your first customer. At the same time, however, it's also reasonable to solve some types of problems that you expect to have in the future. Let's imagine, for example, that your application's user base is growing rapidly. You might determine that to handle the predicted load in N months, you will need to increase the scalability of your architecture. In this scenario, while you don't currently have a problem, there's actual evidence rather than wishful thinking that you will have a problem that needs to be solved.

3.5 Summary

- An application's architecture is a set of structures, consisting of elements and relations, needed to reason about an application
- Architecture is important because it enables an application to satisfy its system quality attributes, which is a measurable and desirable property of the application.
- There are two main categories of quality attributes: development-time quality attributes, which include ease and speed of development and deployment; and runtime quality attributes, which include availability, performance and security.
- Architecture is multidimensional and consists of four static views and a dynamic view.
- The four static views are the domain view (subdomains, which are slices of business functionality); component view (components, which are deployable units containing subdomains), the deployment view (the runtime infrastructure that runs the components), and the build

view (deployment pipelines, which package subdomains into components and deploy them).

- The dynamic view consists of scenarios that describe the application's dynamic behavior by animating the elements of the static views.
- An architectural style is a set of constraints on the elements and relations that comprise an instance of that style. The layer architecture, hexagonal architecture, monolithic architecture and microservice architecture are all examples of architectural styles.
- Architecting is the process of making architectural design decisions. While it should primarily be the responsibility of the development teams, when developing a microservice architecture, an architecture team maybe needed to address cross-concerns that impact multiple services, such as infrastructure.
- Whoever makes an architectural decision should seek advice from people impacted by the decision and experts in the decision's area.
- Build the simplest possible architecture that works and evolve it to solve tangible architectural problems

4 The importance of loose coupling

This chapter covers

- The four types of software coupling
- Why it's important to minimize coupling
- Techniques for designing loosely coupled software

Now that you have learned about the fundamentals of software architecture, let's look at one of the most important architectural principles: loose coupling. Loose coupling is so central to good architecture and design that it deserves a deep dive. That's because an architecture must be loosely coupled to satisfy key quality attributes, such as availability and ease of change. Without it, the application will most likely be difficult to change and prone to failure. If you have ever spent endless meetings trying to coordinate changes with other teams, then most likely you have experienced the pain of tight coupling.

To understand why loose coupling is important, we need to first define what is meant by coupling. Two software elements are coupled if one can affect the other in some way. Coupling manifests itself in various ways. In this chapter, I cover four different types of coupling. Two types of coupling - design-time coupling and build-time coupling - affect how quickly teams can develop and deploy software and so directly impact fast flow. The remaining two types - runtime and infrastructure coupling - determine the application's runtime behavior, most notably, its availability and latency. Let's start by exploring the concept of coupling.

4.1 Introduction to coupling

Let's start with a very abstract definition of coupling, which will refine later:

Coupling is a relationship between pairs of software elements that represents the ability of one element to influence the other element in some way.

It's extremely desirable for elements to be loosely coupled, which means that one element has little or no influence on the other element. The opposite of loose coupling is tight coupling, where one element has a significant influence on the other element. Tight coupling is the cause of numerous development and operational issues, and is best avoided.

But what on earth do the phrases "the ability of one element to influence the other element", "loosely coupled" and "tightly coupled" actually mean? To make sense of these terms, it's important to understand the different types of coupling.

4.1.1 Four flavors of coupling

As figure [4.1](#) shows, there are four different types of coupling.

- Design-time coupling - the likelihood that a change to one element will require a change to the other element.
- Build-time coupling - when changing one element requires another element to be recompiled and retested.
- Runtime coupling - when one element's failure causes another element to fail.
- Infrastructure coupling - when two elements share infrastructure, such as a database, and one element's behavior affects the other element

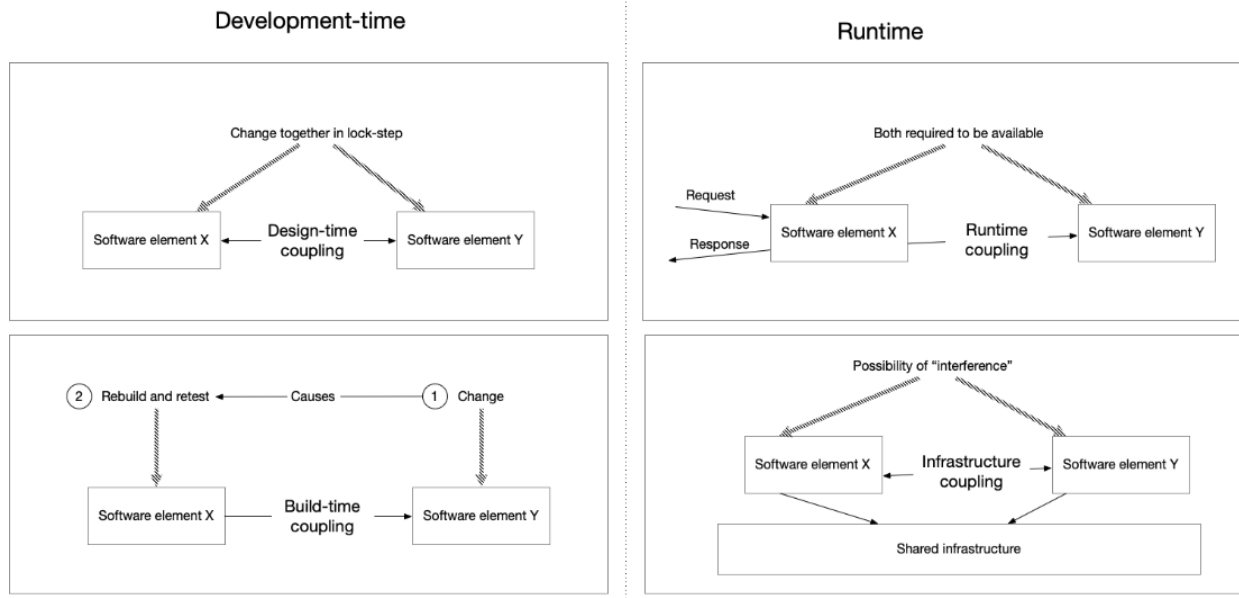


Figure 4.1 There are four different types of coupling: design-time, build-time, runtime, and infrastructure

Each of these types of coupling affects certain architectural characteristics. Design-time coupling and build-time coupling influence its development-time characteristics. Runtime coupling and infrastructure coupling affect the application's runtime behavior.

4.1.2 Why loose coupling matters

Loose design-time and build-time coupling are essential for fast flow. By localizing changes, loose design-time coupling enables teams to work independently and in parallel. It also makes it easier for a team to make changes. Loose build-time coupling is also important, especially for large code bases, because it accelerates the deployment pipeline.

The other two types of coupling are unrelated to fast flow. Instead, they affect the application's runtime behavior. If a service is tightly runtime coupled to another service, then the failure of the second service will cause the first service to fail, which reduces availability. Tight runtime coupling can also increase latency since it introduces additional network

hops. Infrastructure coupling can also reduce availability. If two services share infrastructure, then the behavior of one service could affect the other by, for example, stealing resources.

Let's look in detail at each of these types of coupling in turn, starting with design-time coupling.

4.2 Design-time coupling

The first type of coupling is design-time coupling. As I mentioned in chapter 1, loose design-time coupling is a defining characteristic of the microservice architecture and a key reason why the architectural style supports fast flow. The services - i.e., components - must be loosely design-time coupled, and so a change to one should rarely affect another. Importantly, design-time coupling isn't just a relationship between the components of the component view. It's also a relationship between the elements of the domain view. The classes, packages, and subdomains of the domain view must also be loosely design-time coupled.

In this section, I'll explain the concept of design-time coupling and why loose design-time coupling is essential. You will also learn about cohesion, a closely related concept. I'll describe why it's important to design cohesive software elements, which are elements whose child elements are tightly coupled. After exploring these foundational concepts, we will look at the benefits of loose design-time coupling and how to achieve it.

4.2.1 Design-time coupling and cohesion

An integral part of software design is organizing software elements into a hierarchy that minimizes design-time coupling, whenever possible, and manages it when

unavoidable using the related concept of cohesion. At the lowest level, methods and properties are organized into classes. Those classes are then organized into packages, which can be further grouped into higher-level packages. Beyond this, packages are organized into subdomains. Finally, subdomains are organized into components. Each level of hierarchy is shaped by the twin goals of minimizing design-time coupling and maximizing cohesion. Let's look more closely at each of these concepts starting with design-time coupling.

ABOUT DESIGN-TIME COUPLING

Loose design-time coupling is an ancient concept. For example, the famous computer scientist David Parnas wrote a famous paper about modularization back in 1972. The paper's title is "On the criteria to be used in decomposing systems into modules" (<https://dl.acm.org/doi/10.1145/361598.361623>). The key idea was to design modules in a way such that a change to one module would have minimal impact on other modules. Another early paper called "Structured Design" was written in 1974 by Larry Constantine described the concepts of coupling and cohesion. Over fifty years later, the concepts described in these papers are still extremely relevant.

Design-time coupling refers to the likelihood that a change to one software element will require changes to another. Some design-time coupling in an architecture is inevitable. For example, the `Order Management` subdomain uses the `Delivery Management` subdomain to schedule a delivery for an order. It is, therefore, coupled to the `Delivery Management` subdomain. There's a risk that a change to `Delivery Management` will require a change to `Order Management`. The trick is to design these two subdomains in a way that they are

loosely design-time coupled: future changes to `Delivery Management` rarely affect `Order Management`.

As I describe below in more detail in [Section 4.2.2](#), loose design-time coupling is important because it makes change easier and safer. If two subdomains are loosely design-time coupled, then a developer can change one without having to consider the other. Not only does the ability to make localized changes improve the life of an individual developer, but if the subdomains are owned by different teams, then it enables them to work independently and in parallel. The delivery team can, for example, change the `Delivery Management` subdomain without having to coordinate with the orders team.

ABOUT COHESION

Cohesion is a concept closely related to design-time coupling. It's a principle that organizes the hierarchy of software elements based on how tightly they are design-time coupled. A software element is cohesive if its child elements are tightly design-time coupled. Those child elements that are not tightly design-time coupled should belong to a different parent element. A parent's child elements are tightly design-time coupled to one another but loosely design-time coupled elements with other parents.

This definition of cohesion might sound surprising since tight design-time coupling is so undesirable. But in reality, some amount of tight design-time coupling is unavoidable. And the best way to manage it is for the tightly design-time coupled software elements to have the same parent element. For example, classes that are tightly coupled should be in the same package. Similarly, tightly coupled packages should belong to the same parent package. Tightly coupled subdomains should be in the same component. Organizing

tightly coupled software elements in this way will ensure that changes that inevitably affect multiple elements are still as localized as possible.

Consider, for example, the two designs that model orders and restaurants shown in figure [4.2](#). As I describe in my QConPlus 2021: Takeout burritos and minimizing design-time coupling in a microservice architecture (<https://microservices.io/post/microservices/2021/05/21/qcon-2021-loose-design-time-coupling.html>), menu items in the FTGO application evolved from simply being a name, price and description to supporting numerous configuration options. The `Order LineItems` have become tightly design-time coupled to `MenuItems` and their options. As result, the first design lacks cohesion since `LineItems` and `MenuItems` are in different packages despite being tightly design-time coupled. What's more, those classes are only loosely design-time coupled to other classes in their respective packages such `DeliveryInfo` and `OpeningHours`. The second design, which is more cohesive, has the `LineItems` and `MenuItems` and their options in their own package, which is loosely design-time coupled to the other packages.

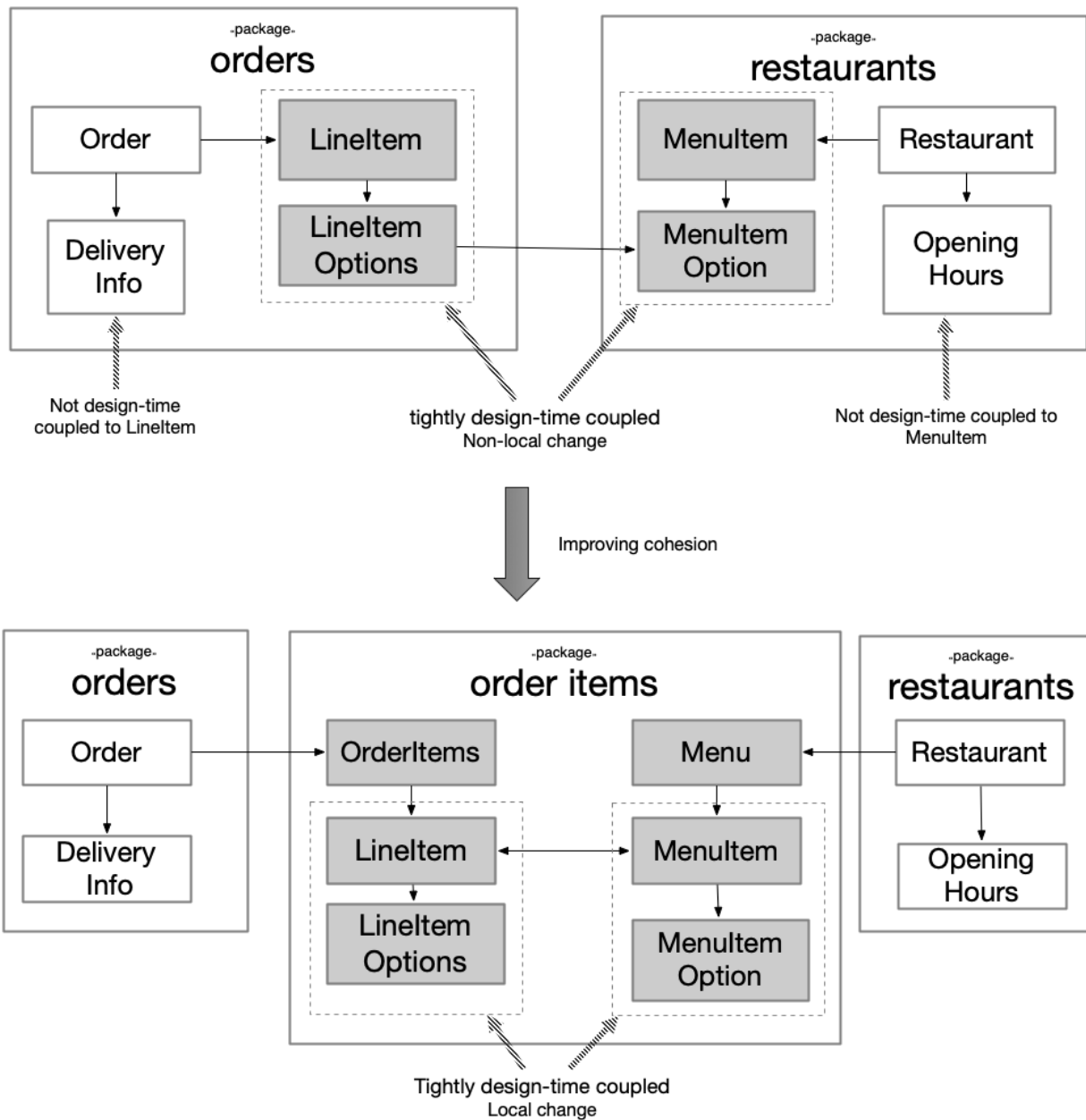


Figure 4.2 You can localize changes, which improves maintainability, by designing loosely coupled packages that are internally cohesive by virtue of being comprised of tightly coupled classes

Localization of changes is not the only benefit of cohesion. Another benefit is that cohesion makes elements easier to change. When you are changing a cohesive element, you will only need to consider child elements will most likely need to change together. Conversely, an element that lacks cohesion

will contain elements that are unrelated to the change that you are making.

A method, for example, that is not cohesive will contain code that is unrelated to the change that you are trying to make. There's more code to understand and more code to test. Similarly, imagine trying to make a change to a subdomain that requires upgrading the technology stack of its component. If the component is not cohesive, then the upgrade might be a much larger task, involving updates of unrelated subdomains.

4.2.2 Benefits of loose design-time coupling

Loose design-time coupling reduces the number of software elements that are impacted by a change. Cohesion ensures that the elements that are impacted are likely to be in the same package or module. As a result, loose design-time coupling has three important benefits:

- makes the software easier and safer to change.
- improves team autonomy
- enables organizations to scale

Let's look at each benefit in turn.

LOOSE DESIGN-TIME COUPLING MAKES CHANGE EASIER AND SAFER

Loose design-time coupling is important because it makes applications easier and faster to change. Consider, for example, two subdomains, shown in figure [4.3](#) - Customer Management and Order Management, which uses Customer Management. Since Order Management invokes the Customer Management's API, it is design-time coupled to that module.

The degree of coupling determines how likely that a change to Customer Management requires a change to Order Management.

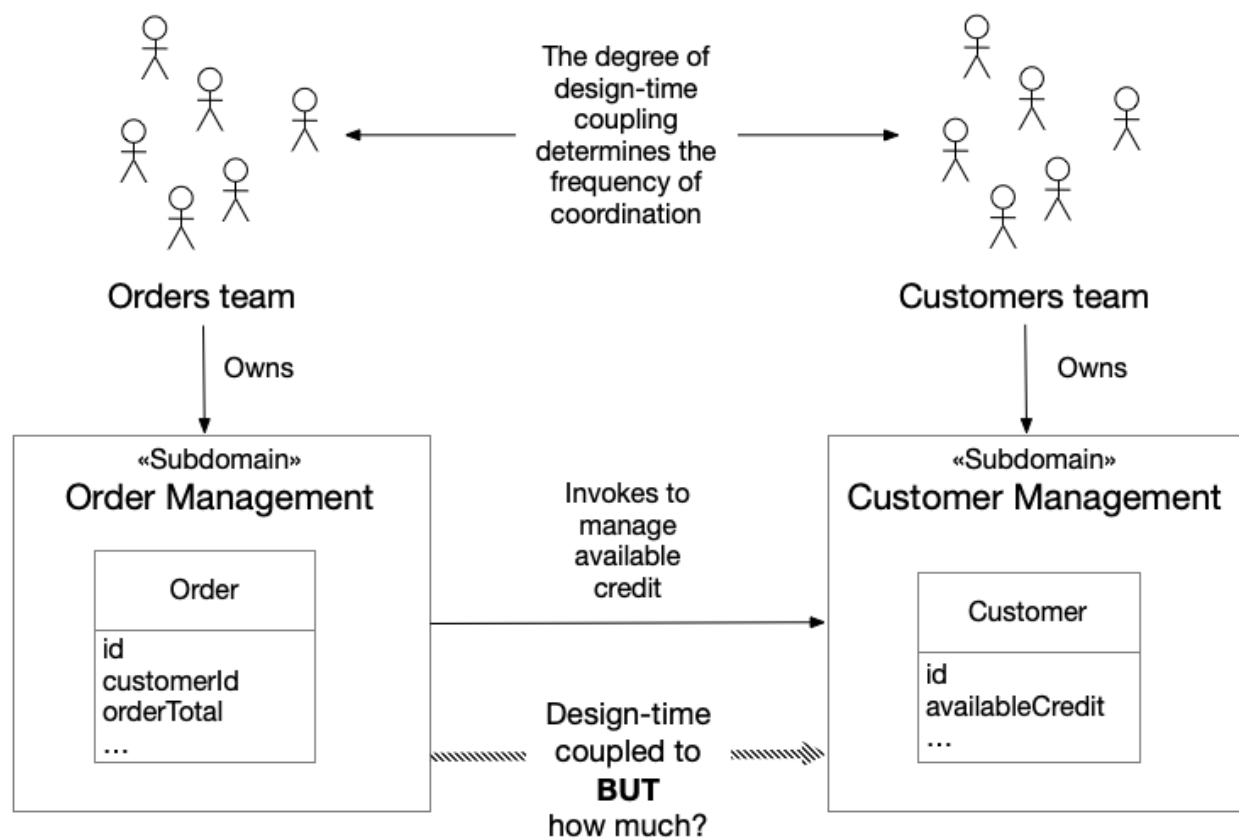


Figure 4.3 The Order Management is design-time coupled to the Customer Management because it invokes the Customer Management's API. The degree of coupling determines how likely a change to Customer Management will require a change to Order Management. Changes that are localized are easier to make than changes that are widespread, especially when multiple teams are involved.

If the Order Management is loosely design-time coupled to Customer Management, then it's unlikely that a change to Customer Management will require a change to Order Management. The change will typically be localized to Customer Management. It might even be localized to a single class within Customer Management. The more localized the change is, the easier and safer it is to make. Especially, when the change is small enough to fit in a single developer's head.

Conversely, if the two modules are tightly coupled, then changing `Customer Management` will most likely affect `Order Management`. Since the change impacts more parts of the application, it's more challenging and riskier to make. The change might even have unintended consequences. Tight coupling is an even bigger problem if the two modules owned by different teams. Let's look at why.

LOOSE DESIGN-TIME COUPLING IMPROVES TEAM AUTONOMY

Another reason why loose design-time coupling is important is that it reduces the likelihood that multiple teams will need to work together to implement the change. Let's imagine, for example, that the `Order Management` and `Customer Management` subdomains are owned by different teams. If those subdomains are tightly coupled, then those two teams will frequently need to work together. That will require meetings to coordinate and plan the work. The `Customer Management` team might need to wait for the `Order Management` team to complete their work. As a result, changes will take longer to implement. The teams will also be less productive due to the overhead of coordination, and one team sometimes having to wait for the other to complete a task.

LOOSE DESIGN-TIME COUPLING ENABLES BUSINESS SUCCESS

Despite being an ancient idea, loose design-time coupling is still very relevant, perhaps more so than ever given the need for fast flow. The authors of *Accelerate*, which is a must-read book about DevOps, have found that business success relies on loosely design-time coupled software. The link between the two is as follows. First, the authors have found that there's a strong correlation between business success and the performance of the software development organization. Second, they have also found that developers in high performing organizations are able to "Complete their work without communicating and coordinating with people outside their team" and "Make large-scale changes to the design of their system without depending on other teams to make changes in their systems or creating significant work for other teams". Being able to work this way requires an architecture that is loosely coupled from a design-time perspective. So in other words, loose design-time coupling makes the business more profitable.

4.2.3 Loose design-time coupling enables organizations to scale

The degree of design-time coupling in an architecture affects how well an organization can scale. With a loosely coupled architecture, the communication overhead is low because the teams mostly work independently. As a result, new teams can be added without significantly increasing the communication overhead.

In contrast, not only does a tightly design-time coupled architecture force teams to spend considerable time

coordinating their work, but it also makes it difficult to grow the organization. That's because every new team that's added to organization increases the communication overhead. Despite having multiple teams, the tight design-time coupling means that the organization behaves like a single large team.

In many ways, the impact of design-time coupling echoes a key insight Fred Brooks's classic book, *The Mythical Man Month* (Addison-Wesley Professional, 1975):

Adding people to a late project makes it later

Brookes observed that the project is delayed because the new people need to be trained and need to communicate with the existing team members. The communication overhead is $O(N^2)$, where N is the size of the team. As the number of people increases, the communication overhead increases quadratically and productivity declines.

More recently, the Accelerate authors found that low performing organizations slowed down (as measured by deployments per day per developer) as they added more people. This finding is consistent with the Fred Brooks's view that adding people to a late project makes it later. However, they found that medium performers stayed the same, presumably because loose design-time coupling minimized the communication overhead. And, remarkably, they found that high performers sped up as they added more people. Deployments per day per developer increased as the number of developers increased.

They wrote:

By focusing on the factors that predict high delivery performance—a goal-oriented generative culture, a modular architecture, engineering practices that enable continuous delivery, and effective leadership—we can scale deployments per developer per day linearly or better with the number of developers. This allows our business to move faster as we add more people, not slow down, as is more typically the case.

— Forsgren et al

Accelerate: The Science of Lean Software and DevOps

4.2.4 Designing loosely design-time coupled software

Loose design-time coupling rarely happens by accident. Achieving it requires deliberate effort and careful design choices. Without thoughtful architectural decisions, systems can easily become tightly design-time coupled, making future modifications costly and complex. Unfortunately, however, designing for loose design-time coupling is challenging because it involves predicting (a.k.a. betting) which aspects of the system will change and which will remain stable.

As figure [4.4](#) shows, stable concepts are represented by APIs, whereas unstable concepts are the implementation details that are hidden by those APIs. If your predictions are correct, then changes will affect implementations but not APIs, whereas if your predictions are incorrect, then changes will ripple through the application.

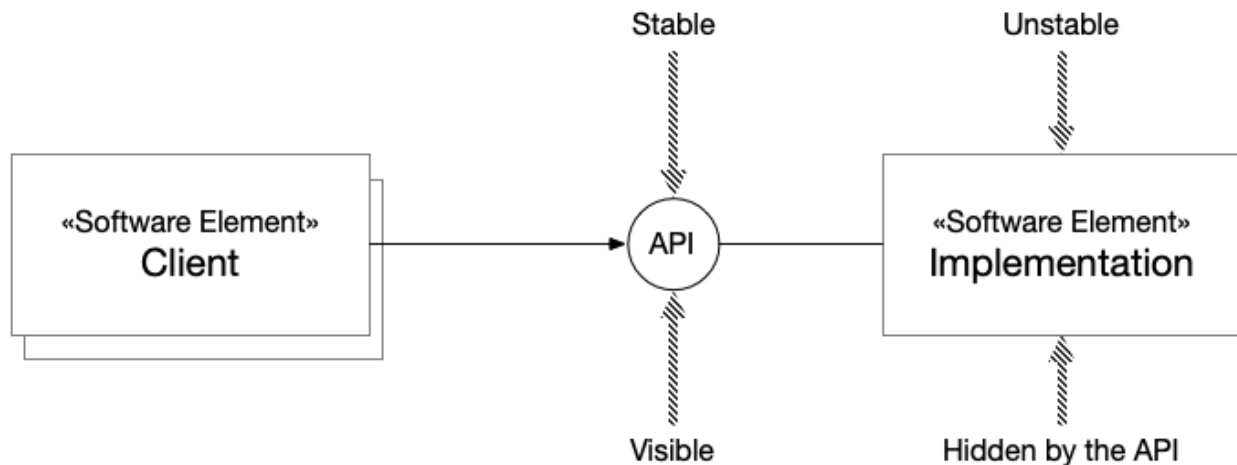


Figure 4.4 Designing an API involves making a bet about what concepts are stable - the API - and what are unstable - the implementation. If you bet correctly, then changes will be localized to the implementation. If you bet incorrectly, then changes will impact the clients.

Predicting what's stable and what is unstable is challenging, as it requires both experience with the domain and an iterative approach to design. It's not uncommon for APIs to be unstable during the early stages of development, as the team learns more about the domain and the requirements of the system. But as the application matures, the APIs should stabilize.

One benefit of the monolith-first approach is that it allows teams to iterate on APIs more easily. Modern IDEs, for example, have automated refactoring capabilities that make it easier and safer to make widespread changes to a code base.

Fortunately, there are principles and guidelines that can help achieve loose design-time coupling:

- DRY (Don't Repeat Yourself) principle
- Iceberg principle
- Dependency management guidelines

Let's look at each one in turn, starting with the DRY principle.

DESIGNING LOOSELY COUPLED SOFTWARE USING DRY

One basic design technique to minimize design-time coupling is the DRY principle

(https://en.wikipedia.org/wiki/Don%27t_repeat_yourself), which stands for Don't Repeat Yourself. The DRY principle states that "every piece of knowledge must have a single, unambiguous, authoritative representation within a system". It requires, for example, that an algorithm should be implemented in a single place. While copy/pasting that code multiple times might be convenient at the time it's done, it means that when the algorithm needs to change, you will need to implement that change multiple times.

Consider an example that I used in one of my talks: changing the order total calculation in the FTGO application. The order total algorithm had been very stable since the application was first developed. It simply calculated the sum of the subtotal, the applicable taxes, a delivery fee, a service fee and a tip. As a result, this business logic had, as figure [4.5](#), been copied and pasted in multiple times. This was not a problem since because it never needed to be changed.

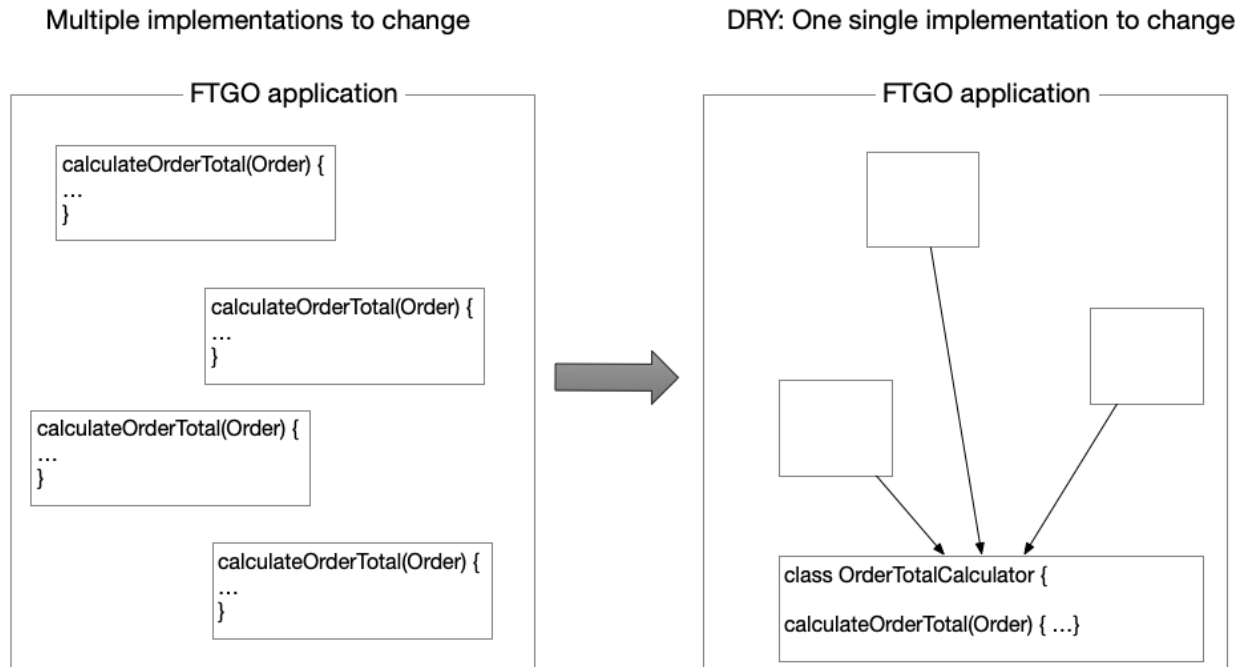


Figure 4.5 Applying the DRY principle replaces multiple implementations of a business rule with a single implementation. As a result, a change only needs to be made in one place, and there's no risk of bugs caused by inconsistent implementations.

But then the pandemic happened, and FTGO decided to add a COVID surcharge to cover the cost of masks for drivers. This seemingly simple change would have been easy to make had the developers applied the DRY principle and implemented the calculation in `OrderTotalCalculator` class. But instead, they had to find and change all the places where the order total was calculated. That was a time-consuming and error-prone process.

One interesting consequence of applying the DRY principle is that now multiple parts of the application depend on the single implementation. For example, in the FTGO application, any code that needs the order total must invoke the `OrderTotalCalculator`. That's typically not a problem in a monolithic code base, but in a microservice architecture the invocations might be expensive network calls. You might be tempted to avoid the communication overhead by putting the `OrderTotalCalculator` in a shared library. But, as I describe

in chapter 7, shared libraries that contain unstable business rules can reduce team autonomy. Each new release of the library forces multiple teams to upgrade their services to the new version in lockstep.

Given that DRY might introduce an undesirable dependency, there are some situations when duplication is acceptable. The principle motivation for applying the DRY principle to the order total calculation is because divergent implementations of the same business rule are a source of bugs. But sometimes, slightly different implementations don't cause bugs. For example, different services could use different versions of a shared utility. You could even have copy and paste duplication if divergent implementations do not cause bugs and the maintenance cost is low.

SOFTWARE ELEMENTS SHOULD LOOK LIKE ICEBERGS

Another principle that helps achieve loose design-time coupling is what I call the *Iceberg principle*, which is illustrated in figure [4.6](#). In the same way that most of an iceberg is below the surface of the water, the surface area of a software element - ie. its API - should be much smaller than the implementation. This is important because, as noted above, what's hidden can easily be changed, whereas what's visible is more difficult to change because it can affect the software element's clients. An API should encapsulate or in other words, hide as much as possible of the implementation.

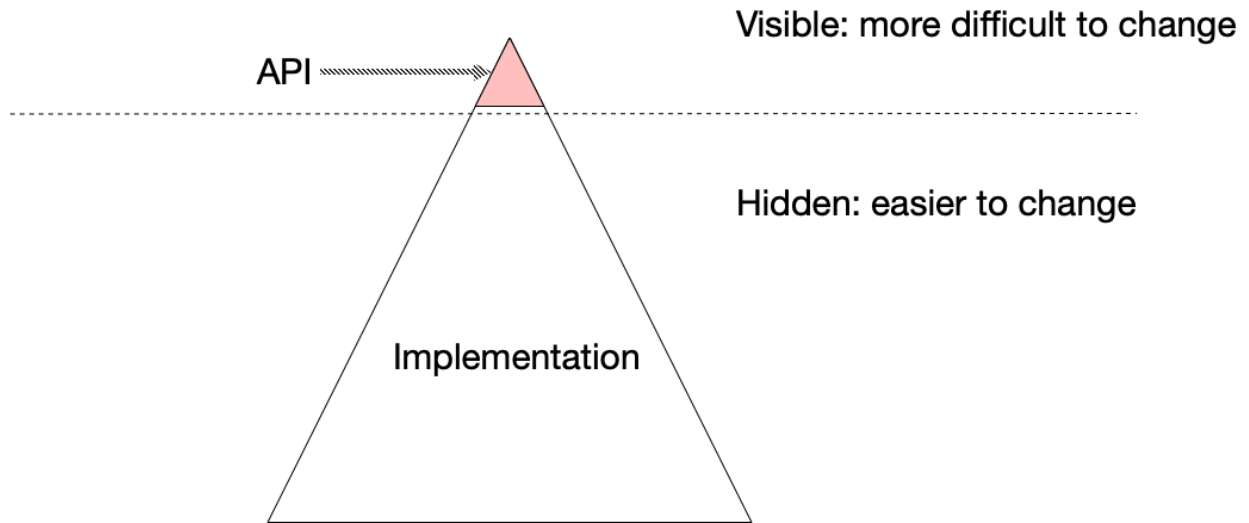


Figure 4.6 Software elements should resemble icebergs: most of the implementation should be hidden by a small, stable API

John Ousterhout in his book *the Philosophy of Software* (Yaknyam Press 2018) has an alternative geometry-based metaphor to describe loosely coupled software elements: they should be deep rather than wide. The width of a software element corresponds to its API. The depth corresponds to its implementation. By designing elements to be deep rather than wide, we minimize what's exposed to clients.

MANAGING DEPENDENCIES

The Iceberg principle minimizes what a software element exposes. But we also need to minimize what a software element consumes. That's because each dependency is a potential reason to change. The more dependencies that a software element has, the less stable it's likely to be.

Consider the example shown in figure [4.7](#). When an order is placed, `Order Management` must notify several other subdomains. There are two ways to do this:

- `Order Management` invoke each subdomain's API

- The subdomains subscribe to an event that `Order Management` publishes, e.g. the Observer pattern.

Each option has different trade-offs when it comes to design-time coupling.

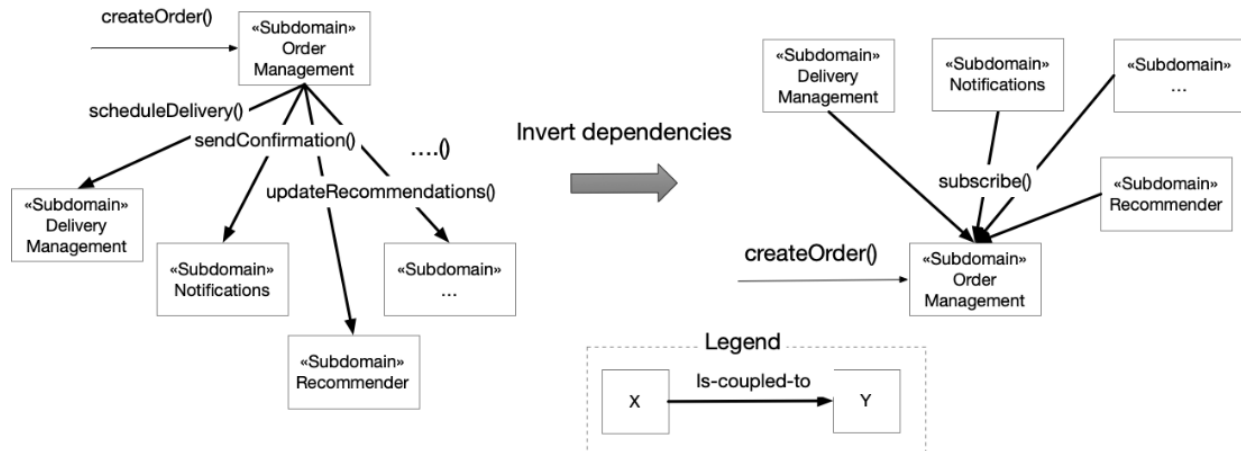


Figure 4.7 There are two ways that `Order Management` can notify other subdomains of a new order: it can invoke each subdomain's API, or it can publish an event that the subdomains subscribe to. The first option design-time couples `Order Management` to the other subdomains. The second option inverts the dependencies so that the subdomains are design-time coupled to `Order Management`. Which option is best depends on the stability of each subdomain's API.

While the first option is straightforward, its main drawback is that `Order Management` is design-time coupled to many other subdomains. Any change to one of their APIs would require corresponding changes to `Order Management`. Unless those subdomains have extremely stable APIs, the large number of dependencies increases the risk that the orders team would need to collaborate frequently with other teams, slowing down development and reducing productivity.

The second option inverts the dependencies so the `Order Management` subdomain is unaware of which other subdomains consume the events that it publishes. From the perspective of the `Order Management` team, this is a big improvement since their subdomain is not design-time coupled to the

consumers. In theory, it has fewer reasons to change. The concern, however, is that the subdomains are design-time coupled to `Order Management`. Changing the `Order Management` subdomain might require one or more of the subdomains to change in lockstep. Which option is better depends on the details of the scenario, including the stability of each subdomain's API. As I discuss in chapter 20 careful analysis is required.

What's more, it's not just the number of dependencies that's important. We also need to take into account the 'surface area' of each dependency that is consumed. The more of the dependency that's consumed, the more likely it is to change. An example of loose design-time coupling is the NASA-style launch status check

(<https://microservices.io/post/architecture/2024/12/16/launch-status-check-loose-coupling.html>). The flight director calls out to each team that is responsible for a particular subsystem, such as the engines, the guidance system, and the fuel system, and they respond with "Guidance is go/no go", etc. If all teams are ready, the launch proceeds; otherwise, it's scrubbed. While each subsystem is internally quite complex, it exposes the human equivalent of a simple and very stable API: `boolean isGo()`.

Figure 4.8 shows two different designs for automating the launch status check. The first option mirrors the NASA approach. The `Flight Management` subdomain simply invokes `isGo()` on each subdomain. It is loosely design-time coupled to each subsystem and unaffected by any changes to them.

The second option is a design where the `Flight Management` subdomain retrieves each subdomain's data and makes the decision itself. This design has the equivalent of the Insider Trading OO design smell

(<https://microservices.io/post/architecture/2024/11/03/redu>

cing-insider-trading-microservices.html), which occurs when one class accesses the internal data of another class. As a result, the flight controller is tightly design-time coupled to the subsystems and will most likely need to change whenever a subsystem changes.

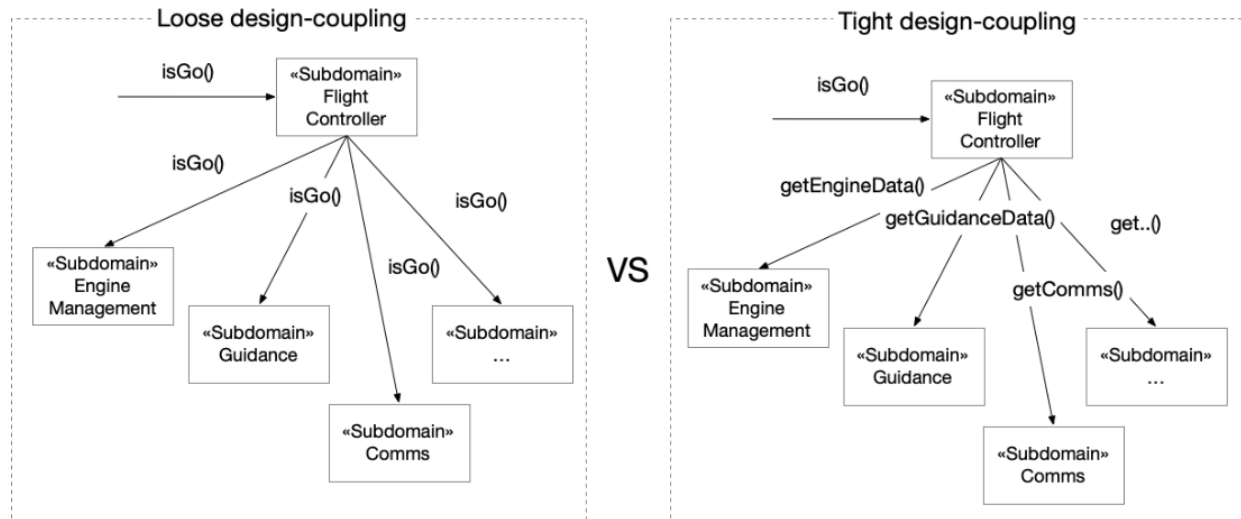


Figure 4.8 Two possible ways to implement the NASA-style launch status check. The first and better option is for Flight Controller to invoke `isGo()` on each subdomain. The second option, which is tightly design-time coupled, is for Flight Controller to retrieve data from each subdomain and perform the calculation itself.

As you have just seen, it's essential to minimize design-time coupling. It's also usually important to follow the DRY principle. Also, software elements should expose as little as possible **and** consume as little as possible. Ensuring that software elements are loosely design-time coupled will improve productivity by increasing team autonomy and making changes easier to implement. Let's now look at another type of coupling that affects productivity: build-time coupling.

4.3 Build-time coupling

The second type of coupling is build-time coupling, which is a type of relationship between the subprojects of a build

project. A build project, which is part of the *repository and deployment dimension of architecture* discussed earlier in chapter 3, organizes the source code for one or more subdomains into build subprojects that build and test components. For example, the build project for the FTGO monolith consists of multiple subprojects, such as `order-management`, `customer-management`, and `delivery-management`. A build subproject, such as `order-management`, is build-time coupled to build subproject `customer-management` if changing `customer-management` requires `order-management` to be recompiled and retested even though it hasn't changed.

Like design-time coupling, build-time coupling also affects development. But rather than influencing how easy it is for a developer to make a change, build-time coupling determines how long the developer who has made a change must wait for feedback. Ideally, only the changed subprojects should be recompiled and retested, which is typically much faster than recompiling and retesting the entire component. Build-time coupling, however, increases the number of subprojects that must be recompiled and retested. In the worst case, the developer might need to wait for the build tool to recompile and retest the entire component!

To better understand the concept of build-time coupling, let's look at a concrete example. There are many build tools, but my favorite build tool is Gradle (<https://gradle.org/>). It's the one that I'm going to describe here, although many others work similarly. I'll first describe build-time coupling in a Gradle project. After that, I'll describe a couple of ways to minimize it.

4.3.1 About build-time coupling in Gradle projects

A Gradle build project, which builds and tests components, usually consists of multiple subprojects. You typically define dependencies between subprojects

(https://docs.gradle.org/current/userguide/dependency_management_for_java_projects.html), which enables one

subproject to use the other project's classes at either compile-time, runtime or both. Because Gradle implements incremental builds

(https://docs.gradle.org/current/userguide/incremental_build.html#header), it will only recompile and retest those

subprojects that are affected by a change: the subproject that's changed and those subprojects that depend on it. It's these dependencies that determine build-time coupling between projects.

Let's imagine, for example, the FTGO application has the Gradle project structure shown in figure [4.9](#). When you run `./gradlew build` after changing the `Money` class in project `money`, which contains the code for the `Money` subdomain, Gradle will run all the tests in the project `money` since it contains the changed file. It will also run the tests of any subprojects that depend on the changed subproject. Specifically, Gradle will also run the tests in `accounting` and `customer-management` since they depend on `money` via the `implementation` dependency. Moreover, Gradle will run the tests in `order-management` since it has a transitive dependency on `money` via `customer-management's api` dependency on `money`.

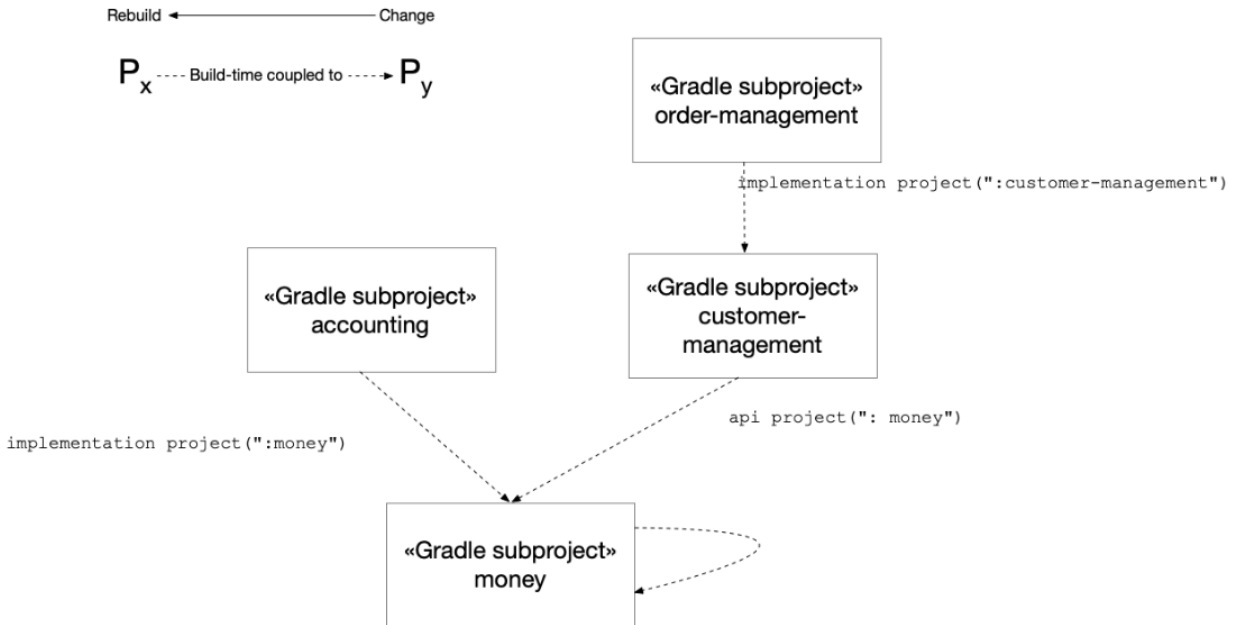


Figure 4.9 The dependencies between Gradle projects create build-time coupling. Changing `money` causes the other projects to be recompiled and retested but changing `customer-management` only requires `customer-management` and `order-management` to be recompiled and retested.

In contrast, changing the `Customer` class in `customer-management` only requires Gradle to recompile and retest the `customer-management` and `order-management` subprojects. Build projects, such as `accounting` and `money`, that do not depend on `customer-management` are unaffected by the change.

4.3.2 A project should only be build-time coupled to stable projects

One way to accelerate builds is to minimize the number of subprojects that depend upon on a subproject that changes frequently. In particular, a subdomain's implementation almost always changes more frequently than its API. Consequently, it makes sense to structure a subdomain as follows:

- API subproject - defines the API that is used by the subdomain's clients

- implementation subproject - the subdomain's implementation

Consider, for example, the `Customer Management` subdomain, which has a `CustomerService` interface. It's extremely likely that the `CustomerService` interface changes far less frequently than its implementation. If, however, the `Customer Management` consisted of a single `customer-management` Gradle project, which was depended upon by its clients, such as `order-management`, then every time the implementation changed, those clients would need to be recompiled and retested.

But, as figure [4.10](#) shows, we can accelerate builds by splitting the `customer-management` project into a `customer-management-api` project, which contains `CustomerService` and related classes, and `customer-management-impl`, which contains the implementation classes. Clients, such as `order-management` would depend on the relatively stable `customer-management-api` project rather than the unstable `customer-management-impl` project. Also, the client's tests would use mocks for the `CustomerService` interface.

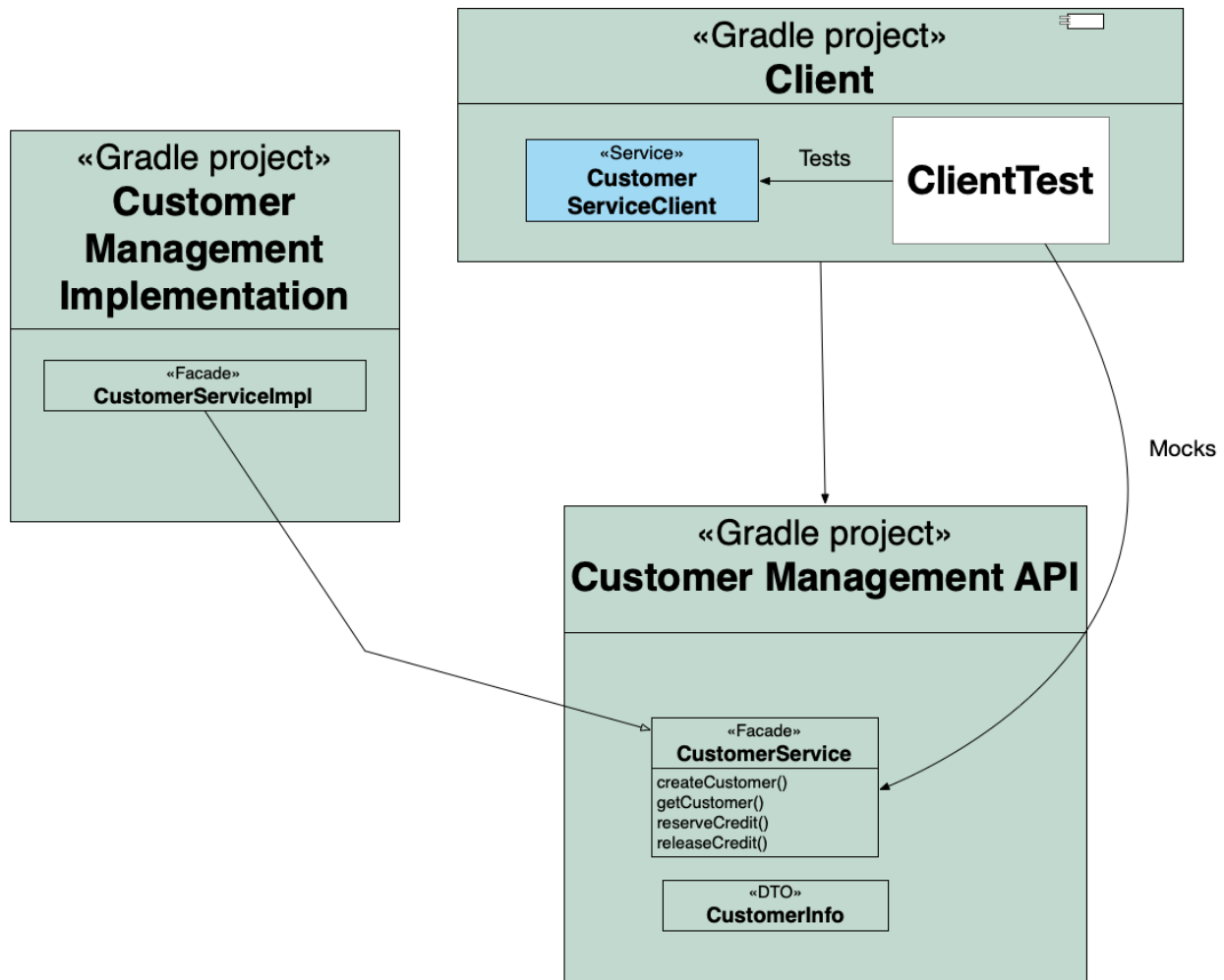


Figure 4.10 Defining an API project and an implementation for a subdomain reduces build-time coupling since the API project changes much less frequently than the implementation project

Using a separate `Customer Management API` project decouples the clients from the implementation. Changes to the `Order Management` subdomain would rarely require its clients to be retested.

4.3.3 Reducing build-time coupling within a Gradle subproject

Another way to speed up builds is to split a Gradle subproject into multiple subprojects. A Gradle build project is build-time coupled to itself. Gradle will run all tests in a

subproject when any of its source files have changed. As a result, it might run tests for code that is unaffected by a change.

For example, consider the `Customer Management Implementation` Gradle project that contains three packages `web`, `persistence`, and `domain` where `web` and `persistence` depend upon `domain` but not on each other. A change to a class in the `web` package would cause classes in all three packages to be retested even though neither `domain` nor `persistence` are affected by the change. To avoid this problem, we can, as figure 4.11 shows, reduce build time coupling by splitting the Gradle project into three subprojects that mirror the packages. The `web` and `persistence` subprojects depend upon the `domain` project but not each other.

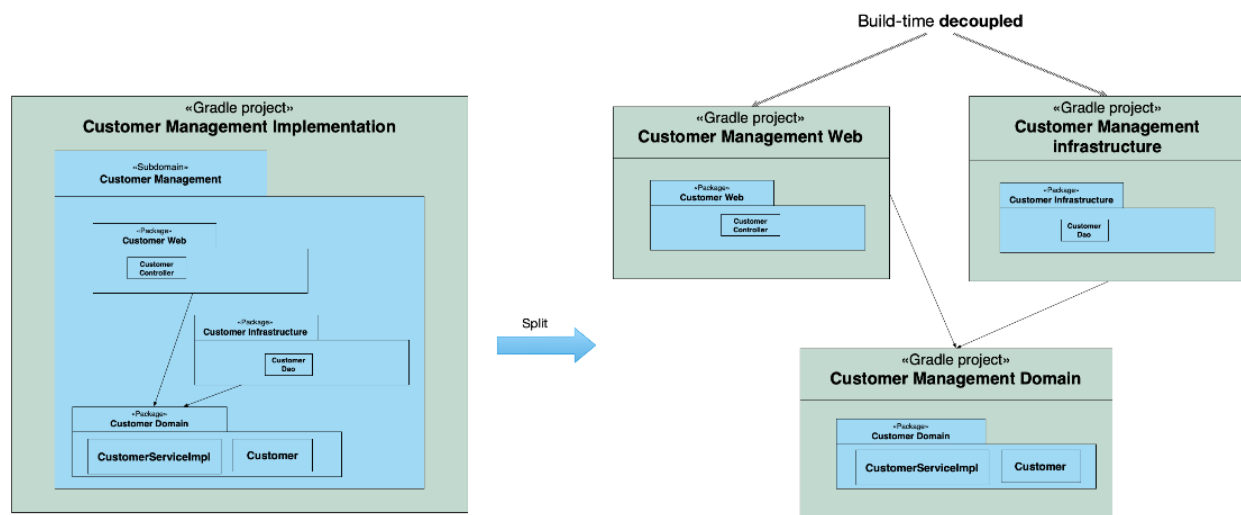


Figure 4.11 Splitting a subdomain’s implementation project into several projects can often reduce build-time coupling

The `web` and `persistence` projects are not build-time coupled and so a change to one will not trigger the execution of the other’s tests. This is especially beneficial because the `persistence` project’s tests use a database and so could be quite slow.

It's important to minimize build-time coupling in a large monolithic architecture since it consists of a single code base. Later in chapter 6, I'll describe some patterns for doing just that. Build-time coupling is a less important concern in a microservice architecture since each service is relatively small. However, it can still be useful and in chapter 13, I'll discuss some patterns for organizing a service's code base, which are partially motivated by the goal of reducing build-time coupling.

Let's now look at a very different type of coupling: runtime coupling.

4.4 Runtime coupling

The third type of coupling is runtime coupling. Unlike, design-time coupling and build-time coupling, which impact development, runtime coupling affects the runtime behavior of a distributed application. In a distributed architecture, it's common for an operation (aka request) to span multiple components. A component, such as the `Order Service`, is runtime coupled to component, such as the `Customer Service`, if `Order Service` cannot respond to request until it receives a response from `Customer Service`. Or, in other words, the failure of `Customer Service` causes `Order Service` to fail. As a result of runtime coupling, the availability of the operation depends on the availability of both components.

Runtime coupling is one of the reasons that the famous computer scientist Leslie Lamport (https://amturing.acm.org/award_winners/lamport_1205376.cfm) once wrote:

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

It's important to pay close attention to runtime coupling when designing a distributed architecture, such as a microservice architecture. Otherwise, you risk building an application that is fragile and unreliable.

Let's first look at an example of tight runtime coupling and see how it impacts the application's availability and latency. After that, we'll look at some strategies for minimizing runtime coupling.

4.4.1 An example of tight runtime coupling

To understand how tight runtime coupling reduces availability and increases latency, let's consider an example. Figure 4.12 shows a simple architecture where the `Order Service` handles a `POST /orders` request by invoking the `Customer Service` to validate the customer and reserve credit. The `Order Service` is runtime coupled to the `Customer Service` because it cannot respond to the `POST /orders` request until the `Customer Service` has responded to the `PUT /customers/{id}` request.

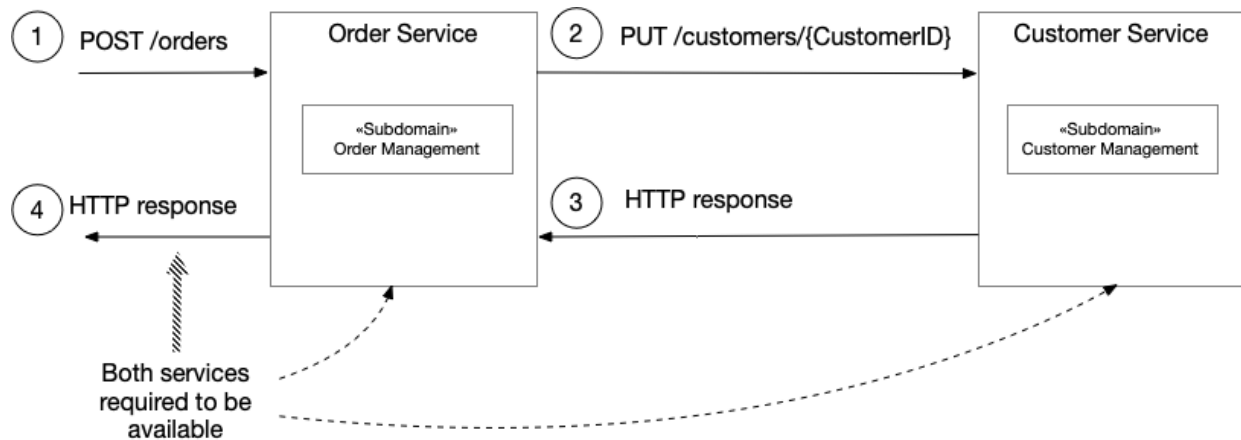


Figure 4.12 The `Order Service` cannot respond to the `POST /orders` request until the `Customer Service` has responded to the `PUT /customers/{id}` request. As a result, the availability of the `POST /orders` request is the product of the availability of the `Order Service` and the `Customer Service`

Since both services must be available in order for the application to handle the `POST /orders` request, the availability of that endpoint is reduced. Mathematically speaking, the availability of the `POST /orders` request is the product of the availability of the `Order Service` and the `Customer Service`. If the `Customer Service` is 99.9% available and the `Order Service` is 99.9% available then the availability of the `POST /orders` request is 99.8%. While even a chain of two services reduces availability, a common anti-pattern is to have long chains of numerous services that are runtime coupled. They behave like Christmas tree lights where a failure of an individual service causes the entire chain to fail.

Tight runtime coupling can also increase latency since the response time is the sum of the request response times. For example, the response time of the `POST /orders` request is the sum of the response times of the `Order Service` and the `Customer Service`.

4.4.2 Siloed organization can lead to tight runtime coupling

Tight runtime coupling often arises in organizations where no one is paying attention to the end-to-end flow of requests. For example, while conducting an architecture review for a client, I investigated the implementation of a key request, which was implemented by two teams. Team A had a couple of services, which then invoked Team B's service. From Team A's perspective, everything seemed fine. But when I talked to Team B, I discovered that they had implemented a cluster of services. As a result, the request was dependent on the availability of many services. There was a risk that the client could not meet their SLA because of this runtime coupling.

4.4.3 Minimizing runtime coupling

There are few different strategies to minimize an operation's runtime coupling:

- Merge an operation's subdomains into a single service
- Move responsibilities between services to reduce distribution
- Implement the operation using asynchronous messaging

Let's look at each one in turn, starting with fewer, larger components.

MERGE AN OPERATIONS' SUBDOMAINS INTO A SINGLE SERVICE

One way to reduce runtime coupling for a given operation by organizing its subdomains into a single service. If an operation is local to a single component, there's no runtime coupling. Consider, for example, the `createOrder()` operation shown in figure [4.12](#), is implemented by the `Order Management` and `Customer Management`, which are each in their own service. We could, for example, make this operation local to a single service by, as figure [4.13](#) shows, merging all three subdomains into a single `Order Service`.

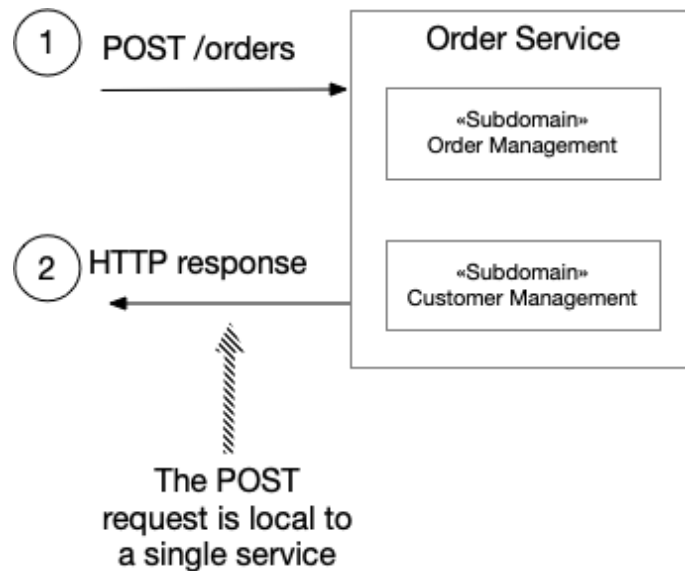


Figure 4.13 Implement `createOrder()` as a local operation by merging the Order Service and Customer Service into a single service

This will ensure that the `POST /orders` request is handled by that service. The drawback of this approach is that the combined service is likely to be larger than is ideal. What's more, if you make every operation local to a single service, there's a risk that you end up with a monolith and its associated drawbacks. Consequently, this approach is best used for frequently invoked operations that must have high availability and low latency.

MOVE RESPONSIBILITIES BETWEEN SERVICES TO REDUCE DISTRIBUTION

The second way to reduce runtime coupling, which requires more careful design, is to shift responsibilities between services instead of simply merging them. We could, for example, move the responsibility for managing available credit from the `Customer Service` to the `Order Service`. The `Order Service` would then be able to validate the customer's credit without invoking the `Customer Service`.

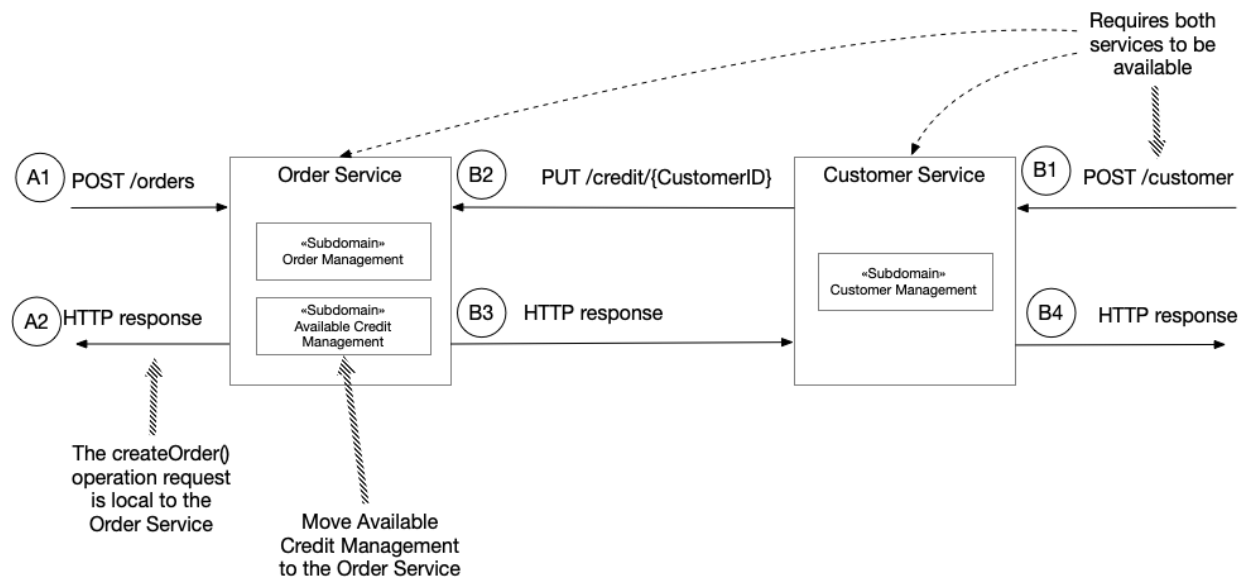


Figure 4.14 Implement `createOrder()` as a local operation by extracting Available Credit Management from the Customer Management subdomain and move it to the Order Service. One notable downside is that creating a customer now involves two services - fortunately, this is a less critical operation

Reassigning responsibilities in this way would reduce runtime coupling and improve availability and latency. One drawback of this approach, however, is that creating a customer now involves two services. However, since it is a less critical operation, it's most likely a reasonable trade-off. A limitation of this approach, which is a type of architectural refactoring that I describe in chapter 20, is that it only works if the subdomain that's moved is used by a single service. If it's used by multiple services, then only one service could access it locally.

ASYNCHRONOUS MESSAGING

The third way to reduce runtime coupling is to use asynchronous messaging. As figure 4.15 shows, the `Order Service` could, for example, handle the `POST /orders` request by sending a message to the `Customer Service` to validate the customer and returning an HTTP response immediately. The

Customer Service would then reply to the message from the Order Service when it's available. When the Order Service receives the reply, it can finalize processing of the Order.

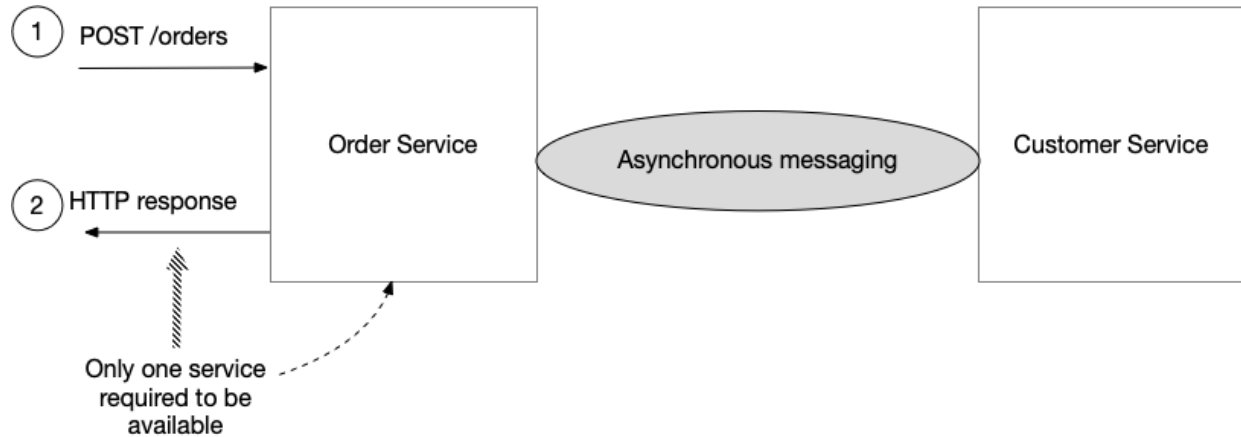


Figure 4.15 The Order Service handles the `POST /orders` request by sending a message to the Customer Service to validate the customer and returning an HTTP response immediately. Eventually, the Customer Service sends a message to the Order Service, which can then either approve or reject the Order

The benefit of this approach is that it increases the availability of the `createOrder()` operation since the Order Service can respond without waiting for the Customer Service. The downside of this approach is that the HTTP response only contains a partial outcome. It does not reflect the outcome of the Customer Service validating the customer and attempting to reserve credit. To determine the outcome, the Order Service's client will need to either periodically poll the Order Service or subscribe to the events published by the Order Service when it finally approves or rejects the Order.

Despite these drawbacks, asynchronous messaging is a powerful technique for reducing runtime coupling. Later in chapters 10 and 11, I describe patterns that use asynchronous messaging to reduce runtime coupling. Let's now look at the other type of coupling that can impact the application's runtime behavior: infrastructure coupling.

4.5 Infrastructure coupling

The fourth type of coupling is infrastructure coupling, and like runtime coupling, it affects the application's runtime behavior. An application's components run on and consume infrastructure. Infrastructure is a broad term and includes the basic compute and networking resources, along with services, such as database servers and message brokers. Components that use the same infrastructure are infrastructure-coupled. Even though they do not directly or indirectly communicate, these components can negatively affect each other's runtime behavior.

It's important, therefore, to minimize infrastructure coupling, especially for highly available components. Let's first look at an example of infrastructure coupling. After that, I'll describe strategies for minimizing infrastructure coupling.

4.5.1 An example of infrastructure coupling

To understand why infrastructure coupling is important, let's consider once more the `Customer Service` and `Order Service` example. While one of the responsibilities of the `Customer Service` is to support the `Order Service` by managing the customer available credit, it would typically also have other responsibilities, such as managing customer accounts. One simple way to deploy these two services, which is shown in figure [4.16](#), is on the same Kubernetes cluster and with a shared database server. However, since the instances of the two services share the same infrastructure, they can potentially affect each other. This can be a problem if a less critical service, such as the `Customer Service`, could prevent a more critical service, such as the `Order Service`, from functioning correctly.

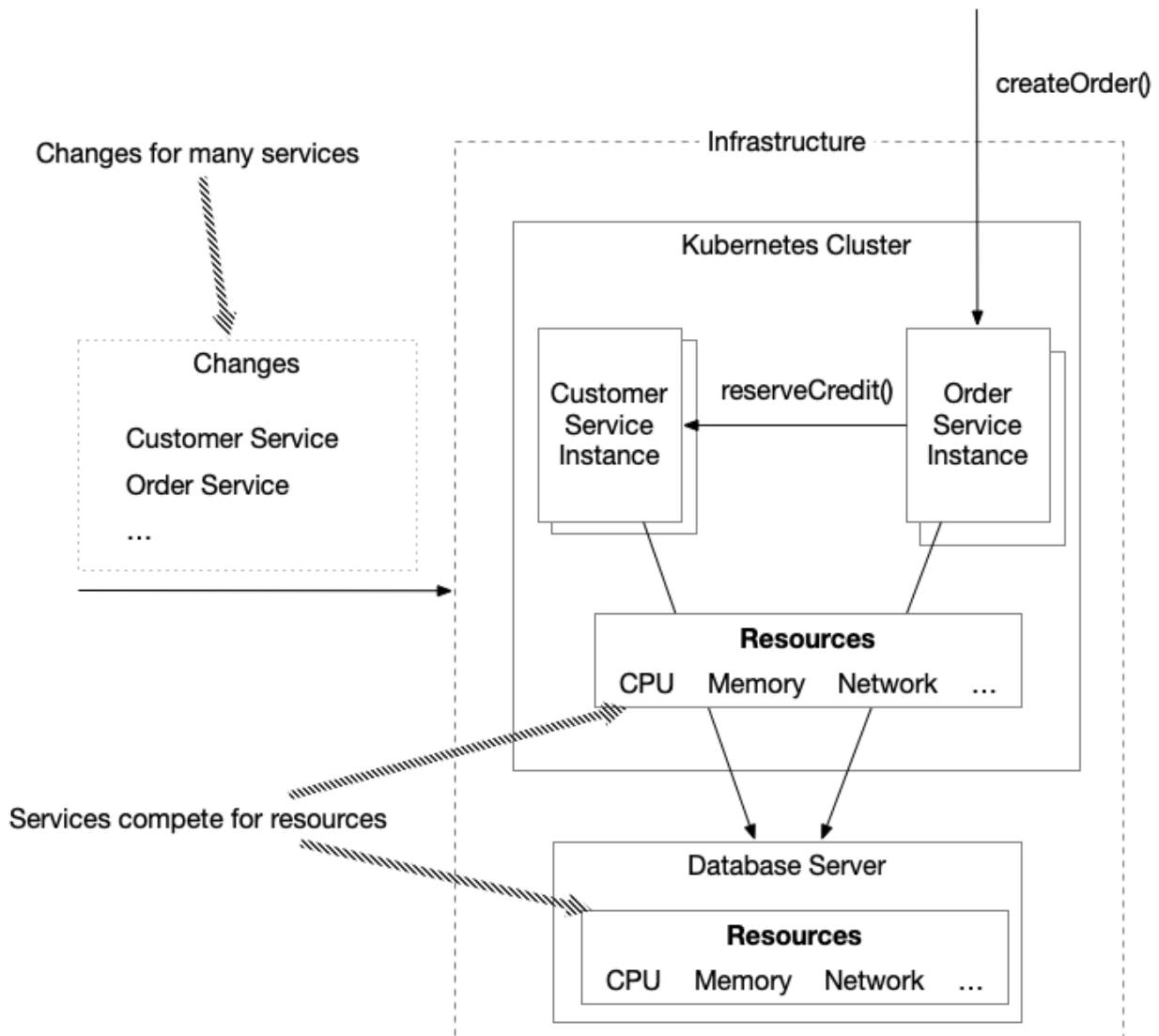


Figure 4.16 If multiple services share the same infrastructure, they can potentially degrade each other’s runtime behavior since they are competing for the same resources. Also, configuration changes for one service can affect another service.

In this example, the two services are competing for resources, such as CPU, memory, and network bandwidth. Also, a configuration change to infrastructure that supports one service could affect the other service. Let’s look at these two problems in more detail.

4.5.2 Two types of infrastructure coupling

There are two different types of infrastructure coupling:

- Competition for resources
- Configuration changes with unintended consequences

let's look at each one in turn, starting with competition for resources.

COMPETITION FOR RESOURCES

The first type of infrastructure coupling is competition for resources. Kubernetes lets you configure limits on the CPU and memory resources that a container can consume.

Provided that you remember to configure limits, the `Customer Service` cannot, for example, starve the `Order Service` of those kinds of resources. However, it could steal all the available network bandwidth. Or, the `Customer Service` could consume all the available database resources, such as connections, or overload the database server with expensive queries. The `Customer Service` could even block the `Order Service` by simply holding a database lock for a long time.

CONFIGURATION CHANGES WITH UNINTENDED CONSEQUENCES

The second kind of infrastructure coupling is related to configuration changes. Complex systems often fail when a configuration change has unintended consequences.

Consequently, in the `Customer Service` and `Order Service` example, there's a risk that a configuration change to the Kubernetes cluster or the database server for one service could accidentally affect the other service.

4.5.3 Minimizing infrastructure coupling

We can minimize the infrastructure coupling of truly critical services through a combination of service design and

deployment infrastructure strategy. First, we should design such services so that they are runtime decoupled from other services. This will ensure the services are unaffected by the failure of other services. Second, we should deploy the critical services on their own infrastructure with separate infrastructure services. This ensures that the critical services are not competing for resources and are not affected by configuration changes made to support other services. By applying these principles, we can increase the availability of critical services.

Let's imagine, for example, that the `Order Service` needs to be highly available. We can use the design option shown earlier in [Section 4.4.3](#) where the `Order Service` tracks the customer's available credit and the `POST /orders` request is local to the `Order Service`. We can then, as figure [4.17](#) shows, deploy this service on a separate Kubernetes cluster and with a separate database server. Key requests, such as `createOrder()`, are handled entirely by the `Order Service` instances and the infrastructure that they use on and run. There's no resource sharing with the other services. Also, the `Order Service` instances are not affected by configuration changes to the other services.

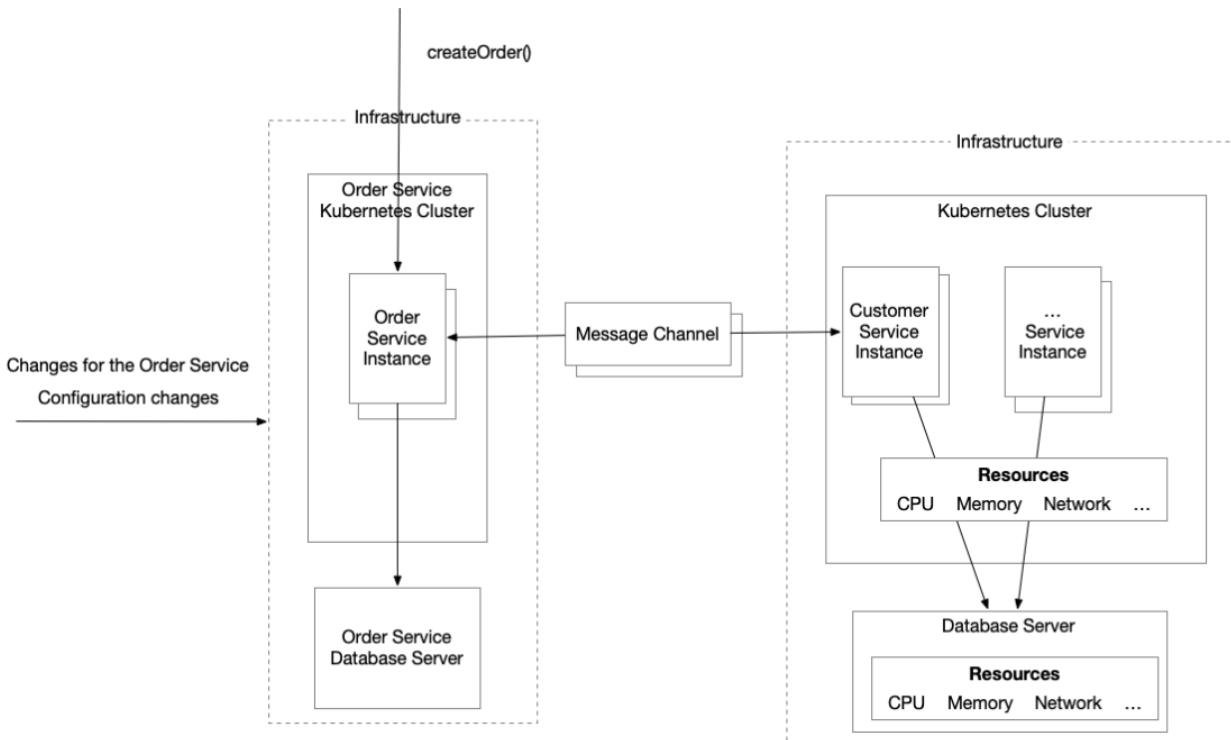


Figure 4.17 To minimize infrastructure coupling and improve availability, services that are critical to the business should be deployed on separate infrastructure and loosely runtime coupled to the rest of the application

Infrastructure coupling is the fourth and final type of coupling that I cover in this chapter. Infrastructure coupling along with runtime coupling affects the application's runtime behavior, whereas design-time and build-time coupling affects the development experience.

4.6 Summary

- Two software elements are coupled if one can affect the other in some way either at development time or runtime
- There are four types of coupling:
 - Design-time coupling - the likelihood that a change to one element will require a change to the other element.

- Build-time coupling - when changing one element requires another element to be recompiled and retested.
- Runtime coupling - when one element's failure causes another element to fail.
- Infrastructure coupling - when two elements share infrastructure, such as a database, and one element's behavior affects the other element
- Loose design-time coupling, which localizes changes by minimizing the number of software elements impacted by a change, is essential for fast flow since it increases team autonomy and makes changes easier to implement.
- Loose build-time coupling, which reduces the number of build subprojects that must also be recompiled and retested when a given build module is changed, is important for a fast deployment pipeline, especially for large codebases.
- Loose runtime coupling is important for high availability and low latency since it reduces the likelihood that a failure or slowness of one component will cause another component to fail or to be slow.
- Loose infrastructure coupling is essential for high availability since it reduces the likelihood that one component will affect another component's runtime behavior by competing for resources or by configuration changes.
- While it's generally desirable to minimize the different types of coupling, it's important to remember that some degree of coupling is inevitable, and eliminating coupling might have significant drawbacks.

5 Architectural requirements for fast flow

This chapter covers

- The five quality attributes that are essential for fast flow
- The importance of these quality attributes
- How an architecture can satisfy the quality attributes
- The role of an application's component architecture in determining how effectively these quality attributes are met

In chapter 1, I briefly explained that achieving fast flow - the continuous delivery of a stream of small changes and rapid feedback one each one - requires an architecture that is loosely coupled at design time and supported by a rapid deployment pipeline. In this chapter, I further explore these architectural requirements through the lens of quality attributes, a concept introduced in chapter 3.

A quality attribute is a measurable, desirable property of an application defined through specific scenarios. I will describe how an architecture must satisfy five key quality attributes:

- Modifiability - ease of making changes to the application
- Evolvability - ease of evolving the application's technology stack
- Testability - ease and speed of verifying that a change is releasable
- Deployability - ease and speed of deploying a change into production

- Observability - ease of understanding the application's runtime behavior and the behavior of its users

For each quality attribute, I'll describe example scenarios and discuss how the architecture can satisfy them. Let's begin by examining the workflow that teams use to implement a flow item, such as a feature or bug fix, which results in a change to the application.

5.1 Introduction to architectural requirements for fast flow

To understand the architectural requirements for fast flow, let's walk through how a developer - or more generally, a team - implements a flow item. As figure [5.1](#) shows, the development workflow is an iterative process, which ends when the flow item is either completed or abandoned. The outer change-deploy-analyze loop consists of the following steps:

1. Make one or more design decisions
2. Implement the changes
3. Deploy the changes by submitting the changes using `git commit/push`, which triggers the deployment pipeline that then tests and deploys the changes
4. Analyze feedback from either the deployment pipeline (if it fails) or from the production environment and end-users

Nested inside the first loop is the edit-test-analyze loop, which is the developer's implementation workflow. The developer makes a change, runs the tests, and analyzes the results.

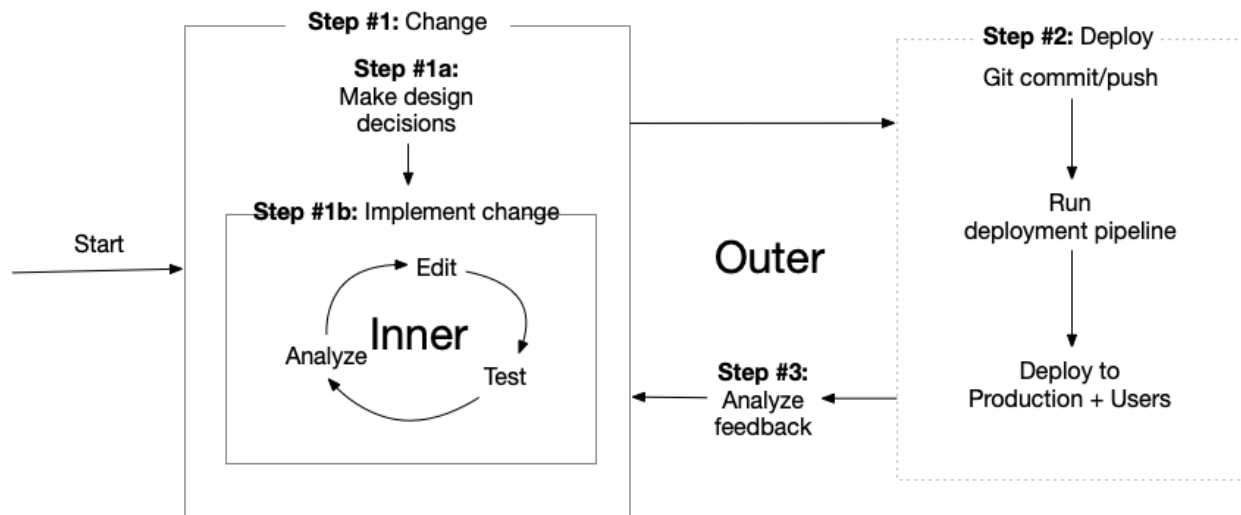


Figure 5.1 The iterative workflow for designing and developing a flow item. It consists of two loops, one nested inside the other. The outer change-deploy-analyze deploys changes into production and analyzes feedback. The change step of the outer loop consists of making a design decisions and the inner edit-test-analyze loop, which is the developer’s workflow for implementing a change.

The scope of flow items varies widely. A flow item might require a simple, local change consisting of just a few lines of code, such as the modification of a business rule. Others, however, might require widespread changes, including architectural modifications. For example, a breaking change to an API can require the API’s clients to be updated. The implementation of complex new feature in microservice architecture might include the creation of new services.

Furthermore, a flow item’s changes aren’t just limited to application code. As I discuss in detail in [Section 5.3](#), a flow item might require changes to the application’s technology stack, such as the upgrade of a library, the language runtime or an infrastructure service. A flow item might also include changing the configuration of the deployment pipeline or deployment infrastructure.

Since the goal is fast flow, every step in the workflow must be both fast and straightforward. We’ll examine each step to

identify what is required to accelerate and simplify it. In doing so, I will highlight the specific quality attributes that are essential for each stage.

5.1.1 Step #1: Making a change

The first step in the outer change-deploy-analyze loop is to make a change. There are two sub-steps in the first step of the workflow: making design decisions and implementing the changes. Let's look at each of these sub-steps in turn.

STEP #1A: MAKE DESIGN DECISIONS

Unless the flow item is very simple, the developer must first make one or more design decisions. Each design decision typically requires evaluating and comparing multiple candidate solutions and selecting the best approach. The **architecture advice process**, which I described in chapter 3, requires a developer to seek advice from two groups of people: people impacted by the decision; and experts in the area of the decision.

As figure [5.2](#) shows, the number of people involved in a decision can vary widely, which in turn affects the time required to make the decision. Decisions that involve only the developer or their team can be made quickly. But decisions that involve multiple teams, perhaps even the entire organization, can take a long time to make.

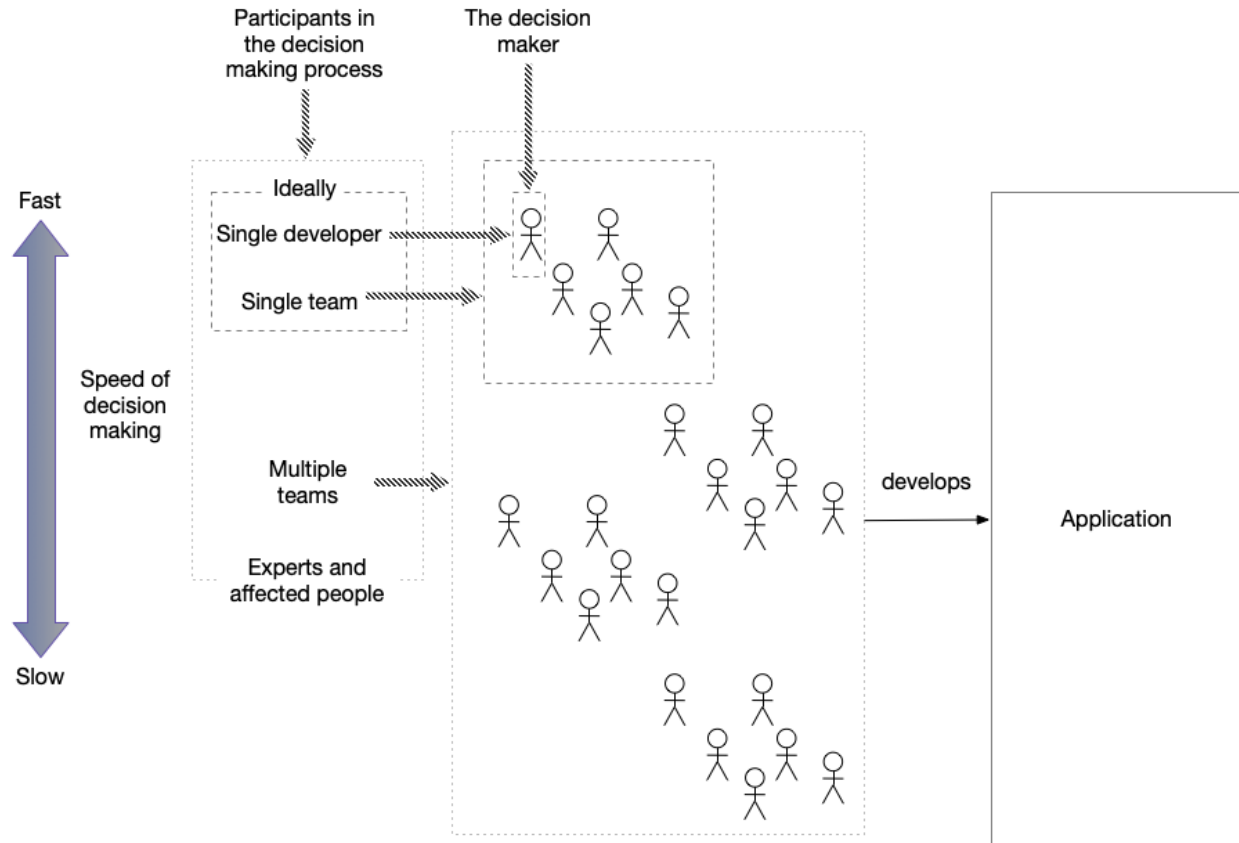


Figure 5.2 The number of people involved in making a decisions varies widely. The more people involved, the longer it takes to make the decision. Ideally, a decision should involve, at most, the decision maker's team.

To enable design decisions to be made rapidly, an application's architecture must have two properties:

- easy to understand and change - developers must be able to quickly identify what changes are needed and determine how to implement them.
- localized changes - minimize the impact of change and the number of people impacted by a decision in order to streamline decision-making. Ideally, decisions must be local to a team.

As figure [5.3](#) shows, these architecture requirements are captured by the following quality attributes:

- Modifiability - This quality attribute reflects how easily and rapidly changes can be made. I describe modifiability in more detail in [Section 5.2](#).
- Evolvability - This quality attribute reflects how easy it is to change the application's technology stack. Evolvability is another flavor of modifiability, but as I explain in [Section 5.3](#), it's distinct enough to warrant separate treatment.

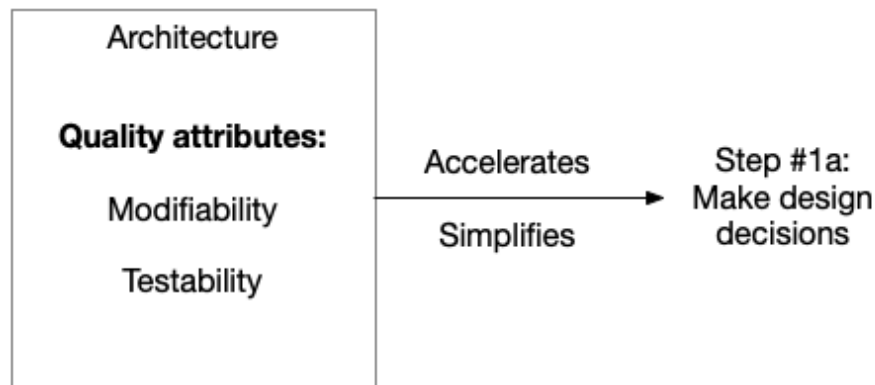


Figure 5.3 The modifiability and evolvability quality attributes determine the speed and ease of making design decisions

Once the design decisions have been made, the next step is to implement the changes.

STEP #1B: IMPLEMENT CHANGES

When implementing the changes required by a flow item, developers spend the majority of their time cycling through the following edit-test-analyze loop:

1. Writing or modifying code including tests
2. Running the tests
3. Analyzing the test results

To iterate rapidly, the application must be easy to understand and change. The tests must be fast and they must run locally. Also, the team must be able to complete

the work with little or no coordination with other teams. They should not, for example, need to wait for another team to make changes to their code base.

As figure [5.4](#) shows, these requirements are captured by three quality attributes. The first two are modifiability and evolvability, which I introduced above. The third quality attribute is testability. Testability reflects how easily and quickly a change can be verified as releasable. I describe testability in more detail in [Section 5.4](#).

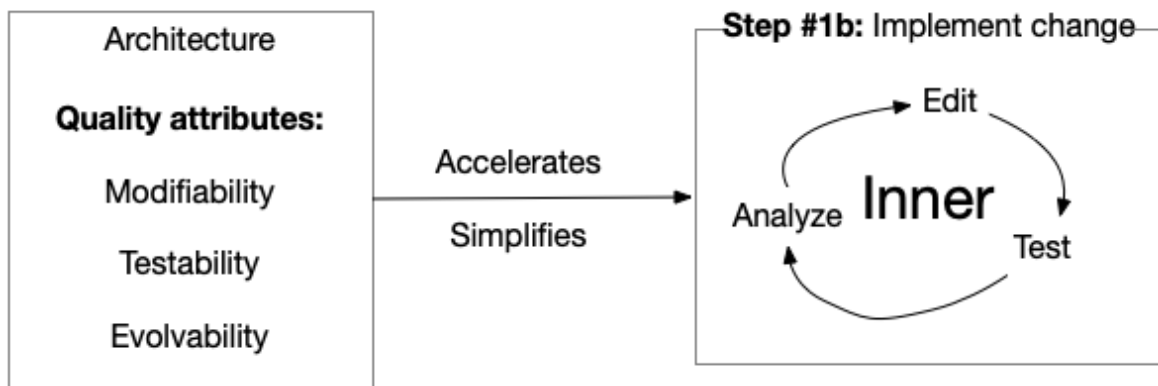


Figure 5.4 The modifiability, evolvability and testability quality attributes determine the speed and ease of implementation

Testability enables developers to iterate rapidly through the edit-test-analyze loop. At appropriate intervals - after completing an iteration of the loop - developers should integrate their work into the trunk to ensure they receive timely feedback on their in-progress changes.

5.1.2 Step #2: deploy changes

The second step in the outer change-deploy-analyze loop is to deploy the changes. Ideally, a developer should *commit* and *push* their changes at least once per day - even if it's still work in progress. A *git push* triggers a deployment pipeline, which runs the tests and, if they pass, deploys the changes into production. To provide the developer with

prompt feedback, the deployment pipeline's lead time - including the wait time and the processing time - must be minimized. Also, the deployment pipeline should also process each set of changes individually without batching.

As figure [5.5](#) shows, these requirements are captured by two quality attributes. The first is testability, which I described above. Second is deployability, which is a quality attribute that describes how easily, quickly and reliably a change can be deployed into production I describe deployability in more detail in [Section 5.5](#).

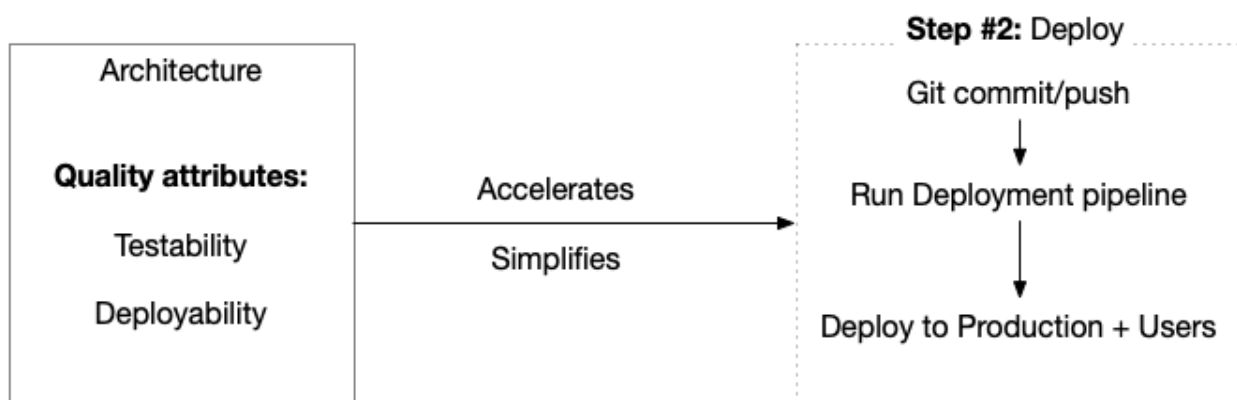


Figure 5.5 A fast deployment pipeline, which provides prompt feedback, is achieved by satisfying the testability and deployability quality attributes

If the deployment pipeline fails, developers need to analyze the results and resolve the problem. And, if it succeeds, developers must analyze the behavior of the application and its users.

5.1.3 Step #3: analyze feedback from production and end-users

Once a change has been deployed, the developers need to understand its behavior in production and its usage by end users. The production environment must, therefore, provide feedback in the form of telemetry - logs, metrics and

distributed traces. It must collect, store and analyze telemetry and make it easily accessible to developers. As figure 5.6 shows, these requirements are captured by the observability quality attribute. Observability is a quality attribute that describes how easy it is to understand the application's runtime behavior. I describe observability in more detail in [Section 5.6](#).

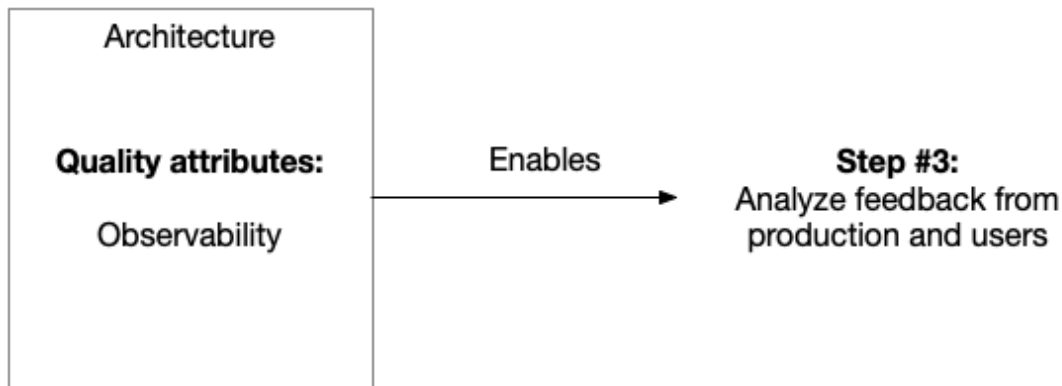


Figure 5.6 Observability is a quality attribute that describes how easy it is to understand both the application's runtime behavior and the behavior of its users.

As you can see, fast flow needs an architecture that satisfies the five quality attributes: modifiability, evolvability, testability, deployability and observability. Let's look at each one of these quality attributes in turn, starting with modifiability.

5.2 Modifiability: ease and speed of change

Since fast flow is about the rapid delivery of changes, an application must be designed to be fast and straightforward to change. It's essential, therefore, that the application's architecture has high modifiability. In this section, I will first describe some modifiability scenarios that capture the requirements for fast flow. After that, I will describe how an architecture can be designed to satisfy those requirements.

5.2.1 Quality attribute scenarios for modifiability

While detailed quality attribute scenarios for modifiability are application-specific, there are some general modifiability requirements for architectures that support fast flow:

1. A team must be able to implement changes without regularly needing to involve other teams in decision-making and implementation.
2. Changes must be localized as possible within a team's code base. In particular, they must be local to a component to avoid the complexity and overhead of coordinating changes across multiple components.
3. Teams must perceive that it's straightforward to change their subdomains.

Let's look at a few quality attribute scenarios that capture these requirements. These scenarios describe an Orders team, which owns the `Order Management` and `Discounts` subdomains. The `Order Management` subdomain is deployed as the `Order Service` and the `Discounts` subdomain is deployed as the `Discount Service`. Here's one quality attribute scenario for modifiability:

The Orders team wants to change a business rule that is implemented by `Order Management` subdomain. This change will be made without coordinating with any other teams and without changing the `Discount Service`. The team will later report via a survey that it was straightforward to make.

While most modifications can be made independently, some changes require collaboration: The following scenario captures this idea:

The Orders team wants to implement new features in the Order Management UI that require changes to the Order Management subdomain and the Order Service's API. In the majority of cases, the API changes are backwards compatible and coordination with other teams is not required. The team will report via surveys that they are able to work autonomously the majority of the time.

Let's now look at how an architecture can be designed to satisfy this modifiability scenario.

5.2.2 Modifiability requires loose design-time coupling and high cohesion

Achieving high modifiability requires an architecture that is designed to loosely design-time coupled and cohesive. As described in chapter 4, loose design-time coupling and high cohesion are key to localizing changes. If an architecture is loosely coupled and cohesive then, as figure [5.7](#) shows, it's more likely that changes affect just part of a subdomain in a single component that's owned by a single team. In comparison, changes in a tightly design-time coupled architecture tend to be widespread, involving multiple components and teams. Such widespread changes will be more time-consuming and more challenging to make compared to small, localized changes in a loosely design-time coupled and cohesive architecture.

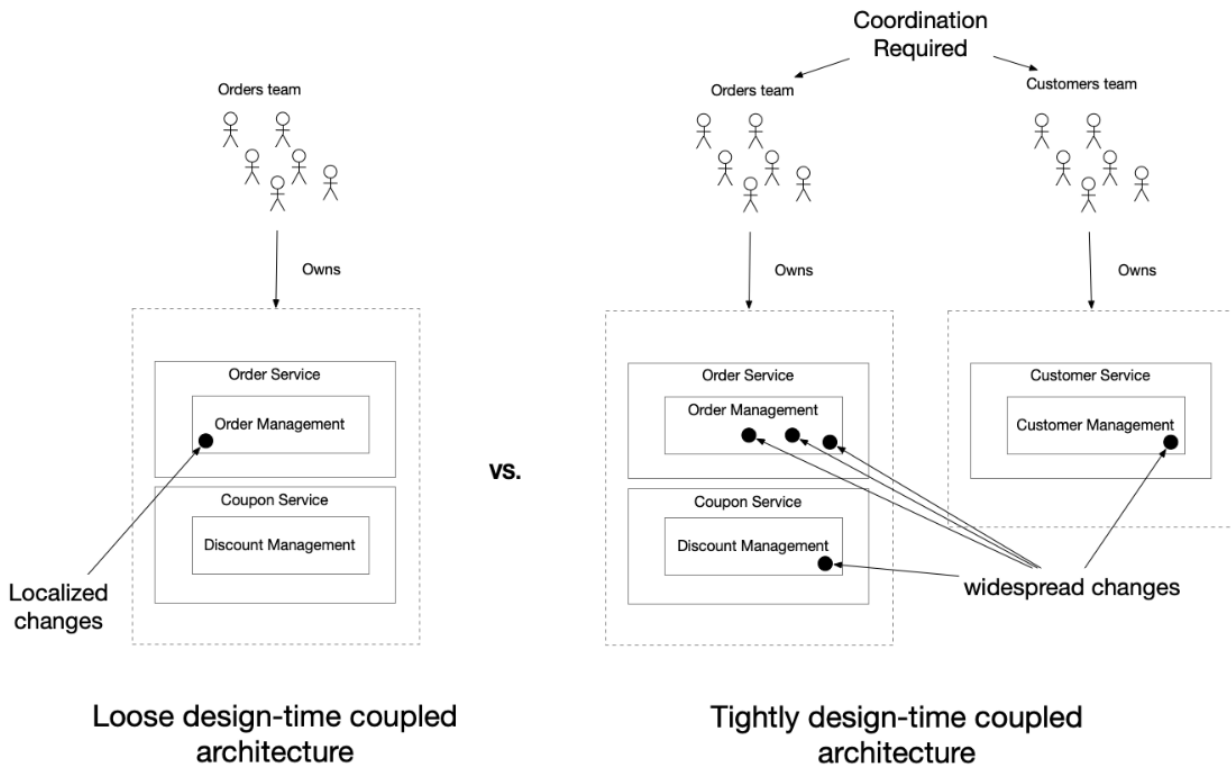


Figure 5.7 High modifiability requires a loosely design-time coupled and cohesive architecture that localizes changes to just part of a subdomain in a single component that's owned by a single team.

Localizing changes by designing for loose design-time coupling and high cohesion improves modifiability in three different ways:

- improves team autonomy
- localizes most changes to a single component
- makes subdomains easier to understand and change

Let's look at each of the ways that modifiability is improved, starting with improved team autonomy.

LOOSE DESIGN-TIME COUPLING IMPROVES TEAM AUTONOMY

First, if subdomains owned by different teams are loosely design-time coupled, then the teams can work mostly

independently. Most design decisions are made within the team. There's no need to regularly seek advice time from other teams. Also, most changes are local to a team's code base. They do not need to wait for other teams to make changes.

LOOSE DESIGN-TIME COUPLING MINIMIZES THE LIKELIHOOD OF CHANGES SPANNING COMPONENTS

The second way that loose design-time coupling improves modifiability is by localizing changes to a single component. When a team owns multiple loosely design-time coupled components, it's likely that a change only affects one of them. The team rarely needs to make complex and time-consuming multi-component changes, which require the careful sequencing and deployment of modifications to components.

LOOSE DESIGN-TIME COUPLING MAKES SUBDOMAINS EASIER TO UNDERSTAND AND CHANGE

The third way that loose design-time coupling improves modifiability is by making subdomains easier to understand and change. If a subdomain is loosely design-time coupled and cohesive, then it's likely that a change will only affect just a small part of the subdomain, perhaps just a few classes. As a result, a developer can quickly and easily determine what needs to be changed and implement those modifications.

Let's now look at the second quality attribute: evolvability, which reflects how easy it is to make changes to an application's technology stack(s).

5.3 Evolvability: ease and speed of changing the technology stack

In June 2023, I was lucky enough to visit the Pantheon in Rome (https://en.wikipedia.org/wiki/Pantheon,_Rome). First built around 126 AD, it's one of the best preserved ancient Roman buildings. It has been in continuous use throughout its almost 1900 year history. Today, despite being constructed with ancient technologies, the Pantheon is a church as well as a major tourist attraction.

In contrast, a software application is not like a physical building. Even in the unlikely event that its requirements never changed, we'd still need to update the application to keep its technology stack current. It's as if yesterday's building materials no longer functioned and needed to be replaced. Consequently, another important software quality attribute is evolvability.

Evolvability is a quality attribute that reflects how easy it is to change an application's technology stack or stacks. In some ways, as I mentioned earlier, evolvability is a special case of modifiability. But what makes it different is that a technology stack upgrade is a cross-cutting change that potentially has a widespread impact. For example, upgrading a library used by a component might impact all the component's subdomains and require the subdomains' teams to collaborate. What's more, there are often unplanned changes to the technology stack, such as when a newly discovered vulnerability in a library requires an immediate upgrade. And more generally, as I describe later in this section, upgrades to the technology stack are driven by factors other than customer requirements.

I begin this section by first exploring the concept of a technology stack. After that, I'll discuss the reasons why you

need to continually upgrade your application's technology stack. Next, I'll describe quality attribute scenarios for evolvability. Finally, I describe how to design an architecture that satisfies those scenarios. Let's first look at the idea of a technology stack.

5.3.1 About technology stacks

An application's technology stack is the set of technologies used to construct and run the application. There are various categories of technologies including:

- Application technologies: libraries and frameworks, and programming language runtimes
- Infrastructure services, such as databases and message brokers
- Development tools and libraries, e.g., testing libraries, such as JUnit, and Gradle and Github Actions workflow
- Deployment environment: cloud infrastructure, e.g., AWS or Azure; and container technology, e.g., Docker, Kubernetes; and Cloud services including infrastructure services and observability tools, such as logging and monitoring

Almost all libraries, frameworks, programming languages, services, and tools are versioned. e.g., using semantic versioning (<https://semver.org/>). An application's technology stack, therefore, consists of a set of technology versions. It might even contain multiple versions of the same technology.

COMPONENTS VS. TECHNOLOGY STACKS

A complex application often consists of multiple technology stacks. As figure [5.8](#) shows, each component can have its

own private technology stack. There are also application-level technology stacks that are shared by multiple components.

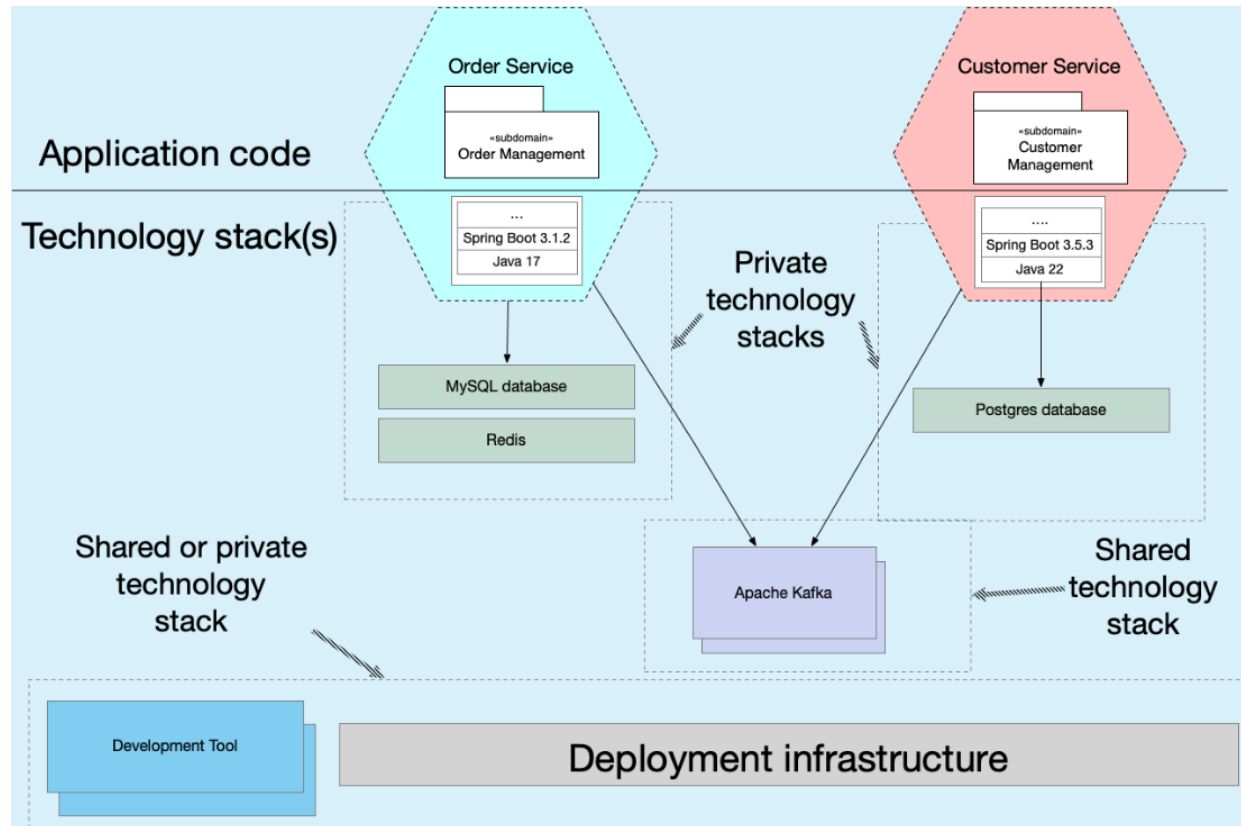


Figure 5.8 An application's technology stack consists of libraries, frameworks, programming languages, infrastructure services, development tools, and deployment environments. Some technologies, such as languages, libraries and some infrastructure services, are private to components. Other technologies might be shared between components, such as message brokers.

A component's private technology stack consists of libraries, frameworks, and a language runtime. It also includes infrastructure services such as databases that are private to that service. The tools and libraries used to develop a component might also be specific to that component. Each of an application's components can use different (versions of) technologies, which is a key benefit of the microservice architecture. In this example, the `Customer Service` uses a new technology stack - Java 22, Spring Boot 3.5.3 and

Postgres, which is the new company standard - while the `Order Service` uses an older technology stack - Java 17, Spring Boot 3.1.2 and MySQL. The `Order Service` also has a specific requirement to use Redis for caching.

In addition to the component-specific parts of the technology stack, some parts of the technology stack are shared by multiple components. The shared parts include infrastructure services that are used by multiple components, such as a message broker. The development tools and deployment infrastructure are usually shared as well. However, different components might use different development tools and/or be deployed to different infrastructure. In this example, both services using Apache Kafka.

A TECHNOLOGY HAS RELATIONSHIPS WITH OTHER TECHNOLOGIES

As figure [5.9](#) shows, a technology (or technology version) can have three types of relationships with other technologies (or technology versions):

- **Requires** - a given technology (version) might require some other technology (version). For example, libraries and frameworks require the runtime of the language that they are written in. Also, many libraries depend on one or more other libraries. Moreover, a technology version might require a version (or range of versions) of some other technology, e.g., Spring Boot 3.x requires Java 17, and version X of library A might depend on version Y of library B.
- **Excludes** - using a technology might prevent the application (or a component) from using other alternative technologies. For example, a component can only have single language runtime (e.g. JVM vs. GoLang vs. ...), or application framework (e.g., Spring Boot vs.

Quarkus vs. ...). Moreover, one technology version might exclude other versions of the same technology. For example, a component's classpath can contain only a single version of a library or framework.

- **Provides** - Sometimes, a technology provides versions for another technology. For example, public cloud providers offer various infrastructure services as a service, but they typically support only specific versions — sometimes not even the latest. Moreover, these providers may eventually drop support for older versions. Consequently, if you rely on such service offerings, the cloud vendor effectively dictates how that portion of your technology stack evolves.

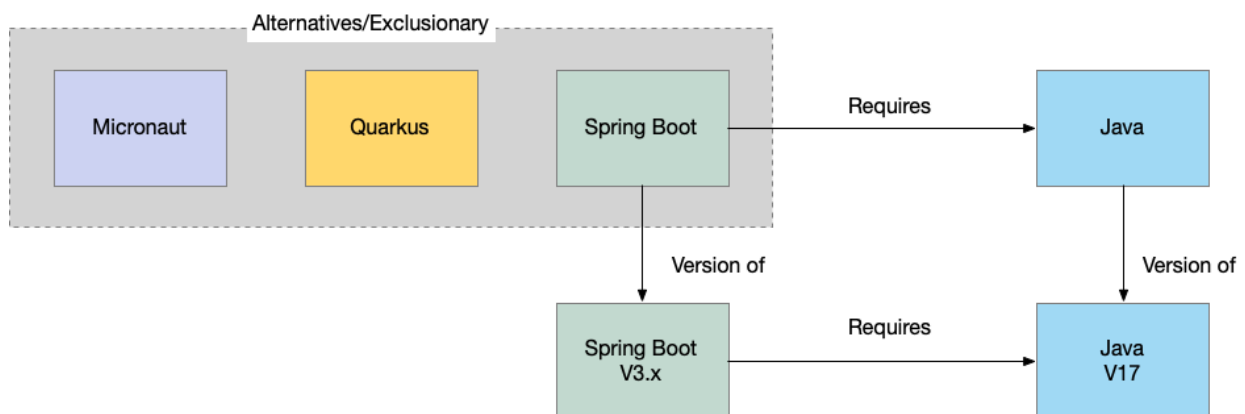


Figure 5.9 There are three types of relationships between technologies: requires, excludes, and provides

ASSEMBLING A TECHNOLOGY STACK IS LIKE SOLVING A JIGSAW PUZZLE

Because of these relationships, assembling a technology stack is often like solving a jigsaw puzzle. It's quite common, for example, for a Java component to have hundreds of transitive dependencies. You need to find the right combination of versions of the various versions that are compatible with each other. It can, for example, be especially tricky to find transitive Maven dependencies that

are compatible. This problem is not, however, limited to Java. For example, I once was unable to upgrade a Kubernetes cluster because a Terraform provider (<https://developer.hashicorp.com/terraform/language/providers>) required an older version.

5.3.2 Technology evolves

The principle challenge with managing an application's technology stack is that technology constantly evolves. Every technology follows a lifecycle, which is shown in figure [5.10](#):

- Introduction - initially just used by early adopters and visionaries.
- Widely used - it becomes used more widely to varying degrees. It might be used by the majority of developers or just a niche.
- Phased out - it dies out completely or, more likely, persists as a "zombie" technology that is used by a small number of customers.

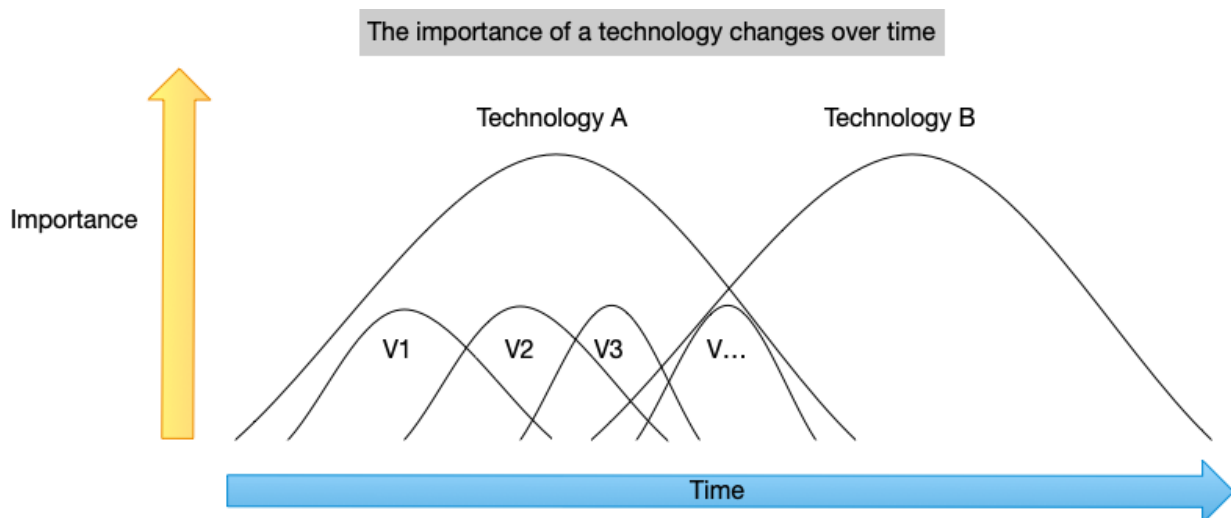


Figure 5.10 Over time, technologies and technology versions come and go

What's more, each version of a technology follows a similar, but shorter, lifecycle. For example, a new version of Java is released every six months. Each release has vendor support for some period of time.

ALIGN THE IMPORTANCE OF A TECHNOLOGY IN YOUR COMPANY TO ITS IMPORTANCE IN THE MARKET

The degree to which a technology (or a technology version) is important in the market determines the availability of support, bug patches, learning resources, and developers who know the technology and want to use it. Therefore, it's essential to align the importance of each technology (or technology version) to your organization and its importance in the marketplace. As figure [5.11](#) shows, a mismatch in its internal importance and market importance can be risky.

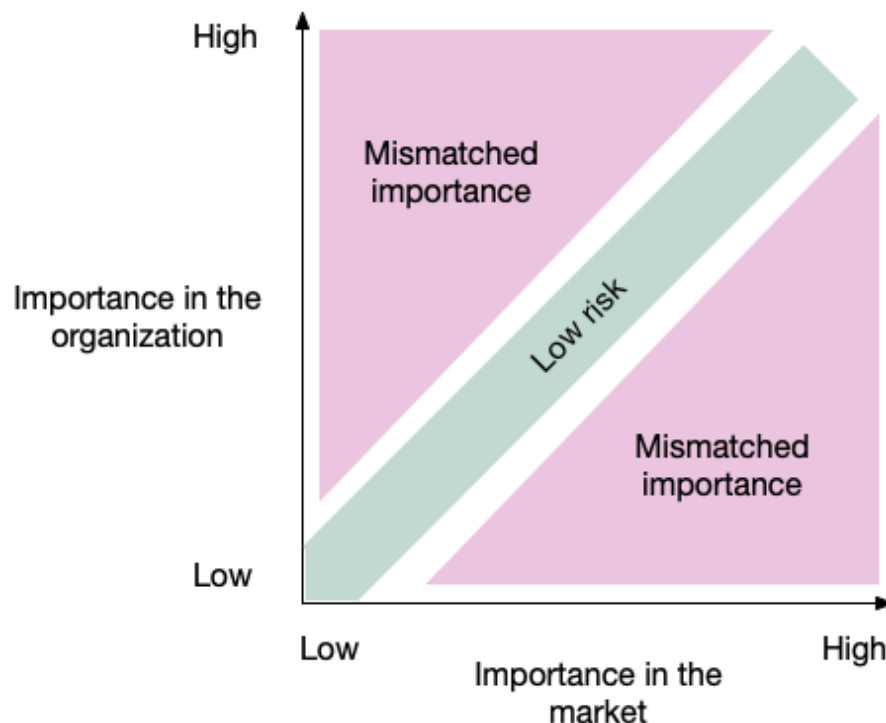


Figure 5.11 The importance of a technology to your organization should be aligned with its importance in the market

For example, you should not use technologies that have become obsolete or are no longer supported by the vendor. Likewise, failing to adopt technologies that have become widely used and well-supported can pose similar risks. Therefore, it's essential that you continually upgrade your application's technology stack to ensure that it remains aligned with the market.

YOU MUST CONTINUALLY UPGRADE YOUR APPLICATION'S TECHNOLOGY STACK

You need to continually upgrade your application's technology stack, especially to fix vulnerabilities. At a minimum, you need to continually upgrade library and framework versions. You also need to upgrade the language runtimes, infrastructure services, and deployment environments. And periodically, you need to consider replacing older technologies with newer ones.

The effort required to implement these upgrades varies considerably. Some upgrades require negligible effort; for example, upgrading to a newer, yet completely backwards compatible version of a library. Other updates require more work. For example, upgrading from Spring Boot 2 to Spring Boot 3 requires upgrading to Java 17 and Jakarta EE 9, which in turn required the tedious replacement of `import javax.` **with** `import jakarta.` in all your source code.

There are also upgrades that require a massive effort, such as switching languages or application frameworks. For example, when Python 2 reached its end-of-life in 2020, organizations were forced to migrate to Python 3, or at least should have. Migration tools handled some aspects of the migration, such as syntax changes - see the Conservative Python Porting Guide (<https://portingguide.readthedocs.io/en/latest/>). But some

organizations discovered that their application used libraries that were not yet ported from Python 2 to Python 3. In these cases, the organization needed to either wait until a Python 3 version was available or change their application to eliminate the dependency on that library.

VULNERABILITIES REQUIRE IMMEDIATE ACTION

The evolution of technologies (and their versions) in the market usually occurs gradually. However, from time to time, critical vulnerabilities are discovered in technologies that require immediate action. In these cases, it is essential to quickly upgrade to a version that includes the necessary fix. For instance, in 2021 a critical vulnerability in the Log4j library (<https://nvd.nist.gov/vuln/detail/cve-2021-44228>) enabled remote code execution, forcing organizations around the world to promptly update their applications to a secure version of Log4j.

ADOPTING NEW TECHNOLOGIES REQUIRES EXPERIMENTATION

Upgrading to a new version of a library or a language runtime is relatively low risk. There might be a few surprises but, in general, the upgrade is straightforward. In contrast, adopting an entirely new technology is a different matter. For example, LinkedIn is awash with posts extolling the virtues of various shiny new technologies. But not only do fans of a given technology overlook its drawbacks, you can never be sure that a technology is a good fit for your organization until you try it. You need to be able to conduct low cost but realistic experiments that determine whether a technology meets the needs of your application.

For example, let's imagine that you are considering adopting Rust. There are three options:

1. Simply write a few toy applications before deciding to use Rust. The problem with this approach is that it's not realistic since you haven't deployed it to production.
2. Spend a year building a complex application that runs in production. While this option is realistic, there's a risk that you will waste six person-years to discover that you prefer a language with a garbage collector.
3. Implement and deploy a single subdomain in Rust. This option is realistic yet relatively low risk since you will only invest a few person-months.

The third option is almost always the best approach.

Let's now look at the evolvability quality attributes that enable developers to easily and quickly evolve an application's technology stack.

5.3.3 Evolvability quality attribute scenarios

There are four main types of technology stack upgrades:

1. Upgrades required by a team that's implementing a feature.
2. Upgrades to keep the technology stack current
3. Emergency upgrades that fix vulnerabilities
4. Low cost yet realistic experiments to evaluate new technologies.

The fourth type of upgrade is essential, but as I explain below, it requires a specific type of architecture. The third type of upgrade is non-negotiable - it simply needs to be done quickly. The first two types of upgrades are also important. Yet there's a risk that an organization will delay making them if they create too much work for too many teams.

To understand the requirements for high evolvability, let's first examine a scenario where evolving an application's technology stack is challenging. Next, I will outline evolvability scenarios that illustrate the requirements for fast flow.

AN EXAMPLE OF WHEN EVOLVABILITY IS DIFFICULT

Let's imagine, for example, that the Orders team, which I introduced in the previous section, decides they need to upgrade the Spring framework in order to implement a new feature. They are one of 10 teams building a monolith. Although the application code changes are confined to the `Order Management` subdomain, the framework upgrade affects all team. As a result, the Orders team must coordinate with the other nine teams and secure their agreement before proceeding.

Each of those other teams needs to

1. Investigate whether any code changes are required
2. Implement any necessary modifications.
3. Verify that their subdomain works with the upgraded library

What's more, the discussion might even involve product management because of the impact on the schedule. The Orders team might not get approval to upgrade the library. They would instead have to find a way to implement the feature using the currently used version.

EXAMPLE EVOLVABILITY SCENARIOS

While detailed quality attribute scenarios for evolvability are application-specific, there are some general evolvability

requirements:

- A team can upgrade a technology stack element used by their subdomain without regularly having to coordinate with other teams
- An organization can quickly and easily upgrade the library used by the application to a supported version.
- An organization can quickly and easily upgrade a technology stack element, such as a library, that contains a vulnerability
- An organization must be able to incrementally upgrade their application's technology stack
- A team must be able to conduct low cost yet realistic experiments to evaluate new technologies

Let's look at a few concrete example evolvability scenarios. The first scenario is for the Orders team:

The Orders team wants to upgrade a library that is used by `Order Management` subdomain. This change will be made without needing to coordinate with any other teams.

The second scenario describes how an organization's security team can quickly and easily fix a vulnerability that doesn't require any code changes:

An organization's security team learns that there's a critical vulnerability in a 3rd party library and need to upgrade the version of the library used by their application. No code changes are required, so it will take less than a day to make and test change and deploy the new version of the application.

What's notable about this scenario is that it requires the architecture to satisfy the testability and deployability quality attributes, which I describe below.

The third and fourth scenarios describe how large upgrade tasks can be broken into a series of small tasks, one per team.

An organization wants to upgrade to a new version of a library that is widely used by their application. Some code changes are required. The upgrade will be done incrementally, one team at a time. Each team changes their subdomain's code, tests the change and deploys it.

An organization wants to migrate from Oracle to Postgres. Each team migrates their subdomain to Postgres, tests the change and deploys it. They only need to coordinate with operations who manage the database infrastructure.

Each team is able to change, test and deploy their code with minimal coordination.

The fifth scenario is when a team wants to evaluate a new technology:

An team wants to evaluate Rust by implement (or reimplementing) a subdomain in Rust. The team implements the subdomain, tests it and deploys it. The team then evaluates the new technology. This experiment should be low cost and only require a few person-months of development effort.

Let's now look at how to design an architecture that can satisfy these evolvability scenarios.

5.3.4 Architecting for high evolvability

In order for a large application developed by many teams to have high evolvability, it must have an architecture that consists of multiple components. As figure [5.12](#) shows, a

multi-component architecture limits the scope of many technology stack changes to a single component. What's more even when a change affects every component, it can be made incrementally, one component at a time. The main drawback of a multi-component architecture is that being able each component is a double edged-sword. Some types of upgrades, such as upgrading a library used by every component, must be done separately to each component.

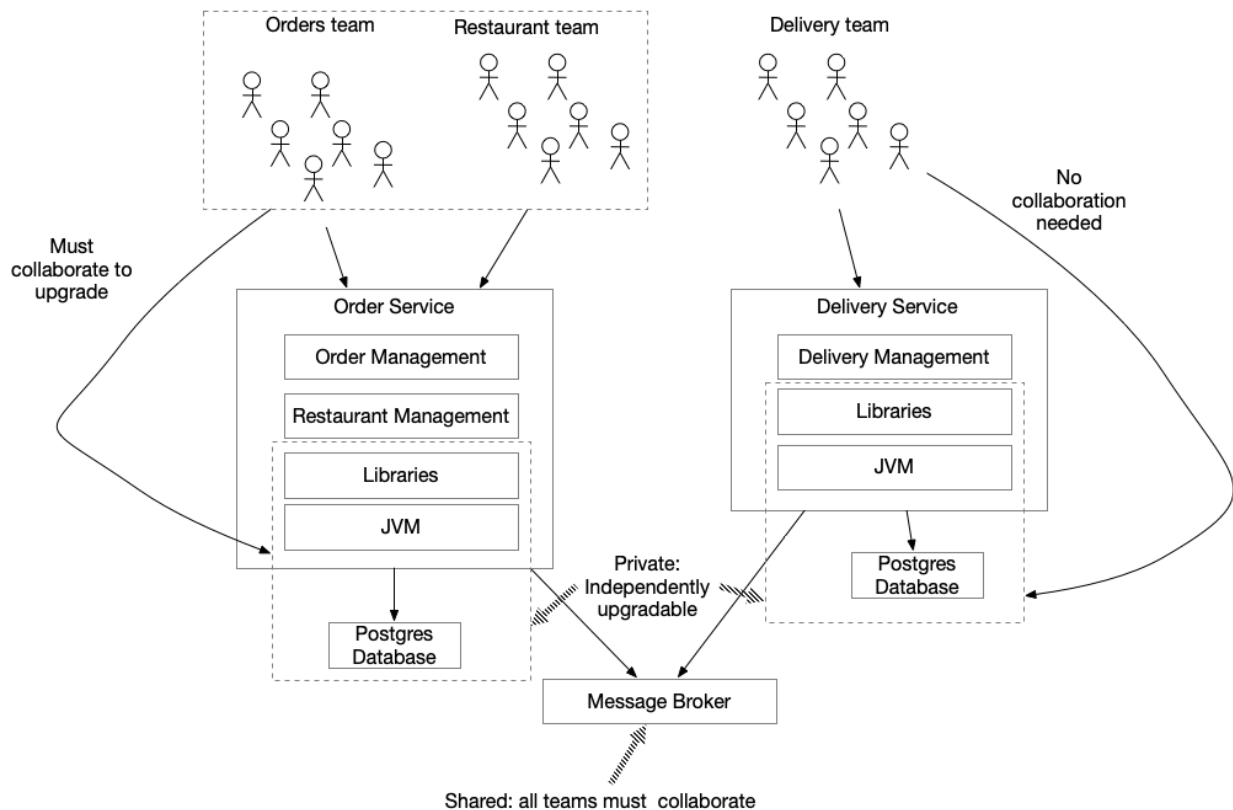


Figure 5.12 A multi-component architecture significantly improves an application's evolvability. Upgrading the private elements of a component's technology stack affects just that component and its team or teams. Typically, only upgrades of shared technology stack elements require many teams to collaborate.

Let's first look at each of these benefits in more detail. After that, I describe the drawback of multi-component architecture and how to mitigate it below.

HOW A MULTI-COMPONENT ARCHITECTURE IMPROVES EVOLVABILITY

A multi-component architecture improves evolvability in the following ways:

- reduces the scope of many technology stack upgrades
- potentially reduces the impact of a vulnerability
- enables upgrades to be broken up into a series of smaller upgrades
- makes it possible to conduct low cost yet realistic experiments that evaluate a new technology

Let's look at each benefit in turn.

The first benefit of a multi-component architecture is that it reduces the scope of many technology stack upgrades. Upgrades of those technology stack elements that are private to a component only affect that component. Only those teams that develop the component need to agree on the upgrade. For example, if a component is developed by a single team, then they can upgrade the component's technology stack elements without needing to coordinate with other teams. Only technology stack elements that are used by multiple components, such as a message broker, require many teams to coordinate.

The second benefit of a multi-component architecture is that it can reduce the impact of a vulnerability. If a library with a vulnerability is only used by a subset of an application's components, then only those components need to be upgraded. This will typically be easier and faster than upgrading the entire application, especially when the technology stack upgrade requires code changes.

The third benefit of a multi-component architecture is that upgrades can be broken up into a series of smaller component-specific upgrade tasks. Let's imagine that you need to upgrade the application to use a newer version of the Spring framework or the JVM. That's potentially a large amount of work. A single component application needs to be upgraded in its entirety in one go, which forces the teams to focus exclusively on that task. In a multi-component architecture, however, each team can upgrade their components independently, which makes it easier to interleave the upgrade tasks with other flow items.

The fourth benefit of a multi-component architecture is that it enables low cost yet realistic experiments for evaluating new technologies. A team can easily implement a subdomain using a candidate technology and deploy it as a separate service. If the experiment succeeds, the team has identified a new technology to use in their application. But if the experiment fails, and they didn't like the new technology, it's not a disaster. In either outcome, the organization is learned a valuable lesson for a relatively low price.

HOW A MULTI-COMPONENT ARCHITECTURE CAN REDUCE EVOLVABILITY

One drawback, however, of a multi-component architecture is that certain upgrades must be performed repeatedly, once for each component. Consider, for example, upgrading to a newer version of the Spring framework. In a single component architecture, this upgrade requires a one-time change to the application's build logic (and perhaps plumbing code). In contrast, a multi-component architecture requires the same change to be made to each component, which can be more time-consuming.

Fortunately, it's straightforward to write scripts that automate changes to Git repositories. Furthermore, in chapter 17, I describe the Microservice chassis pattern, which centralizes build logic, plumbing code and the management of dependency versions, and make some kinds of upgrades easier.

5.4 Testability: ease and speed of verifying releasability

In chapter 1, I explained how a fast deployment pipeline is a key architectural requirement for fast flow. A fast deployment pipeline relies on a combination of two quality attributes: testability and deployability. In this section, I focus on testability, while the next section will cover deployability.

As I mentioned earlier in [Section 5.1](#), testability is a quality attribute that reflects how quickly and easily a change can be verified as releasable. Both the developer and the deployment pipeline must perform this verification. The speed of the edit-test-analyze loop - where developers write and run tests - plays a large role in determining their productivity. Ensuring that tests are both easy to write and fast to execute is therefore essential. Likewise, when the deployment pipeline is triggered to verify and deploy changes, it's important that it provides rapid feedback to developers. Given the importance of easy test authoring and fast execution, testability is critical for achieving fast flow.

5.4.1 Quality attribute scenarios

While detailed quality attribute scenarios for testability are application-specific, there are some general testability requirements:

- When implementing a change, the developer should find it straightforward to write automated tests that verify the change
- The tests run during the edit-test-analyze loop must execute quickly
- The pre-commit tests - those that run before `git push` - must also execute quickly
- The deployment pipeline's lead time, which consists primarily of waiting time and test execution time, must be under X minutes

Let's look at some example quality attribute scenarios for testability requirements. First, here's a scenario that captures the requirement for fast tests that are executed during the edit-test-analyze loop:

A developer on the Orders team changing a business rule that is implemented by `Order Management` subdomain. After making a change, they run the tests in IntelliJ IDEA. The tests complete within 2 seconds. The developers report on surveys that it's generally straightforward to write tests for their subdomain.

Next, here's a quality attribute scenario for testability that describes the requirement for fast and local pre-commit tests:

A developer on the Orders team wants to `git push` the revised business rule. They run the pre-commit tests in IntelliJ IDEA. The tests complete within 1 minute.

Finally, here's a scenario that captures the requirement for a fast deployment pipeline:

A development does a `git push`. The deployment pipeline is triggered and the tests complete within 10 minutes. The developer receives feedback that the change is releasable.

Let's now look at how an architecture can satisfy these testability scenarios.

5.4.2 Architecting for high testability

Achieving high testability requires an architecture that is designed to satisfy the three dimensions of testability shown in figure [5.13](#):

- Automatable - the application must be tested using automated rather than manual tests
- Fast running tests - the tests should execute very quickly to provide fast feedback
- Running tests locally - the tests must run on a developer's machine rather than requiring a separate test environment

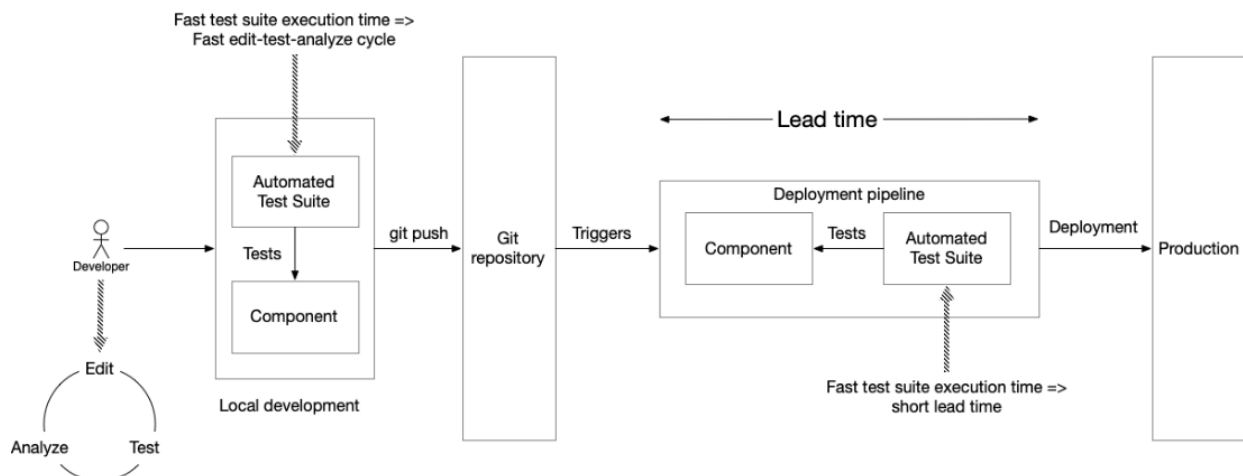


Figure 5.13 Fast flow requires an architecture that is testable: software elements must be tested using fast running automated tests that run locally on a developer's laptop.

Architecture plays an important role in determining how well the application satisfies these dimensions of testability. Let's look at each dimension in turn, starting with the need for automated tests.

5.4.3 Testable using automated tests

Automated testing is one of the main foundations of modern software development. Manual testing is typically too slow and error-prone to support the volume of changes in a fast flow organization. Manual testing, especially when done by a separate QA team, slows down development. Developers need to run automated tests while doing development. Moreover, when a developer pushes a change, they need feedback faster than manual testing can provide. It's essential, therefore, that developers write and run automated tests that are sufficient to determine releasability.

An application's architecture must make it easy for developers to write automated tests. In fact, a useful concept is *Design for testability*: the design of the software should enable easy testing. There are three important design strategies that make testing easier:

- Architect small, loosely design-time coupled elements
- Eliminate or limit non-determinism
- Use patterns such as dependency injection and functional-style programming

Let's look at each one in turn.

SMALL, LOOSELY DESIGN-TIME COUPLED ELEMENTS

The first design strategy is to structure the architecture as a collection of small, loosely design-time coupled elements.

Simple elements are easier to test than complex elements because they require fewer, and simpler tests. Elements with stable APIs are easier to test than those with unstable APIs, since the tests, which are API clients, need to be updated less often.

ELIMINATE OR LIMIT NON-DETERMINISM

The second design strategy is to eliminate, or, at least limit, non-determinism. In order to be verifiable using automated tests, the application should be deterministic. Non-deterministic code can cause tests to be flakey, so should be separated from deterministic code. Ideally, the non-deterministic code should be in its own component with its own, sometimes flakey, deployment pipeline.

DEPENDENCY INJECTION AND FUNCTIONAL-STYLE PROGRAMMING

The third strategy is to use design techniques, such as dependency injection and functional-style programming. Dependency injection simplifies testing by allowing troublesome dependencies to be replaced with a test doubles. For example, if a software element has a dependency on an external service that complicates testing, a test could be simplified by injecting the element with a test double that simulates the service.

Functional-style programming structures the code as side effect free functions that have no dependencies. This approach greatly simplifies testing, as a test simply invokes a function and makes assertions about the return value. Consequently, there is no need to configure and inject test doubles.

5.4.4 Testable using fast running tests

Not only should tests be automated, they must also be fast. When a developer implements a flow item, they change one or more source files in one or more build projects. To validate a change, one option is to execute the test suite for every build project. However, for a large application, this could be too slow or require significant and expensive testing infrastructure.

A better and faster approach would be to execute the test suites of only the build projects that are affected by the change, as shown in figure 5.14. The affected projects include not only the build projects that contain the changed files, but also build projects that depend on them.

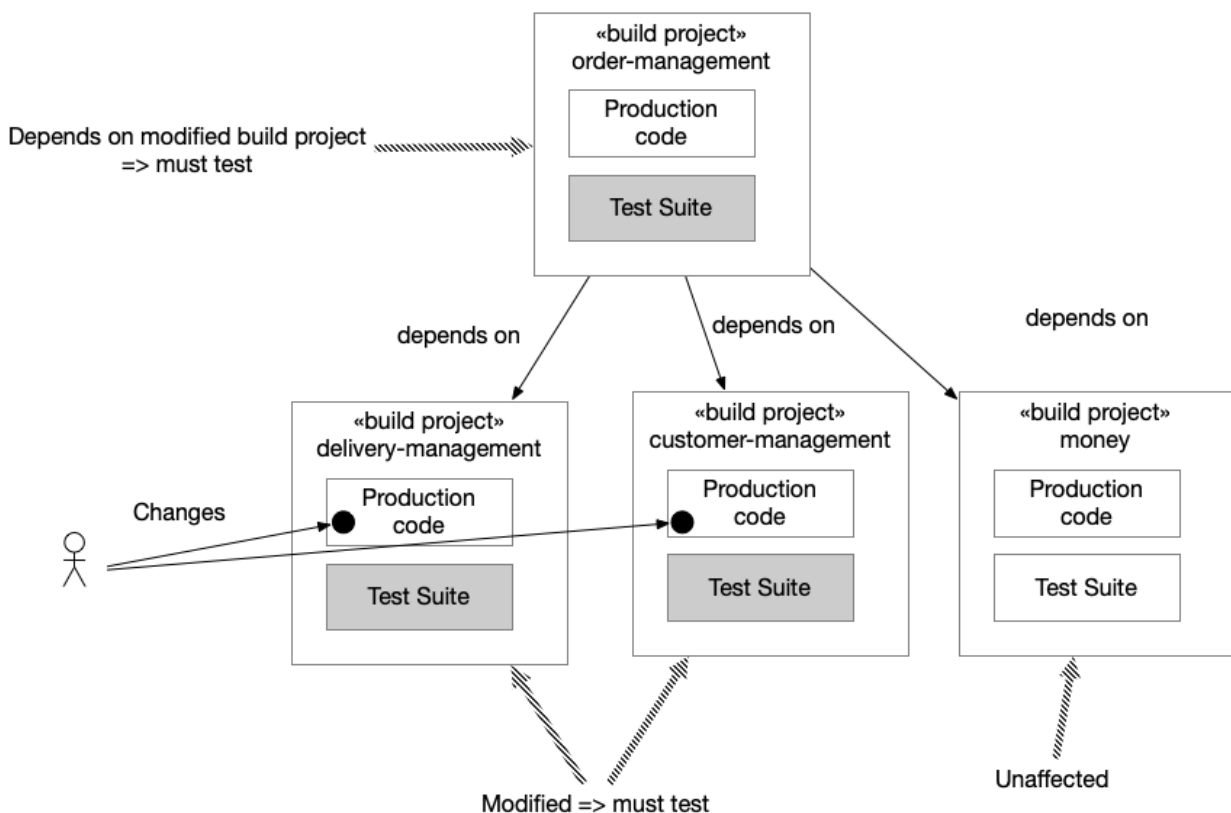


Figure 5.14 The time required to validate a change can be reduced by retesting only the build projects that are affected by the change: build projects containing the modified source files, and those that depend on them.

The test execution time is the sum of each test suite's execution time. As I described in chapter 3, parallelized and clustered builds can sometimes reduce the overall time. However, that can be expensive and complex to implement. Also, local development environments typically have limited resources.

A better approach is to use design techniques that minimize the number of build projects needing retesting after a change while also reducing the execution time of each test suite. Several architectural characteristics help achieve these goals:

- Incremental builds
- Loose build-time coupling
- Loose design-time coupling
- Design for testability
- Multi-component architecture

Let's examine each one in turn, starting with incremental builds.

INCREMENTAL BUILDS

One key requirement is a build tool that supports incremental builds. A build tool should only recompile and retest those build projects that are affected by change rather than wasting time retesting those parts of the application that are unaffected by the change. For example, as I described in chapter 4, Gradle implements incremental task execution. It will only re-execute a test task that previously succeeded if either its test sources or its test classpath have changed. As a result, Gradle only retests the build projects containing the changed files and other projects that have

those projects on their test classpath. This usually takes much less time than retesting everything.

LOOSE BUILD-TIME COUPLING

Incremental task execution can dramatically accelerate the tests. We can further speed up testing by minimizing build-time coupling - a type of coupling, which I described in chapter 4. While a build project that contains changed files must obviously be tested, carefully organizing source files and build projects can reduce build-time coupling, and minimize the number of dependent projects. For example, as shown in figure 5.15, rather than having a subdomain's clients depend on its frequently changing implementation build project, they should instead depend on the more stable API build project. This approach ensures that the client projects don't need to be retested when the implementation changes.

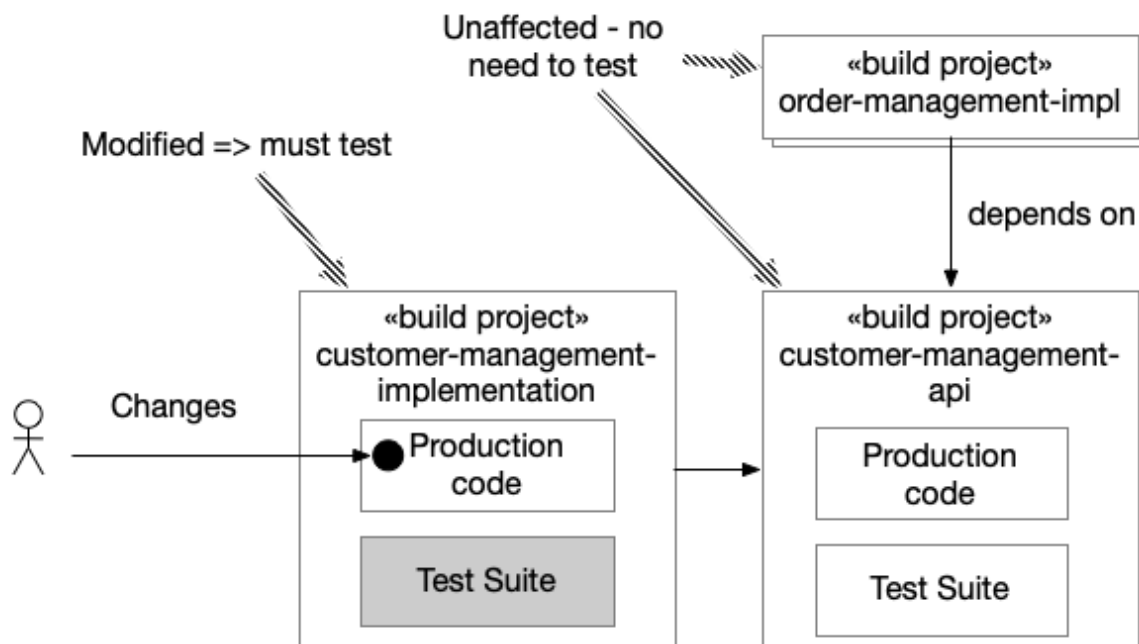


Figure 5.15 A subdomain's clients should depend on the more stable API build project rather than the less stable implementation build project. In this example, the `order-management-impl` build project does not need to be retested when the `customer-management-impl` build project changes.

In general, organizing build projects to minimize dependencies on unstable projects can significantly accelerate the tests.

LOOSE DESIGN-TIME COUPLING

Loose design-time coupling together with high cohesion, also plays a role in speeding up the tests. In an architecture that has these properties, changes tend to be localized to a small number of build projects, perhaps only one. This in turn reduces the number of build projects affected by those changes. As a result, fewer test suites need to be executed to validate the changes.

DESIGN FOR TESTABILITY

Design for testability techniques, such as dependency injection and functional-style programming, also accelerate the tests. As I describe in chapter 15, they allow you to replace slow integration tests, which involve heavyweight infrastructure services, such as databases and message brokers, with faster and simpler, unit tests that run entirely in memory. Dependency injection enables a test to inject a class with a mock DAO that simulates database access. Similarly, complex business logic written in a functional style without dependencies can also be validated with fast running unit tests.

MULTI-COMPONENT ARCHITECTURE

An architecture that structures the application as two or more loosely, design-time coupled components, each with their own deployment pipeline, also speeds up the tests. When a change is local to a component, only that component's tests need to be run, which is much faster than

running all the application's tests. Also, a component's dependencies can be simulated with test doubles, further simplifying and accelerating the tests. Moreover, if each component has its own deployment pipeline, then it's likely that the pipeline's wait time will be low.

5.4.5 Running testing locally

Not only must tests be fast, but a developer should not have to use a separate test environment unless, as I describe below, it's absolutely necessary. A remote testing environment should be avoided because shipping changes to it slows down the developer's edit-test-analyze loop. Local testing also avoids the overhead and complexity of initializing a possibly shared, and hence unstable, remote testing environment to a known state. In fact, the Accelerate authors have identified the ability to test locally as a key capability of high performance development.

The application's component architecture plays a major role in whether local testing is practical. If the application is small, then the developer can run the entire application and its tests on their laptop. But if the application is large and depends on many infrastructure services, a developer's laptop might not have enough resources to run the tests quickly.

There are a couple of solutions. One option is to buy larger laptops for the developers. Another option is for the application to have a multi-component architecture where each component can be developed and tested independently. Let's look at each option in turn.

BUY BIGGER LAPTOPS

One way to enable the local development of a large application is to simply provide developers with more resources. You can, for example, at the time of writing, buy a MacBook with 16 cores and 128G of memory. Alternatively, you could use a Cloud Development Environment, such as GitPod (<https://www.gitpod.io/>), or Github Codespaces (<https://github.com/features/codespaces>). GitPod, for example, integrates IDEs that run on your laptop, such as IntelliJ IDEA, with a development environment running in an AWS EC2 instance. As a result, developers have access to a powerful development environment without needing a powerful laptop.

In both cases, however, you are paying for expensive resources. Your CFO might not be happy. What's more, while a cloud environment might offer massive EC2 instances, your application might not even run well on the largest available laptop. You might, therefore, want to consider an alternative approach: a multi-component architecture.

USE A MULTI-COMPONENT ARCHITECTURE

For many large applications, a better way to enable developers to develop and test locally is to use a multi-component architecture. However, its benefits are realized only when developers can run just the component they are working on. Running multiple components is likely to require more resources than a single component architecture. Instead, the architecture should enable a developer to complete their work by running only a single component locally. Not only does a smaller component require fewer resources, but it might also require a subset of the application's infrastructure services.

For example, as figure [5.16](#) even though the `Order Service` is one of many services, a developer on the Orders team just

needs to run the `Order Service` and its infrastructure services, such as a database server and a message broker. Any dependencies, such as the `Customer Service`, are simulated using test doubles that are various techniques, including consumer-driven contract testing, which I describe in chapter 16.

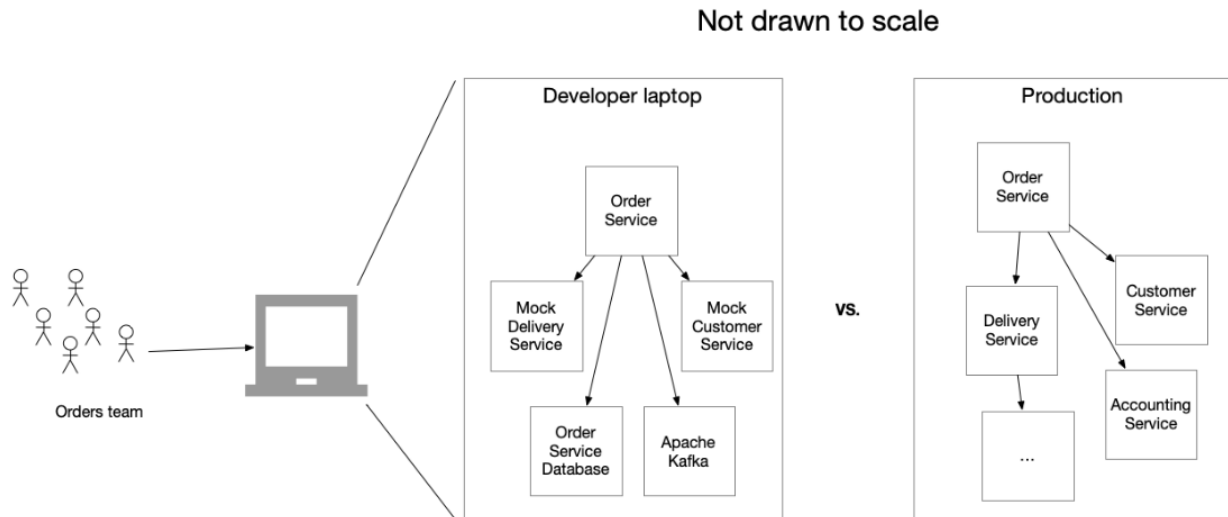


Figure 5.16 A multi-component architecture enables a developer to work efficiently by running only the component that they are working on.

A multi-component architecture enables a developer to work efficiently by running only the component that they are working on.

USE REMOTE TESTING ENVIRONMENTS WHEN YOU MUST

Unfortunately, however, there are scenarios that require a remote testing environment. For example, let's imagine that your application implements webhook endpoints that are invoked by third-party application. While it's possible to simulate the third-party application, you might need to run tests that trigger real webhook requests. Routing such requests to a developer's laptop is typically infeasible and requires a dedicated remote testing environment.

Fortunately, another benefit of a multi-component architecture is that it enables you to isolate those parts of the application that cannot be tested locally in their own components. For instance, if the webhook-related functionality is packaged as a separate component, it can be tested in a special test environment by the component's deployment pipeline, while the other components can be tested locally by the developer.

Testability is a critical quality attribute that underpins both a fast deployment pipeline and rapid local development. Next, we examine deployability — another quality attribute that enables a fast deployment pipeline.

5.5 Deployability: ease, speed and reliability of deployments

Deployability is a quality attribute that along with testability determines the speed of the deployment pipeline. Specifically, it describes how quickly, easily and reliably an updated component that's been tested by the pipeline can be deployed into production. In a fast flow organization, changes are deployed at least once per developer per day, and so it's important for an application to have high deployability.

Deployability defines the requirements for the deployment mechanism that is triggered by the deployment pipeline after it has validated a change. The deployment mechanism is responsible for deploying the updated component into production. The details of the deployment mechanism including whether it is part of the deployment pipeline or separate from it, and how it's triggered depend on the deployment technology used. For example, as I describe in chapter {chapter-kubernetes}, the workflow to deploy a

Spring Boot application into a Kubernetes cluster consists of the following steps:

1. Building a container image containing the application and publishing it to a container registry
2. Updating the production environments Kubernetes YAML to reference the newly published version of container image
3. Triggering a rolling update of the Kubernetes Deployment, which replaces the old service instances with new services instances

In that chapter, I describe a deployment mechanism that implements that workflow. This mechanism is split into two parts: one that runs in the deployment pipeline, and another that runs in the Kubernetes cluster.

Let's first look at quality attribute scenarios for deployability. Afterward, I will describe how an architecture can achieve high deployability.

5.5.1 Deployability quality attribute scenarios

Here are several quality attributes scenarios that capture deployability requirements. The first scenario describes how an updated component is quickly deployed into production without any downtime:

A deployment pipeline finishes testing a component and triggers a deployment. The updated component is deployed into production within 5 minutes. No SLAs are violated.

The second scenario describes when a configuration bug is detected in production and the update is rolled back:

A deployment pipeline finishes testing a component and wants to deploy it. The updated component is deployed into production but fails to start due to a configuration problem. The startup failure is detected, and the deployment is rolled back. No SLAs are violated.

The third scenario describes when a performance problem is detected in production and the update is rolled back:

A deployment pipeline finishes testing a component and wants to deploy it. The updated component is deployed into production, but REST endpoints exhibit unacceptably high latency because of an optimized query. The problem is detected and the deployment is rolled back. No SLAs are violated.

Let's now look at how an architecture can satisfy these deployability scenarios.

5.5.2 Architecting for high deployability

Achieving high deployability requires an architecture that satisfies the dimensions of deployability shown in figure [5.17](#). These characteristics are primarily determined by the application's deployment architecture.

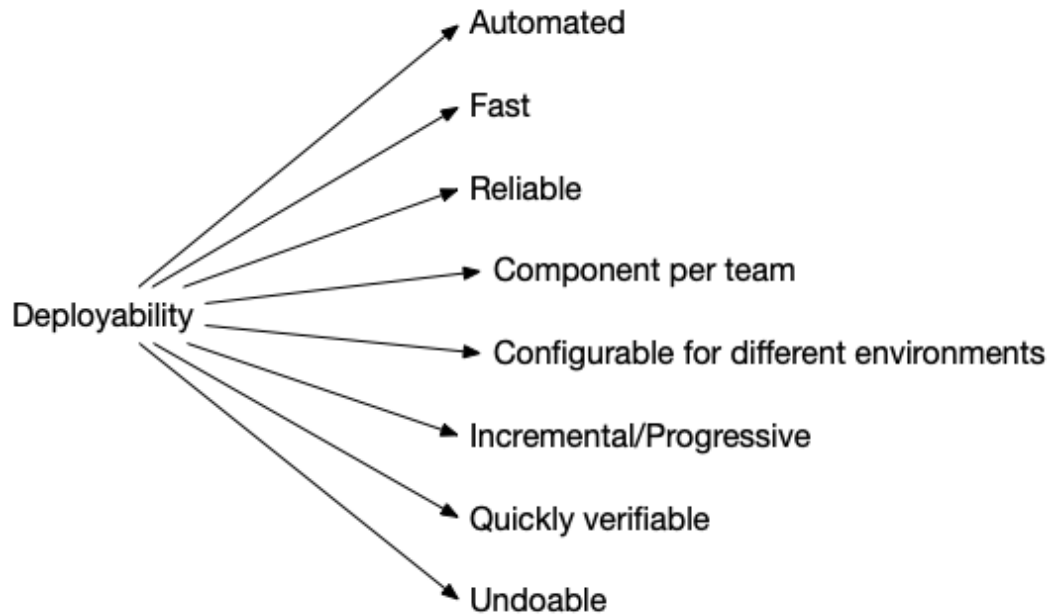


Figure 5.17 The dimensions of deployability that are important for fast flow

Let's look at each dimension in turn.

AUTOMATED DEPLOYMENTS

After a deployment pipeline has tested a component, a human could manually update production using either the command line or a GUI console. However, manual deployments are tedious, slow and error-prone. It's unlikely that a human could keep up with the continuous stream of changes that are typical in a fast flow organization. They would become an obstacle to fast flow. And what's worse, they are likely to make mistakes.

A much better approach is to implement the DevOps technical practice of automated deployments. Specifically, the deployment pipeline should automatically deploy the updated component into production. Automated deployments are faster and more reliable than manual deployments. They also reduce a team's cognitive load since

they don't need to remember the steps required to deploy the component.

FAST DEPLOYMENTS

The deployment time is part of the deployment pipeline's lead time. To minimize lead time, the time interval between the tests finishing and the updated component handling production traffic should be as short as possible. Additionally, a fast deployment mechanism enables the deployment pipeline to handle a continuous stream of changes from many developers without becoming a bottleneck.

As figure [5.18](#) shows, the deployment process consists of three main steps:

1. Package the component in a production-ready format, such as a container image, and publish it to a registry
2. Configure the production environment to use the updated component, triggering the rollout of the new component including the provisioning of resources
3. Waiting until the component instances are ready to handle production traffic

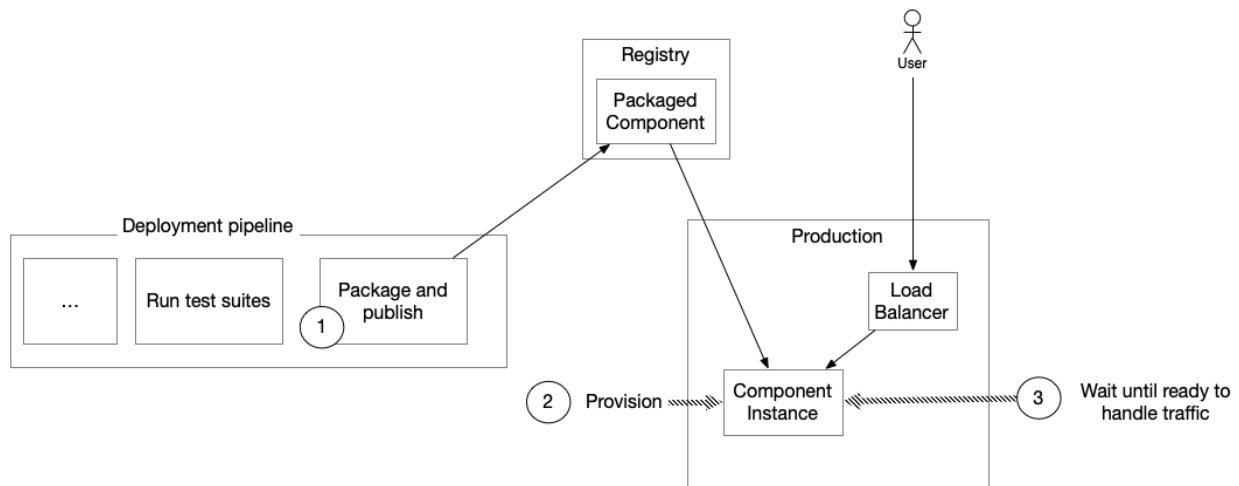


Figure 5.18 The deployment process consists of three steps: (1) the deployment pipeline publishes a component in a production ready package to a registry, (2) production is configured to use the updated component and (3) the component instances are started and become ready to handle production traffic

Ideally, the deployment process should be fast, taking only a few minutes. Most notably, the deployment infrastructure should provision resources rapidly, especially since it's common to use a progressive deployment mechanism (described below), where old and new versions of a component run alongside one another. As described in chapter 18, one of the key benefits of containers is that they enable fast deployments.

Also, the component instances must quickly start up and be ready to handle traffic. It's not uncommon for large, complex Java components to take a few minutes to startup. Slow startup times increase the deployment time, and consequently the lead time. It's important to reduce the startup time. One benefit of a multi-component architecture is that each component is likely to be smaller and simpler than a monolith, which will reduce the startup time.

A multi-component architecture also accelerates deployments by parallelizing them. The deployment mechanism typically needs to finish deploying one change

before it can start deploying the next change. If changes are pushed frequently in a single component architecture, the deployment mechanism will be constantly busy and is likely to increase the wait time for changes. A multi-component architecture avoids this problem because each component has its own deployment pipeline, which runs in parallel with other deployment pipelines. Also, each component's deployment pipeline handles a fraction of the overall changes so it's unlikely to be a bottleneck.

RELIABLE

Just like tests, deployments should be reliable. Deployments should not fail randomly, especially given their high frequency, because unreliable deployments waste valuable time diagnosing and fixing issues. Automation is one way to eliminate human error and significantly improve the reliability of deployments. There's also a design aspect, however.

Releasing a new version of a component typically involved terminating old processes and starting new ones. But it should, however, not cause visible failures. For example, the production environment should implement connection draining to ensure that in-flight (HTTP) requests are completed before the old instances are terminated. Additionally, long-running background tasks, such as batch jobs that process large volumes of data, should be designed to be interruptable and resumable, in order to not delay upgrades or losing any work.

COMPONENT PER TEAM

Yet another important finding by the Accelerate authors is that high performance teams can deploy their changes without coordinating with other teams. One obvious way to

promote team autonomy is through a multi-component architecture, where each team has their own component with its own deployment pipeline. This architecture enables a team to deploy their changes independently, without needing to coordinate with other teams.

CONFIGURABLE FOR DIFFERENT ENVIRONMENTS

A key requirement is that a component can be deployed to multiple environments without needing to be rebuilt each time. For example, the deployment pipeline might deploy the component in a series of environments, such as test and staging environment, before it's finally deployed to production. We should not need to rebuild the component for each environment. Instead, as shown figure [5.19](#), a component should be built once and then deployed to multiple environments.

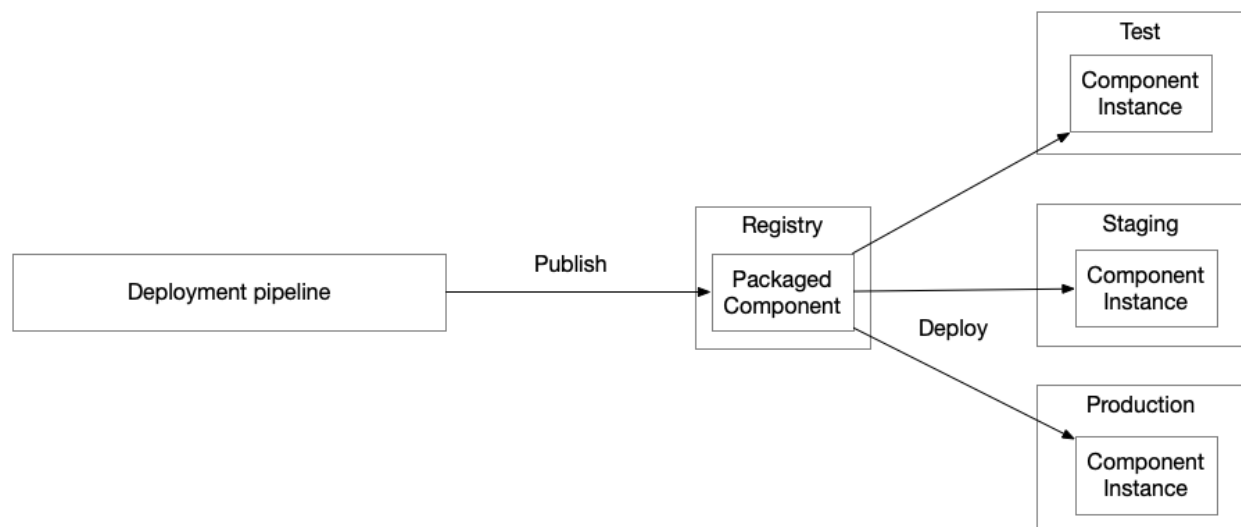


Figure 5.19 The packaged component, which is built once by the deployment pipeline, can be deployed to multiple environments

The solution is to avoid hard-wiring any information about the runtime environment into the component. All configuration data, such as the database hostname and credentials, must be passed into the component at runtime.

A common approach, for example, is for a component to get database credentials from environment variables. In chapter 17 I describe the External Configuration pattern, which solves this problem.

INCREMENTAL/PROGRESSIVE DEPLOYMENTS

No matter how thoroughly we test our software, production bugs are inevitable. Therefore, it's essential to mitigate the impact of a buggy deployment on users. One effective strategy is to decouple deployment—running an updated component in production—from release—routing traffic to the updated component. This approach allows for a progressive rollout, where the updated component is gradually exposed to an increasing percentage of users, thereby reducing potential disruptions.

For example, one common strategy is to do canary releases, where the updated component is deployed alongside the current version. Initially, only a small percentage of the traffic is routed to the updated component, with the share gradually increasing until it handling 100% of the traffic. If at any point, the telemetry (see observability below) from the updated component indicates a problem, the deployment is rolled back. I'll discuss incremental releases in more detail in chapter {chapter-kubernetes}.

QUICKLY VERIFIABLE

Another key requirement is the ability to quickly determine whether a deployment succeeded or failed. One very basic indicator of success is the newly deployed component becoming ready to handle production traffic. However, mere readiness does not guarantee that it actually works. The component might, for example, be misconfigured and respond to every HTTP request with an HTTP 500.

Alternatively, an inefficient query might cause the component to respond slowly.

Observability, which is discussed below, plays a central role in determining whether a deployment was successful. The deployment mechanism should analyze the new component's telemetry - the result of handling traffic - to determine if it's running successfully. If a component is, for example, exhibiting a high error rate or high latency, the deployment mechanism should treat the deployment as having failed.

UNDOABLE

If a deployment fails, the deployment mechanism should be able to quickly roll back the deployment. Ideally, rolling back should be as fast and as simple as routing traffic back to the still running older component instances. Alternatively, the deployment mechanism will need to restart the previous version of the component, which typically takes longer and exacerbates the impact of the failure.

5.6 Observability: ease of understanding an application's runtime behavior

The fifth and final quality attribute that is essential for fast flow is observability. Observability reflects how easy it is to understand the behavior of the application and its users. It's important for the following reasons:

- Feedback from production and users is a key part of fast flow
- Failed deployments must be quickly detected and rolled back

- Production issues must be quickly diagnosed and fixed

In this section, I first discuss quality attribute scenarios for observability. After that, I describe how an architecture can satisfy the observability requirements.

5.6.1 Quality attribute scenarios for observability

Let's look at a couple of quality attribute scenarios that describe observability requirements. The first scenario captures the requirement that the deployment mechanism must be provided with metrics so that it can determine the outcome of a deployment:

During deployment, a component emits metrics, such as error rate and latency, that are collected by the observability infrastructure. These metrics are provided to the deployment mechanism, which uses them to determine whether the deployment was successful or if it should be rolled back.

The second scenario captures the requirement that teams must be provided with telemetry so they can understand the behavior of the application and end users:

The application's components emit telemetry that is collected by the observability infrastructure. This telemetry is provided to developers who use it to understand the behavior of their components and of users.

Let's now look at how these scenarios can be satisfied.

5.6.2 Architecting for observability

There are a couple of strategies for achieving high observability:

- Implement observability patterns
- Use a multi-component architecture

Let's first look at the observability patterns.

IMPLEMENT OBSERVABILITY PATTERNS

A good definition of technical observability is from *What Is Observability? Key Components and Best Practices* (<https://www.honeycomb.io/blog/what-is-observability-key-components-best-practices>)

Observability (sometimes referred to as o11y) is the concept of gaining an understanding into the behavior and performance of applications and systems. Observability starts by collecting system telemetry data, such as logs, metrics, and traces. More important is how that telemetry is analyzed to diagnose issues, understand the interconnectivity of dependencies, and ensure reliability.

This definition focuses on how observability ensures that an application is reliable. What it doesn't mention is that, since many requests are initiated by user actions, telemetry also provides insights into user behavior. The telemetry can reveal, for example, whether a feature that is considered to be valuable is actually being used.

In chapter 17 I describe several observability patterns, including log aggregation, distributed tracing, and application metrics. Each pattern consists of instrumentation code within the component and infrastructure services. The

instrumentation code gathers and emits the telemetry. The infrastructure collects, stores and analyzes the telemetry.

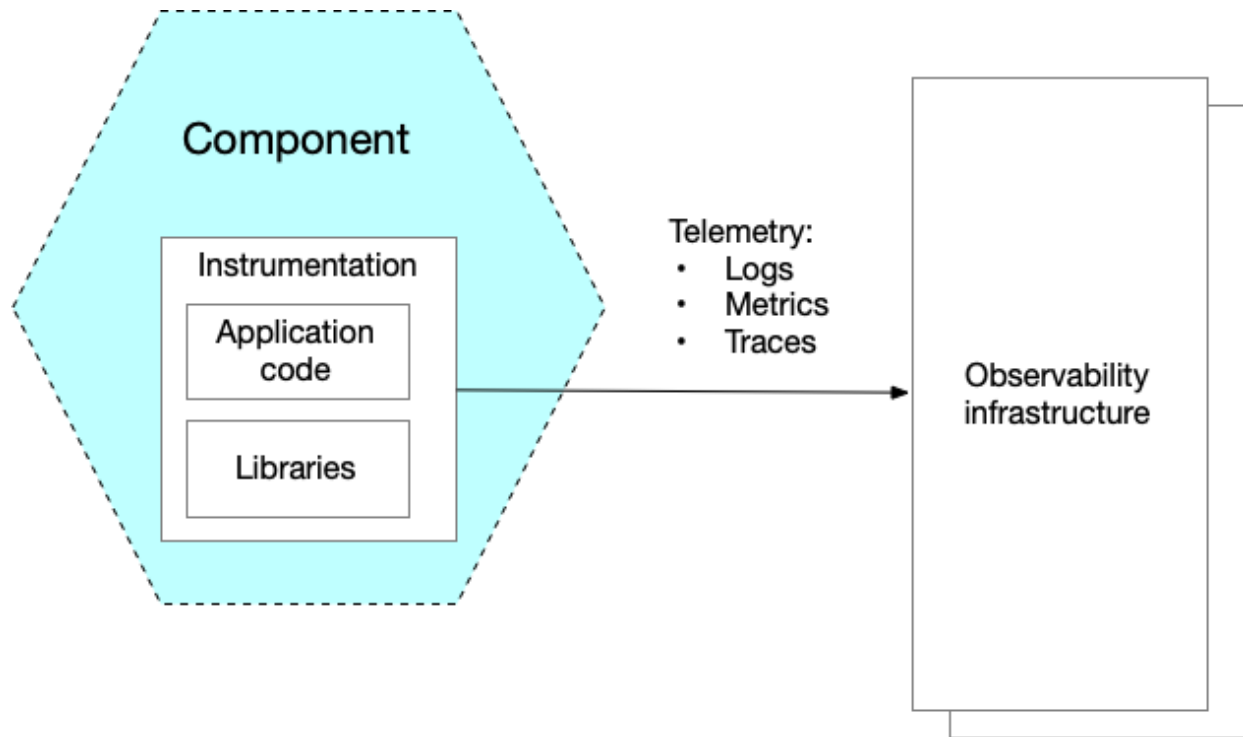


Figure 5.20 Observability requires collaboration between developers and operations. Developers create services that are instrumented to emit telemetry. Operations provide the infrastructure services that collect, store and analyze the telemetry.

As figure [5.20](#) shows, observability is the result of collaboration between developers and operations. Operations are responsible for the infrastructure services, such as log aggregation and monitoring, that collect, store and analyze telemetry. Developers are responsible for the instrumentation code that emits the telemetry. For some types of telemetry, developers simply need to include the appropriate library. But, for others, such as domain-specific metrics or application-level logging, they must write code.

USE A MULTI-COMPONENT ARCHITECTURE

Telemetry is collected for each component instance. Consequently, some metrics—particularly those related to resource utilization—reflect only the aggregate behavior of the entire component. This aggregation can obscure the performance and behavior of individual subdomains when a single component contains multiple subdomains. If necessary, you can improve observability by using a multi-component architecture. It will provide finer-grained telemetry, perhaps at the subdomain level, which can make it easier to understand the application's behavior.

5.7 Summary

- Fast flow requires an architecture that satisfies five quality attributes: modifiability, evolvability, testability, deployability, and observability. The first four attributes - modifiability, evolvability, testability, and deployability - enable developers to rapidly deliver changes to customers. The fifth attribute - observability - is important for two reasons: it ensures the safety of deployments, and, it provides developers with feedback from production and users.
- Modifiability reflects how easily changes can be made to the application. It is achieved through designing an architecture that is loosely design-time coupled and cohesive.
- Evolvability describes how easy it is to upgrade the application's technology stack. For large applications, it is achieved by using a multi-component architecture which allows for most architectural decisions to be made separately for each component and for upgrades to be done incrementally.
- Testability is a quality attribute that reflects how easy and fast it is to verify that a change is releasable. It is

achieved through designing an architecture that is modular, loosely design-time coupled, and deterministic.

- Deployability is a quality attribute that reflects how easy and fast it is to deploy a change into production. It is achieved through designing an architecture that supports deployments that are automated, fast, reliable, can be done without affecting SLAs, and can be automatically rolled back when there's a problem.
- Observability reflects how easy it is to understand the behavior of the application and its users. It is primarily achieved through designing an architecture that implements the observability patterns. A multi-component architecture can be used to improve the observability of large and complex applications.
- An application's component architecture - one component or many - influences how well a large application can satisfy the evolvability, testability, deployability, and observability quality attributes, a key differentiator between microservice and monolithic architectures.

6 The monolithic architecture

This chapter covers

- The benefits and limitations of the monolithic architecture
- How well a monolithic architecture satisfies the quality attributes for fast flow
- Strategies for structuring a monolith that improve team autonomy
- Design techniques for reducing build-time coupling and accelerating the deployment pipeline

In chapter 5, I described the architectural requirements for fast flow in terms of five quality attributes: modifiability, evolvability, testability, deployability and observability. Throughout that chapter, I discussed the role that an application's component architecture - and its build architecture - plays in satisfying these quality attributes. In particular, I showed how a multi-component architecture is often necessary to meet these requirements in large applications developed by large organizations.

However, despite its limitations, a single component architecture, such as the monolithic architecture, is still a good choice for many applications. After all, it avoids much of the complexity inherent in distributed, multi-component architectures, such as the microservice architecture. In this chapter, I describe the monolithic architecture and how well it satisfies the architectural requirements for fast flow. You will also learn how to improve a monolith's ability to support

fast flow by carefully structuring the parts of the architecture that the monolithic architecture style leaves unspecified. In particular, I show how to structure a monolith to improve team autonomy and reduce build-time coupling.

Let's start by defining the monolithic architecture and reviewing its benefits and drawbacks.

6.1 About the monolithic architecture

The monolithic architecture is a simple architectural style that structures an application as a single component. As shown in figure 6.1, the code resides in single code repository and it is built and deployed by a single deployment pipeline. The application often has a single relational database.

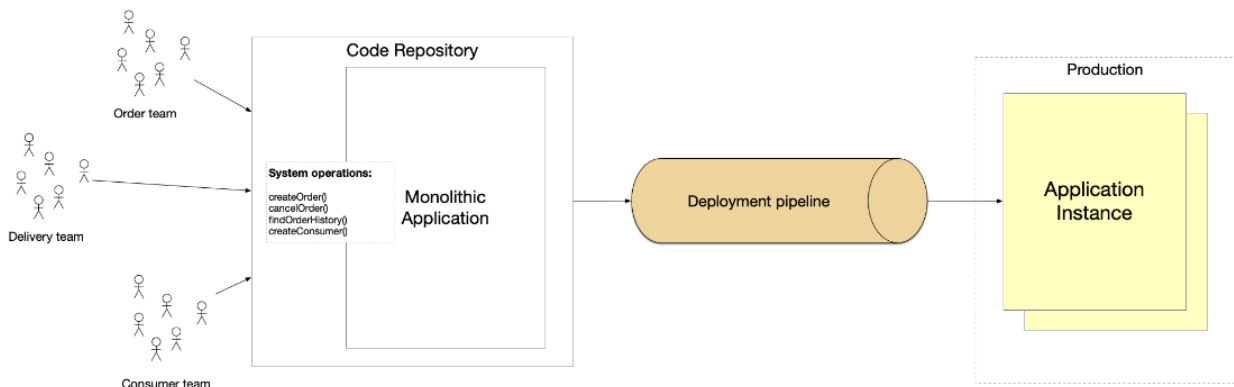


Figure 6.1 The monolithic architecture is an architectural style that structures the application as a single component in a single code repository that's built and deployed by a single deployment pipeline

Because of its simplicity, the monolithic architecture is a suitable architecture for many applications - especially early in its lifetime. And as I describe in sections [Section 6.2](#) and [Section 6.3](#), there are design techniques that can improve a monolith's ability to support fast flow.

In this section, I describe the benefits of the monolithic architecture and its drawbacks - including why its ability to support fast flow declines as the application and its organization grow. Let's start by looking at the monolithic architecture's benefits.

6.1.1 Benefits of the monolithic architecture

The monolithic architecture has several benefits:

- Simplicity of a single code base
- System operations are easier to implement, understand and maintain because they are local to a single component
- Simplified deployment since there's only a single component
- Simplified troubleshooting of runtime issues

Let's look at each one in turn.

SIMPLICITY OF A SINGLE CODE BASE

One important benefit of the monolithic architecture is that developing a monolith, which has a single code base, is generally much simpler than a distributed application. Most notably,

- *Easy to make radical changes to the application* - You can change the code and the database schema, build, and deploy.
- *Simple to develop* - IDEs and other developer tools are focused on building a single application.
- *Compile-time enforcement of APIs* - when using a compiled language, such as Java or Rust, the compiler verifies that interfaces are used correctly instead of

needing to use more complex techniques, such as contract testing, which I discuss in chapters 15 and 16.

SYSTEM OPERATIONS ARE EASIER TO IMPLEMENT, UNDERSTAND, AND MAINTAIN

Another important benefit of a monolithic architecture is that system operations are, by definition, local to a single component. Not only do most operations not incur the overhead of communication over a network, but they can be implemented as a single transaction. As a result, they are typically easier to implement, understand and troubleshoot.

As I describe in chapter 10, such operations are Atomic Consistent Isolated Durable (ACID). Operations have a consistent view - the 'C' in ACID - of the data. The database ensures that concurrently executing operations are isolated from one another - the 'I' in ACID. In contrast, in a microservice architecture, operations that span services are implemented as a series of transactions in each of the services. Consequently, they are not automatically isolated from one another, and can access inconsistent data.

There are, of course, exceptions to the one transaction per system operation rule. Bulk operations, such as batch jobs, are often intentionally split into multiple transactions in order to improve error handling, performance and scalability. Additionally, as I describe below in section [Section 6.2](#), it's sometimes beneficial to split the implementation of an operation into multiple parts, each of which is a separate transaction. For example, the `createOrder()` operation could be split into a synchronous part that creates an `Order` and sends an event to a message broker, and one or more event handlers that complete the creation of the `Order` by, for example, sending a confirmation email.

Most operations can, however, be implemented as single transactions. Moreover, it's important to note that this is a decision that's entirely up to the developer - they are not forced to do this because the operation spans multiple components.

SIMPLIFIED DEPLOYMENT

Another benefit of the monolithic application is that because it consists of a single component, some aspects of deployment are usually straightforward. Unlike with a multi-component architecture, a monolith has a single technology stack. As a result, deployment is relatively simple - usually a set of load balanced component instances that are deployed using one of the technologies described in chapters 18 and {chapter-kubernetes}.

However, even though there's a single component, the monolithic architecture supports more sophisticated deployment architectures that can, for example, allow different subdomains to be scaled separately. For example, let's imagine that the FTGO application's `Restaurant Management` subdomain has a large in-memory database of restaurants in order to support fast queries. Deploying a single cluster of component instances would likely be inefficient, as each instance would require a significant amount of memory—even if most subdomains don't need it. A more efficient approach is a multi-cluster deployment, which is shown in figure [6.2](#), where each cluster handles different requests and runs on appropriately sized hardware.

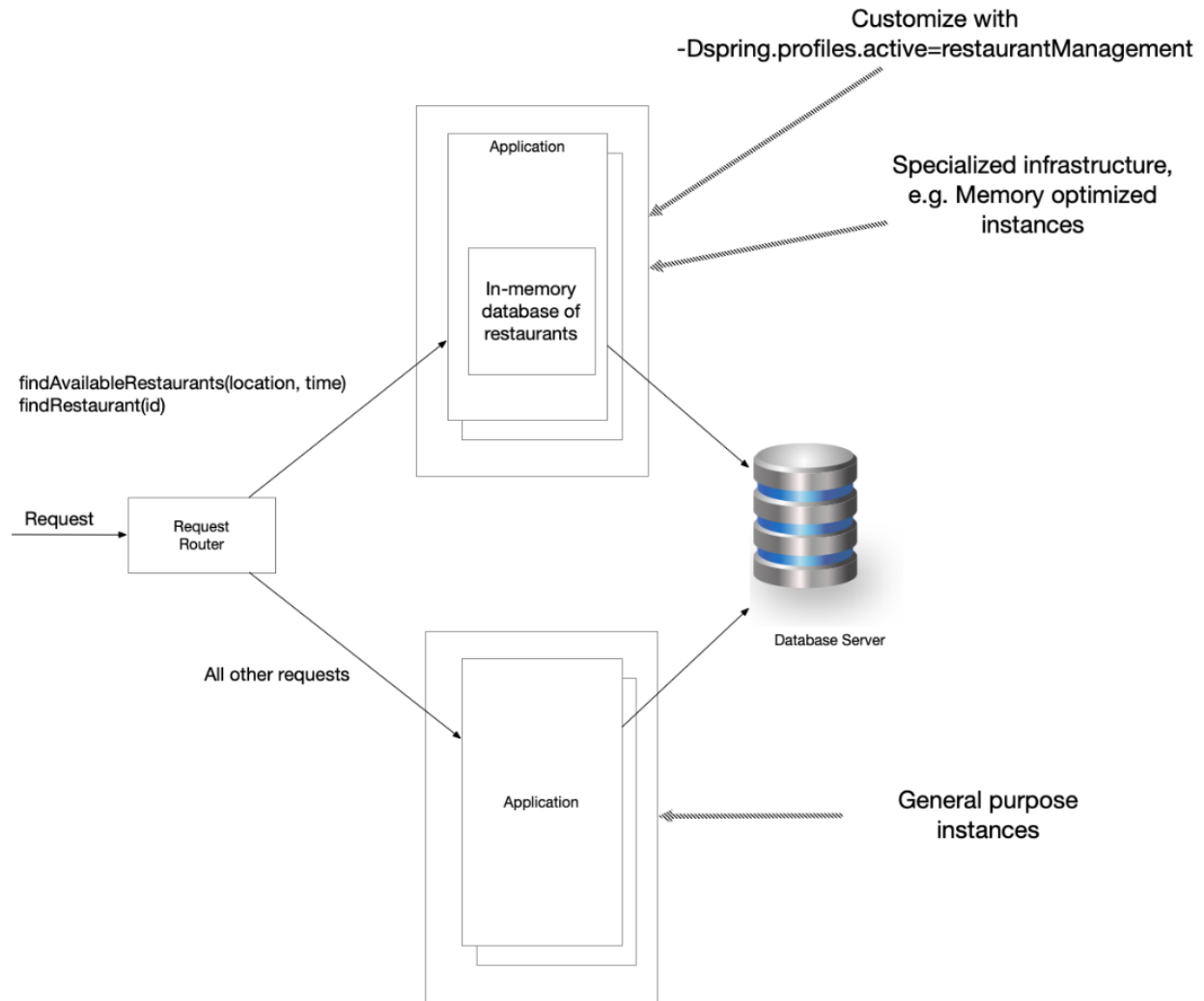


Figure 6.2 A monolithic application can be deployed as multiple clusters that share the same database but are optimized for different workloads. Most of the application runs on a cluster of general purpose instances, while the `Restaurant Management` subdomain, which has a large in-memory database, runs on a cluster of memory-optimized instances.

Both clusters share the same database but are optimized for different workloads. One cluster, which runs on machines with lots of memory, handles queries for the `Restaurant Management` subdomain. The other cluster, which runs on regular machines, handles all other requests. A Spring Boot application could use Spring Profiles (<https://docs.spring.io/spring-boot/reference/features/profiles.html>) to manage the activation of different application functions. The restaurant

database cluster could, for example, use a `restaurantManagement` profile to activate the in-memory database functionality.

SIMPLIFIED TROUBLESHOOTING

Another benefit of the monolithic architecture is simplified troubleshooting. Since operations are typically local to a single component instance, all relevant log entries for a request are in a single log file. As a result, troubleshooting runtime problems is more straightforward than in a distributed architecture where the relevant log entries are often spread across multiple files.

An exception to this occurs when an operation is split across multiple component instances. In such cases, the application must implement some of the observability patterns described in chapter 17, such as log aggregation and distributed tracing, to enable effective troubleshooting.

As you have seen, the monolithic architecture has numerous benefits. The problem is that an application and its organization can outgrow the monolithic architecture. Eventually, the monolithic architecture becomes an obstacle to fast flow. To understand why, let's look at how well a monolith satisfies the five quality attributes for fast flow.

6.1.2 The monolithic architecture and the quality attributes for fast flow

In chapter 5, I described five architectural quality attributes for fast flow: modifiability, evolvability, testability, deployability and observability. A small monolith developed by a small organization can often satisfy these attributes quite well. The challenge, however, is with large monoliths developed by large organizations. In those scenarios, the

monolithic architecture can be an obstacle to fast flow. To understand why, let's examine how well the monolithic architecture supports each of these quality attributes, starting with modifiability.

MODIFIABILITY: REQUIRES DISCIPLINED DESIGN

Modifiability is an essential quality attribute for fast flow. As described in chapter 5, modifiability is achieved by having an architecture whose domain and component dimensions are loosely design-time coupled and cohesive. Fortunately, there's no reason why a monolith's domain view cannot be loosely design-time coupled - it's simply a matter of applying good design principles. Also, since a monolith consists of a single component, there's no possibility of design-time coupling within the component view.

One challenge with ensuring that a monolithic architecture is loosely design-time coupled, however, is maintaining the discipline to continually practice good design. The monolithic architecture is inherently very forgiving. You can, for example, add a new class almost anywhere in the code base and it will work. Schedule pressure can encourage developers to take shortcuts that create tight design-time coupling. Over time, these compromises can accumulate as technical debt, increasing the risk that the application devolves into a big ball of mud.

Later in section [Section 6.2](#), I'll describe how another challenge with the monolithic architecture is that the patterns that we typically use to organize the domain view, such as a layered architecture, don't scale to support the growing complexity. These patterns work well in a small monolith, but as it grows, they fail to provide the loose design-time coupling and cohesion that fast flow requires. In

that section, I describe a more effective alternative: the modular monolith.

EVOLVABILITY: DECLINES AS THE APPLICATION AND ITS ORGANIZATION GROW

In order to support fast flow, a monolith must be highly evolvable. However, as I described in chapter 5, when a large application is developed by a large organization, high evolvability is best achieved through a multi-component architecture. Since a monolithic architecture consists of a single component, its evolvability declines as both the application and the organization grow.

As figure [6.3](#) shows, upgrading the technology stack of a large monolith often requires coordination across all teams, which is time-consuming and slows down development. Widespread changes to a large code base are major undertakings that cannot easily be done incrementally. Even critical upgrades—such as those required to fix security vulnerabilities—can take a long time, increasing operational risk.

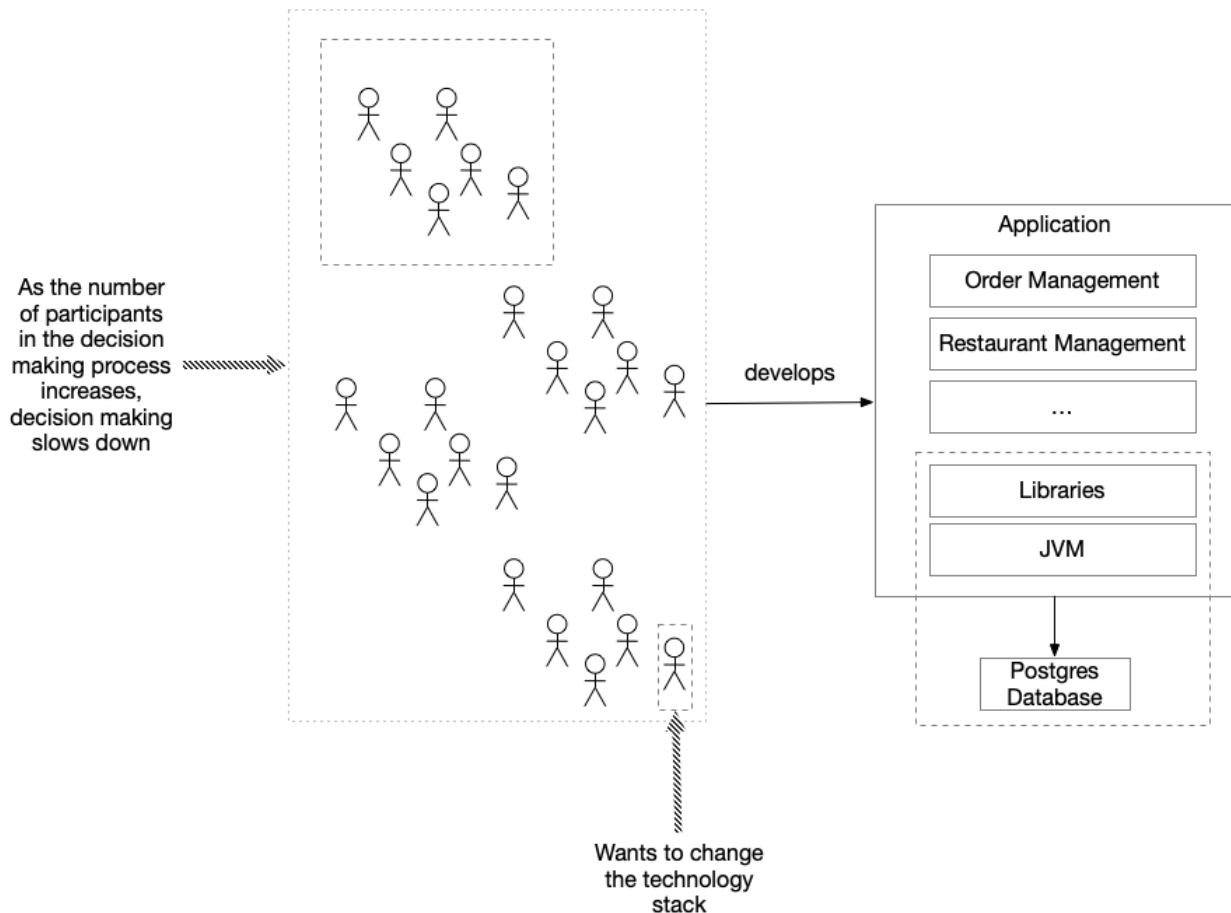


Figure 6.3 If a developer wants to upgrade the technology stack of a monolith, they must coordinate with all teams. If there are many teams, decision making can be extremely time-consuming and slows down development.

Additionally, the monolithic architecture prevents teams from conducting low-cost, low-risk experiments with new technologies, since introducing a new technology typically affects the entire application rather than an isolated part. As a result, a large monolith developed by a large organization cannot easily satisfy the evolvability scenarios described in chapter 5. Over time, there is a significant risk that its technology stack becomes increasingly obsolete.

TESTABILITY: DECLINES AS THE APPLICATION AND ITS ORGANIZATION GROW

Testability is another quality attribute that is essential for fast flow. A small monolith developed by a small team can be highly testable. But, as I described in chapter 5, when a large application is developed by a large organization, high testability is best achieved through a multi-component architecture. Since a monolithic architecture consists of a single, large component, its ability to support the three dimensions of testability - automated testing, fast tests and locally running tests - decline as both the application and the organization grow.

There are three reasons why testability declines. The first reason testability declines is that, as figure [6.4](#) shows, the deployment pipeline can become a bottleneck because testing a monolith can take a long time. Although individual build projects can have fast-running test suites, the total execution time is high. End-to-end test suites, in particular, tend to be slow due to the application's size and long startup time. Designing to minimize build-time coupling can help, as can build acceleration technologies, such as parallelization and clustering. Even so, some changes will still trigger long test runs. To make matters worse, as the organization grows and the volume of changes increases, the deployment pipeline can become a bottleneck, unable to keep up with demand.

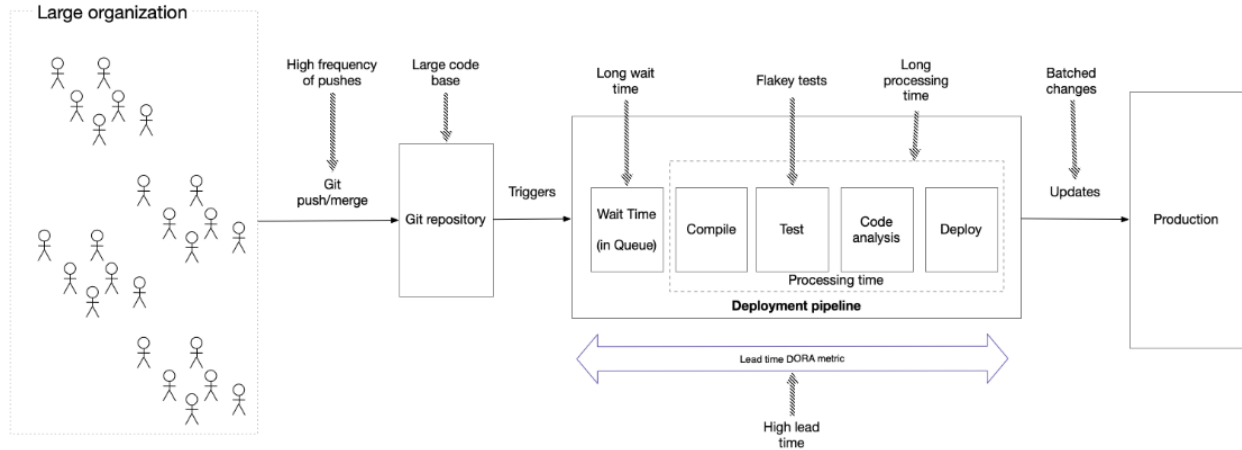


Figure 6.4 Testability declines as the application and its organization grow: the application takes a long time to test, there's a large volume of changes, and the deployment pipeline becomes a bottleneck. Changes are either queued, leading to longer lead times, or batched together, making production issues harder to troubleshoot. Flakey tests force developers to waste time analyzing test failures to determine whether the failure was caused by a real bug or by non-determinism.

The second reason is that the monolith architecture does not allow for non-deterministic modules to be isolated in their own components. As a result, non-deterministic behavior can affect the entire application and lead to intermittent test failures in the deployment pipeline. This flakiness forces developers to waste time analyzing test failures to determine whether the failure was caused by a real bug or by non-determinism.

The third problem is that a large monolith can become impractical for developers to test locally on their laptops. While a deployment pipeline can run on a cluster of machines, a developer's laptop is typically a single machine, with limited resources. Consequently, the tests might run slowly, or they might not run at all. One solution is, as I mentioned in chapter 5, to provide each developer with either a large laptop or a cloud IDE that runs on a large machine. However, this approach introduces significant additional costs.

The declining evolvability and testability of a monolithic architecture present a significant obstacle to achieving fast flow. But unfortunately, they are not the only obstacle. Another obstacle to fast flow when using the monolithic architecture is a decline in deployability.

DEPLOYABILITY: DECLINES AS THE APPLICATION AND ITS ORGANIZATION GROW

Deployability is another quality attribute that's essential for fast flow. Figure [6.5](#) shows how well the monolithic architecture can satisfy the eight dimensions of deployability described in chapter 5. Regardless of size, a monolith can satisfy the following four dimensions of deployability: automated deployments, configurable for different environments, incremental/progressive deployments, and undoable deployments. A small monolith can also satisfy the requirement for a component-per-team provided its developed by a single team. In many cases, it can also satisfy the remaining three dimensions of deployability: fast deployments, reliable deployments, and quickly verifiable deployments. However, like testability and evolvability, a monolith's ability to meet these dimensions declines as the application and the organization grow.

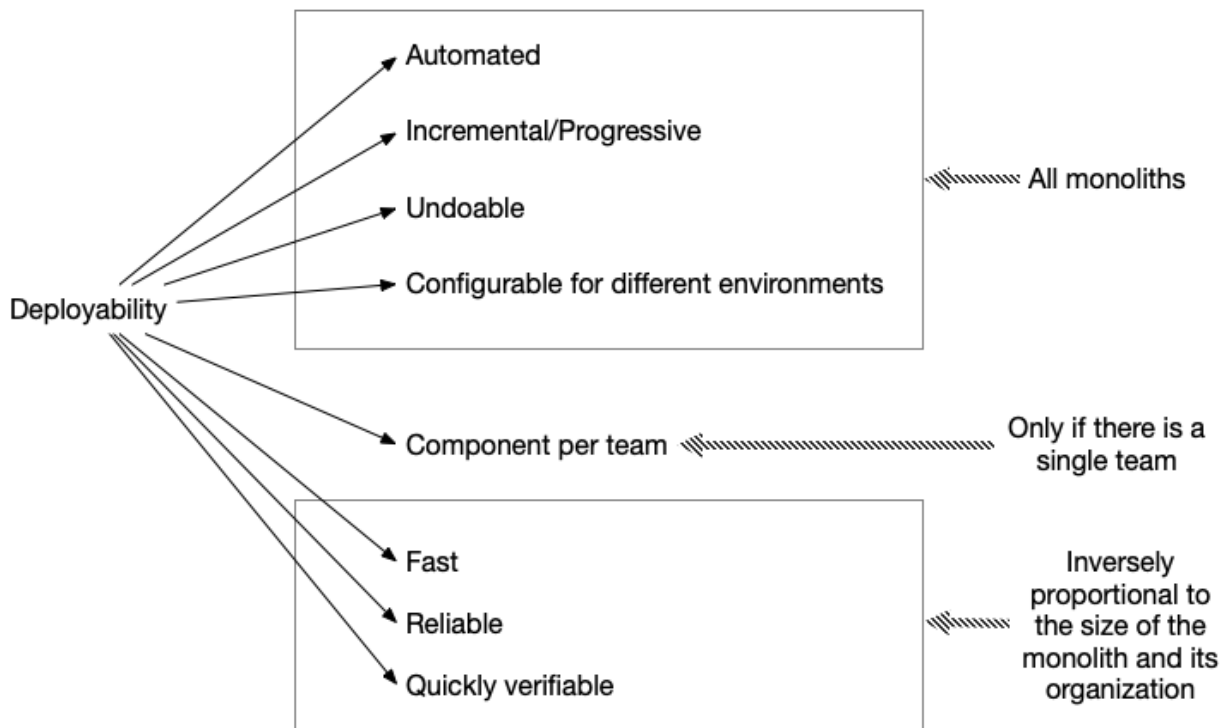


Figure 6.5 A monolith can satisfy four dimensions of deployability, but its ability to satisfy the three other dimensions declines as the application and the organization grows. Also, it can only satisfy the requirement for a component-per-team if there's a single only.

Let's examine in more detail how a monolith struggles to satisfy those three dimensions of deployability as it and its organization grow. The first problem is that the speed of deployments declines. Deployments are often fast for a small monolith developed by a small organization. However, as the monolith and the organization grow, the deployment step of its deployment pipeline increasingly becomes a bottleneck. While build acceleration technologies, such as parallelization and clustering, can accelerate the tests, deployments must be done sequentially. Larger applications also take longer to deploy due to increased startup times. Some canary release strategies even require the application to be started twice—once for the canary and again for the full rollout—further slowing the process.

To make matters worse, slow deployments combined with high test execution time increase the deployment pipeline's

utilization, reducing its ability to handle the high volume of changes produced by a large organization. As a result, the deployment pipeline becomes overloaded: changes are either queued—leading to longer lead times—or batched together, making production issues harder to troubleshoot.

The second problem is that deployment reliability declines as the monolith grows. Because every deployment updates the entire application—even if the change itself is small—the risk of unintended consequences increases significantly. In a large, complex monolith, a seemingly minor change can inadvertently impact unrelated parts of the system due to hidden dependencies, shared configuration, or gaps in test coverage. As a result, the frequency of deployment-related failures tends to rise.

The third problem is that the ability of teams to deploy their changes autonomously declines as the organization grows. Since a monolith consists of a single component, because all teams contribute to the same code base. They cannot deploy their changes independently. One team could, for example, break the build, which prevents other teams from deploying their changes. The problems caused by the lack of independence increase as the organizations grow.

The fourth and final problem is that time to verify a deployment increases as the monolith grows. For example, canary releases observe the behavior of the new version of the application in production before rolling it out to all users. Essentially, the application is tested using production traffic. The challenge with a complex monolith is that since it implements so much functionality, it can take longer to be confident that the new version works. It might even be impractical to test the entire application as part of a canary release. You might, for example, introduce a bug that won't be detected until a batch job runs days later.

OBSERVABILITY: DECLINES AS THE APPLICATION GROWS

Observability is the fifth and final quality attribute essential for achieving fast flow. A monolithic application can be observable by simply implementing the observability patterns described in chapter 17. However, the monolithic architecture has a significant drawback: since the application consists of a single component, telemetry such as resource utilization and performance metrics are aggregated across all subdomains. Consequently, visibility into the behavior of individual subdomains is limited. As the monolith grows larger and more complex, understanding and diagnosing issues within specific subdomains becomes increasingly difficult.

6.1.3 Other drawbacks of monolithic architecture

Declining evolvability, testability, deployability, and observability are not the only drawbacks of the monolithic architecture. Two other significant drawbacks are:

- Inability to support multiple technology stacks
- Inability to segregate subdomains by their characteristics

Let's look at each of these drawbacks in turn.

INABILITY TO SUPPORT MULTIPLE TECHNOLOGY STACKS

Earlier in section [Section 6.1.2](#), I described how the evolvability of a monolithic architecture declines as the application and its organization grow. A related drawback is that since a monolith consists of a single technology stack,

all code must use the same language runtime and supporting libraries. For example, a Java monolith must use JVM-compatible languages. This limitation prevents teams from using the best tool for each subdomain even though other languages or runtimes might offer better performance, a more appropriate abstraction, or access to a specialized ecosystem. But in a monolith, mixing languages or runtimes is not possible.

INABILITY TO SEGREGATE SUBDOMAINS BY THEIR CHARACTERISTICS

Another drawback of the monolithic architecture is its inability to segregate subdomains based on their characteristics. As I describe in chapter 20, subdomains differ in characteristics such as business criticality, and security. In a microservice architecture, it is often advantageous to isolate subdomains with different characteristics into separate services. For example, subdomains that handle sensitive data or implement business-critical functionality can be deployed as separate services that are isolated from the rest of the application. A monolithic architecture, however, lacks this flexibility: all subdomains are packaged and deployed together, without the structural separation that services provide.

6.1.4 The monolithic architecture pattern language

As we've seen, the monolithic architecture can become a significant obstacle to fast flow as both the application and its organization grow. Some limitations — such as the use of a single technology stack — are inherent and unavoidable. However, there are ways to mitigate some of the other drawbacks through careful design.

Figure 6.6 presents a pattern language that enhances a monolith's ability to support fast flow by reducing both design-time and build-time coupling. It defines two key patterns for structuring the application:

- The Technology-Oriented Monolithic Architecture pattern - a common but problematic structure that leads to tight coupling and poor modularity.
- The Modular Monolith pattern - a better alternative that organizes the application around domain concepts and improves modifiability, testability, and deployability.

In addition to these structural patterns, the pattern language includes several supporting patterns that reduce build-time coupling by organizing the monolith's build projects.

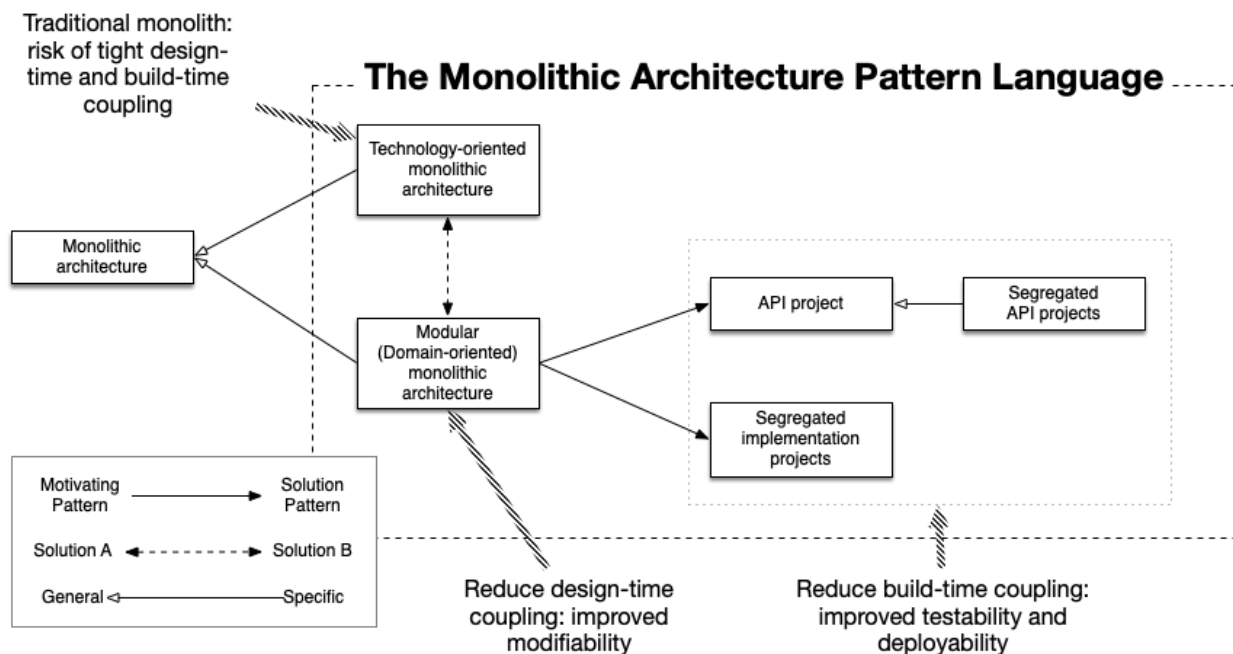


Figure 6.6 The monolithic architecture pattern language: patterns that improve a monolith's ability to support fast flow by reducing its design-time and build-time coupling

Let's begin by examining the Technology-Oriented Monolithic Architecture pattern and its drawbacks. Then we'll explore how the Modular Monolith pattern addresses those issues

and enables a more maintainable, scalable application structure.

6.2 Modularizing the monolith

Despite having the reputation of being difficult to maintain, a monolithic architecture can, as I described earlier in [Section 6.1](#), be designed to be loosely design-time coupled and cohesive. One obstacle, however, to achieving this is the way that the domain dimension of a monolith is traditionally structured: a technology-oriented architectural style. This style often leads to a domain view that is tightly design-time coupled and lacks cohesion. Worse still, a technically oriented architecture often increases build-time coupling, which further reduces testability and deployability. Let's first take a closer look at what is meant by a technology-oriented architecture style and its drawbacks. After that, I'll introduce the modular monolith architectural style and explain why it's better.

6.2.1 A technology-oriented architectural style for the domain architecture

The Monolithic architectural style is only a partial definition of the application's architecture. It organizes the application's component view as a single component and constrains the build view to consist of a single deployment pipeline. This architectural style does not specify any constraints for the application's domain view or the other aspects of the build view. As a result, developers use a variety of complementary architectural styles to structure a monolith's domain architecture.

Common architectural styles for structuring the domain architecture include the layered architecture and the hexagonal architecture, which I described in chapter 3. A

shared trait of these styles is that they are primarily technology-oriented. Their top-level architectural elements reflect technical concerns—such as controllers, services, entities and repositories—rather than modeling concepts from the business domain. That’s even true with the hexagonal architecture, which emphasizes domain logic but still organizes the architecture around technical roles. To understand the challenges posed by a technology-oriented domain architecture, let’s take a closer look at a layered monolith.

THE TRADITIONAL LAYERED MONOLITH

Let’s look at how to structure a monolithic version of the FTGO application using the layered architecture style. The FTGO application consists of various teams and subdomains, including:

- `consumers` - manages consumers
- `orders` - manages orders created by consumers
- `notifications` - delivers notifications (e.g. email, SMS, etc) to consumers

As figure [6.7](#) shows, the layered architecture style organizes the elements of these subdomains into three layers:

- `web` - the user interface and/or API logic
- `domain` - the business logic
- `persistence` - the database access logic or, more generally, infrastructure logic

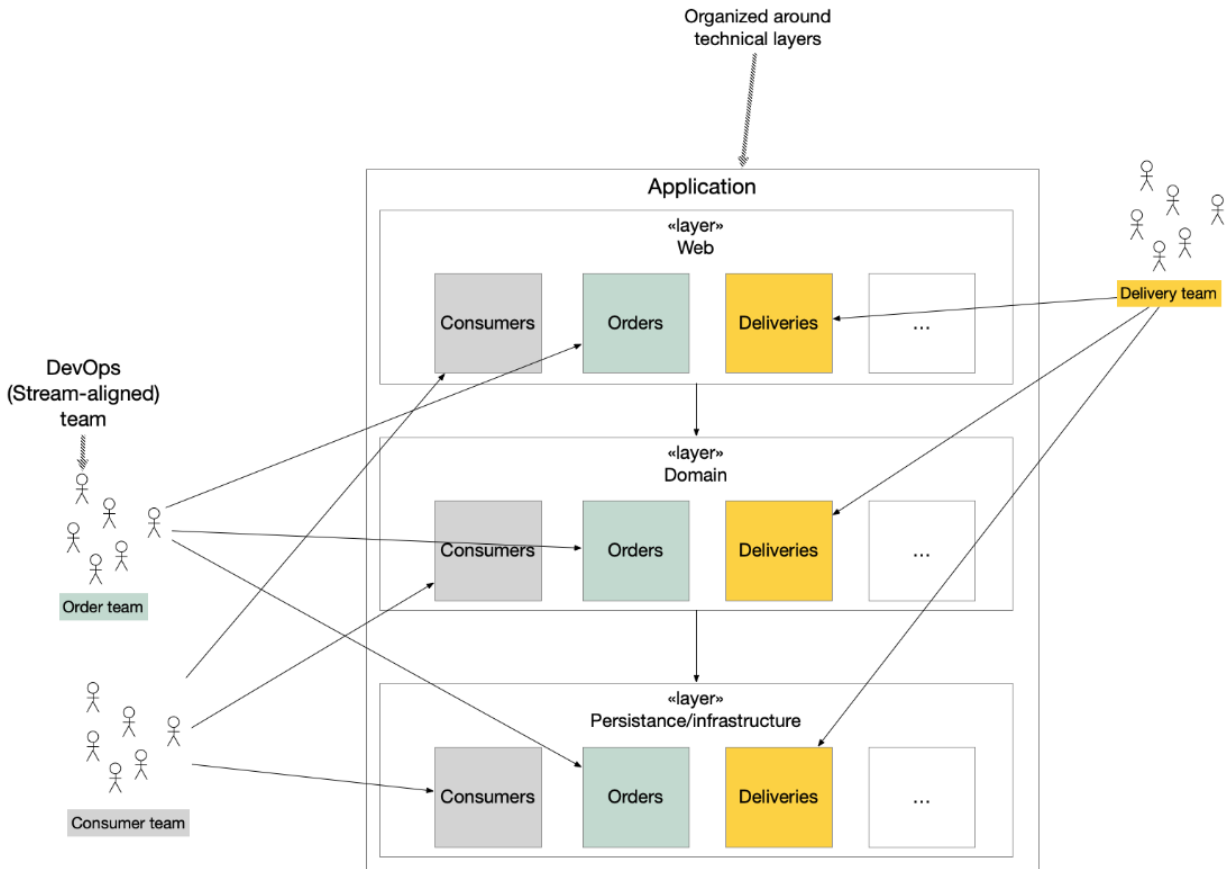


Figure 6.7 In a technology-oriented layered architecture, the code for each subdomain is scattered across the technical layers. As a result, there's a mismatch between the application architecture and the subdomains and the teams that develop them.

As you can see, the code for each subdomain is scattered across the technical layers. For example, the persistence layer would contain classes such as `ConsumerRepository`, and `OrderRepository`. Similarly, the domain layer would contain classes such as `Consumer`, and `Order`. Finally, the web layer would contain classes such as `ConsumerController`, and `OrderController`.

DEVELOPING A MONOLITH

Over the years, I've started the development of many monolithic applications by doing the equivalent of:

```
mkdir application  
cd application  
mkdir web domain persistence
```

After creating those directories, I then configured the Maven or Gradle project with sub-projects for each of those three directories.

This structure works well for small applications, but as the number of developers grows, it becomes increasingly problematic. There are four main problems:

- Layers don't have well-defined responsibilities
- Tight design-time coupling
- Layered architecture doesn't reflect the team structure
- Excessive build-time coupling

Let's look at each of these problems in turn.

LAYERS DON'T HAVE WELL-DEFINED RESPONSIBILITIES

The first problem is that the domain architecture consists of three layers regardless of its size. The responsibilities of each layer just keep growing. The business logic layer implements the business logic for the entire application.

TIGHT DESIGN-TIME COUPLING REDUCES MODIFIABILITY

Another problem with a layered architecture is tight design-time coupling, which reduces modifiability. Subdomains, such as `Consumer Management`, are spread across all three layers, making it more likely that changes will ripple through all the layers. For example, adding a new attribute to the `Consumer` entity requires changing HTTP request handlers, the business logic and the persistence logic. The non-local nature of changes makes the application harder to change and slows down development.

THE LAYERED ARCHITECTURE DOESN'T REFLECT THE TEAM STRUCTURE

Another problem with a layered architecture is that it doesn't mirror the structure of the application's development organization. For example, as I mentioned earlier, the FTGO organization has teams organized by subdomain - such as `consumers`, `orders` and `deliveries` - not technical layers, such as `web`, `domain` and `persistence`. As a result, teams are forced to work across all the layers, which can be a source of friction that reduces productivity.

EXCESSIVE BUILD-TIME COUPLING SLOWS DOWN TESTING

The third major problem with the layered architecture is that its excessive build-time coupling will slow down the edit-test-analyze loop and the deployment pipeline. If each layer is a Gradle build project, then changing a class in that layer will cause both the layer and those layers above it to be rebuilt and retested. Since the layers are large, small changes can often result in large builds and long test times. For instance, a minor change to a domain logic class, such as `Order` results in the retesting of the entire `domain` layer and the `web` layer.

6.2.2 Introduction to the modular monolith

An alternative - and often better - way to organize a monolith is to structure it around the subdomains rather than the technical concepts. This approach is known as the modular monolith architectural style.

For example, the development of the application described earlier would start by doing the equivalent of:

```
mkdir application
cd application
mkdir main consumers orders deliveries main
cd consumers
mkdir ...
...
```

The application's top-level directories - except for `main` - are named after subdomains rather than technical concepts. These directories represent what I call **domain modules**, a concept that I'll explain in more detail later. The `main` module is the application's entry point and depends on the domain modules. In a Spring Boot application, for example, the `main` module contains the main class that starts the application.

For example, the modular monolith version of the FTGO application, which is shown in figure [6.8](#), consists of numerous domain modules including `consumers`, `orders` and `deliveries`.

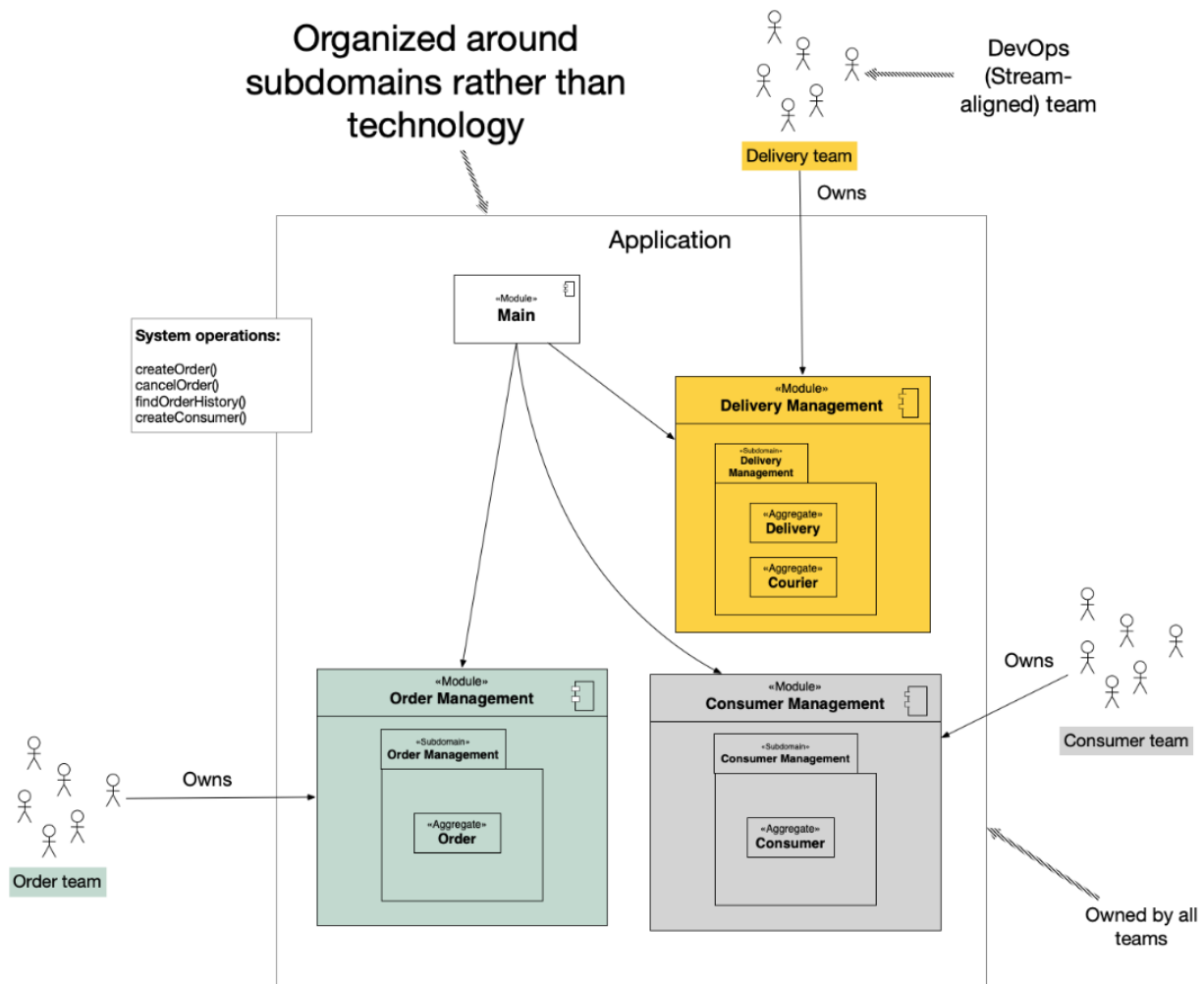


Figure 6.8 A modular monolith consists of domain modules corresponding to the application's subdomains. Consequently, it's aligned to the organization's structure.

Each domain module contains the `web`, `domain`, and `persistence` logic for the subdomain. The `consumers` domain module would, for example, contain packages, such as `com.ftgo.application.consumers.web`, and `com.ftgo.application.consumers.domain`.

DOMAIN-ORIENTED INSTEAD OF MODULAR ARCHITECTURE?

The name 'modular monolith' implies that the alternative approach somehow lacks structure. That, however, is not the case. The layered architecture certainly has structure: technology-oriented layers. Perhaps, a better term for the modular monolith is 'domain-oriented architecture'.

Interestingly, the Hexagonal architecture, which I introduced in chapter 3, is often described as domain-centric. From one perspective, it is. The domain logic is at the center of the architecture. Yet at the same time, it's organized around technical concepts, such as adapters, ports, and the domain - not concepts from the domain itself.

A key benefit of this architectural style is that the domain architecture mirrors the organization's structure. Each team has clear ownership of the code base corresponding to their subdomains. This alignment enhances autonomy and accountability. However, this is just one of the benefits of the modular monolith architectural style. Let's now take a closer look at this architectural style, beginning with the design of domain modules.

6.2.3 Domain module design

The building block of the modular monolith architectural style is the **domain module**. A domain module corresponds to a subdomain and is a vertical slice through the architecture. It includes business logic along with supporting web/api and infrastructure logic. Each domain module is owned by a single team, ensuring clear responsibility and alignment with organizational boundaries.

You must make several key decisions when designing a domain module:

1. Define the domain module's internal domain architecture
2. Design the API it exposes to other domains
3. Decide how to organize the domain module's database schema
4. Structure the domain module's classes and packages into build projects

I will cover build project organization in section [Section 6.3](#). Let's look at the other three design decisions in turn, starting with the domain module's domain architecture.

THE DOMAIN MODULE'S DOMAIN ARCHITECTURE

Since a domain module is a vertical slice through the architecture, it makes sense to organize its packages into a hexagonal architecture. For example, figure [6.9](#) shows structure of the `Consumer Domain Module`. This domain module consists of the following elements:

- Consumer REST API - an inbound adapter that implements a REST API
- Domain logic - the subdomain's business logic
- Consumer DAO - an outbound adapter that accesses the database

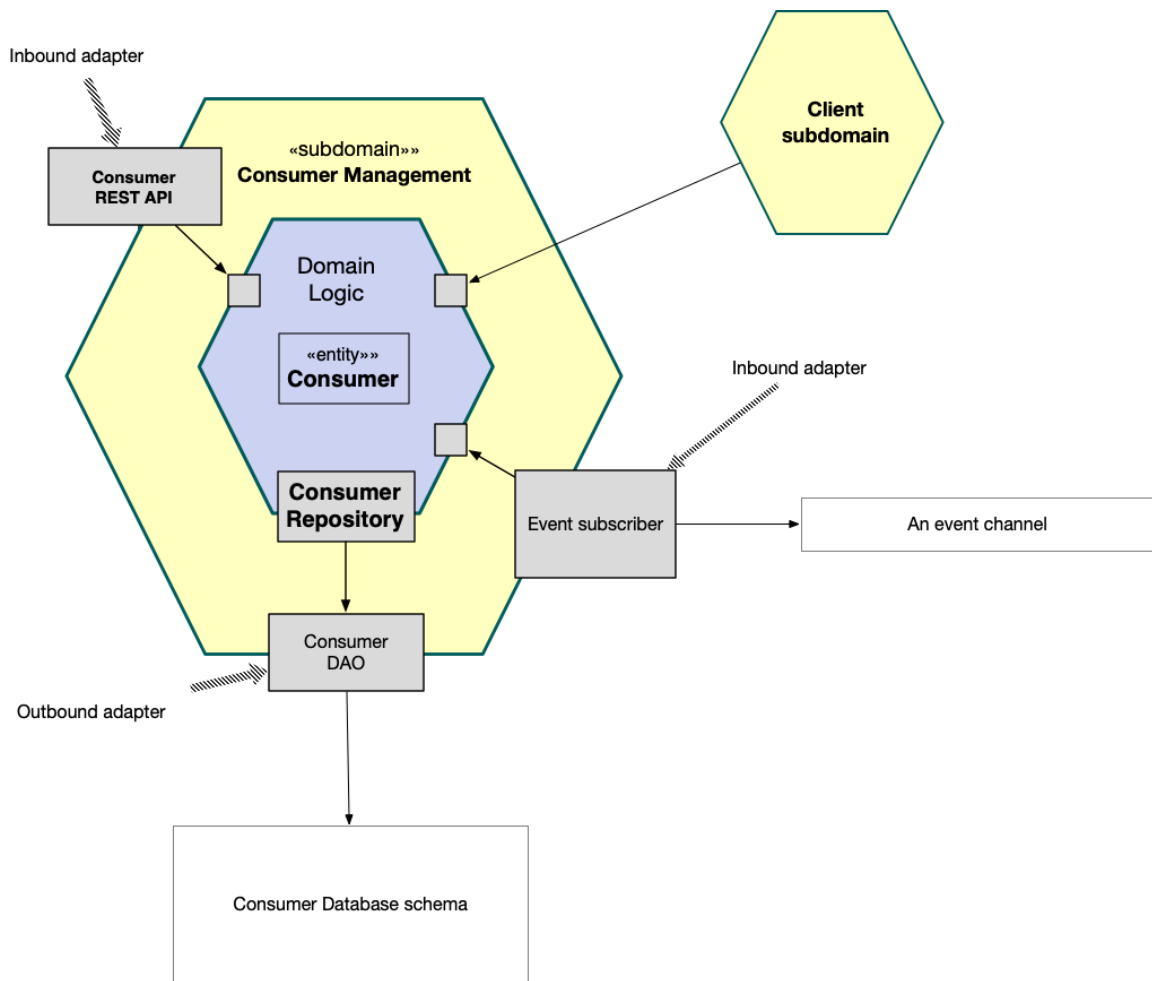


Figure 6.9 Domain module has a hexagonal architecture

The elements of this architecture consist of the packages shown in figure [6.10](#). The root package for the subdomain is `com.ftgo.consumers`, which contains the following sub-packages:

- `consumers.restapi` package - the REST API inbound adapter including HTTP controller classes, such as `ConsumerController`
- `consumers.domain` - the root package for the business logic. It contains entities such as the `Consumer` class, and repositories, such as `ConsumerRepository`
- `consumers.infrastructure` - the root package for the adapter classes. This package contains classes such as

the `ConsumerDao` class, which implements the `ConsumerRepository` interface.

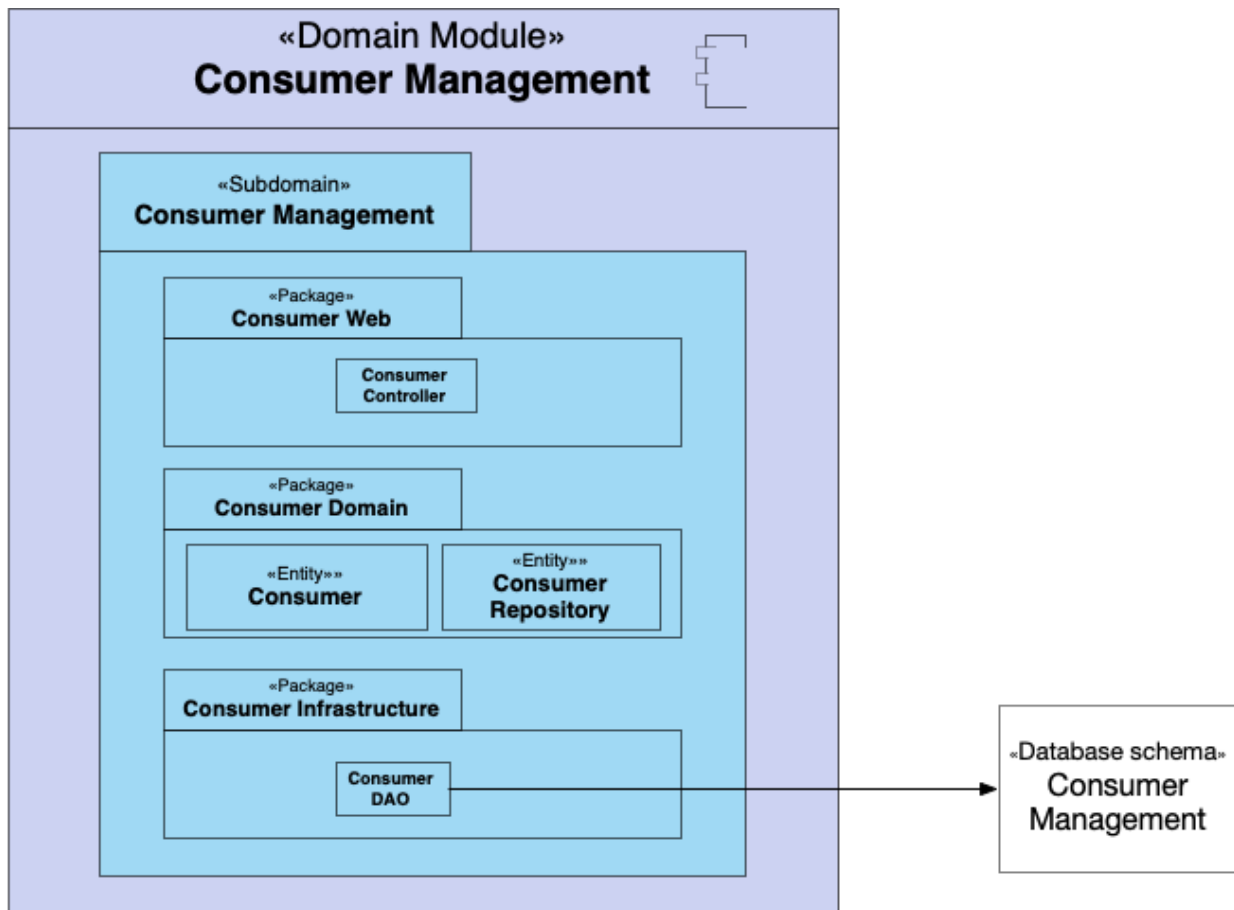


Figure 6.10 A domain module consists of various sub-packages, including `restapi`, `domain`, and `infrastructure`. The `restapi` package contains the REST API inbound adapter, the `domain` package contains the business logic, and the `infrastructure` package contains the outbound adapters.

A domain module's adapters are responsible for handling communication with the outside world — including external systems, users, and other components. In addition to these external interfaces, a domain module often exposes an internal API that is used by other domain modules within the same application. Let's take a closer look at the nature of this internal API.

DOMAIN MODULE API DESIGN

Loose design-time coupling is essential for fast flow. A domain module must have, therefore, a small, stable API that encapsulates its internal logic. Consider, for example, the `Consumer Domain Module`, which contains business entities such as `Consumer` and `ConsumerRepository`. The `createOrder()` operation, which is implemented by the `Order Domain Module`, must invoke the `Consumer Domain Module` in order to reserve and release credit. The `createOrder()` operation could invoke the `Consumer` and `ConsumerRepository` directly as shown in figure [6.11](#). However, this would create tight design-time coupling between the two domain modules.

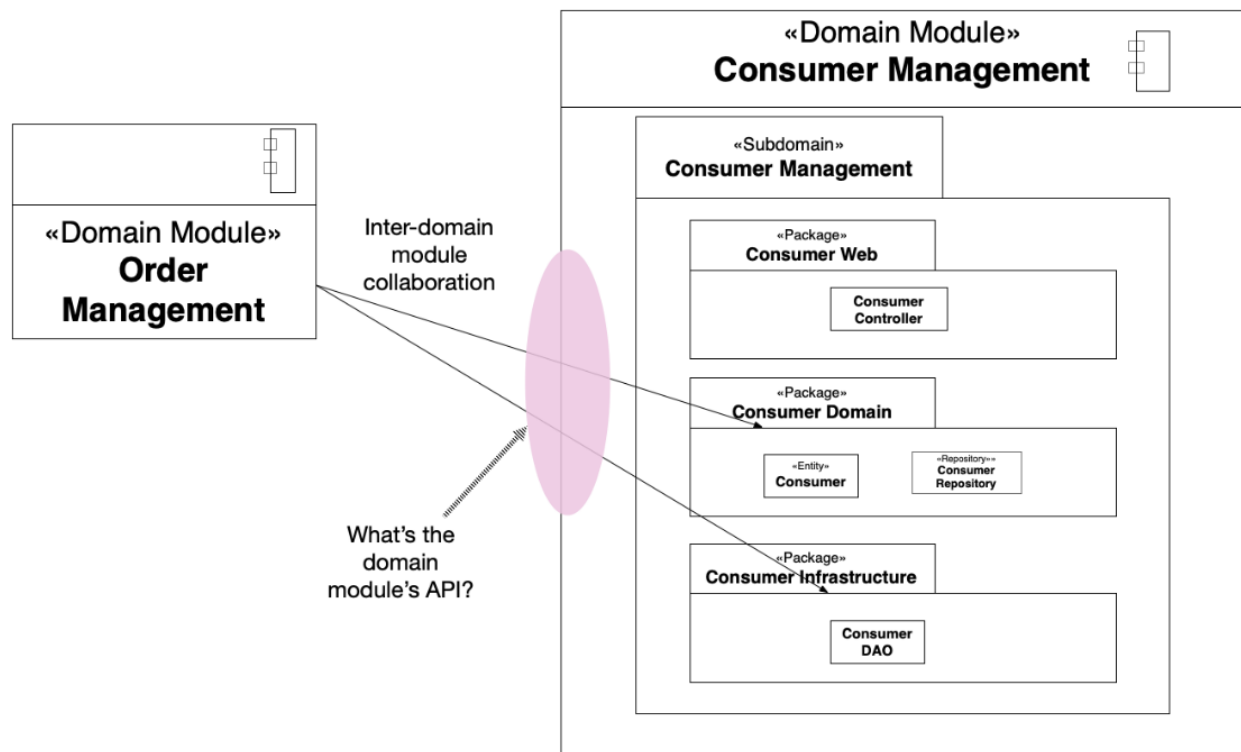


Figure 6.11 `Order Domain Module` could directly invoke the `Consumer` entity and `ConsumerRepository` in order to reserve and release credit. The drawback of this approach is that it creates tight design-time coupling between the two domain modules.

The problem with this approach is that the `Order Domain Module` is not restricted to invoking functionality intended to be public. It can, for example, invoke any public method of either the entity or the repository. Assumptions that the

`Consumer` entity is a JPA entity even leaks into the `Order Domain Module`. Moreover, it's challenging to mock an API that consists of entities and repositories.

A better approach, which reduces design-time coupling, is for a domain module to have a facade-style API that defines operations for reserving and releasing credit as shown in figure [6.12](#). When needed, a facade uses Data Transfer Objects (DTOs) to exchange data with its clients. The `Consumer Domain Module` would have, for example, a `ConsumerService` interface, which defines methods such as `reserveCredit()` and `releaseCredit()`.

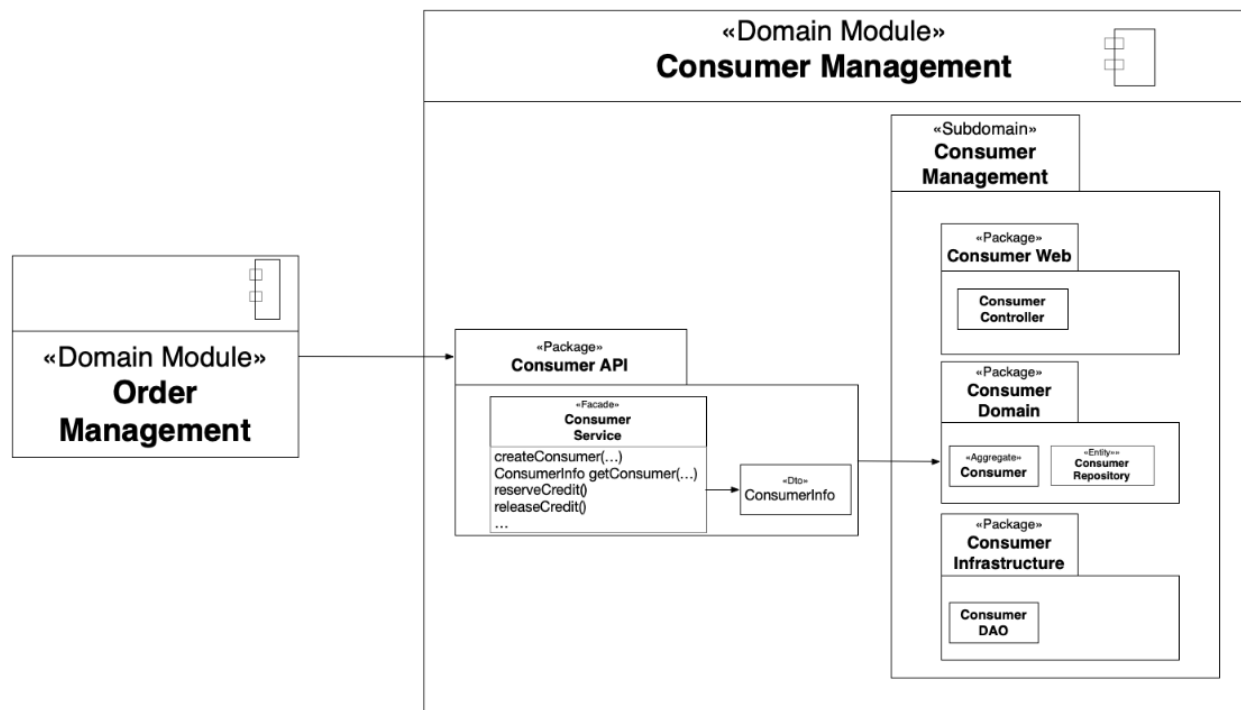


Figure 6.12 The `ConsumerService` facade encapsulates the implementation details of the `Consumer Domain Module` and reduces design-time coupling. Moreover, an `Order Domain Module` can easily mock the `ConsumerService` facade.

The `Consumer Domain Module` is now encapsulated behind the `ConsumerService` facade. The `Order Domain Module` domain no longer has a direct dependency on the `Consumer` entity or the

ConsumerRepository. Moreover, the tests for the Order domain Module can easily mock the Consumer Service.

DOMAIN MODULE DATABASE DESIGN

A facade-style API is just one aspect of ensuring that domain modules are loosely design-time coupled. A domain module's database schema should also be treated as a private implementation detail. As figure [6.13](#) shows, the Order Domain Module should not, for example, have direct access to the Consumer Domain Module's database schema. All collaboration between domain modules should be through APIs.

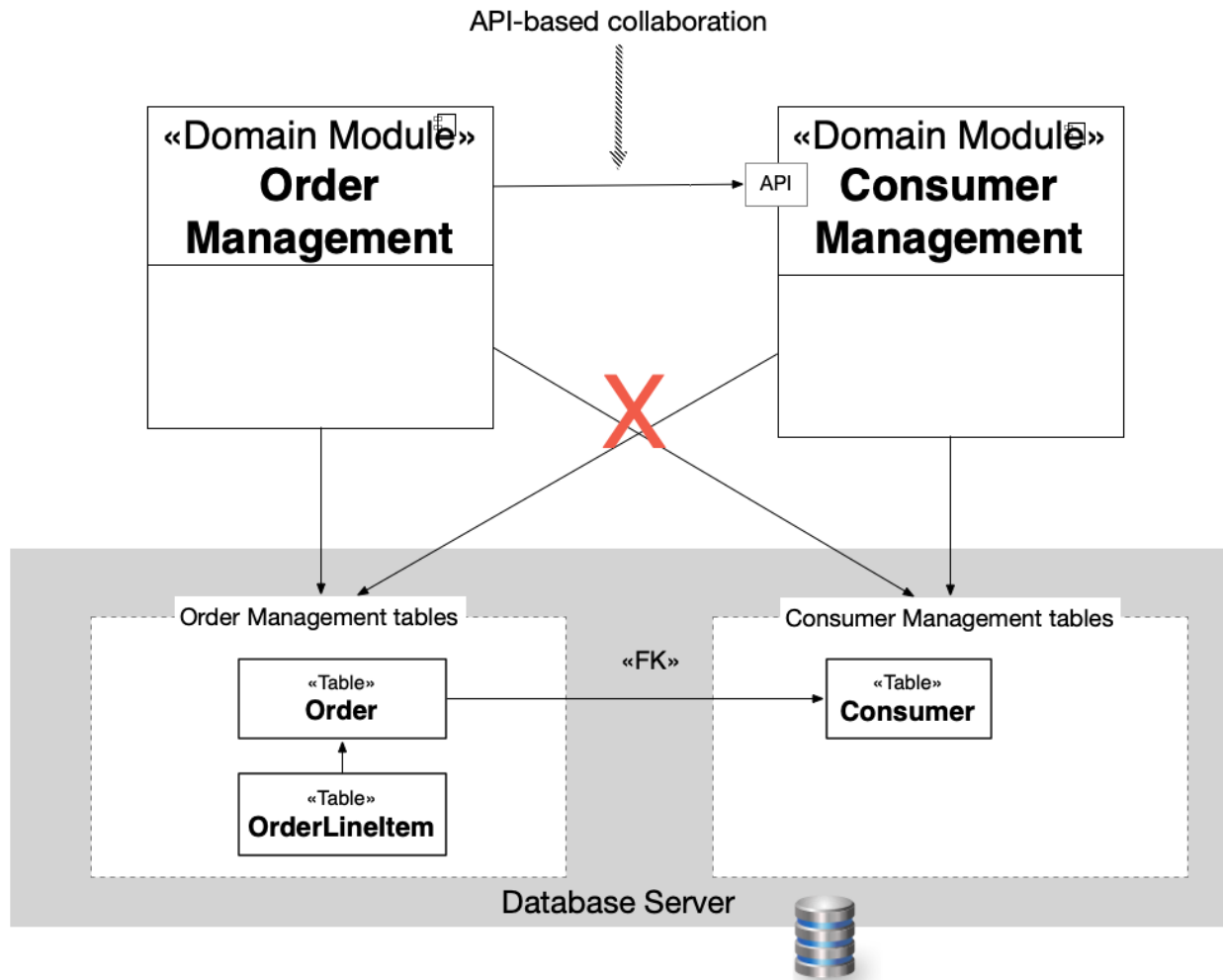


Figure 6.13 Domain modules should collaborate through APIs rather than direct database access. This reduces design-time coupling and improves modularity.

The drawback of API-only collaboration is that sometimes it is complicated and less efficient than direct database access. Consequently, it's best to be pragmatic and to allow some `SELECT` statements to cross domain module boundaries. For example, figure [6.14](#) shows a module that implements a query that spans the `Consumer Management` and `Order Management` subdomains in order to efficiently retrieve a consumer's order history.

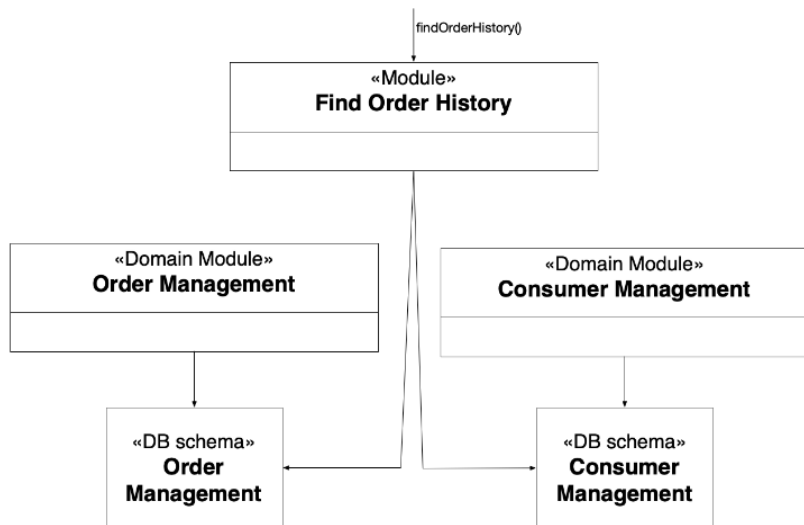


Figure 6.14 Sometimes ease of implementation and efficiency outweigh the need for modularity. In this example, a query that spans multiple domain modules is implemented using a single SQL statement that joins multiple tables.

The query consists of a `SELECT` statement than joins multiple tables including those belonging to the `Consumer Domain Module` and the `Order Domain Module`.

It's important to avoid writing database update statements - such as `INSERT`, `UPDATE`, `DELETE` statements - that span multiple boundaries because doing so is likely to duplicate or bypass the business logic that enforces data consistency. For example, it would be a bad idea if the `Order Domain module` directly updated a consumer's available credit, since it would have to duplicate business logic in the `Consumer Domain Module`. The exception is bulk updates that need to violate modularity in order to be efficient.

6.2.4 Designing system operations that span domains

A system operation is implemented by one or domain modules. Designing a single domain module operation is straightforward. For example, the `createConsumer()` operation is implemented by the `Consumer Domain Module`, which exposes

a `POST /consumers` REST endpoint and implements the business logic internally.

In contrast, designing a multi-domain module operation is more complicated. You first need to decide which domain module is the initial entry point - the one that handles the external request. Next, you must decide how the entry point domain module collaborates with the other domain modules involved in the operation.

Consider, for example, the `createOrder()` operation, which spans multiple modules, including the `Order Domain Module`, `Consumer Domain Module`, and `Notification Domain Module`. The `Order Management module` is a natural choice for handling the `POST /orders` HTTP request since it owns the core logic for creating and managing orders. However, `Order Management module` must also collaborate with the `Consumer Management domain` to reserve credit for the order and with the `Notification Management domain` to send a confirmation email to the consumer.

These interactions require a number of design decisions to be made, such as how domain modules should communicate and how to manage transactions across modules. Let's look at these design issues in more detail starting with domain module collaboration.

DOMAIN MODULE COLLABORATION

When a system operation spans multiple domain modules, you must choose one or more collaboration mechanisms. Each mechanism has different trade-offs in terms of complexity, and both design-time and runtime coupling. Figure [6.15](#) shows the three different ways that a domain module, such as `Order Domain Module`, could collaborate with the other domain modules to implement the `createOrder()`

operation. The three domain module collaboration patterns are:

- Invoke - one domain module calling another
- Observe - one domain module subscribing to notifications from another
- Asynchronous messaging - one domain module sending messages to another

The domain module collaboration patterns

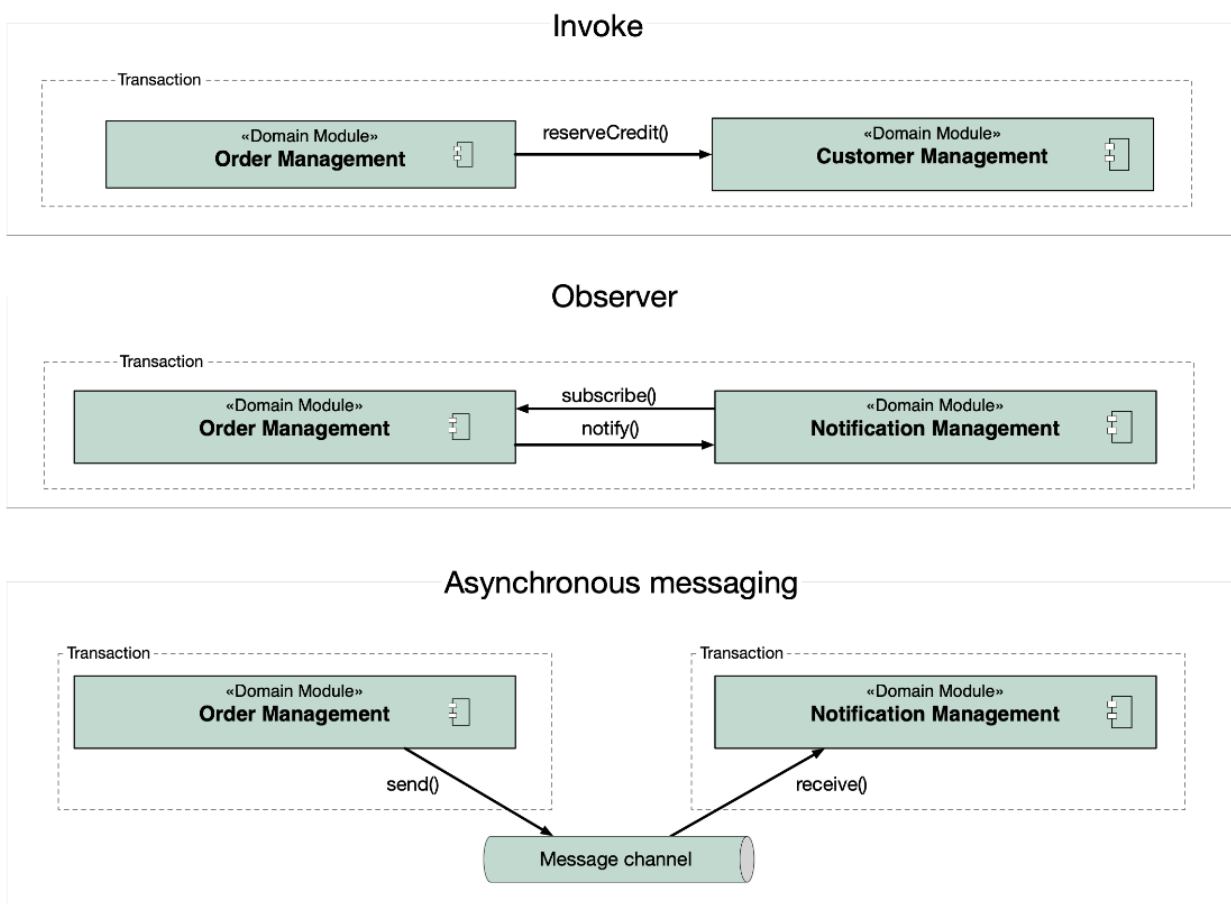


Figure 6.15 There are three domain module collaboration patterns: Invoke, one domain module calling another; Observe, one domain module subscribing to notifications from another; and Asynchronous messaging, one domain module sending messages to another.

Let's look at each pattern in turn.

THE INVOKE PATTERN

The most common and, by far the simplest, collaboration mechanism is a simple method call. One domain module can simply invoke a method on another domain module. As figure 6.16 illustrates, `Order Management` can invoke the `reserveCredit()` method on the `Consumer Management` domain module:

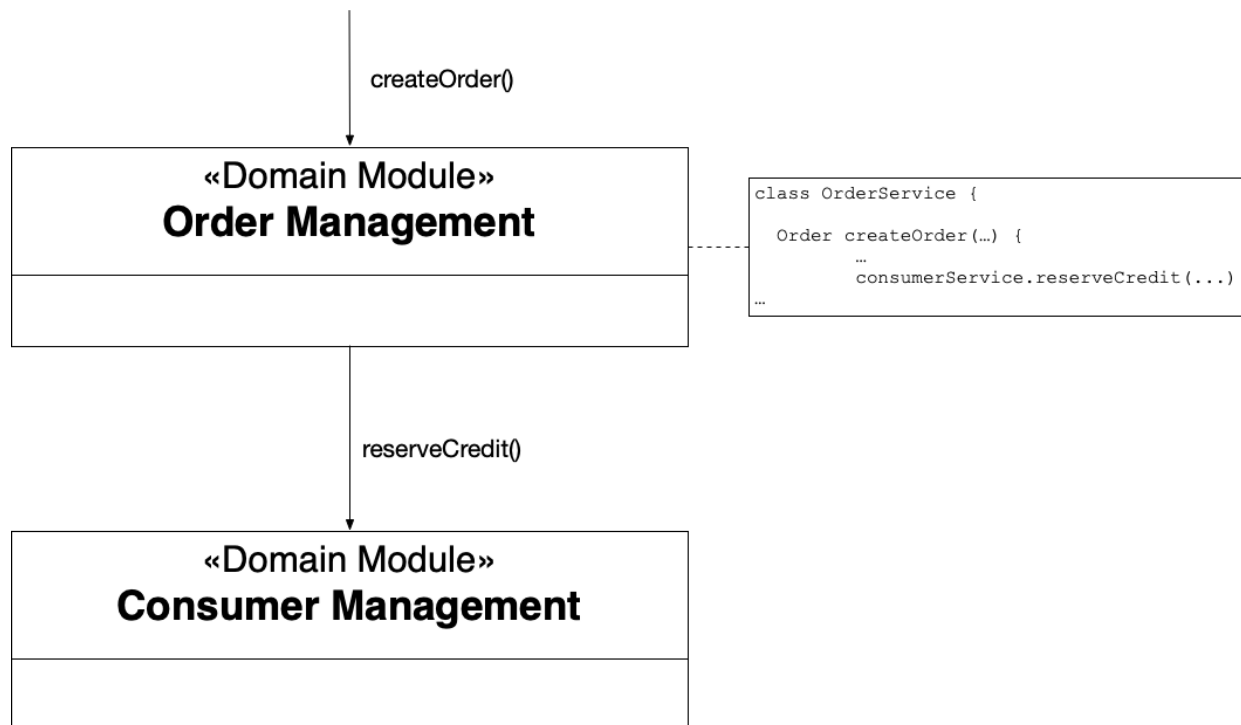


Figure 6.16 The Invoke pattern is the most common and simplest collaboration mechanism. For example, the `Order Domain Module` can invoke the `reserveCredit()` method on the `Consumer Management` domain module.

This is a simple and familiar mechanism, which is one reason why it's so widely used. Moreover, it also works well with the transaction-per-operation strategy, which I describe below.

One drawback, however, of invocation-based collaboration is that the calling domain module is design-time coupled to the domain modules that it invokes. Even if they are loosely design-time coupled, the risk of a lock step change is

proportional to the number of dependencies. This is especially undesirable if a domain module is coupled to domain modules that are unrelated to its core responsibilities. For example, the `Order Domain Module`'s core responsibility is managing orders. While invoking the `Consumer Management` domain module to reserve credit is an integral part of creating an order, invoking the `Notification Management` domain module to send a confirmation email is not. Responsibilities such as sending emails are secondary and should not be tightly integrated.

Moreover, it should be possible to add a post-order creation action without modifying `Order Domain Module`. In other words, `Order Domain Module` should conform to the Open-Closed Principle (TODO-REF): it should be open for extension but closed for modification. One effective way to achieve this separation of concerns is the Observer pattern, which I discuss next.

Another drawback of invocation-based collaboration is that the caller is effectively runtime coupled to the callee - it cannot continue until the method call returns. This means, for example, that the `Order Domain Module` cannot complete the creation of the `Order` until the `Consumer Domain Module` domain module has reserved credit and the `Notification Domain Module` domain module has sent a confirmation email.

Given that reserving credit is an integral part of creating an order, it makes sense for the `Order Domain Module` to wait for the `Consumer Management` domain module to reserve credit. However, it doesn't make sense for the creation of the order to be delayed by problems sending a confirmation email. Later on, I will describe how to solve this problem with asynchronous messaging.

THE OBSERVER PATTERN

The second collaboration pattern is the Observer pattern. This pattern consists of the following elements:

- Subject - the software element that is being observed and notifies its registered observers of changes
- Observer - an element that is notified by the subject

The observer pattern is a simple, in-memory event publish/subscribe mechanism. What's more, the subject can invoke each observer within its transaction. As a result, the observer pattern works well with the transaction-per-operation strategy, which I describe below.

As figure [6.17](#) shows, the `Order Management` domain module, for example, could publish a notification when an `Order` is created. Other domain modules, such as the `Notification Management`, can subscribe to the notification, and react accordingly.

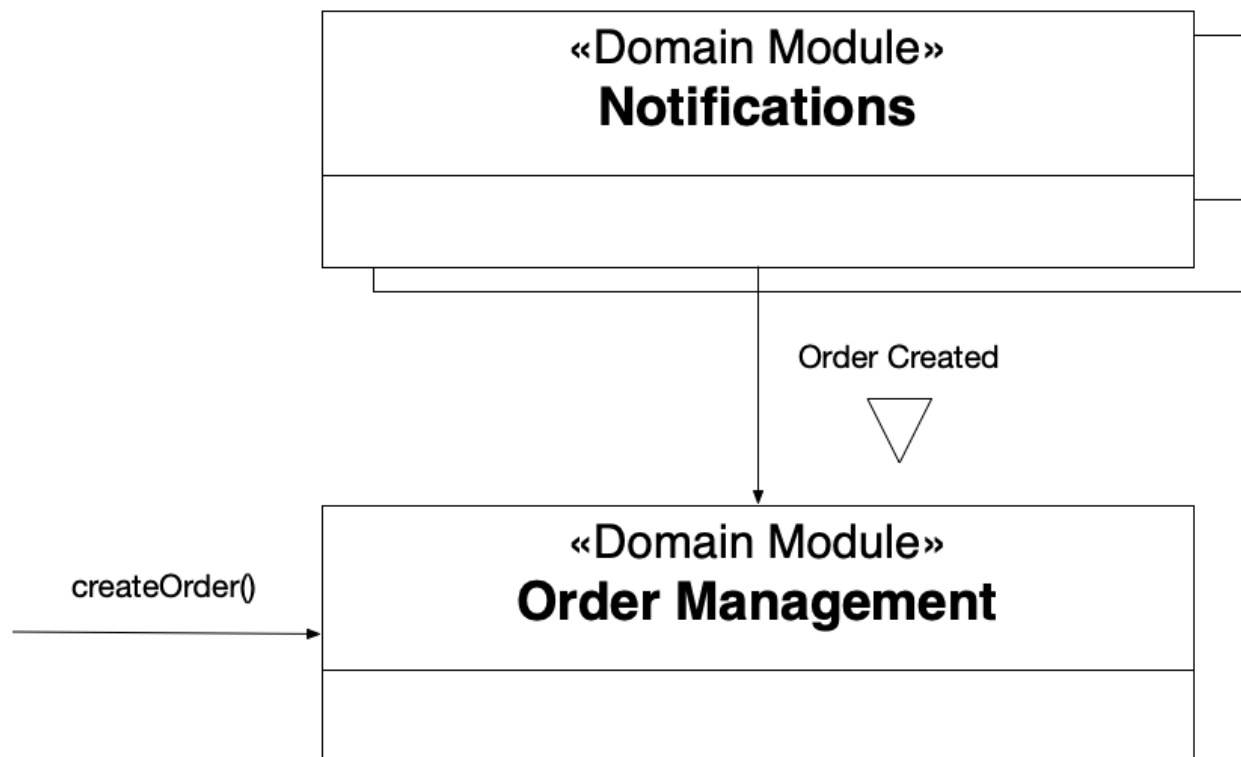


Figure 6.17 Another simple domain modular collaboration pattern is the **Observer pattern**. For example, the `Order Management` domain module can **notify** `Notification Management` **when it creates an** `Order`.

The observer pattern decouples `Order Management` from those other domain modules. However, those other modules are, instead, design-time coupled to `Order Management`. Whether this is an improvement depends on the specific situation.

Another issue is that the `Order Management` is still runtime coupled to the observers. After all, they are invoked synchronously within the subject's transaction. While invoking the observers synchronously simplifies transaction management, it makes it possible for slow observer to affect the execution of the operation. To avoid this problem, it sometimes makes sense to use the third collaboration pattern: asynchronous messaging.

ASYNCHRONOUS MESSAGING PATTERN

The third collaboration pattern is asynchronous messaging. When using this approach, domain modules collaborate by exchanging messages over message channels, which are typically implemented by a message broker. Quite often those messages are events, but as I explain in chapter 3, there are other types of messages. Asynchronous messaging is useful because it can eliminate runtime coupling between domain modules.

For example, as I mentioned earlier, one drawback of having the `Order Domain Module` collaborate with `Notification Domain Module` via direct invocation or the Observer pattern is that it creates runtime coupling. Because the `Notification Domain Module` invokes an external email service - which may be down or slow - there's a risk that it will block the `Order Domain Module`.

A better solution is to split the `createOrder()` operation into a synchronous part and an asynchronous part. The asynchronous part, which handles the HTTP request, creates a `Order`, publishes an event to a message channel, and returns an HTTP response to the client. The asynchronous part consists of one or more event handlers that perform follow-up actions, such as invoking external applications, including the email service.

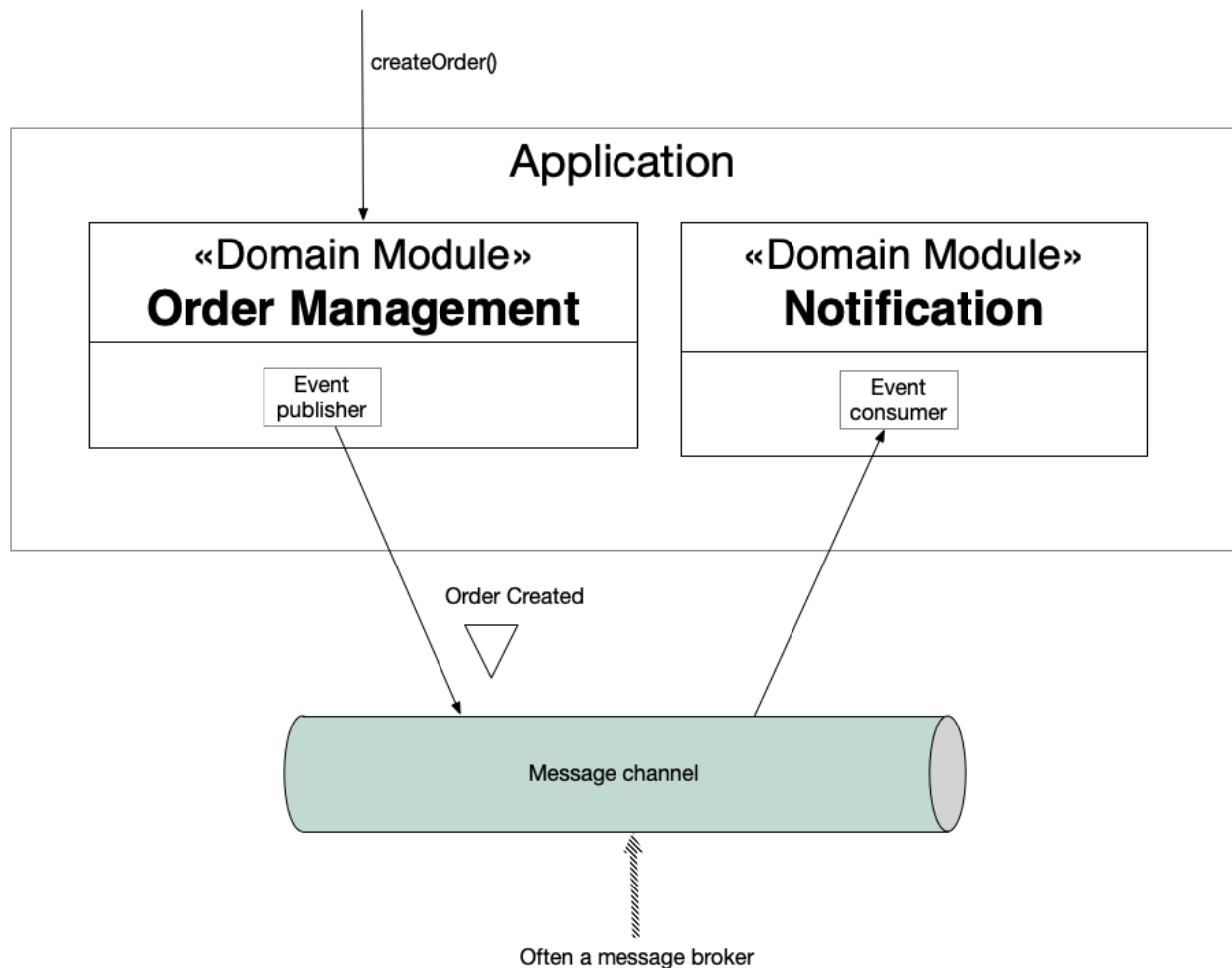


Figure 6.18 The `createOrder()` operation can be decoupled from notification delivery but publishing an `Order Created` event to a message channel. The `Notification Management` domain module can then subscribe to the event and send a confirmation email.

Implementing `createOrder()` in this way decouples `Order` creation from the interactions with the external systems. The `Order` can still be created, even if the email service is unavailable, improving overall reliability. This asynchronous approach also enables the use of task queues for scalable background processing. Additionally, it also enables different parts of an operation to be executed on infrastructure with hardware resources suited to their specific needs.

On the surface, using messaging channels might seem straightforward. However, it's a far more complex

collaboration mechanism than simple method calls or the observer pattern. It requires careful application of patterns such as atomic event publication, and message deduplication, which I describe in chapter 3. Transaction management is particularly challenging because the operation consists of multiple transactions - an approach which is considerably more complex than a single transaction, as I explain below.

TRANSACTION MANAGEMENT

In addition to deciding how domain modules should collaborate to implement a system operation, you must also determine the operation's transaction management strategy. There are three main transaction management strategies to choose from:

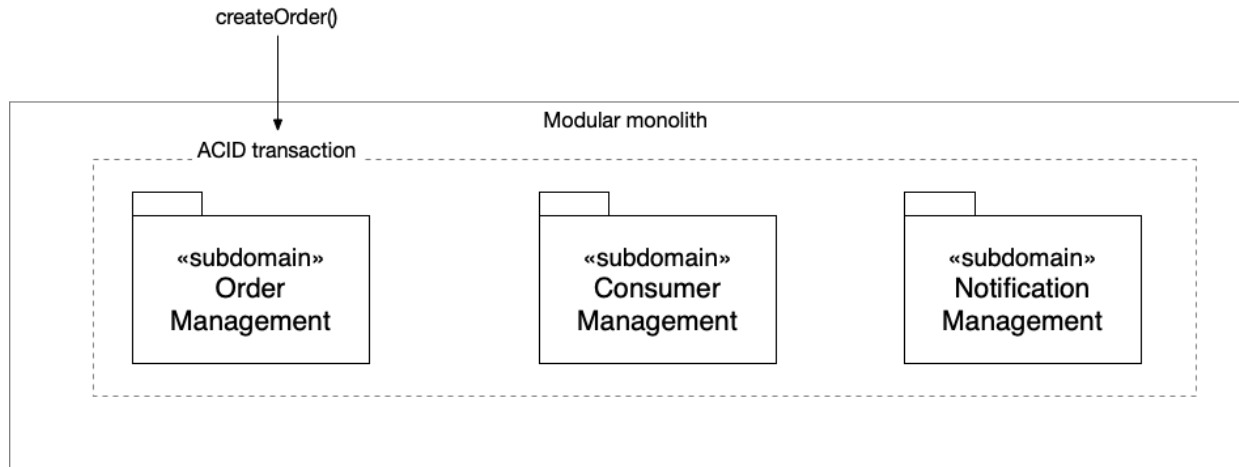
- Transaction-per-operation
- Transaction-per-domain module
- Requirements-driven transaction boundaries

The first two strategies structure transactions around architectural elements - operations and domain modules. The third strategy designs an operation's transaction management based on the operation's requirements. It's important to keep in mind that the collaboration and transaction management strategies are intertwined. For example, the Invoke and Observer collaboration patterns work well with the transaction-per-operation strategy. But when using the asynchronous messaging strategy, splits the operation into multiple transactions.

Let's look at each transaction management strategy in turn.

TRANSACTION-PER-OPERATION

The simplest approach is transaction-per-operation, which is shown in figure [6.19](#). In fact, one of the biggest benefits of the monolithic architecture is that a system operation that spans multiple subdomains can be implemented as a single ACID transaction. For example, the `createOrder()` operation, which spans `Order Management`, `Consumer Management` and `Notification Management`, can use a single transaction to create an order, reserve credit, and send a confirmation email. Moreover, popular frameworks such as Spring provide declaration transaction management, which makes transaction management very simple.



Simplest

```

@Transactional
public ... createOrder(...) {
    // Simple sequential logic
    consumerManagement.reserveCredit(...);
    var order = new Order(...);
    orderRepository.save(...);
    orderObservers.notify(...);
    ...
}

```

Figure 6.19 The transaction-per-operation strategy implements a system operation as a single ACID transaction. It's the simplest and most common approach for implementing transactions in a monolithic architecture.

The `@Transactional` annotation specifies that `createOrder()` executes within a transaction. If the operation fails, then the database transaction is automatically rolled back.

The transaction-per-operation strategy is a straightforward and effective way to ensure data consistency. It's my preferred option unless an operation's requirements dictate a multi-transaction approach. It does mean, however, that some transactions will span multiple domain modules, which some developers have argued violated modularity. Still, for

many system operations, the simplicity of this strategy outweighs those concerns.

TRANSACTION-PER-DOMAIN MODULE

The second strategy is the transaction-per-domain module. One benefit of this strategy is that it reinforces the idea that the domain modules are loosely coupled. The main drawback is that it's much more complicated to implement. Most notably, it requires the use of the Saga pattern, which I describe in detail in chapter 10. For example, figure [6.20](#), the design of the `createOrder()` command that uses transaction-per-domain module strategy. This design uses a Saga, which consists of a sequence of ACID transactions in each of the three subdomains.

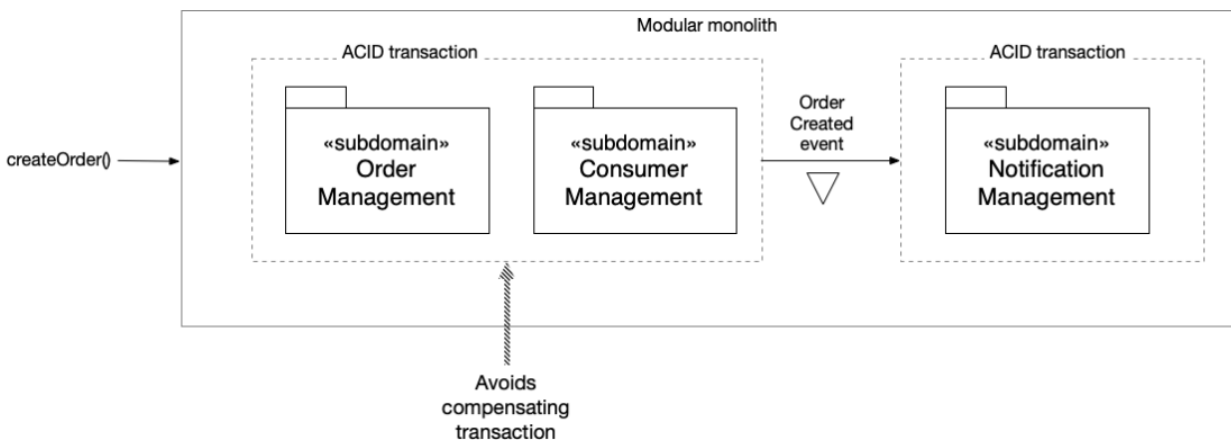


Figure 6.20 The `createOrder()` operation is implemented as a Saga, which consists of a sequence of ACID transactions in each of the three subdomains. Because reserving credit can fail, the `Order Domain Module` must use a compensating transaction to undo any updates that it did before invoking the `Consumer Domain Module`.

The challenge with such a design is because reserving credit can fail, the `Order Domain Module` must use a compensating transaction to undo any updates that it did before invoking the `Consumer Domain Module`. Compensating transactions can be complicated to implement and understand. In fact, the

transaction-per-domain module has some downsides of the transaction model used by the microservice architecture without any of the benefits.

Fortunately, however, not all sagas are complicated. Figure [6.21](#) shows an alternative design for the `createOrder()` operation that avoids the need for a compensating transaction despite consisting of multiple transactions. The first transaction spans the `Order Domain Module` and `Consumer Domain Module`, while the second transaction spans the `Notification Domain Module`. Since any business rule violation can only occur within the first transactions, there's no need for a compensating transaction.

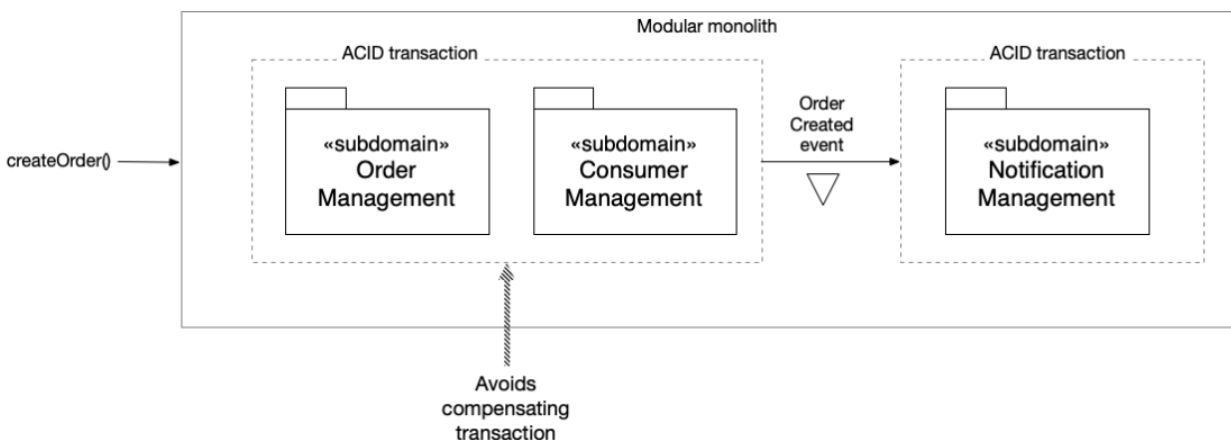


Figure 6.21 The `createOrder()` operation is implemented using a simpler saga that does not use compensating transactions. The first transaction spans the `Order Domain Module` and `Consumer Domain Module`, while the second transaction spans the `Notification Domain Module`. Any business rule violation can only occur within the first transactions, so there's no need for a compensating transaction.

As you can see, while some sagas are relatively simple, others — particularly those involving compensating transactions — can be complicated. Unless there is a truly compelling reason to do so, I recommend avoiding the transaction-per-domain module model when it involves compensation logic.

REQUIREMENTS-DRIVEN TRANSACTION BOUNDARIES

The first two transaction models—transaction-per-operation and transaction-per-domain module—define transaction boundaries based on architectural elements. In contrast, a third option is to design transaction boundaries based on the specific requirements of each operation. This approach acknowledges that certain operations have characteristics or constraints that make them unsuitable for single-transaction execution, even if they span only one domain module.

Two common examples of operations that benefit from requirement-driven transaction boundaries are:

- Batch jobs - while a batch job could theoretically be implemented as a single long-running transaction, it is often intentionally split into smaller transactions to improve error handling, scalability, and performance.
- Operations that invoke external systems - external systems can rarely participate in an application's transaction. As a result, an application must often commit its local transaction before invoking an external system, especially if the data passed to the external system is derived from that transaction. After the system receives a response, it may begin a new transaction to complete the remaining work.

This transaction management strategy ensures consistency and reliability, even in the face of unreliable or slow external services.

6.2.5 Testing strategy for a modular monolith

One of the key advantages of the modular monolith over a technology-oriented monolith is improved testability. By organizing the application around loosely design-time

coupled domain modules, it becomes possible to test each module independently. This isolation reduces the scope of changes and testing, which in turn shortens the edit-compile-analyze loop and accelerates the deployment pipeline. Moreover, section [Section 6.3](#) describes design techniques that minimize build-time coupling — both between and within domain modules. These techniques further speed up testing by reducing the number of projects that need to be rebuilt and retested after a change.

In this section, I'll briefly describe a testing strategy for a modular monolith. It's similar to the microservices testing strategy described in chapters 15 and 16. To accelerate and simplify testing:

- Tests are pushed down the test pyramid as far as possible.
- Domain modules are tested in isolation.
- Application tests, which are often slow, are kept to a minimum

Let's first look at testing domain modules.

DOMAIN MODULE TESTING PATTERNS

As you might expect, each domain module has a test suite that verifies its behavior. The test suite consists of three types of tests:

- Unit tests - in-memory tests for non-trivial code, such as algorithms
- Integration tests - tests for the domain module's adapter classes, such as HTTP Controllers, DAOs, etc.
- Module tests - tests for the entire module that use test doubles for the domain modules that it depends upon

with

For example, the test suite for the `Order Domain Module` includes unit tests for the `Order` entity and integration tests for the JPA-based persistence logic. The module level tests are implemented using Spring Boot's testing support.

APPLICATION TESTING

In addition to the per-domain module tests, there is a test suite for the entire application. The tests package the application as a deployable unit, such as a container image. These tests typically do not repeat the testing done by the domain module-level tests. Instead, they simply verify that the application's `@SpringBeans` are wired together correctly.

For example, the application-level tests for the FTGO application are written using the Test Containers framework (<https://testcontainers.com>), which builds and runs the application's container image.

6.3 Minimizing build-time coupling to improve testability and deployability

One of the major drawbacks of the monolithic architecture is that a large code base can take a long time to build. This can significantly impact both testability and deployability, making the monolith an obstacle to fast flow. Fortunately, a key advantage of the modular monolith is that it can have significantly less build-time coupling than a traditional layered monolith. The key is to structure each domain module's build projects in a way that minimizes build-time coupling.

In this section, I'll describe how to structure each domain module to reduce build-time coupling both between domain modules and within domain modules. This can significantly reduce the amount of testing needed to validate a change, which in turn accelerates the edit-test-analyze loop and the deployment pipeline. Let's start by first exploring the concept of **physical design principles**.

6.3.1 About physical design principles

Domain modules are part of the application's domain architecture, but they must be organized into build projects, which belong to the component architecture. A common and seemingly straightforward approach is to map each domain module to a single build project. However, this simple one-to-one mapping often results in high build-time coupling—both between modules and within a module—which can slow down testing and deployment.

To understand why, consider a simple example with two domain modules: `Consumer Domain Module` and `Order Domain Module`, where the later uses the former. When mapped one-to-one, each one becomes a Gradle project - `consumer-domain` and `order-domain` - then `order-domain` depends upon `consumer-domain`. As a result, any change to `consumer-domain` will trigger the recompilation and retesting of `order-domain`. In a real-world application, such a change could cascade through multiple transitive dependencies, significantly increasing build time. This slows down both the edit-test-analyze loop for developers and the deployment pipeline.

A better approach is to organize domain modules into build projects using 'physical design principles'. Whereas logical design is concerned with concepts such as naming and design-time coupling, physical design is concerned with structuring the build architecture in way that minimizes

build-time coupling. It applies two key ideas from chapter 4 that minimize build-time coupling both between and within projects. The first idea is that a build project should only depend upon stable projects. The second idea is that build projects should consist of elements that need to be tested together.

PHYSICAL DESIGN PRINCIPLES

One of my favorite books from the 1990s is Large Scale C++ Design by John Lakos (Addison-Wesley, 1996). While much of the book is C++-specific (not my most favorite language), it makes an important contribution: physical vs. logical software design. Logical software design is the main focus of software design techniques, such as object-oriented design and Domain-Driven Design: naming, assignment of responsibilities to classes and packages, class hierarchies, encapsulation, loose (design-time) coupling, cohesion etc. Ideas that are essential if we want our software to be understandable and maintainable.

Lakos argues, however, that for large-scale software, logical design is insufficient. We also must consider the physical design of the software, which is the structure of the application's physical entities, such as source files, Gradle projects/Maven modules - the file system artifacts consumed by the build tools. That's because physical design determines, among other things, build-time coupling between modules. Lakos describes insulation techniques (analogous to encapsulation techniques for reducing design-time coupling) that can be used to reduce compile-time coupling.

In this section, I describe three techniques for applying these ideas to minimize build-time coupling:

- Define build projects for domain module APIs
- Apply the Interface segregation principle
- Segregate implementation build projects

Let's examine each one of these techniques in turn.

6.3.2 Define build projects for domain module APIs

A great technique for reducing build-time coupling is to split a domain module build project into separate API and implementation build projects. Clients only depend on the API build project, not on the implementation. As a result, they are build-time coupled to the more stable API rather than the constantly changing implementation.

For example, figure [6.22](#) shows how the `Consumer` domain is split into `Consumer API` build project and a `Consumer Domain` implementation project. Clients, such as `Order Domain Module`, depend on the `Consumer API` build project. The clients are tested using test double for the `ConsumerService` interface. Changes to the `Consumer Domain` implementation project do not trigger the retesting of the `Consumer` domain's client.

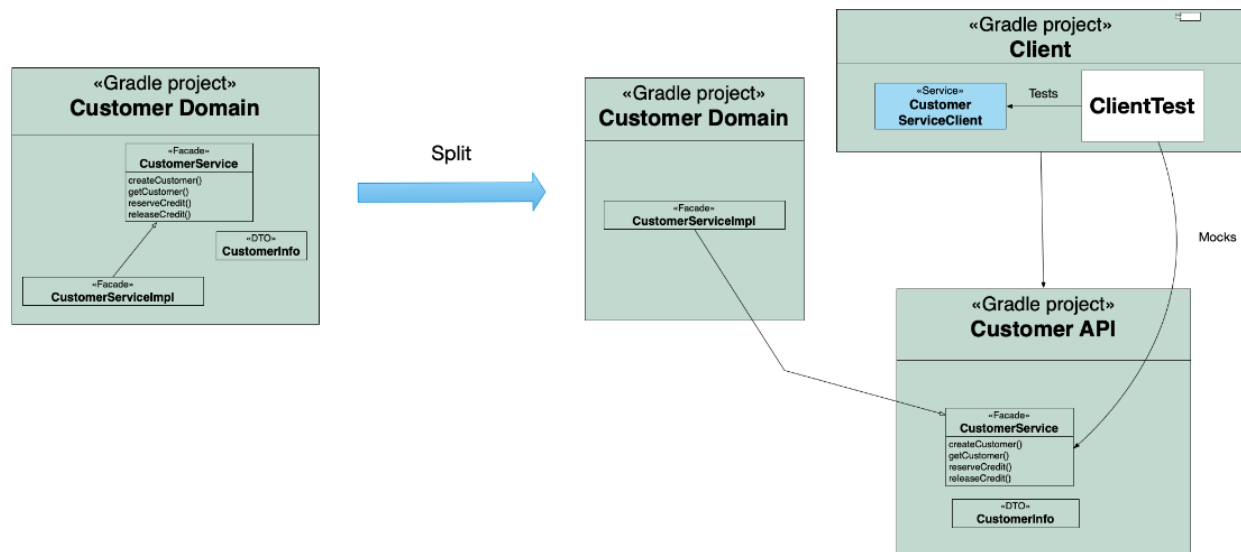


Figure 6.22 Reduce build-time coupling by splitting a domain module into an API build project and an implementation build project. Since clients only depend upon the API build project, they do not need to be retested when the implementation changes.

Structuring each domain module as an API build project and implementation build project significantly reduces build-time coupling. Clients depend only on relatively stable interfaces, which minimizes the scope of retesting after a change. This improves test execution time, shortens the edit-test-analyze cycle, and increases the throughput of the deployment pipeline.

6.3.3 Apply the Interface segregation principle

Defining a separate build project for a domain module's API is a good first step towards reducing build-time coupling. But consider the `ConsumerService` interface: the `createConsumer()` method is not used by the `orders` domain module. As a result, there's still unnecessary build-time coupling between the `Consumer Management` and `Order Management` domains. For example, if you add an attribute to `Consumer` entity, which in turn adds a new parameter to the `ConsumerService.createConsumer()` method, the `orders` module

will still be rebuilt and retested - even though it doesn't use this method.

To eliminate this unnecessary build-time coupling, we can apply the Interface segregation principle (<https://microservices.io/post/architecture/2023/09/12/how-modular-can-your-monolith-go-part-4-physical-design.html#applying-the-interface-segregation-principle-isp>), which states that no client should be forced to depend on methods it does not use. Applying this principle splits the `ConsumerService` interface into two interfaces: a smaller `ConsumerService` interface and a newer `CreditManagement` interface, each of which is in a separate API build project. These two build projects depend on a third build project that contains common types.

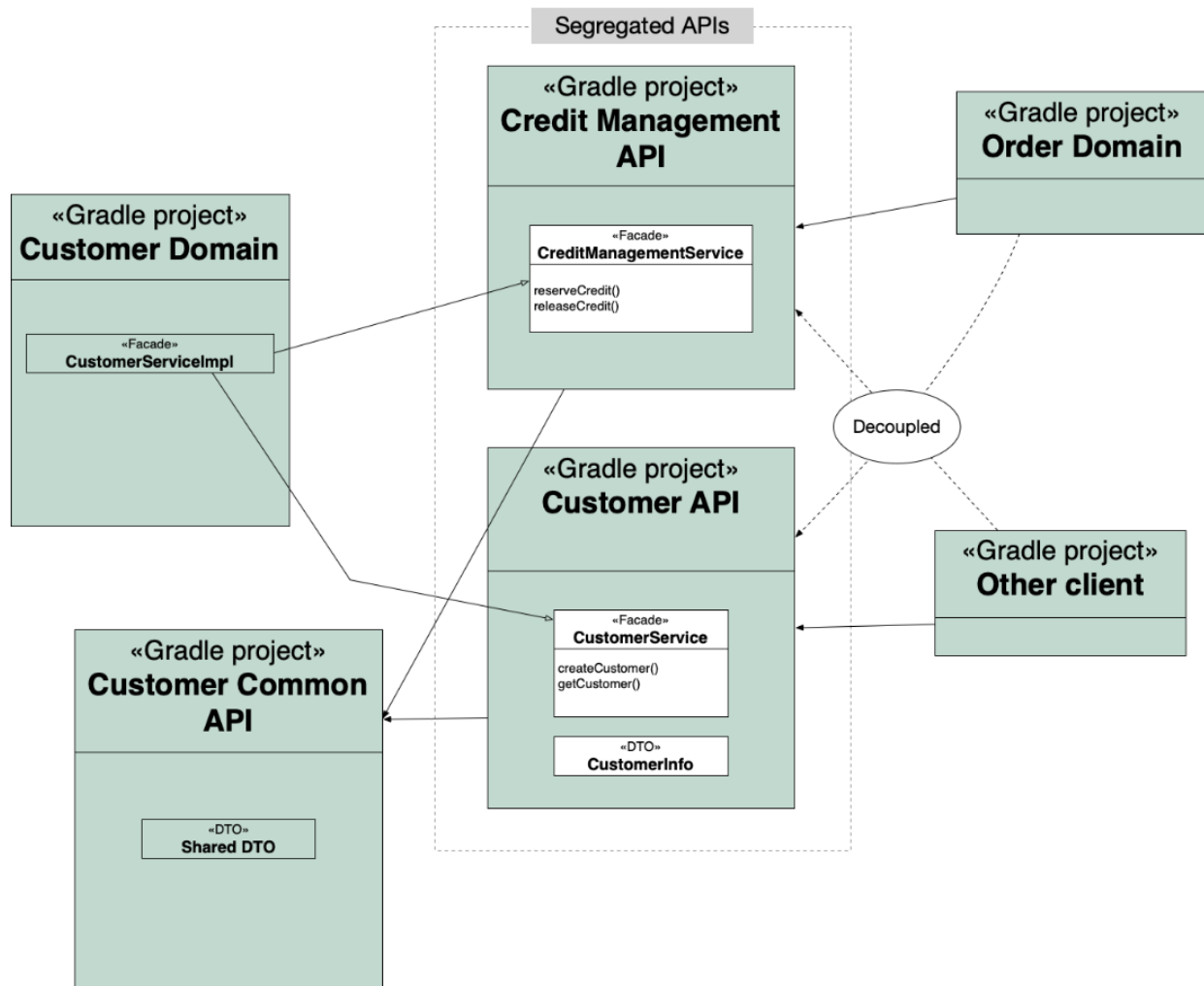


Figure 6.23 Reduce build-time coupling to an API by applying the Interface segregation principle and replacing a single domain API build project with several build projects.

By splitting the domain API build projects, clients are only build-time coupled to the APIs that they actually use. As a result, a client only needs to be retested when the API that it uses changes.

6.3.4 Segregate implementation build projects

Defining API build projects and applying interface segregation principle reduces build-time coupling between domain modules. However, build-time coupling within a domain module's implementation build project can still slow

down testing. A change to any class within that build project triggers the execution of all tests, including slower ones such as persistence tests. Even though the domain module's clients are not retested, testing changes to a complex implementation can be time-consuming. This slows down the edit-test-analyze cycle and deployment pipeline.

Consider, for example, the `Order Management` domain module. It consists of three sub-packages: `domain`, `web` and `persistence`. The `web` and `persistence` packages depend on the `domain` but not on each other. A change to one cannot affect the other from a design-time perspective. But if these three packages are within the same build project, then they are build-time coupled to one another. Changing `web` will cause `persistence` to be retested and vice versa.

To avoid unnecessary test execution and accelerate the edit-test-analyze cycle and deployment pipeline, it's best to segregate these sub-packages into separate implementation build projects. For instance, as figure [6.24](#) shows, the `web` and `persistence` packages should be placed in their own build projects that both depend on the `domain` build project.

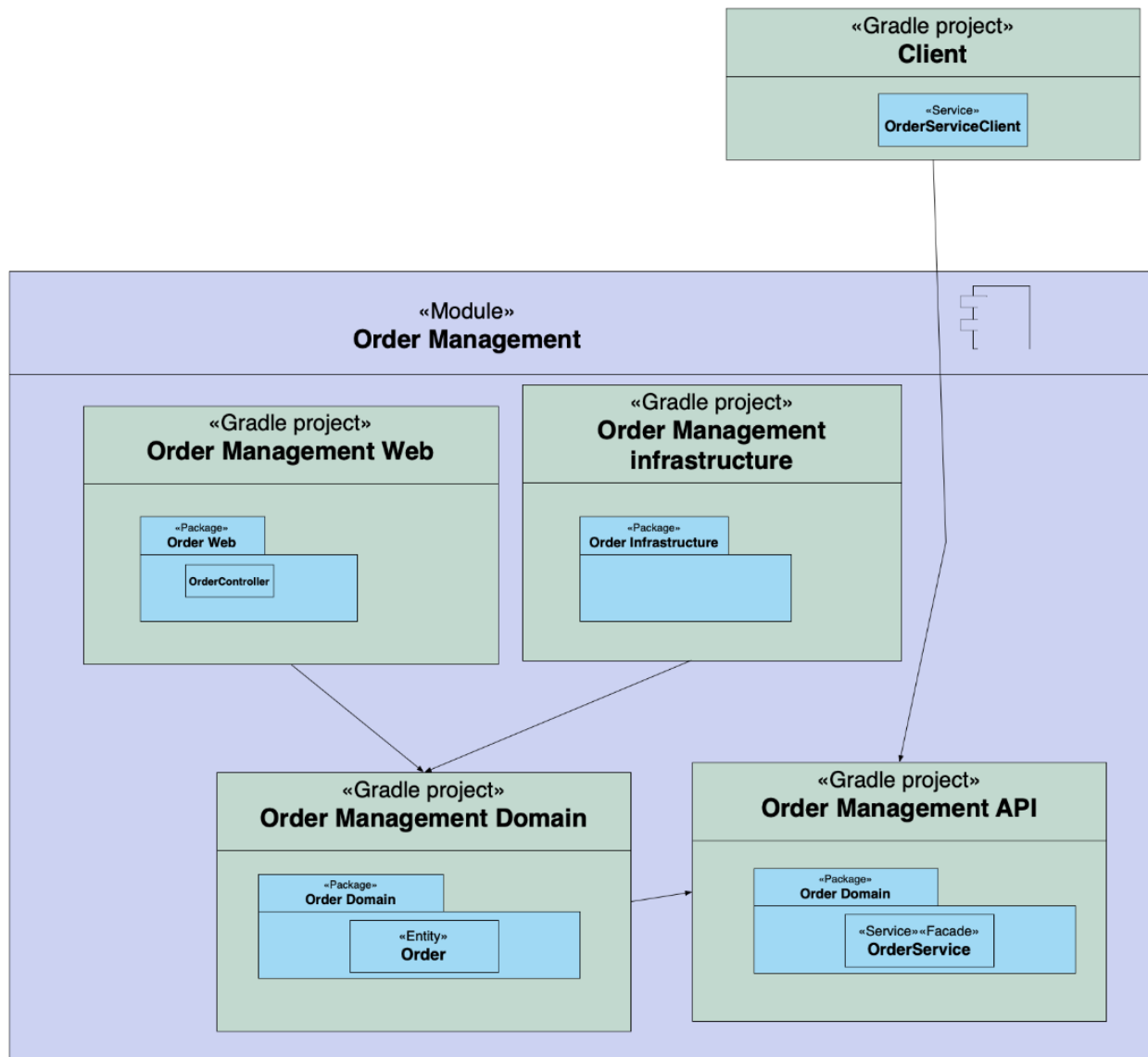


Figure 6.24 The implementation of a domain module should be split into multiple build projects in order to reduce build-time coupling and build times.

This structure eliminates unnecessary build-time coupling between unrelated concerns and ensures that only the tests for the changed code are re-executed. As a result, both local development and the deployment pipeline can provide faster feedback.

6.4 The limitations of the modular monolith

The modular monolith is a significant improvement over the traditional layered monolith. It is structured to better manage complexity, increase team autonomy, and accelerate the deployment pipeline. At the same time, it's still a monolith - with all the limitations that entails. What's more, the modular monolith is not always, as some claim, a stepping stone to the microservice architecture. Let's examine each of these issues in detail.

6.4.1 A modular monolith is still a monolith

Despite its advantages, a modular monolith remains a monolith: a single code base, a single technology stack and a single deployment pipeline. There's a limit to how modularity can improve testability and deployability. While, modularity can reduce average build times, it cannot reduce the worst case build time, such as those that occur when a widely used API or dependency is changed. As the application and its organization grow, the lack of testability and deployability will eventually become an issue. Evolvability is limited due to a single technology stack, and observability is coarse-grained. Eventually, your application and organization may outgrow the monolithic architecture. When that happens, you will need to use the microservice architecture in order to achieve fast flow.

6.4.2 The myth of the microservices-ready modular monolith

The modular monolith is sometimes described as a stepping stone to the microservice architecture. There's a common belief that when the time is right, each domain module can be extracted into a service. On the one hand, as I describe in chapter 21, loose design-time coupling and modularity helps. But on the other hand, there are two obstacles to refactoring a modular monolith into microservices: Domain

modules interact in ways that are not suitable for network-based communication.

- Domain modules interact in network unfriendly ways
- Splitting operations into multiple transactions is not straightforward

Let's look at each obstacle in turn.

DOMAIN MODULES INTERACT IN WAYS THAT ARE NOT SUITABLE FOR NETWORK-BASED COMMUNICATION

The first obstacle is that modules often use patterns of interaction that work well within a process but are poorly suited to a distributed architecture. Method calls between modules are incredibly efficient, while service-to-service communication over a network introduces latency, runtime coupling, and complexity. If you naively replaced method calls with service calls, the application's performance and reliability might drop significantly.

SPLITTING OPERATIONS INTO MULTIPLE TRANSACTIONS IS NOT STRAIGHTFORWARD

The second obstacle is that operations in a modular monolith typically use on a single ACID transaction that spans multiple modules. In a microservice architecture, however, transactions are local to each service. As you'll see in chapter 10, this means that a system operation that was once implemented as a single transaction must be split into multiple transactions. This transformation is non-trivial. It often requires rethinking the operation's design and implementing coordination mechanisms such as the Saga pattern, which I describe in chapter 10. Splitting transactions introduces complexity around consistency,

concurrency, error handling, and rollback—complexity that doesn't exist within a monolith.

6.5 Summary

- The monolithic architecture is a simple architectural style that structures the application as a single component with a single code base and a single deployment pipeline
- Since a monolith consists of a single code base, development is often simpler; widespread changes are easier to make and can be deployed with a single `git push`; and most operations can be implemented as ACID transactions.
- The monolithic architecture is a good choice for many applications, especially those that are smaller and developed by a small number of teams
- However, large complex applications developed by many teams can outgrow the monolithic architecture: the single deployment pipeline can become a bottleneck; and some types of changes, such as those to its technology stack, can require time-consuming coordination across many teams
- A modular monolith structures the code base around the application's subdomains, which improves team autonomy and reduces build-time coupling
- Careful build project design to further minimize build-time coupling can significantly accelerate the deployment pipeline for many changes - though it cannot reduce worst case build times
- In theory, a modular monolith can be designed to be microservices-ready, but in practice, it's likely that it will be difficult to extract modules into services because of how they collaborate and use transactions

- A modular monolith is still a monolith, and some applications and their organizations will eventually outgrow the monolithic architecture

7 The Microservice architecture: enabling fast sustainable flow

This chapter covers

- Why the microservice architecture enables fast flow development at scale
- The drawbacks of the microservice architecture
- An overview of the microservice architecture pattern language

In the previous chapter, I described how the monolithic architecture is often a good choice. But sometimes - especially when the business-critical application becomes successful - an application and its development organization can outgrow its monolithic architecture. As a result, it becomes impossible to satisfy the five software quality attributes required for fast flow: maintainability, testability, deployability, observability, and evolvability. In such a scenario, it makes sense to migrate to a microservice architecture.

Chapter 1 introduced the microservice architecture. It defines this architecture style, explains how it enables fast flow, and describes its benefits and drawbacks. This chapter builds on that foundation. It digs deeper into the definition of the microservice architecture, drawing on the concepts from chapter 3, which covered architecture fundamentals, and chapter 4, which described the four flavors of loose coupling. This chapter also takes a closer look at how the

microservice architecture supports the five quality attributes required for fast flow described in chapter 5. It examines in more depth the drawbacks and challenges of adopting and using the microservice architecture. Finally, this chapter introduces the microservice architecture pattern language — a set of patterns for solving recurring design and implementation challenges.

Let's begin by briefly recapping the definition of microservice architecture.

7.1 What is the microservice architecture exactly?

The term "microservice architecture" is widely used but often inconsistently defined. It means different things to different people. It's often used loosely to describe systems built from multiple services, without much agreement on what qualifies as a microservice architecture. There's also an excessive focus on the size of services.

This ambiguity makes it harder to reason about the architecture's benefits, drawbacks, and trade-offs. Let's first recap my definition of the microservice architecture from chapter 1. After that, we will look at other commonly used definitions. I'll compare them to mine and explain why I believe my definition better captures the essential characteristics of the microservice architecture. This comparison sets the foundation for the rest of the chapter, where I dive deeper into the microservice architecture, explore its benefits, drawbacks and associated pattern language.

7.1.1 My definition of the microservice architecture

In chapter 1, I defined the microservice architecture as follows:

The microservice architecture is an architectural style that organizes an application's subdomains into two or more components (a.k.a. services) that are independently deployable and loosely coupled.

I also described how that definition is a bundle of five key concepts:

- It's an architectural style
- The application consists of two or more services, each of which is a component, a.k.a. deployable unit
- The services consist of subdomains
- The services are loosely coupled
- The services are independently deployable

This definition, however, is not the only definition in use. Let look a couple of other popular definitions.

7.1.2 Some other popular definitions

The original and widely cited definition comes from Martin Fowler and James Lewis

(<https://martinfowler.com/articles/microservices.html>)

In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.

This isn't the only definition. Adrian Cockcroft (<https://www.slideshare.net/slideshow/dockercon-state-of-the-art-in-microservices/42344423#42>) offers a much more concise definition:

Loosely coupled service-oriented architecture with bounded contexts

There are also definitions that emphasize the micro in microservices, suggesting that services should be tiny — perhaps only 10-100 lines of code, or no more than two weeks of development work. One motivation for having small services is replaceability as a defining characteristic: a service should be small enough to throw away and rewrite without significant effort.

While these definitions highlight important aspects of the microservice architecture, they don't fully capture the qualities that I believe are essential — especially for enabling fast, sustainable flow. Let's take a closer look at how the Fowler and Lewis's definition compares with mine.

7.1.3 Comparing two definitions of the microservice architecture

Let's first consider the similarities between my definition and Fowler's and then look at the differences. Here are the concepts that both definitions share:

- Organized around business concepts - both my definition and Fowler's describe how services are modeled on the business. Fowler uses the term business capabilities, whereas my definition uses the term 'subdomains', which are implementable slices of business functionality. However, chapter 20 describes how subdomains can be derived from business capabilities.
- Independent deployability - both definitions describe independent deployability
- Automated deployment - while my definition doesn't explicitly mention automation, chapter 5 describes how fast flow requires an automated deployment pipeline
- Multiple technology stacks - while my definition doesn't explicitly mention the possibility of multiple technology stacks, it's implied by the fact that services are components.

As you can see, the two definitions have a lot in common.

There are, however, some important differences:

- Size - I don't explicitly mention the size of services. While the name 'microservice' implies that services should be small, I believe that is a source of confusion that's led some organizations to create excessively complicated, fine-grained architecture. The goal is to have services that satisfy the fast flow quality attributes defined in chapter 5 - not small services.
- Replaceability - while evolvability, which includes the ability to upgrade or change a service's technology stack is important, I don't consider replaceability an important goal of the microservice architecture, especially if it

requires lots of little services. Moreover, even though a component has a single technology stack, any of its elements - such as a subdomain or an element of a subdomain, such as a package - are replaceable if the element preserves its API.

- Communication technologies - I don't reference a particular communication technology since, as I describe in chapter 3, there are a wide variety of options, including REST, and messaging with different trade-offs.
- Loose coupling - my definition adds loose coupling (along with independent deployability) as a defining characteristic of the microservice architecture, since it's essential for fast flow (and, more generally, good software). I described the concept of loose coupling in chapter 4. Later in [Section 7.3.2](#), I describe why it's particularly important in a microservice architecture.

Now that we have compared and contrasted the two definitions of the microservices, let's explore the microservice architecture in more detail.

7.2 There's more to the microservice architecture than just services

The definition of the microservice architecture described earlier is a good start. Services are one aspect of the microservice architecture - after all, 'service' is in the name. But this definition only scratches the surface. Most notably, it only focuses solely on the component view of the architecture, as described in chapter 3.

The microservice architecture has three other important aspects:

- System operations - system operations exist in the space between services. They are especially important because distributed system operations - operations that span services - are much more complicated than local ones and can constrain how subdomains are partitioned into services.
- Build architecture dimension - the requirement for independently deployable services also shapes the build dimension: source repositories, build projects, and deployment pipelines.
- Service-team relationship - The right relationship between services and teams is essential for fast flow.

In many ways, the definition of microservices is multidimensional - in the same way that architecture itself is multidimensional.

Let's look at each of these aspects of the microservice architecture, starting with what a service is and how services are organized within a microservice-based application.

7.2.1 Services in a microservice architecture

Figure [7.1](#) shows the component view of a simple microservice architecture. The application consists of three services - Order Service, Delivery Service, Consumer Service - along with an API Gateway.

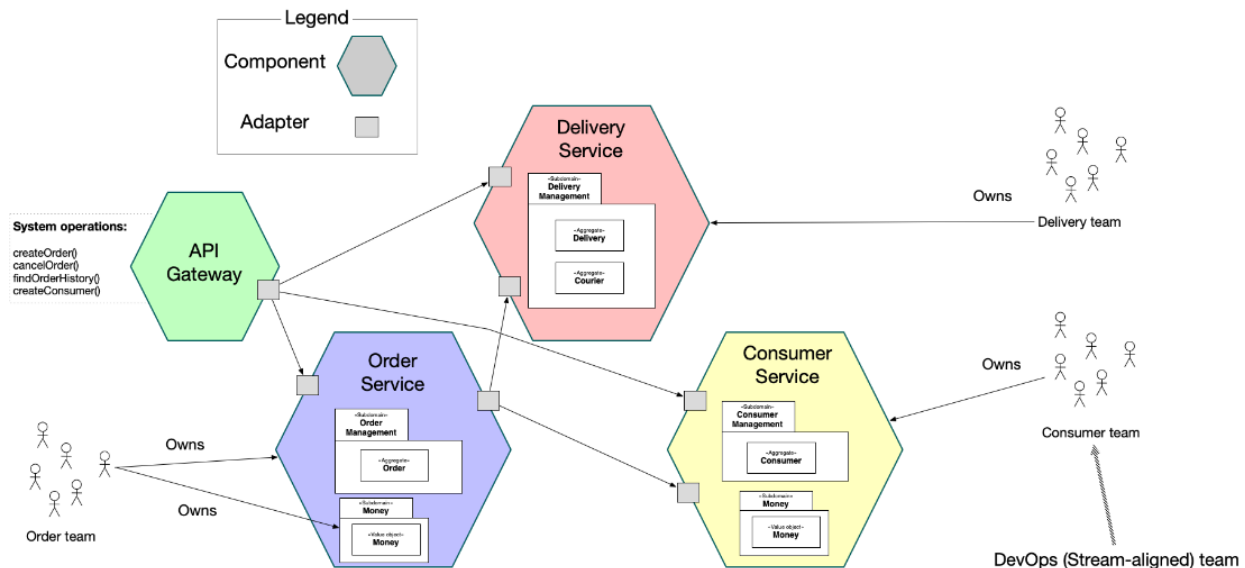


Figure 7.1 This application which consists of three services - Order Service, Delivery Service and Consumer Service - is a simple example of a microservice architecture, which is an architectural style that organizes an application's subdomains into two or more components (a.k.a. services) that are independently deployable and loosely coupled

In practice, a microservice architecture usually has many more services. But during a migration from a monolith, an application may contain just a few:

- the monolith - a huge service
- services extracted from it
- service that implement new functionality

This architecture also has an API Gateway, which routes external traffic to the services. The API Gateway pattern is covered in more detail in chapter 12.

This example illustrates the high-level structure of a microservice architecture. Let's now take a closer look at the services themselves — how they're defined, how they interact, and how they relate to subdomains.

7.2.2 The structure of a service

Figure 7.2 shows the structure of an individual service. A service consists of one or more subdomains. It has an API, which consists of operations and published events. The service might consume the APIs of other services. It typically has a database. However, it's a private implementation detail, hidden behind the service's API.

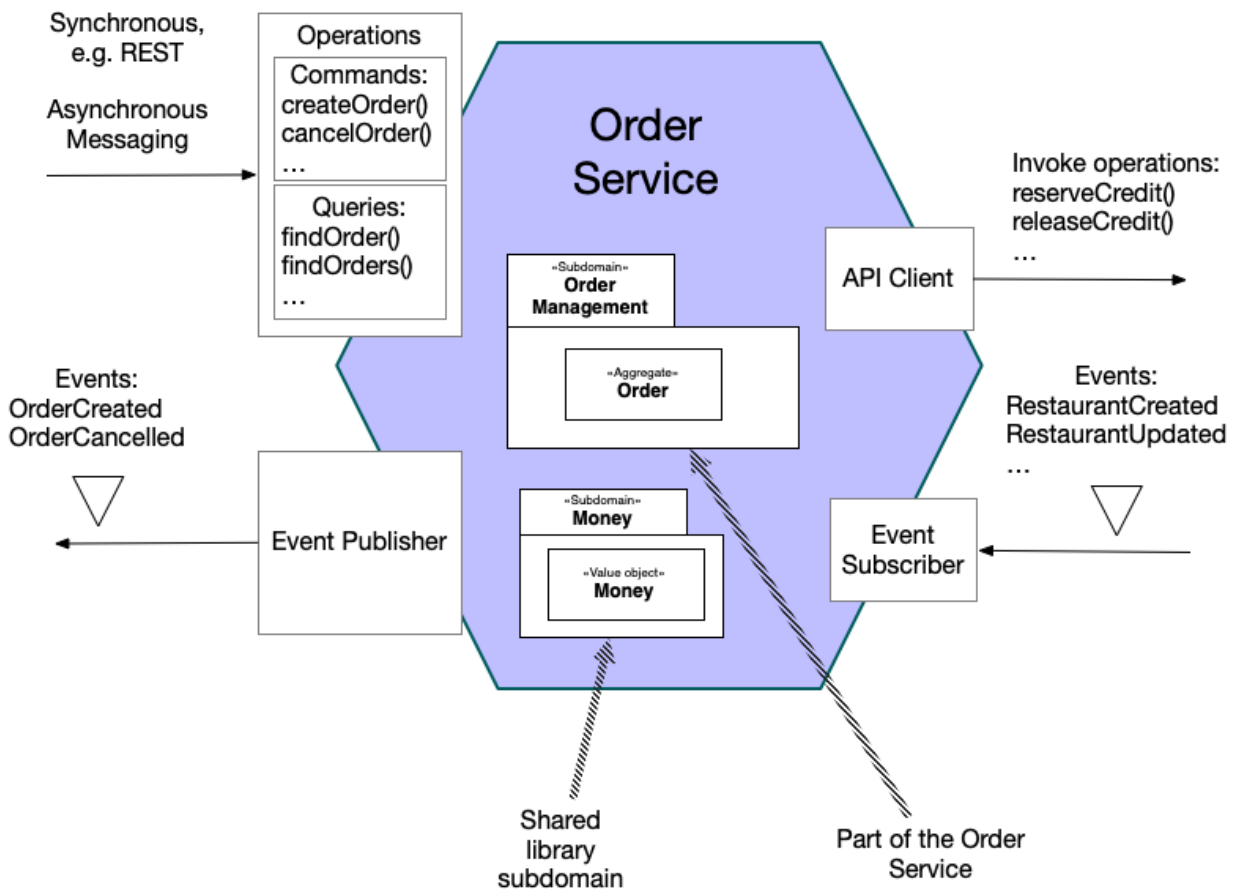


Figure 7.2 A service consists of one or more subdomains. It has an API consisting of operations and events. It collaborates with other services by invoking their operations and consuming their events.

The `Order Service` consists of the `Order Management` and `Money` subdomains. The `Order Management` subdomain resides solely in the `Order Service`. The `Money` subdomain is, however, a shared library used by multiple services.

A service's API consists of operations and events. Operations can be invoked synchronously or asynchronously. Some of a

service's operations might correspond to system operations. For example, the `Order Service` has a `createOrder()` operation, which is the implementation of the `createOrder()` system operation. Similarly, the `Consumer Service` has a `createConsumer()` operation, which corresponds to the `createConsumer()` system operation.

Other service operations exist only to enable collaboration with other services. For example, the `Consumer Service` has a `reserveCredit()` operation, which is invoked by the `Order Service`'s `createOrder()` operation to reserve credit for an order.

7.2.3 Operations in a microservice architecture

System operations are a fundamental part of a microservice architecture. As figure [7.3](#) shows, a microservice architecture has two kinds of system operations:

- local operations - handled entirely by a single service
- distributed operations - span multiple services

A distributed operation has an entry point, which is the service that initially handles the request. That service then collaborates with one or more other services to fulfill the request.

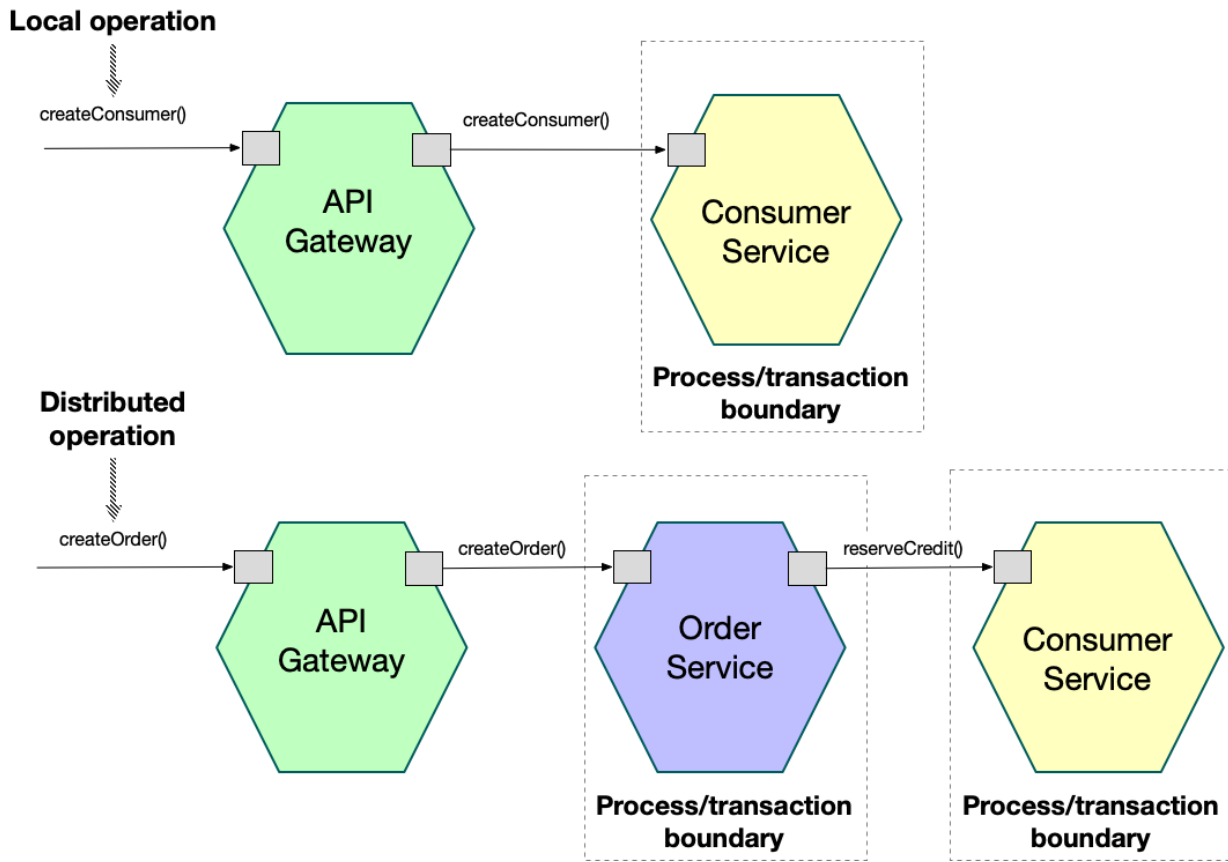


Figure 7.3 A microservice architecture has two types of system operations: local operations, such as `createConsumer()`, that are implemented by a single service, and distributed operations, such as `createOrder()`, that involve collaboration between services

Distributed operations are usually much more complicated to implement than local operations. I'll explain the reasons why in detail in [Section 7.5.1](#).

7.2.4 Build architecture dimension

Another important aspect of the microservice architecture is the build architecture dimension. You can structure the build architecture dimension of a microservice architecture in a few different ways, but a key requirement is that it must support independently deployable services, which I describe in more detail in [Section 7.3.1](#).

One common build architecture - the multi-repo - is shown in figure 7.4. Each service has its own code repository, build project and deployment pipeline. Not only is each service tested and deployed independently, but you can scale the build architecture by simply adding more repositories and pipelines. The main drawback is that making global changes, such as dependency upgrades, can be difficult to coordinate across multiple repositories.

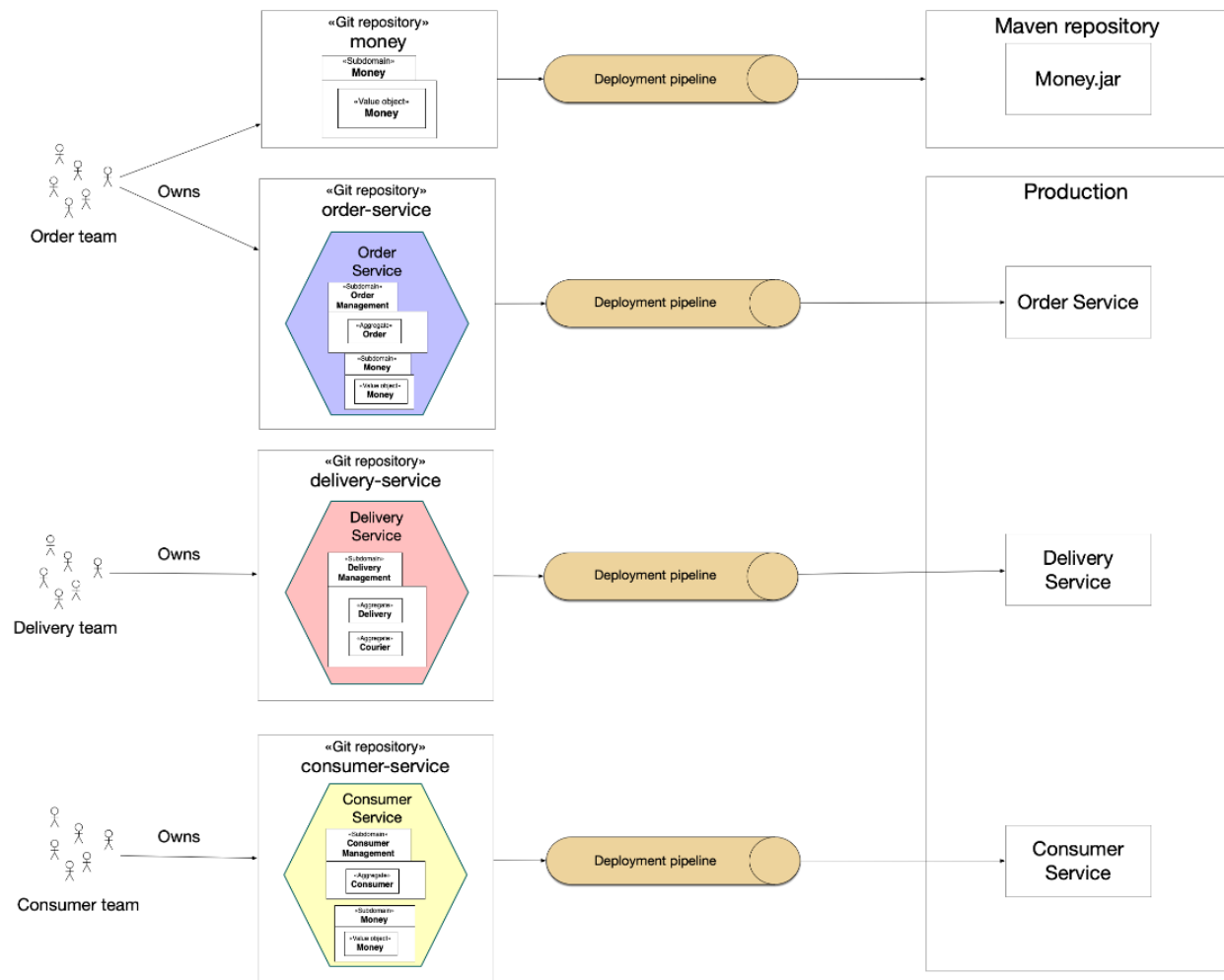


Figure 7.4 Each service has its own deployment pipeline, which builds, tests and deploys the service. The Money Library subdomain has its own deployment pipeline that builds and tests the library and publishes it to a Maven repository.

An alternative build architecture is the monorepo, which I introduced in chapter 3. A monorepo is a single code

repository that contains all of the services. Its main benefit is that all the code is in one place, which makes it easier to make global changes. However, it can be challenging to implement truly independently deployable services in a monorepo. It can also be challenging to scale to a large number of developers who make frequent changes to a large code base. It typically requires investing heavily in the build system, which can be a distraction from delivering business value.

7.2.5 Services $\Leftrightarrow$ teams

Another important aspect of the microservice architecture is the relationship between services and teams. Ideally, each service is owned by a single team. Since the service is loosely design-time coupled to other services, the team is free to develop, test and deploy the service independently of other teams, which maximizes the team's autonomy and so accelerates the flow of features into production.

It's important to keep in mind, however, that there's not a strict one-to-one relationship between services and teams. A service might have multiple subdomains, each of which might be owned by a different team. For example, in architecture shown in figure [7.1](#), the `Order Service` consists of just one subdomain, which is owned by the `Order Management` team. However, the `Consumer Service` consists of two subdomains, which are owned by two different teams. The `Consumer Management` subdomain is owned by the `Consumer Management` team and the `Money` subdomain, which is a shared library, is owned by the `Order Management` team.

In this example, `Money` is a shared library. But, as I explain in chapter 20, there are other scenarios where two non-library subdomains that are owned by different teams need to be collocated in a single service. For example, the cost of

communication over a network might be so high that it prevents the subdomains from being packaged as separate services, even if doing so it reduces the autonomy of the two teams.

These additional aspects — system operations, build architecture, and the relationship between services and teams — highlight that there's more to the microservice architecture than just a collection of loosely coupled services. With that broader context in place, we can now look at two essential characteristics of the microservice architecture.

7.3 Essential characteristics of the microservice architecture

So far, we've looked at the structure of services, how they relate to subdomains and teams, how system operations are implemented, and how services are built and deployed. These are all important aspects of a microservice architecture. But to achieve its promised benefits, an architecture's services must be independently deployable and loosely coupled. Without these characteristics, you risk ending up with a distributed monolith — an architecture that combines the downsides of both monolithic and distributed systems. Unfortunately, this is a mistake many organizations make.

Let's start by examining what it means for services to be independently deployable. After that, we'll look at the importance of loosely coupled services.

7.3.1 Essential characteristic #1: Services are independently deployable

In chapter 1, I introduced the concept of independently deployable services and explained their importance in a microservice architecture. I described how there are several aspects to the concept of independently deployable services: A service is independently deployable if:

- It is a component - a deployable unit.
- It has its own deployment pipeline - one that builds, tests and deploys the service.
- It is production-ready after it has been tested in isolation.

To understand what it means for a service to be independently deployable - and why that matters - let's look at each of these aspects in turn. After that, we'll look at the benefits of independently deployable services and how that affects acceptance testing.

A SERVICE IS A COMPONENT

The simplistic definition of 'independently deployable' is a service that it is a component. As I described in chapter 3, examples of components include, executable JAR file, operating system executable, a container image, or a Zip file containing an AWS Lambda function. This is quite different from a library, such as regular JAR file, that needs to be packaged with other libraries to form a component.

However, while packaging a service as a component satisfies the literal definition of 'independently deployable', it is woefully inadequate. Let's look at another aspect of independently deployable services: a deployment pipeline per service.

A SERVICE HAS ITS OWN DEPLOYMENT PIPELINE

In order for a service to be independently deployable, it must have its own deployment pipeline. Without its own deployment pipeline, a service can only be deployed with one or more other services, which requires coordination between the teams that own the services and reduces their autonomy. Moreover, a shared deployment pipeline means that the failure of one service's tests could prevent the other services from being deployed.

As mentioned earlier, while it's possible to have a monorepo - a single repository that contains all the services - the most straightforward build architecture for supporting independent deployability is the multi-repo, as shown earlier in figure [7.4](#). In this example, the `Order Service`, `Delivery Service` and `Consumer Service` each have their own code repository and deployment pipeline that builds, tests and deploys the service. The `Money Library` subdomain has its own code repository and deployment pipeline that builds, and tests the library and publishes it to a Maven repository.

SERVICES ARE PRODUCTION READY AFTER BEING TESTED IN ISOLATION

The third and final aspect of independently deployable services is that they are production-ready after being tested in isolation. As figure [7.5](#) shows, what emerges from the service's deployment pipeline that is a service that can and should be deployed into production. The service's tests will invoke the service in the same way as its clients. And, if the service has collaborators, then they are simulated using test doubles (<http://xunitpatterns.com/Test%20Double.html>).

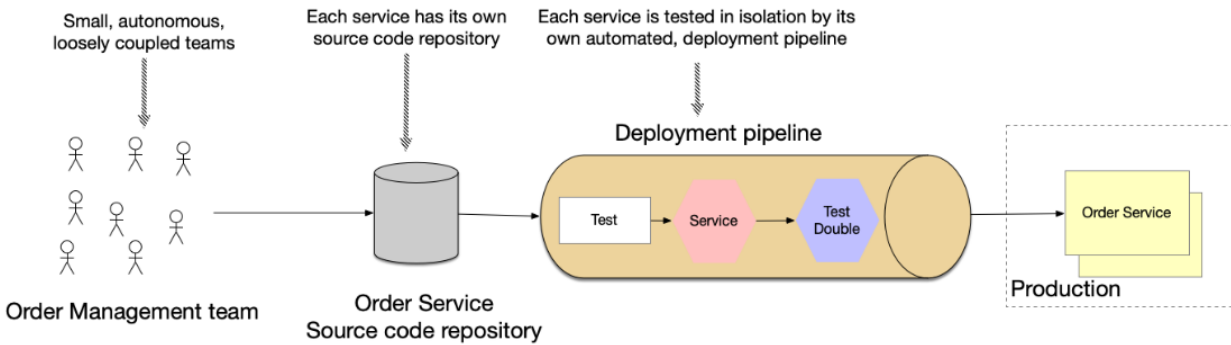


Figure 7.5 A service must be independently deployable: packaged as a deployable unit, have its own deployment pipeline and be production-ready after it has been tested in isolation.

One challenge with testing in isolation is correctly simulating the behavior of the service's clients and collaborators. Tests must invoke the API - e.g., REST endpoints and request and response schemas - that's expected by the service's clients. Similarly, each test double must simulate the collaborator's API.

There are two parts to the solution. The first is intentionally designing services to be testable in isolation. APIs must be simple, well-defined and stable. This will ensure that clients and collaborators are easier to simulate.

The second part to the solution is to use contract testing, which I introduce below. Contract tests accurately simulate a service's clients and collaborators. They will fail unless a service implements the API that clients expect and uses the collaborators' APIs correctly.

Now that we have covered the three aspects of independently deployable services, let's look at how to verify that services can communicate even though we do not test them together.

USE CONTRACT TESTS TO VERIFY SERVICE INTERACTIONS

One obvious benefit of end-to-end testing is that it verifies whether services - such as the `Order Service` and `Consumer Service` - collaborate correctly. For example, if the `Consumer Management` team makes a breaking change to the `Consumer Service`'s API, end-to-end tests will fail if the `Order Service` hasn't yet been updated to use the new API. But if services are independently deployable, you need a way to verify that they collaborate correctly that does not involve testing them together.

The solution is to use consumer-driven contract tests. As you will see in chapters 15 and 16, contract tests verify that a service - such as the `Consumer Service`, and its clients, including the `Order Service` - agree on the service's API, even though they are tested separately. A contract test will fail if a change to the service's API breaks the contract, or if a client uses the service's API in a way that violates the contract.

One downside of contract tests is the overhead of maintaining them. But this effort is worth it because contract testing makes independently deployability possible. I once encountered an organization that abandoned contract tests because they were "too much work" and fell back on application-level end-to-end tests. But I strongly suspect that the root cause was their excessively fine-grained and tightly design-time coupled architecture. The fine-grained architecture increased the number of contracts that needed to be maintained, while the tight design-time coupling meant the contracts were constantly changing. This example highlights the importance of the microservice architecture's other essential characteristic: loose coupling, which I describe shortly.

BENEFITS OF INDEPENDENTLY DEPLOYABLE SERVICES

Independent deployability has two main benefits:

- It accelerates, simplifies and increases the reliability of the deployment pipeline
- It reduces the need for cross-team coordination

Let's look at each of these benefits in turn.

The first benefit of independent deployability is that it accelerates, simplifies and increases the reliability of the deployment pipeline by eliminating slow, complicated, and unreliable end-to-end tests. Imagine writing an end-to-end test for a service that retrieves orders from the `Order Service`. Not only would it need to deploy the `Order Service`, but it would have to deploy and initialize all of its dependencies. It would, for example, need to create a `Consumer` in the `Consumer Service` before it could create an `Order` in the `Order Service`. Such a test is likely to be slow and complicated.

Worse, it might fail randomly due to the complexity of the application. Teams would need to investigate the failure to determine whether it can safely be ignored. This introduces a frustrating manual step into what should be a fully automated deployment pipeline. Application-level end-to-end tests that are run before releasing the entire application are even worse.

In contrast, a service-level test that tested the service in isolation would only need to deploy the `Order Service`, create one or more orders, and then query the `Order Service`. Any dependencies of the `Order Service` that are needed to create orders, such as the `Consumer Service`, would be simulated using test doubles. The test would be relatively simple, fast and reliable.

The second benefit of independent deployability is that it reduces the need for cross-team coordination. When services are tested together, a failure caused by one team's change can block another team's release. Teams must also collaborate to debug failures of application-level end-to-end tests. This tight coupling slows down the teams and limits each team's ability to work independently.

Together, these benefits make independent deployability a major enabler of fast, sustainable flow. It allows teams to move quickly and safely, releasing changes without waiting on or being blocked by other parts of the organization. Without it, the benefits of microservices are much more difficult to achieve. Yet, as I describe below, a common antipattern of microservices adoption is an organization insisting on testing the application's services together.

As you can see, independent deployability is an important characteristic of the microservice architecture. Without it, you have a distributed monolith. Yet sadly, I've encountered numerous organizations that insist on testing the application's services together. Let's look at one of the most common obstacles to independently deployable services: application-level user acceptance tests.

7.3.2 Essential characteristic #2: Services are loosely coupled

The second essential characteristic of the microservice architecture is that the services are loosely coupled. As I described chapter 4, there are four flavors of coupling:

- Design-time coupling - the likelihood that a change to one software element will require a change to the other element.

- Build-time coupling - when changing one element requires another element to be recompiled and retested.
- Runtime coupling - when one element's failure causes another element to fail.
- Infrastructure coupling - when two elements share infrastructure, such as a database, and one element's behavior affects the other element

Let's look at each of these flavors of coupling in the context of a microservice architecture, starting with design-time coupling.

SERVICES MUST BE LOOSELY DESIGN-TIME COUPLED

Design-time coupling between services is a serious concern because of the high cost of lock step changes - changes that must be coordinated across multiple services, or worse, across multiple teams. Tight design-time coupling significantly reduces an application's modifiability is a major obstacle to fast flow.

For example, let's imagine that the `Consumer Management` team needs to make a breaking change to the `Consumer Service's` API. They can't simply make the change and deploy the new version since that would break its clients - such as the `Order Service` - and cause a production outage. Instead, the team must:

1. Add a new version of the `Consumer Service's` API alongside the current version.
2. Ask the teams that own the client services - including the `Order Service's` team - to migrate to the new version.
3. Once all clients have migrated, the `Consumer Management` team can delete the old version.

As you can see, breaking API changes are a major burden - a topic I explore further in chapter 3. And, as I describe below, one of the inherent risks of the microservice architecture. That's why loose design-time coupling is essential.

SERVICES SHOULD BE LOOSELY RUNTIME COUPLED

Runtime coupling can also be a significant issue in a microservice architecture. Tight runtime coupling doesn't prevent an architecture from enabling fast flow. But as chapter 4 describes, it increases latency and reduces availability. You must carefully design your service architecture to prevent runtime coupling from becoming a problem. When defining a service architecture, you might find that to avoid runtime coupling, you will need to colocate multiple subdomains in a single service. Chapter 20 discusses this in more detail.

BUILD-TIME COUPLING IS USUALLY NOT AN ISSUE

Build-time coupling is usually not an issue in a microservice architecture. Since services are independently deployable components that are built and tested separately, there is no build-time coupling between services. There might be build-time coupling within a service, but the codebase is typically small, and the deployment pipeline remains fast. But if necessary, you can often accelerate the pipeline by applying physical design principles I described in chapter 6 to organize the codebase.

INFRASTRUCTURE COUPLING IS MOSTLY A MATTER OF CHOICE

The degree of infrastructure coupling depends on the chosen deployment architecture. For example, an organization can choose whether to use a shared database server or a database service for each service. This decision involves trade-offs between the benefits of shared infrastructure and the drawbacks of infrastructure coupling.

We've now looked at the essential characteristics of the microservice architecture: independently deployable services and loose coupling. These are not just abstract ideals — they're what make the architecture valuable in practice. Let's now examine the benefits of a well-designed microservice architecture.

7.4 Benefits of the microservice architecture

The microservice architecture offers numerous benefits. Its most important benefit is that it enables fast flow. There are, however, several other benefits:

- Enables an organization to scale
- Improves the developer experience
- Enables segregation of subdomains by characteristics
- Supports multiple technology stacks

In this section, I describe each benefit in more detail. But first, let's look at how the microservice architecture enables fast flow.

7.4.1 Enables fast, sustainable flow

The biggest benefit of the microservice architecture is that it enables fast flow by satisfying the quality attributes for fast flow: modifiability, evolvability, testability, deployability and

observability. Let's look at how the microservice architecture satisfies each of these attributes.

HIGH MODIFIABILITY DUE TO LOOSE DESIGN-TIME COUPLING

One key reason why the microservice architecture supports fast flow is that it has high modifiability. By definition, the services are loosely design-time coupled. A change to one service rarely requires a change to another service. As a result, the teams that own the services work independently of one another with a high degree of autonomy.

Of course, designing services that are loosely design-time coupled requires discipline and effort. Moreover, there's a risk that as a microservice architecture evolves, design-time coupling will increase. You must be vigilant and proactive in managing dependencies between services.

HIGH EVOLVABILITY DUE TO A MULTI-COMPONENT ARCHITECTURE

Another reason why the microservice architecture supports fast flow is that it has high evolvability. As chapter 5 describes, a multi-component architecture, such as the microservice architecture, is more evolvable than a monolithic architecture. The application's technology stack can evolve incrementally, one component - or in this case, one service - at a time. Upgrades are broken down into smaller, per-team efforts that are easier to schedule and prioritize based on risk and need. Also, as I describe below in [Section 7.4.2](#), the microservice architecture enables each service to use a technology stack that is best suited for what it needs to.

HIGH TESTABILITY DUE TO SMALLER, INDEPENDENTLY TESTABLE SERVICES

The third reason the microservice architecture supports fast flow is that it has high testability. Since the application consists of services that are small and testable in isolation without complex and slow end-to-end tests, it's almost always possible to verify behavior using fast-running automated tests that execute locally. Each service also has its own deployment pipeline, which typically handles a relatively low volume of changes. As a result, testing is unlikely to become a bottleneck.

Moreover, the microservice architecture allows subdomains that are inherently difficult to test — such as those requiring manual testing, slow-running test suites, or specialized environments — to be isolated in their own services. This ensures that the testability of other services is not compromised.

HIGH DEPLOYABILITY

The fourth reason the microservice architecture supports fast flow is that it has high deployability. Each service is relatively small and independently deployable, which makes deployments fast and reliable. Teams can deploy their services without coordinating with others and perhaps concurrently. As a result, it's unlikely that deployment will be a bottleneck and changes can flow into production frequently and reliably.

HIGH OBSERVABILITY

The fifth and final reason the microservice architecture supports fast flow is that it has high observability. Since each service instance emits a stream of telemetry, the

resulting data is more fine-grained than in a monolithic architecture, where it's emitted by each application instance. This granularity provides deeper insights into the application's behavior. For example, if a service contains a single subdomain, its telemetry is specific to that subdomain. This can make it easier to understand resource utilization and diagnose problems.

As you can see, the microservice architecture enables a large application - developed by many teams - to satisfy the architectural requirements for fast sustainable flow. While enabling fast flow is the primary benefit, it's not the only one. Let's look at the other benefits, starting with how it enables an organization to scale effectively.

7.4.2 Supports multiple technology stacks

Not only does the multi-component nature of the microservice architecture improve evolvability, but it also enables each service to use a technology stack that's best suited for what it needs to do - the so-called, right tool for the right job strategy. This does not mean, however, complete anarchy. To properly benefit from the freedom to choose, an organization must still carefully manage the set of approved technologies.

Another related benefit of the microservice architecture is that experimenting with new technologies is easier and safer. For example, a team can implement a service using a new technology stack and deploy it into production. If the experiment succeeds, the organization has validated a new stack for future use. Otherwise, if the experiment is unsuccessful, then it's not a disaster. If it fails, the impact is limited — valuable lessons are learned with relatively little effort. The team can then simply reimplement the service

using the existing, proven technology stack when convenient.

7.4.3 Enables an organization to scale

Another important benefit of the microservice architecture is that it enables an organization to scale effectively without sacrificing fast flow. Even a well-structured monolith eventually becomes an obstacle to fast flow as the organization grows. A single deployment pipeline cannot handle the volume of commits and becomes a bottleneck, and tight design-time coupling makes matters worse. Adding developers can actually slow things down due to the increased overhead of coordinating the work of multiple teams. The book *Wiring the Winning Organization* by Gene Kim and Dr. Steven J. Spear (IT revolution press, 2023) describes how the pace of software delivery at Amazon.com slowed to a crawl in 2002 despite their growth.

In contrast, the microservice architecture enables sustainable scale. New teams can take ownership of new services with minimal coordination costs — something that's impossible to achieve in a tightly coupled system. Each service has its own deployment pipeline, and so the build architecture scales as the organization grows.

As a result, the organization can continue to deliver features rapidly, even as it grows. *Wiring the Winning Organization* described how Amazon overcame its software delivery problems by adopting a microservices-style architecture - referred to in the book as modularization. By 2025, Amazon was reportedly doing 130,000 deployments per day.

7.4.4 Improves the developer experience

The microservice architecture also improves the developer experience. As figure 7.6 shows, teams mostly work independently and only occasionally need to collaborate with other teams to implement distributed operations. For example, they might be asked to enhance their service's API, or they might need to request changes to another team's service. As a result, a developer can focus primarily on their service's business logic and is largely shielded from the complexities of the rest of the application.

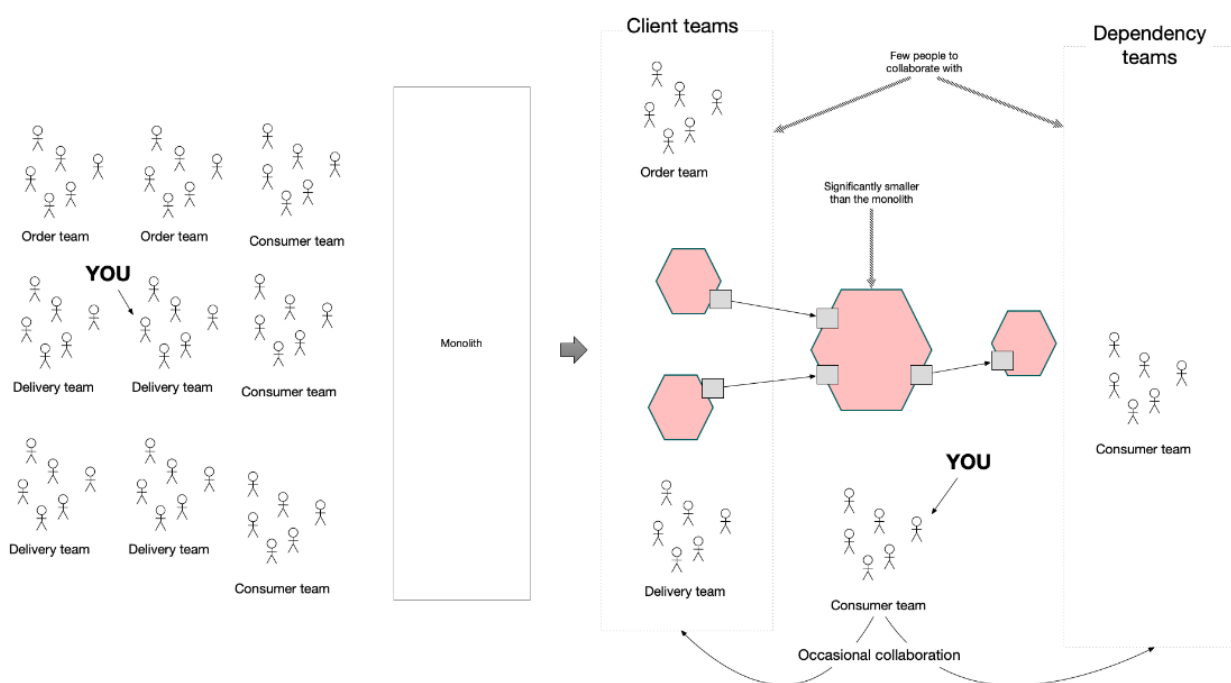


Figure 7.6 From the perspective of a service developer, their world is far simpler than if they were working on a monolith. They mostly work independently with their team-mates and only occasionally need to collaborate with other teams to implement distributed operations.

Because a service developer is, in effect, working on a small, self-contained application, they benefit from a significantly improved developer experience. Their cognitive load is significantly reduced thanks to the smaller code base. They receive much faster feedback because the service's tests and deployment pipeline execute quickly. And, they belong to a small team that works mostly independently and only occasionally interacts with other teams. As a result,

developers experience more flow and are much more productive.

7.4.5 Enables segregation of subdomains by characteristics

Yet another benefit of the microservice architecture is that it enables you to improve an architecture by segregating subdomains based on their characteristics. A subdomain has the following characteristics:

- resource requirements
- domain-specific software development regulations
- business criticality
- security requirements
- Domain-driven design (DDD) type (i.e., core, supporting, generic)

Putting subdomains with differing characteristics into separate services can sometimes improve the architectures in various ways including ease of development, scalability, availability, and security. Let's look at each characteristic in turn starting with resource requirements.

RESOURCE REQUIREMENTS

A subdomain requires certain resources, such as CPU, GPU, memory, network bandwidth, storage, etc. It also needs a certain number of instances to achieve the desired throughput and availability. However, the unit of scaling in an application is a service - or more precisely, a service instance. As a result, when a service contains multiple subdomains with differing resource needs, those subdomains are scaled together, which risks over provisioning resources for some of them. By packaging those subdomains as

separate services, you can scale them independently by, for example, using different EC2 instance types. This approach improves resource utilization and can reduce costs, especially when operating at high scale.

To understand this benefit, consider the scenario shown in figure [7.7](#). There are two subdomains:

- `Order Management`, which has 'normal' resource needs
- `Fraud Detection`, which needs GPUs to run machine learning algorithms

To achieve the desired throughput, the application needs 10 instances of `Order Management`, but only 5 instances of `Fraud Detection`.

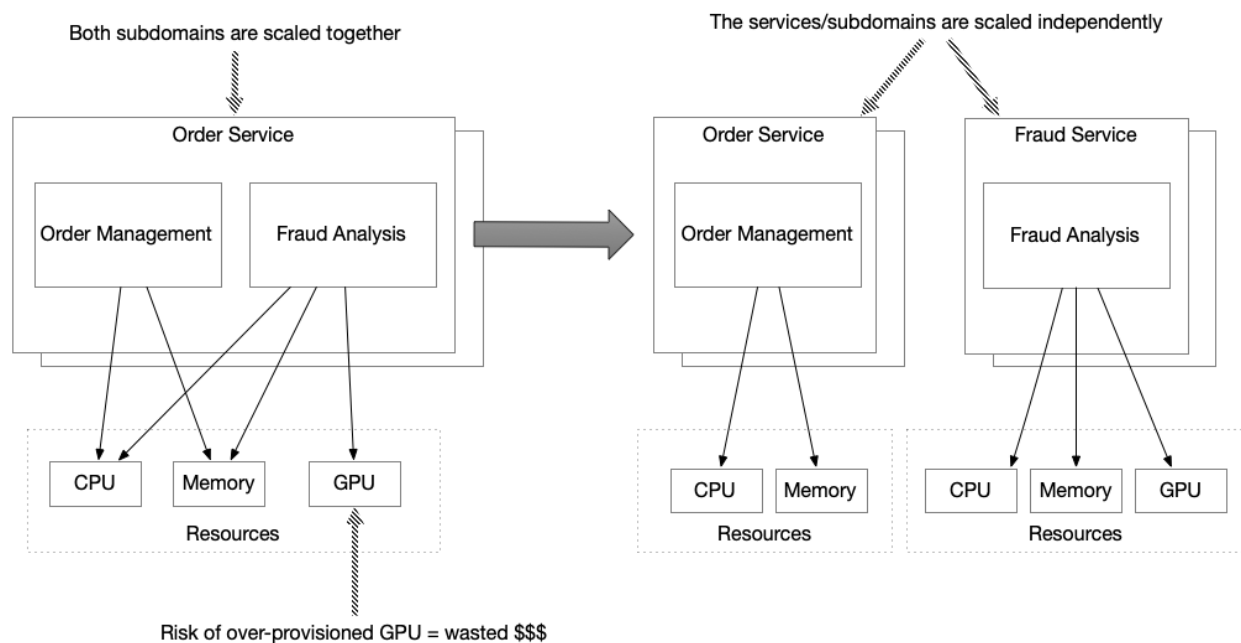


Figure 7.7 Subdomains within the same service are scaled together, which can require over-provisioning of some resources. Putting them in separate services enables them to be scaled independently which improves resource utilization and reduces cost.

If both subdomains are packaged as an `Order Service`, we need to provision 10 instances of the service. This means

that 5 instances of `Fraud Detection` will be underutilized, which is particularly costly if those instances require specialized hardware. For example, `p4d.24xlarge` EC2 instances, which have GPUs, are 8x the price of `m5.24xlarge` instances. By segregating these two subdomains into separate services, you can scale them independently - greatly reducing the cost of scaling.

DOMAIN-SPECIFIC SOFTWARE DEVELOPMENT REGULATIONS

Software development in some domains is subject to strict regulations. For example, if you are developing software that's classified as Software as a Medical Device, your development process must comply with standards such as ISO 13485 and ISO/IEC 62304. These standards have requirements that are incompatible with DevOps-style development.

However, in many applications, only a subset its subdomains are regulated. In such cases, you can segregate the regulated subdomains into separate services. This enables you to develop the unregulated subdomains using DevOps practices without being constrained by the stricter standards.

BUSINESS CRITICALITY - AVAILABILITY

In addition to optimizing resource allocation and complying with regulatory requirements, it's often helpful to segregate subdomains based on their business criticality to improve availability and reduce operational risk. That's because some of your application's subdomains are likely to be more business-critical than others. For example, if you are a credit card payment processing company then the `Credit Card`

`Processing` subdomain that handles credit card authorization and charge requests is critical to the business. That's how you make money, and you most likely have strict SLAs with your customers. Any significant downtime of a service containing a business-critical subdomain would be catastrophic.

In contrast, other subdomains, such as `Merchant Account Management`, are likely to be less critical. But if it's packaged with the `Credit Card Processing` subdomain in the same service, then it can affect the availability of the `Credit Card Processing` subdomain. A change to `Merchant Account Management` might, for example, could introduce a bug into the `Credit Card Processing` subdomain. This risk arises because the two subdomains share the same codebase and library dependencies. There's also a risk of resource contention, where the less critical `Merchant Account Management` subdomain consumes memory or CPU needed by `Credit Card Processing`.

We can, therefore, improve availability by segregating subdomains by their business criticality. The credit card payment application could, for example, have a `Payment Service`, which contains only the `Credit Card Processing` subdomain. This service has its own code base, runs on its own infrastructure and is runtime decoupled from the other services. As a result, it's unaffected by problems in other parts of the application. Moreover, developers working on less critical functionality don't risk unintentionally impacting high-criticality services.

Segregating services by business criticality can protect vital functionality from disruption and ensure higher availability for the most important parts of your system. Next, let's look at how security requirements can also influence service boundaries.

SECURITY

The security of some applications can be improved by segregating subdomains that manage sensitive data, such as Personally Identifiable Information (PII), from those that don't. When sensitive data is isolated in its own service, it can only be accessed via the service's API, which acts as a security boundary. In addition, you can use network-level security controls - such as firewall rules, service mesh rules, and private subnets - to restrict access further. This architectural separation reduces the attack surface and makes it harder for attackers to access sensitive data.

DDD SUBDOMAIN TYPE

In Domain-Driven Design (DDD), subdomains are classified into the following types:

- Core - the business logic that's unique to your business and provides competitive advantage. Core subdomains should be the focus on your development efforts.
- Supporting - the business logic that's common to many businesses but often requires customization. Ideally, you should be able to buy this functionality from a third party but sometimes you must write it yourself.
- Generic - the business logic that's common to many businesses but doesn't require customization. Generic subdomains should be bought from third parties.

You can accelerate the development of the core domains by packaging them as separate services. This isolates them from non-core subdomains, allowing teams to develop them independently and without distraction. For example, a core subdomain's service's technology stack can be upgraded without having to consider unrelated supporting or generic

functionality. This separation boosts the organization's ability to iterate rapidly on what's essential, without being slowed down by subdomains that don't drive differentiation.

As you've just seen, the microservice architecture enables fast, sustainable flow; improves the development experience; and improves the architecture by segregating subdomains by their characteristics. But as you might expect, it's not a silver bullet. This architecture has several drawbacks. Let's take a look at them.

7.5 Drawbacks of the microservice architecture

As I've just described, the microservice architecture has numerous benefits - benefits that stem from structuring an application as a set of loosely coupled, independently deployable services. Unfortunately, however, the fact that it's a distributed architecture is a double-edged sword. The very properties that make it so powerful also mean that it has significant drawbacks.

Chapter 1 briefly touched on some of these drawbacks. This section takes a deeper look at them, along with some other challenges:

- Distributed operations are complicated
- Operational challenges
- Implementing changes that span services is complicated
- Designing a good microservice architecture is hard
- Successfully adopting a microservice architecture is hard

Let's look at each drawback, starting with the complications of distributed operations.

7.5.1 Distributed operations are complicated

One significant drawback of the microservice architecture is that designing distributed operations is much more complicated than local operations. A local system operation can execute with a transaction and simply call methods on various classes - just like in a monolith. But as figure 7.8, services in a microservice architecture are both process and transaction boundaries. As a result, a distributed operation - one whose subdomains reside in different services - will span those boundaries, which makes it far more complicated to implement.

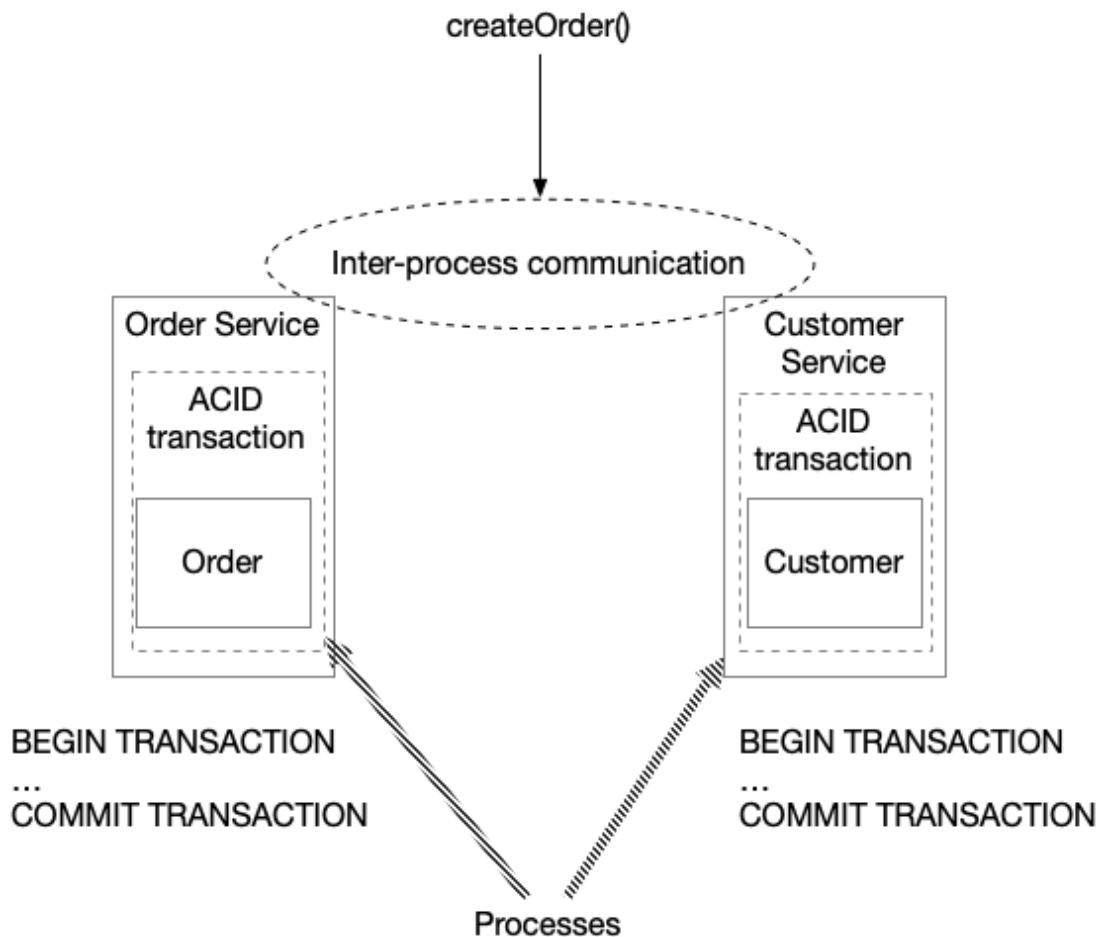


Figure 7.8 Services are process and transaction boundaries, so implementing a distributed operation is much more complicated than a local operation that simply spans different classes in different packages.

When designing a distributed system operation, you must contend with the following challenges:

- The cost and latency of network communication
- The risk of runtime coupling
- The complications of eventual consistency

And, these challenges are mostly unavoidable. Even if you design an architecture so that the most important operations are local to a single service, there will always be some operations that span services.

Let's look at each of the challenges of distributed operations in more detail.

THE COST AND LATENCY OF NETWORK COMMUNICATION

One challenge with distributed operations is the significant cost and latency of network communication. Compared to a method call - measured in nanoseconds - communication over a network is staggeringly expensive, typical in the milliseconds. What's more, a network bandwidth is finite and over-utilization will increase latency and reduce throughput.

As a result, it's essential to consider the cost of distribution when designing a microservice architecture. However, it's important to remember that the relative cost is not the same as actual impact. What matters is whether a distributed operation meets its Service Level Objective (SLO). If its latency is acceptable, then the cost of communication might not be an issue.

The real challenge arises when making an operation distributed would prevent it from meeting its SLO. This scenario involves a potentially difficult trade-off between

performance and the benefits of distribution, such as team autonomy. For example, imagine that `Order Management` and `Consumer Management` subdomains were packaged as separate services. If doing so causes `createOrder()` to exceed its latency SLO then you have two options:

- Accept the high latency and tolerate the impact on user experience
- Combine the two subdomains into a single service so that it can meet its SLO

The second option enables `createOrder()` to meet its SLO, but it comes at a cost: it reduces the autonomy of the two teams that own those subdomains. They must coordinate changes and deployments which can slow down development.

Designing a microservice architecture means making these kinds of trade-offs deliberately and with full awareness of their consequences.

RISK OF RUNTIME COUPLING

Another challenge with distributed operations is the risk of runtime coupling. As I described chapter 4, runtime coupling occurs when one service cannot respond to a request - the invocation of a system operation - without a receiving a response from another service. This dependency reduces availability and increases latency.

When designing a microservice architecture, it's essential to assess each distributed operation's potential for runtime coupling. In some cases, you can reduce or even eliminate an operation's runtime coupling by carefully defining service boundaries. For other operations, runtime coupling may be unavoidable unless you modify the operation's semantics to

allow partial outcomes or make the operation asynchronous. It's important to remember that it's only an actual problem if it causes an operation to not satisfy its SLOs.

It's important to remember that runtime coupling is only a problem if it prevents an operation from meeting its SLOs. If an operation is sufficiently performant and available, then any runtime coupling might not be an issue. But if it introduces significant latency or reduces availability, then you need to consider architectural changes.

COMPLICATIONS OF EVENTUAL CONSISTENCY

Another major challenge with distributed operations is transaction management. System operations typically read and write persistent data, and so need to be transactional. In a monolith, system operations are simply executed within an ACID database transaction (e.g., using Spring's `@Transactional`). If the operation is successful, the transaction is committed. Otherwise, it is rolled back.

But in a microservices architecture, transaction management is far more challenging. To understand why, consider the following scenario:

- `Consumer Service` - implements the local operation `createConsumer()`
- `Order Service` - implements the distributed operation `createOrder()`, which invokes `Customer Service.reserveCredit()` to reserve credit for the customer

A local operation, such as `createConsumer()`, can execute within a standard ACID transaction, just like in a monolithic application. The real challenge, however, is with distributed system operations, such as `createOrder()`, which span multiple services.

Implementing transaction management for distributed operations is complicated because of the need loose design-time coupling - a defining characteristic of the microservice architecture. Services must not access each other's databases directly. Instead, they collaborate via APIs. What's more, each service can have its own database server in order to improve scalability and reduce infrastructure coupling.

The traditional solution to distributed transactions is the two-phase, commit (2PC) protocol. However, as you will see in chapter 10, 2PC is not a good choice for modern applications. It's essentially a form of runtime coupling that can lead to availability and latency problems. In addition, many modern databases do not support 2PC.

A better approach - shown in [7.9](#) - is to limit the scope of ACID transactions to within a service and, implement distributed operations as a sequence of ACID transactions. This approach to transaction management is known as eventual consistency.

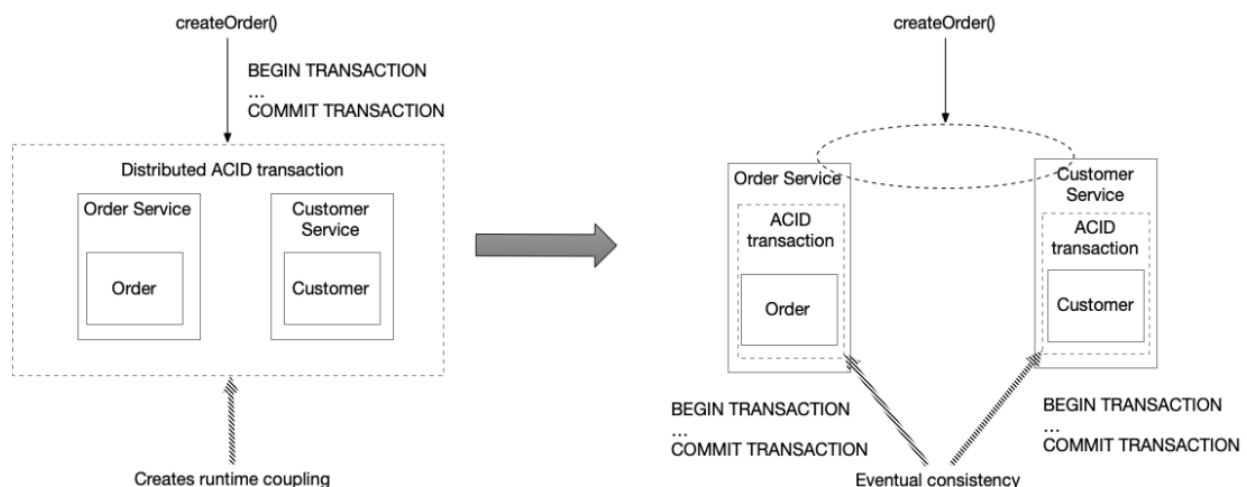


Figure 7.9 Distributed transactions are replaced with a sequence of local transactions coordinated by asynchronous messaging

During the execution of an eventually consistent, distributed command, the application is temporarily in an inconsistent state. For example, the `Order Service` creates and persists an `Order` before the customer's credit is reserved in the `Consumer Service`. The main benefit of eventual consistency is that it improves availability. The application can still respond to requests even if one or more services are unavailable. For example, the `Order Service` can respond to an `HTTP POST /orders` request even if the `Consumer Service` is unavailable.

Eventual consistency is not without its drawbacks:

- It's a more complicated transaction model that requires careful design
- Distributed system operations cannot rely on the database for locking and rollback.
- You must sometimes write code to undo previously committed changes
- You need to handle concurrency at the application level
- Distributed queries cannot provide a strong consistent view of across multiple services.

Fortunately, these challenges are often manageable and worth the trade-off for increased availability.

To implement a distributed operation, you need to apply one or more of the four service collaboration patterns. There are two patterns for implementing distributed commands, Saga and Command-side replica. The other two patterns implement distributed queries: They are API composition and CQRS. These patterns are briefly introduced in [Section 7.6.2](#). Chapter 10 describes the Saga and Command-side replica patterns in detail and chapter 11 describes the API composition and CQRS patterns.

As you can see, designing distributed operations is complicated. And because these operations span multiple services, evolving them sometimes requires coordinated changes to several services. Unfortunately, implementing such cross-service changes is another major challenge of the microservice architecture. Let's look at why.

7.5.2 Implementing changes that span services is complicated

One of the great strengths of the microservice architecture is that making a change to one or more subdomains within that are in the same service is faster and easier than making the equivalent change in a monolith. Since each service is small and independently deployable, changes can be developed, tested, and released without coordination with other teams.

Yet ironically, the inverse is also true: implementing changes that span multiple subdomains in multiple services is more complicatedly complicated than making the equivalent change in a monolith. In a monolith, a large change can be made by a single commit or pull request. In contrast, in a microservice architecture requires such changes to be carefully planned and sequenced to avoid production outages.

Let's consider a concrete example. Suppose the `Order Management` team is developing an enhancement to the `Order Service` that requires the `Consumer Management` team to change the `Consumer Service`'s `reserveCredit()` operation to return the customer's available credit. This is an example of a backwards compatible change to the `Consumer Service`'s API. Old clients can continue to use the updated API by simply ignoring the new `availableCredit` property in the response. But from a deployment perspective, there's an ordering

dependency. The updated `Consumer Service` must be deployed before the `Order Service` can use it.

Now let's imagine that instead of a backwards compatible change, the enhancement requires a breaking change to the `Consumer Service`'s API. The `Consumer Management` team cannot simply deploy the breaking change since that would cause a production outage. Instead, a sequence of non-breaking changes must be made:

1. The `Consumer Management` must add a new version of the `Consumer Service` API alongside the old version. One common approach is to prefix the URL paths with the version, e.g. `/v1/...` vs. `/v2/...`.
2. The client teams, including the `Order Management` team, must migrate their services to the new API version - perhaps after nudging from by the `Consumer Management` team
3. Once all clients had migrated to the new version of the `Consumer Service` API, the `Consumer Management` team can then delete the old API version.

As you can see, rolling out a breaking change to a service's API is much more complicated and time-consuming than the equivalent change in a monolith.

These kinds of cross-service changes highlight one of the core trade-offs of microservices: you gain autonomy and speed for local changes, but you pay for it when implementing distributed changes across subdomains. A key design goal is to define service boundaries so that most changes are local to a single service. Unfortunately, achieving this is another challenge of the microservice architecture.

7.5.3 Designing a good microservice architecture is challenging

Software design is largely about naming and grouping things, and applying the principles of loose design-time coupling and high cohesion, which I described in chapter 4. While it's inherently challenging, it's especially challenging to design a good microservice architecture that enables fast flow while being performant and highly available. That's because services are process and transaction boundaries. As a result, the decision to place two subdomains that are invoked by the same system operation in separate services is far more consequential than, say, putting two classes in different packages.

It makes the microservice architecture very unforgiving of design mistakes. It's far too easy to create a distributed monolith: services that are (a) tightly design-time coupled so changes ripple across services, and (b) tightly runtime coupled so a failure in one service cascades into another. Careful, rigorous design is required to maximize the benefits of the microservice architecture and minimize its drawbacks.

As you will see in chapter 20, designing a microservice architecture means navigating a complex design space filled with design alternatives. For example, let's imagine that you are implementing a new feature that introduces a new subdomain and a new operation that spans both the new subdomain and one or more of the existing subdomains. A key design issue is: which service should contain the new subdomain? If the architecture consists of N services, then there are $N+1$ options - an existing one or a new service.

And for each one of those possible service designs, there are likely multiple ways to implement the system operation. As you will learn in chapters 10 and 11, there are often multiple

ways to implement a system operation - multiple candidate patterns and multiple ways to implement each pattern - each of which involves different trade-offs. What's more, one design candidate might simplify the implementation of the operation at the expense of a less than ideal service design, or vice versa. These trade-offs often conflict, and evaluating the design space requires thoughtfulness and discipline.

7.5.4 Operational challenges

Not only is designing a microservice architecture challenging, but so is deploying and operating it. A monolith's deployment architecture consists of multiple instances of a single application. In contrast, the runtime architecture of a microservice architecture consists of multiple instances of multiple services - each potentially using different versions of different technology stacks. Moreover, microservice-based systems can exhibit emergent behavior—unexpected system-level issues that arise from interactions between independently developed and deployed services. These issues are often difficult to understand and troubleshoot.

Fortunately, as you will see in chapters 18 and {chapter-kubernetes}, there are many tools and techniques that can help. Container technologies allow a production environment to treat a service as a black box that hides its technology stack. Sophisticated orchestration platforms, such as Kubernetes, simplify the deployment and operation of services. And, modern observability tools make it easier to observe and troubleshoot a microservices-based application. Nevertheless, these tools can only help so much, and so some operational challenges still remain.

Despite these operational challenges, many organizations still pursue the microservice architecture for its long-term

benefits. But deciding when and how to adopt it — especially in an existing system — is yet another challenge.

7.5.5 Embracing independent deployability is a significant change

Another challenge with the microservice architecture is that many organizations struggle to embrace independent deployability. This often is because organizations insist on using application-level user acceptance tests. Let's look at what they are, why they are a problem and what's the alternative approach to test that enables independent deployability.

THE PROBLEM WITH APPLICATION-LEVEL USER ACCEPTANCE TESTS

Application-level user acceptance tests are typically written from the perspective of a user. Each one often implements a scenario from a user story or even a customer's journey through the application.

For example, the acceptance test suite for the customers and orders example might include tests for various order creation scenarios, including creating an order for a customer with sufficient credit; creating an order for a customer with insufficient credit; and creating an order for a customer that doesn't exist. It might also include edge cases, such as creating an order whose total is exactly the same the customer's available, along perhaps one cent over and one cent under.

While acceptance tests work well for a monolith, they typically span multiple services in a microservice architecture - violating the requirement for independent deployable services. To properly benefit from the

microservice architecture, organizations need to rethink acceptance tests.

TESTING SERVICES TOGETHER ⇒ BUILD A MONOLITH

A common antipattern of microservices adoption is an organization insisting on testing the application's services together. This is a form of the distributed monolith antipattern, which combines the complexity of microservices with the friction of the monolith. It's best to either improve the architecture so services can be tested independently OR simply build a monolith.

THE ALTERNATIVE TO APPLICATION-LEVEL USER ACCEPTANCE TESTS

To make service independently deployable, you must replace application-level user acceptance tests with service-level user acceptance tests that test each service in isolation, as shown in figure [7.10](#).

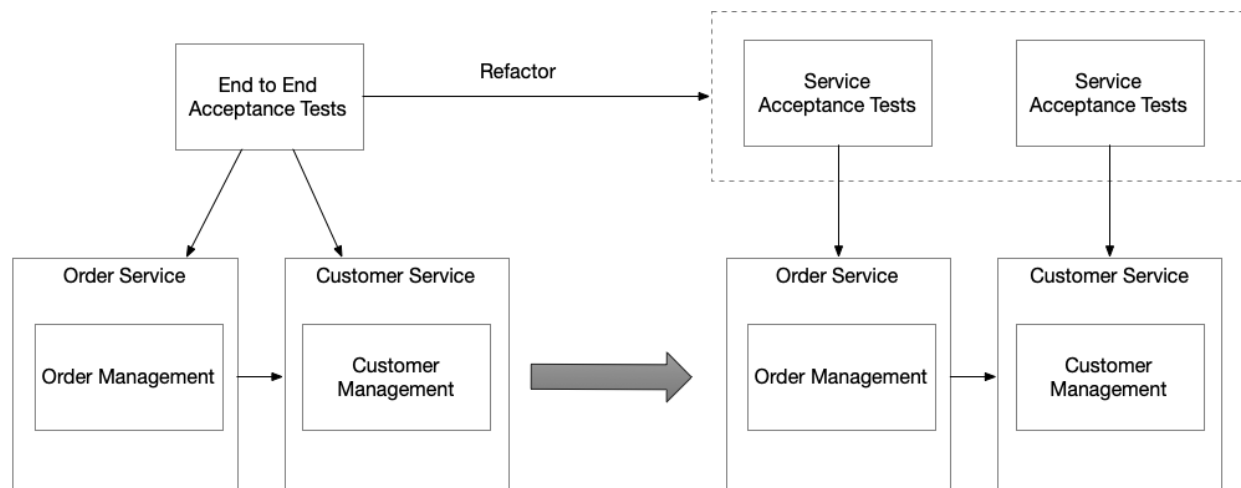


Figure 7.10 Acceptance tests that span multiple services and would be end-to-end tests must be refactored to service-level acceptance tests that test each service in isolation.

Service-level acceptance tests are a form of white box tests that leverages knowledge of how the services collaborate. For example, the acceptance tests for the `Order Service` would verify that `createOrder()` invokes `reserveCredit()` with the expected arguments and handles the various outcomes - success, unknown customer and insufficient credit - correctly.

Here are some possible acceptance tests for the `Order Service`:

createOrder(customerId, orderTotal)			
createOrder() arguments	expected reserveCredit(customerId, orderTotal) arguments	expected reserveCredit() outcome	expected createOrder() outcome
(101, 1234)	(101, 1234)	Credit reserved	Success
(101, 12340)	(101, 12340)	Credit Limit exceeded	Fail-Credit-Limit-Exceeded
(999, 1234)	(999, 1234)	Unknown customer	Fail-Unknown-Customer

These test cases verify that the `createOrder()` operation calls `reserveCredit()` with its arguments and maps the response to its own response.

Similarly, the acceptance tests for the `Consumer Service` would call `createConsumer()` and verify that `reserveCredit()` behaves as expected.

reserveCredit(customerId, amount)		
Customer's available credit	Amount	Expected outcome
1234	1234	Success
1234	1233.99	Success
1234	1234.01	Fail-Credit-Limit-Exceeded
-	-	Fail-Unknown-Customer

In many ways, the structure of the acceptance tests mirrors the application's architecture. As the architecture is decomposed into services, the application-level acceptance tests are likewise decomposed into service-level acceptance tests that are simpler, faster to execute, and more reliable. What's more, as you will see in chapters 15 and 16, it's often a good idea to implement many service-level acceptance tests as fast-running unit tests that can be executed locally.

7.5.6 Refactoring a monolith to microservice architecture is challenging

Another challenge with the microservice architecture is migrating a monolith to the microservice architecture. Sometimes it makes sense to use the microservice architecture for a greenfield project. However, most greenfield applications don't have the problems that microservice architecture solves and so the monolithic architecture is a better choice. In practice, most organizations adopt the microservice architecture when they

realize that their application has outgrown its monolithic architecture.

While there are strategies and techniques for migrating a monolith to microservices - see chapter 21 - the migration to microservices is usually extremely challenging. You need to untangle legacy code, and the size and complexity of the monolith makes the process very time-consuming. Organizations often spend many years migrating their monolith to a microservices-based architecture. Nevertheless, the benefits of the microservice architecture make the migration worthwhile.

7.6 An overview of the microservice architecture pattern language

Building software consists of making numerous architectural and design decisions using a deliberative design process, which I described in chapter 3. A key challenge, however, is that this process requires knowledge and experience. You must understand the context, the problem you are trying to solve, the criteria you will use to evaluate candidate solutions, and the candidate solutions.

Deliberative design is especially challenging when using a complicated architecture style, such as the microservice architecture. Yet at the same time, it's vital that you do so. That's where patterns come in. They not only provide you with concrete candidate solutions but also help you clarify your context and evaluation criteria. In fact, the deliberative design process itself draws inspiration from the patterns movement.

To help guide your decision-making, I've created the microservice architecture pattern language. Throughout the rest of the book, I reference this pattern language as I teach

you about the microservice architecture. Let's look at what a pattern language is and why it's helpful.

7.6.1 Patterns and pattern languages

A *pattern* is a reusable solution to a problem that occurs in a particular context. The concept originated in real-world architecture, but it has since become foundational in software architecture and design. The concept of a pattern was created by Christopher Alexander, a real-world architect, who also introduced the concept of a *pattern language*: a collection of related patterns that solve problems within a particular domain. His book *A Pattern Language: Towns, Buildings, Construction* (Oxford University Press, 1977) describes a pattern language for architecture that consists of 253 patterns. The patterns range from solutions to high-level problems, such as where to locate a city ("Access to water"), to low-level problems, such as how to design a room ("Light on two sides of every room"). Each pattern is a proven solution that arranges physical elements - ranging in scope from cities to windows.

Alexander's ideas inspired the software community to adopt the concept of patterns and pattern languages. The landmark book *Design Patterns: Elements of Reusable Object-Oriented Software* (Addison-Wesley Professional, 1994), by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides is a collection of object-oriented design patterns and helped popularize patterns among software developers. Since the mid-1990s, software developers have documented numerous software patterns. A *software pattern* solves a software architecture or design problem by defining a set of collaborating software elements.

Let's imagine, for example, that you're building a banking application that must support a variety of overdraft policies.

Each policy defines different limits and the fees. This problem can be solved using the Strategy pattern, which is a well-known pattern from the classic *Design Patterns* book. The Strategy pattern consists of three parts:

- A strategy interface called `Overdraft` that encapsulates the overdraft algorithm
- One or more concrete strategy classes, one for each particular context
- The `Account` class that uses the algorithm

Because the Strategy pattern is an *object-oriented* design pattern, the elements of the solution are classes. Later in this section, I describe *higher-level* design patterns, where the solution consists of collaborating services.

Patterns are valuable not only because they provide reusable solutions but also because a pattern must describe the context within which it applies. The idea that a solution is specific to a particular context and might not work well in other contexts is an improvement over how technology used to typically be discussed. This is especially important in software architecture, where, for example, solutions work for Netflix might not be the best approach for smaller-scale applications.

But the value of a pattern goes far beyond requiring you to consider the context of a problem. A well-defined pattern describes other critical yet frequently overlooked aspects of a solution. A typical pattern structure includes three especially valuable sections:

- Forces
- Resulting context
- Related patterns

Let's look at each of these, starting with forces.

FORCES: THE ISSUES THAT YOU MUST ADDRESS WHEN SOLVING A PROBLEM

The *forces* section of a pattern describes the forces or concerns that you must address when solving a problem in a given context. Forces can conflict, so it might not be possible to solve all them. Which forces are more important depends on the context. You have to prioritize solving some forces over others. For example, you often want code to be easy to understand and have good performance. Code written in a reactive style has better performance than synchronous code, yet is often more difficult to understand. In some contexts, performance is more important than understandability; in others, the opposite is true. By explicitly listing the forces, you make clear which issues need to be solved.

RESULTING CONTEXT: THE CONSEQUENCES OF APPLYING A PATTERN

The *resulting context* section of a pattern describes the consequences of applying the pattern. It consists of three parts:

- *Benefits* — The benefits of the pattern including the forces that have been resolved
- *Drawbacks* — The drawbacks of the pattern including the unresolved forces
- *Issues* — The new problems that have been introduced by applying the pattern

The resulting context provides a more complete and less biased view of the solution, which enables better design

decisions.

RELATED PATTERNS: THE FIVE DIFFERENT TYPES OF RELATIONSHIPS

The *related patterns* section of a pattern describes the relationship between the pattern and other patterns. There are five types of relationships between patterns:

- *Predecessor* — A predecessor pattern is a pattern that motivates the need for this pattern. For example, the Microservice architecture pattern is the predecessor to the rest of the patterns in the pattern language, except the monolithic architecture pattern.
- *Successor* — A pattern that solves an issue that has been introduced by this pattern. For example, if you apply the Microservice architecture pattern, you must then apply numerous successor patterns, including service discovery patterns and the Circuit breaker pattern.
- *Alternative* — A pattern that provides an alternative solution to this pattern. For example, the Monolithic architecture pattern and the Microservice architecture pattern are alternative ways of architecting an application. You pick one or the other.
- *Generalization* — A pattern that is a general solution to a problem. For example, in chapter 12 you'll learn about the different implementations of the Single service per host pattern.
- *Specialization* — A specialized form of a particular pattern. For example, in chapter 12 you'll learn that the Deploy a service as a container pattern is a specialization of Single service per host.

In addition, you can organize patterns that tackle issues in a particular problem area into groups. The explicit description

of related patterns provides valuable guidance on how to effectively solve a particular problem. Figure 7.11 shows how the relationships between patterns are visually represented.

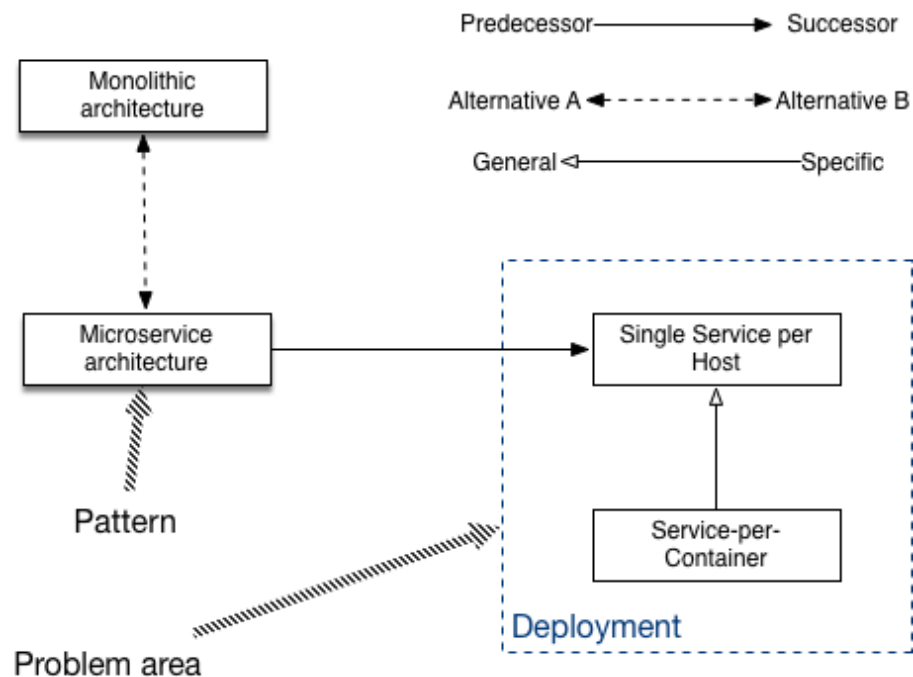


Figure 7.11 The visual representation of different types of relationships between the patterns: a *successor* pattern solves a problem created by applying the *predecessor* pattern; two or more patterns can be *alternative* solutions to the same problem; one pattern can be a *specialization* of another pattern; and patterns that solve problems in the same area can be grouped, or *generalized*.

A collection of patterns related through these relationships sometimes forms what is known as a pattern language. The patterns in a pattern language work together to solve problems in a particular domain. In particular, I've created the microservice architecture pattern language. It's a collection of interrelated software architecture and design patterns for microservices.

Let's take a look at the pattern language.

7.6.2 Overview of the microservice architecture pattern language

The microservice architecture pattern language is a collection of patterns that help you architect an application using the microservice architecture. Figure 7.12 shows the high-level structure of the pattern language. The pattern language first helps you decide whether to use the microservice architecture. It describes the monolithic architecture and the microservice architecture, along with their benefits and drawbacks. Then, if the microservice architecture is a good fit for your application, the pattern language helps you use it effectively by solving various architecture and design issues.

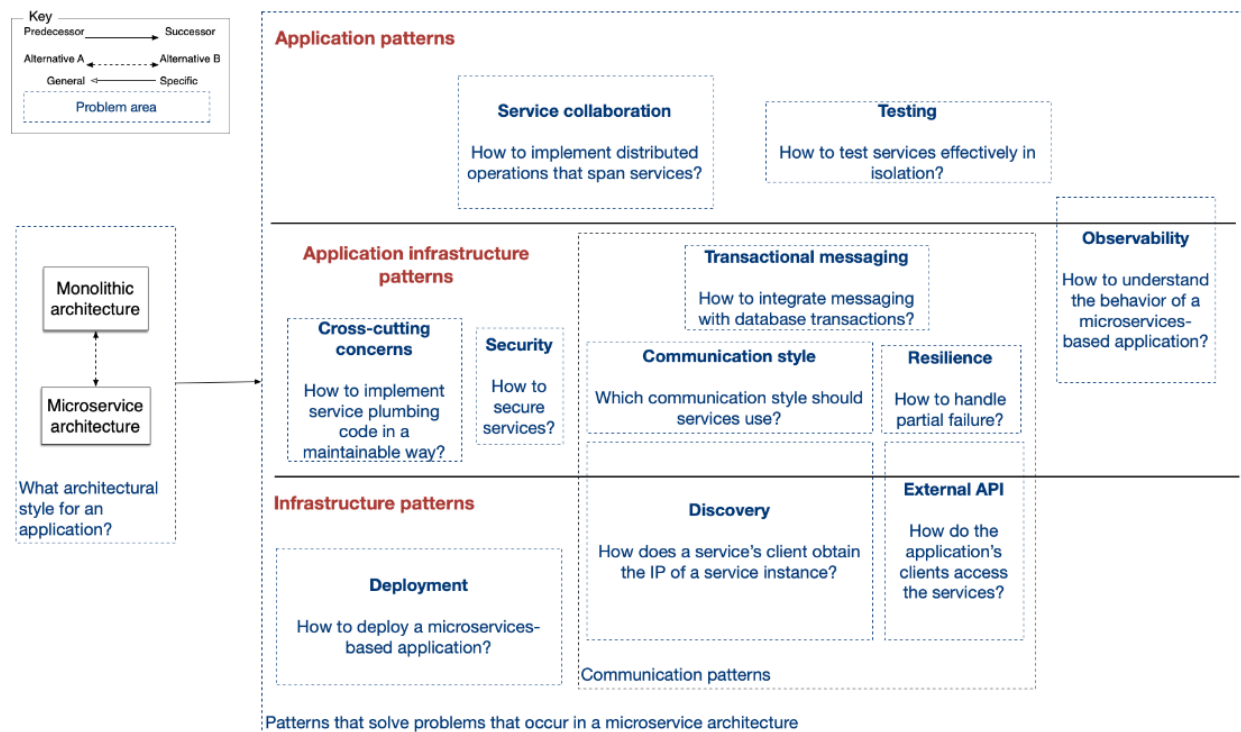


Figure 7.12 A high-level view of the microservice architecture pattern language showing the different problem areas that the patterns solve. On the left are the application architectural style patterns: Monolithic Architecture and Microservice Architecture. The other groups of patterns all solve problems created by choosing the Microservice architecture pattern.

The pattern language consists of several groups of patterns. On the left in figure [7.12](#) is the application architecture patterns group, which consists of the Monolithic architecture pattern and the Microservice architecture pattern. These are the patterns discussed in the previous chapter and this chapter. The rest of the pattern language consists of groups of patterns that solve problems introduced by using the Microservice architecture pattern.

The patterns are also organized into three layers:

- *Infrastructure patterns* — These solve problems that are mostly infrastructure issues outside of development.
- *Application infrastructure patterns* — These are for infrastructure issues that also impact development.
- *Application patterns* — These solve problems faced by developers when developing services.

These patterns are grouped based on the kinds of problems they solve. Let's look at the main groups of patterns.

SERVICE COLLABORATION PATTERNS

As I mentioned earlier in this chapter, a distributed operation, which is an operation that spans multiple services, must be implemented using a combination of the service collaboration patterns. There are four service collaboration patterns:

- Saga - implements a distributed command as a series of local transactions in each of the participating services
- Command-side replica - maintains a local, read-only replica of data from another service so it can be efficiently queried by a command

- API Composition - implements a distributed query by querying multiple services that own the data and aggregating the results
- CQRS - implements a distributed query by maintaining a read-only, efficiently queryable replica that combines data from multiple services

These patterns are described in detail in chapters 10 and 11. Chapter 10 describes the command patterns and chapter 11 describe these querying patterns.

COMMUNICATION PATTERNS

Because the microservice architecture is a distributed system, the service collaboration patterns must be implemented using inter-process communication (IPC) mechanisms. Additionally, an application's clients need a mechanism for communicating with the services. Figure [7.13](#) shows the communication patterns, which are organized into five groups:

- *Communication style* — What kind of IPC mechanism be used: synchronous (e.g., remote procedure calls (RPC) or REST) or asynchronous messaging?
- *Transactional messaging* — How should message sending and receiving be integrated with database transactions that update business data?
- *Reliability* — How can synchronous communication between services remain reliable even when services are unavailable or there are network issues?
- *Discovery* — How does a client of a service discover the network address of a service instance in order to send a request (e.g. a REST call)?
- *External API* — How do external clients communicate with the application's services?

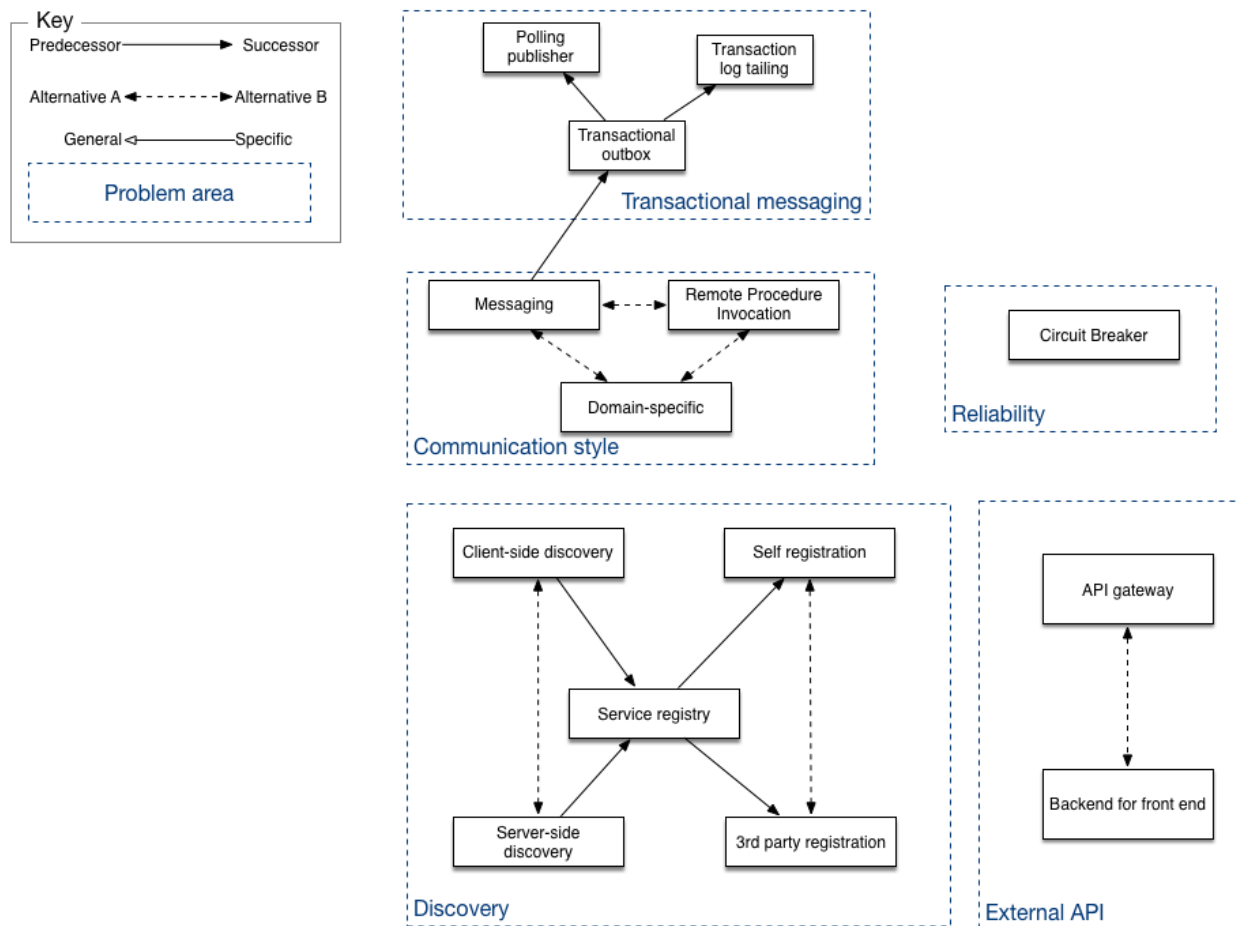


Figure 7.13 The five groups of communication patterns that underlie the service collaboration and enable an application's clients to communicate with its services.

Chapter 3 covers the first four groups: communication style, discovery, reliability, and transaction messaging. Chapter 12 looks at the external API patterns.

In addition to defining communication patterns, the microservices pattern language also includes security patterns to address the challenges of securing communication in a distributed architecture.

SECURITY PATTERNS

In a microservice architecture, users are typically authenticated by the API gateway. It must then pass

information about the user, such as their identity and roles, to the services it invokes. A common solution is to apply the Access token pattern. In this pattern, the API gateway passes an access token - such as JWT (JSON Web Token) - in each request to a service. The service validates the token and extracts information about user. Chapter 11 discusses the Access token pattern in more detail.

PATTERNS FOR THE AUTOMATED TESTING OF SERVICES

The microservice architecture makes individual services easier to test because they're much smaller than the monolithic application. At the same time, though, it's important to test that the different services work together while avoiding using complex, slow, and brittle end-to-end tests that test multiple services together. Here are patterns for simplifying testing by testing services in isolation:

- *Consumer-driven contract test* — Verify that a service meets the expectations of its clients
- *Consumer-side contract test* — Verify that the client of a service can communicate with the service
- *Service component test* — Test a service in isolation

Chapters 15 and 16 describe these testing patterns in more detail.

SERVICE DEPLOYMENT PATTERNS

Deploying a monolithic application isn't always easy, but it is relatively straightforward: you build and deploy a single application, often running multiple instances behind a load balancer. In contrast, deploying a microservices-based application is much more complex. It might consist of tens

or even hundreds of services, potentially written in different languages and frameworks. There are many more moving parts that need to be coordinated and managed.

Figure 7.14 shows the deployment patterns that address these challenges.

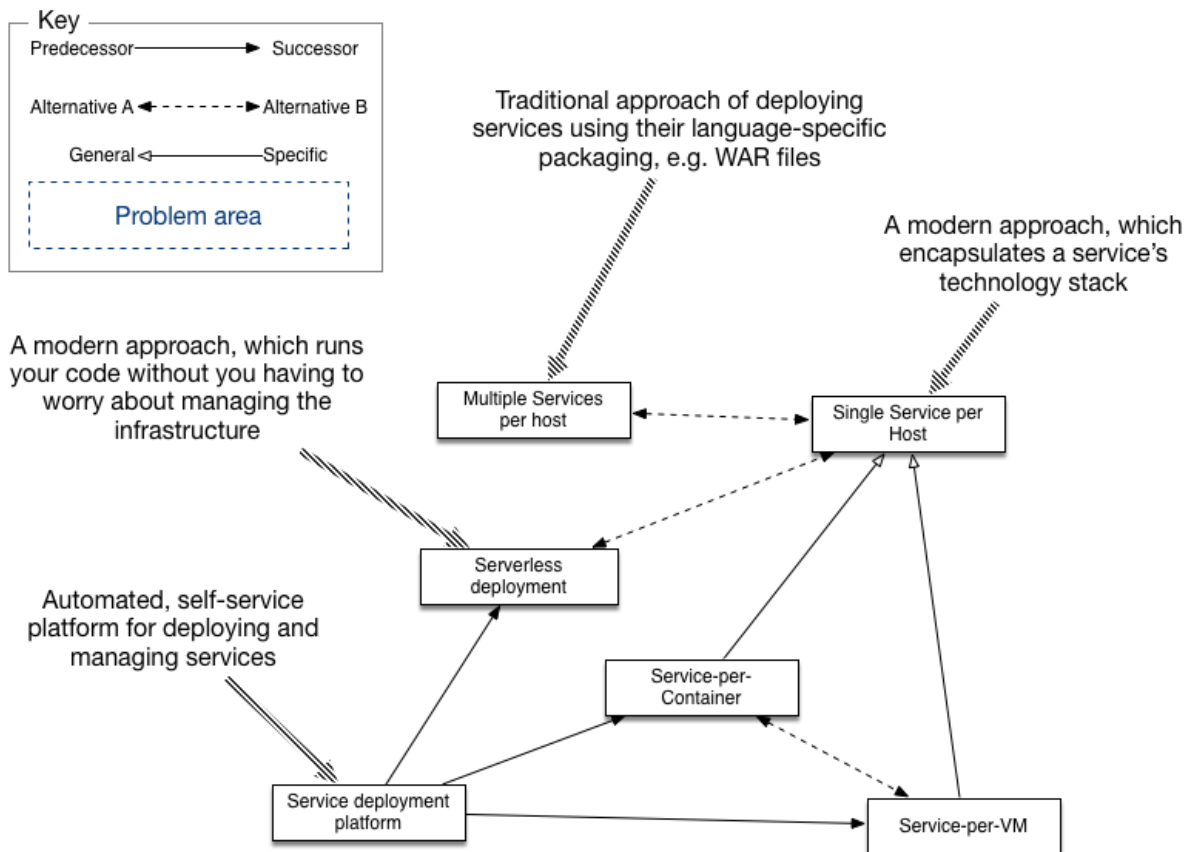


Figure 7.14 Several patterns for deploying microservices. The traditional approach is to deploy services in a language-specific packaging format. There are two modern approaches to deploying services. The first deploys services as VM or containers. The second is the serverless approach. You simply upload the service's code and the serverless platform runs it. You should use a service deployment platform, which is an automated, self-service platform for deploying and managing services.

The traditional, and often manual, way of deploying applications in a language-specific packaging format, for example, WAR files, doesn't scale to support a microservice architecture. You need a highly automated deployment

infrastructure. Ideally, you should use a deployment platform that supports GitOps, which is a way to manage infrastructure and applications using Git. With GitOps, the desired system state — defined by a set of configuration files - in Git, and automation ensures the running system matches what's in Git. Developers make changes simply by updating the Git repository, and the deployment platform automatically applies them. The deployment platform will typically be based on virtual machines (VMs), containers, or serverless technology. Chapter 12 looks at the different deployment options in more detail and chapter {chapter-kubernetes} describes how to deploy services using Kubernetes.

OBSERVABILITY PATTERNS PROVIDE INSIGHT INTO APPLICATION BEHAVIOR

A key part of operating an application is understanding its runtime behavior and troubleshooting problems such as failed requests and high latency. While understanding and troubleshooting a monolithic application isn't always easy, it helps that requests are handled in a simple, straightforward way. Each incoming request is load balanced to a particular application instance, which makes a few database calls and returns a response. For example, if you need to understand how a particular request was handled, you can usually look in that instance's log file.

In contrast, understanding and diagnosing problems in a microservice architecture is much more complicated. A request can bounce around between multiple services before a response is finally returned into a client. As a result, there isn't one log file to examine. Similarly, diagnosing latency problems are more difficult because there are multiple suspects.

To address these challenges, you should design observable services using these patterns:

- *Health check API* — Expose an endpoint that returns the health of the service
- *Log aggregation* — Log service activity and gather logs into a centralized logging server, which provides searching and alerting
- *Distributed tracing* — Assign a unique ID to each external request and trace it across services
- *Exception tracking* — Report exceptions to an exception tracking service, which deduplicates exceptions, alerts developers, and tracks the resolution of each exception
- *Application metrics* — Maintain metrics, such as counters and gauges, and expose them to a metrics server
- *Audit logging* — Record user actions for security and compliance purposes

Chapter 11 describes these patterns in more detail.

PATTERNS FOR HANDLING CROSS-CUTTING CONCERNS

In a microservice architecture, each service must implement numerous cross-cutting concerns, including observability, discovery, security and externalized configuration (e.g. database credentials at runtime). Each service also needs build logic (e.g. Maven or Gradle) along with a deployment pipeline. Reimplementing these concerns from scratch when developing a new service would be too time-consuming and error-prone. This would lead to code duplication and inconsistent implementations across services.

A better approach is to apply the Service Template and Microservice Chassis patterns. A service template is source

code template that a developer can quickly and easily copy to start developing a new service. It's built on top of a microservice chassis - a framework that handles the cross-cutting concerns. The chassis minimizes the size of the service template and reduces code duplication across services. Chapter 11 describes these patterns in more detail.

7.7 Summary

- Although the microservice architecture is an architectural structure that organizes an application's subdomains into two or more loosely coupled and independently deployable services, there's more to it than just services. Other important aspects of the architecture include:
 - System operations, which are either local to a service or span multiple services. They can constrain how services boundaries are defined.
 - Deployment pipelines, along with code repositories and build projects
 - The relationship between services and teams
- The microservice architecture has numerous benefits - many of which stem from services being loosely coupled and independently deployable - including:
 - Enabling fast flow by supporting all five of the quality attributes needed for fast flow
 - Improving the developer experience
 - Supporting for multiple technology stacks
 - Enabling organizations to scale
 - Improving architectural characteristics, such as availability, security, and scalability, by segregating subdomains by their characteristics
- The distributed nature of the architecture is a double-edged sword that creates significant challenges,

particularly around system operations that span services. Distributed system operations are more difficult to implement because of issues like latency, partial failure, and eventual consistency.

- Successfully adopting the microservice architecture requires deliberative design to avoid creating a distributed monolith - an application consisting of tight coupled services that combines the worst aspects of both architectures.
- Teams must embrace the concept of independently deployable services, which means testing services in isolation instead of using application-level acceptance tests
- The microservice architecture pattern language consists of patterns for deciding whether the microservice architecture is a good fit for an application, and, if it is, solving the numerous architecture and design issues it introduces.

8 Interprocess communication in a microservice architecture

This chapter covers

- The Interprocess communication (IPC) technologies that are the building blocks of the service collaboration patterns
- Defining and evolving APIs, including how to safely make breaking changes
- The two main communication styles
- Improving the resiliency of Remote procedure invocation (RPI)-based communication
- Implementing correct, scalable, and resilient asynchronous messaging
- Integrating synchronous APIs with asynchronous implementations

As you saw in chapter 7, the microservice architecture is a distributed architecture made up of many services. Some but not all operations span multiple services, and those distributed operations must be implemented using the service collaboration patterns, described in chapters 9 and 10. Because service instances are typically processes running on multiple machines, the service collaboration patterns use IPC technologies, such as REST or asynchronous messaging.

Of course, a monolithic application also uses IPC, but only to communicate with the outside world - browser applications,

mobile clients, external services, other applications, and so on. Internally, the application's elements directly interact through language-level method or function calls. In a microservice architecture, every interaction between services happens across process boundaries and so IPC plays a much more central role.

There's no shortage of IPC mechanisms to choose from. But the choice of an IPC mechanism is an important architectural decision that can affect key quality attributes, such as availability, and performance. What's more, as I explain in this chapter and chapter 9, IPC mechanisms even intersect with transaction management. Choosing wisely requires understanding the trade-offs, especially how the choice of IPC mechanism affects runtime coupling.

This chapter explores various IPC options, including REST and messaging, and discusses the trade-offs. We'll look at both synchronous and asynchronous communication, including REST and messaging. We'll examine design issues such as API definition and evolution, message formats, and the implementation of resilient clients. And finally, we'll see how asynchronous messaging can improve service availability by reducing runtime coupling. Let's start with an overview of interprocess communication in microservice architecture.

8.1 Overview of interprocess communication in a microservice architecture

As figure [8.1](#) shows, there are many IPC technologies to choose from. Services can use synchronous request/response mechanisms such as HTTP-based REST or gRPC. Alternatively, they can use asynchronous, message-

based communication mechanisms implemented by message brokers, such as Apache Kafka or RabbitMQ. There are also a variety of different message payload formats. Services can use human-readable, text-based formats such as JSON or XML, or they can use more compact binary formats, such as Avro or Protocol Buffers.

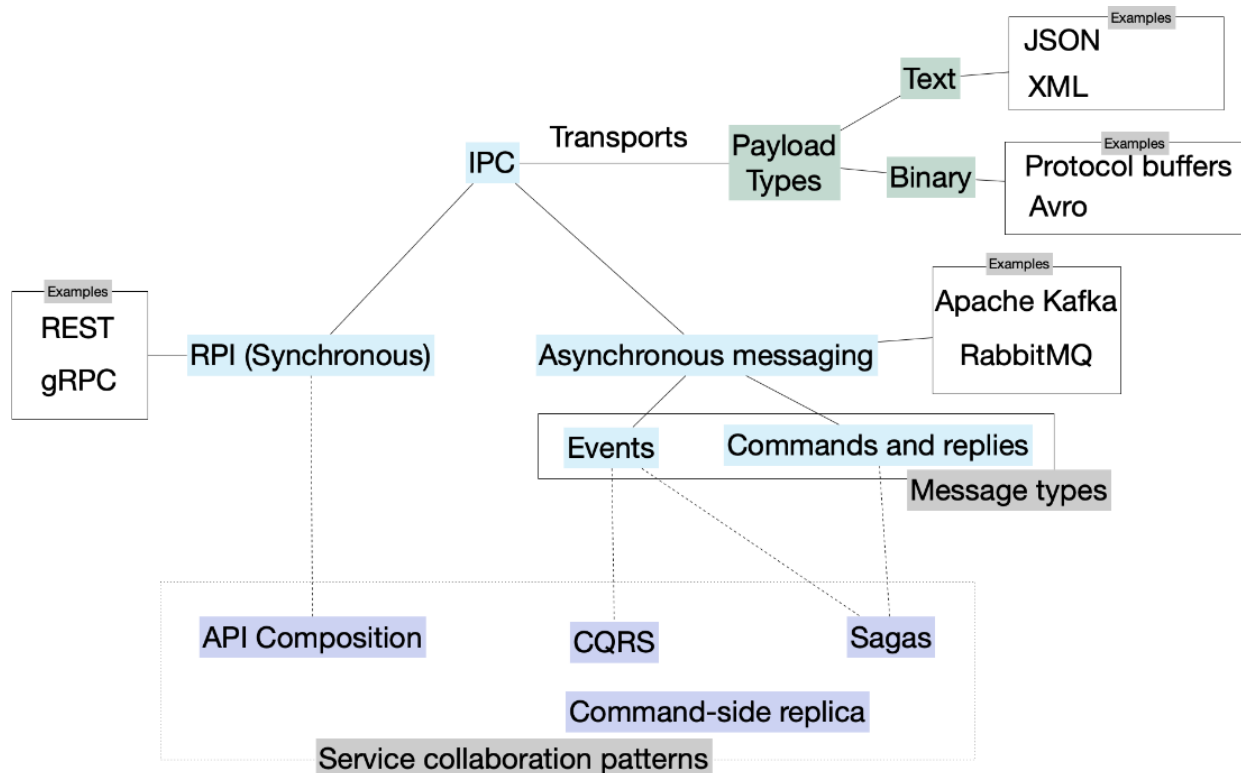


Figure 8.1 The taxonomy of IPC communication styles, implementation technologies including message payload formats, and their relationships with the service collaboration patterns

Before diving into specific technologies, I want to bring up several design issues you should consider. I begin with interaction styles - a technology-independent way of describing how services communicate. Next, I explain how to define APIs and specify their Service Level Objectives (SLOs). Then, I'll briefly discuss the critical topic of API security — covering secure communication, authentication, and authorization. After that, we'll explore strategies for evolving APIs safely as services change over time. I then

describe the design choices involved in selecting message payload formats. Finally, I discuss how to develop API clients that consume those APIs.

Let's begin by looking at interaction styles.

8.1.1 Interaction styles

It's useful to first think about how a service and its clients interact before selecting an IPC mechanism for the service's API. Starting with the interaction style will help you focus on the requirements and avoid getting mired in the details of a particular IPC technology. Also, as I describe throughout this chapter, the choice of interaction style impacts the application's availability. Furthermore, as you'll see in chapters 14 and 15, it helps you select the appropriate integration testing strategy.

There are a variety of client-service interaction styles. As table [8.1](#) shows, they can be categorized along two dimensions. The first dimension is whether the interaction is one-to-one or one-to-many:

- *One-to-one* — Each client request is processed by exactly one service.
- *One-to-many* — Each request is processed by multiple services.

The second dimension is whether the interaction is synchronous or asynchronous:

- *Synchronous* — The client expects a timely response from the service and might even block while it waits.
- *Asynchronous* — The client doesn't block, and the response, if any, isn't necessarily sent immediately.

Table 8.1 The various interaction styles can be characterized in two dimensions: one-to-one vs one-to-many and synchronous vs. asynchronous

	one-to-one	one-to-many
Synchronous	Request/response	-
Asynchronous	Asynchronous request/response One-way notifications	Publish/subscribe Publish/Async responses

Next, let's look at each category in more detail.

ONE-TO-ONE INTERACTION STYLES

There are three types of one-to-one interactions:

- *Request/response* — A service client makes a request to a service and waits for a response. The client expects the response to arrive in a timely fashion. It might even block while waiting. This interaction style tends to create tighter runtime coupling between services.
- *Asynchronous request/response* — A service client sends a request to a service, which replies asynchronously. The client doesn't block while waiting because the service might not send the response for a long time.
- *One-way notifications* — A service client sends a request to a service, but no reply is expected or sent.

It's important to remember that the synchronous request/response interaction style is mostly orthogonal to

IPC technologies. A service can, for example, interact with another service using request/response style interaction with either REST or messaging. Even if two services are communicating using a message broker, the client service might be blocked waiting for a response. It doesn't necessarily mean they're loosely coupled. That's something I revisit later in this chapter when discussing the impact of inter-service communication on availability.

ONE-TO-MANY INTERACTION STYLES

There are two types of one-to-many interactions:

- *Publish/subscribe* — A client publishes a notification message, which is consumed by zero or more interested services.
- *Publish/async responses* — A client publishes a request message and then waits for a certain amount of time for responses from interested services.

In practice, most services use a combination of these styles. For example, many FTGO services expose synchronous APIs for external clients, publish events for downstream consumers, and subscribe to events from other services. Let's look at how to define a service's API.

8.1.2 Defining APIs in a microservice architecture

APIs, or interfaces, are central to software development. An application consists of elements, and each element has an interface that defines the set of operations that its clients can invoke. A well-designed interface exposes useful functionality while hiding the implementation. It enables elements to loosely design-time coupled (see chapter 4) so

that an element's implementation can be changed without affecting its clients.

ABOUT SERVICE APIS

In a monolithic application, an interface is typically specified using a programming language construct such as a Java interface. A Java interface defines the methods that a client can invoke, while the implementation remains hidden. Moreover, because Java is a statically typed language, if the interface or the client change in incompatible ways, the application simply won't compile.

Interfaces are equally important in a microservice architecture, but they look very different. As figure [8.2](#) shows, the APIs that connect services are language-neutral, network-based contracts rather than in-process method or function calls.

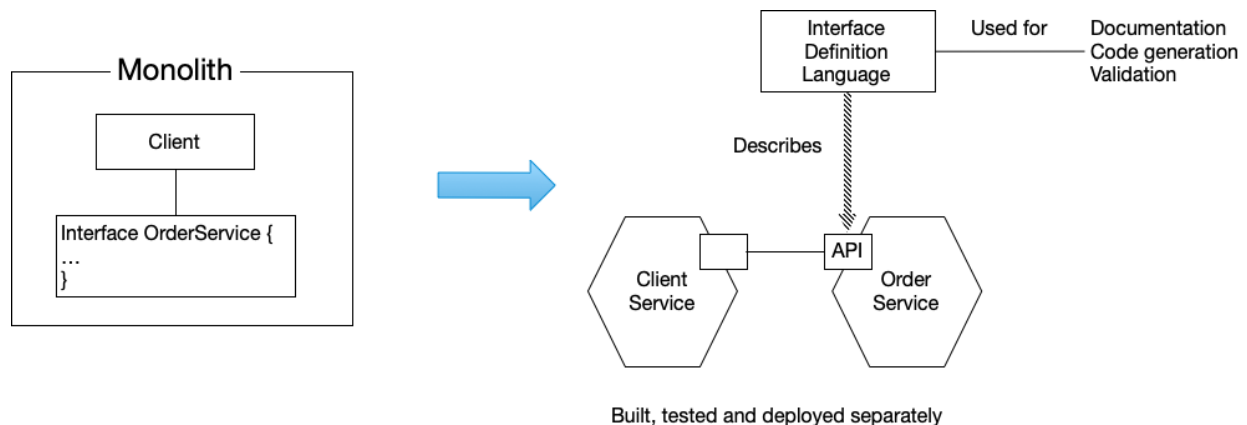


Figure 8.2 In monolith, interfaces are programming language constructs, such as Java integrations, where as in a microservice architecture where the services are developed and deployed independently, APIs are defined by an interface definition language (IDL).

The exact nature of a service's API depends on which IPC mechanism it uses. If it uses HTTP, the API consists of URLs, HTTP verbs, and the request and response payload formats.

If it uses messaging, the API consists of message channels, message types, and message payload formats.

DEFINE SERVICE APIS USING AN IDL

The challenge with developing and using service APIs is that they aren't defined using a simple programming language construct. By definition, a service and its clients aren't compiled together. If a new version of a service is deployed with an incompatible API, there's no compiler to catch the error—only a production outage.

To reduce this risk, a service's API should be defined using an Interface Definition Language (IDL), such as Open API for REST APIs, or AsyncAPI for messaging. An IDL serves several purposes:

- It defines the API in a precise, language-independent way.
- It provides a single source of truth for client and server code generation.
- It enables automated validation, documentation, and compatibility checks.

In other words, the IDL formalizes the contract between a service and its clients, and reduces the risk of incompatibilities.

PRACTICE API-FIRST DEVELOPMENT

Moreover, there are good arguments for using an API-first approach to defining services

(<https://www.postman.com/api-first/>):

1. Write the interface definition

2. Review the interface definition with the client developers
3. Iterate on the API definition
4. Implement the service

Doing this up-front design increases your chances of building a service that meets the needs of its clients.

API-FIRST DESIGN IS ESSENTIAL

Even on small projects I've seen problems occur because components don't agree on an API. For example, on one project the backend Java developer and the AngularJS frontend developer both said they had completed development. The application, however, didn't work. The REST and WebSocket API used by the frontend application to communicate with the backend was poorly defined. As a result, the two applications couldn't communicate!

CREATE AN API STYLE GUIDE

Organizations should create an API style guide that helps ensure that all services within an organization expose APIs that are predictable, discoverable, and easy to use. It defines common conventions for naming, error handling, pagination, versioning, and the use of HTTP methods or message payload formats. Consistency reduces the cognitive load for both developers and consumers — once they learn the conventions of one service, they can quickly understand others. A well-defined style guideline also simplifies documentation, tooling, and automated validation.

In distributed systems, an API style guideline plays a critical role in maintaining consistency across independently developed services. Without shared conventions, each team

might make slightly different design choices, leading to friction, duplication, and increased integration complexity. By adopting a common set of standards — whether RESTful, gRPC, or event-driven — teams can evolve APIs independently while still preserving a unified experience for clients and internal platforms.

The **Service Chassis**, which is described in chapter 16, enforces and automates many aspects of the API style guideline. It provides reusable components such as standardized error responses, logging and tracing interceptors, authentication filters, and health check endpoints. By embedding these cross-cutting concerns in the chassis, teams can focus on implementing business logic while remaining compliant with organizational API conventions. In this way, the Service Chassis acts as a practical mechanism for institutionalizing the API style guideline across all services.

8.1.3 APIs have Service Level Objectives (SLOs)

APIs are not just functional contracts; they also represent operational promises. When a client calls a service, it expects not only correct behavior but also a certain level of reliability and responsiveness. A service API must, therefore, satisfy various non-functional requirements, also known as **software quality attributes**, such as availability, performance, scalability, security, and reliability. To make these requirements concrete and verifiable, teams define metrics and targets, respectively known as service level indicators (SLIs), and service level objectives that quantify the desired service behavior.

ABOUT SERVICE LEVEL INDICATORS (SLIS), SERVICE LEVEL OBJECTIVES (SLOS), AND SERVICE LEVEL AGREEMENTS (SLAS)

Three inter-related key concepts quantify an API's desired behavior:

- Service Level Indicators (SLIs) - quantitative metrics that capture aspects of the API's real-world behavior, such as latency, error rate, or uptime.
- Service Level Objectives (SLOs) (<https://sre.google/sre-book/service-level-objectives/>) - target values for those SLIs that express the desired level of reliability or performance.
- Service Level Agreements (SLAs) - SLOs with contractual commitments, often with penalties or credits for violations.

An SLO defines the target for an SLI, while an SLA formalizes that target for clients.

THE ROLE OF OBSERVABILITY

SLOs are meaningless without observability. Teams must instrument their APIs to collect the metrics that drive SLIs. The observability infrastructure continuously collects telemetry - metrics, logs and traces - and compares against these objectives, enabling early detection of reliability issues. Observability practices and tooling are described in more detail in chapter 16.

Let's now look at some example SLOs.

EXAMPLE SLOS

Table [8.2](#) lists some example SLOs for a service. Each of these examples turns an abstract quality expectation — such as "the service should be fast and reliable" — into a clear,

testable performance target that can be monitored and acted upon.

Table 8.2 Some example SLOs for a service, each of which is a well-defined performance target

SLO Type	Description	Example Target
Availability	Ensures that the API is reachable and functioning for consumers.	At least 99.9% of requests succeed over a rolling 30-day period.
Latency	Measures how quickly the API responds to requests.	95% of requests complete within 200 milliseconds.
End-to-End Processing (Asynchronous)	Measures how long it takes for published events to be processed by downstream consumers.	99% of events are processed successfully within one minute.
Data Freshness	Ensures that derived views, caches, or replicas reflect recent updates.	Replicated data is updated within 60 seconds of a change.

These examples illustrate how SLOs make service quality measurable and actionable. However, defining targets is only part of the picture — teams also need a way to manage the inevitable deviations from those targets. This is where the concept of error budget comes in, providing a quantitative mechanism to balance reliability with delivery speed and innovation.

USING ERROR BUDGETS TO DETERMINE PRIORITIZE

An important concept related to SLOs is the error budget — the allowable amount of unreliability a service can

experience without breaching its objectives (<https://sre.google/workbook/error-budget-policy/>). For example, if an API has an availability SLO of 99.9%, its monthly error budget is the remaining 0.1% of failed or unavailable requests (about 43 minutes of downtime per month). Error budgets are a way to determine a team's priorities. When the service operates within its budget, teams can safely release new features or perform risky changes. But when the budget is exhausted, development priorities shift toward improving stability and addressing reliability issues. By tracking how quickly the error budget is consumed, teams gain an objective signal for when to slow down deployments, improve observability, or invest in resilience engineering.

8.1.4 API Security

Security is a critical aspect of any API design. Even a perfectly designed API is dangerous if it allows unauthorized access or leaks sensitive data. Security is a broad topic, but three key concerns are

- Secure communication - protecting data in transit from interception or tampering
- Authentication - verifying the identity of the caller
- Authorization - controlling what authenticated identities are allowed to do

Let's look at each one of these in turn.

SECURE COMMUNICATION

Secure communication ensures that data exchanged between clients and services cannot be intercepted or altered. In practice, this means that all API traffic should use Transport Layer Security (TLS). TLS encrypts data in transit,

prevents man-in-the-middle attacks, and verifies the authenticity of the service endpoint.

Establishing secure communication is the foundation of API security: without it, even strong authentication and authorization mechanisms can be bypassed by network-level attacks.

AUTHENTICATION

Authentication verifies who is making a request. APIs commonly use access tokens — such as OAuth 2.0 bearer tokens or JSON Web Tokens (JWTs) — to authenticate clients and securely convey identity claims. Depending on the system, authentication may involve end-user credentials, client certificates, or federated identity providers that issue tokens trusted across multiple services.

AUTHORIZATION

Authorization determines what an authenticated identity is permitted to do. It defines the actions and resources accessible to users or services once their identity is established. Authorization policies are often enforced using role-based or attribute-based access control models. For infrastructure services, similar principles apply. For example, Kafka Access Control Lists (ACLs) can restrict which producers and consumers are allowed to publish or subscribe to specific topics.

8.1.5 Evolving APIs

APIs invariably change over time as new features are added, existing features are modified, and sometimes old features are removed. In a monolithic application, it's relatively straightforward to change an API and update all the callers.

If you're using a statically typed language, the compiler helps by giving a list of compilation errors. The only challenge may be the scope of the change - it might take a long time to change a widely used API.

In a microservices-based application, changing a service's API is far more difficult. A service's clients are other services that are developed and deployed independently, usually by other teams. In some cases, the clients may even be other applications owned by other organizations. You usually can't force all clients to upgrade in lockstep with the service. Also, because modern applications are rarely down for maintenance, you will typically perform a rolling upgrade of your service, meaning both the old and new versions of a service will be running simultaneously. What's more, as I describe in section [Section 8.3.4](#), in an asynchronous messaging architecture, a new receiver must be able to handle messages sent by an old sender.

It's important to have a strategy for dealing with these challenges. How you handle a change to an API depends on the nature of that change. There are also important differences between evolving a synchronous RPI (see section [Section 8.2.2](#) and [Section 8.2.3](#)) and evolving an asynchronous messaging-based API (see section [Section 8.3.4](#)).

USE SEMANTIC VERSIONING

The Semantic Versioning specification (<http://semver.org>) is a useful guide to versioning APIs. It's a set of rules that specifies how version numbers are used and incremented. Semantic versioning was originally intended to be used for versioning of software packages, but you can use some of its concepts when versioning APIs in a distributed system.

The Semantic Versioning specification (Semvers) requires a version number to consist of three parts: `MAJOR.MINOR.PATCH`. You must increment each part of a version number as follows:

- `MAJOR` — When you make an incompatible change to the API
- `MINOR` — When you make backward-compatible enhancements to the API
- `PATCH` — When you make a backward-compatible bug fix

For example, as of the time of writing, the current version of Spring Boot is 3.5.6.

There are a couple of places you can use the version number in an API. If you're implementing a REST API, you can, as mentioned in section [Section 8.2.2](#), use the major version as the first element of the URL path. Alternatively, if you're implementing a service that uses messaging, you can include the version number in the messages that it publishes. The goal is to properly version APIs and to evolve them in a controlled fashion.

Let's look at how to handle minor and major changes.

MAKING MINOR, BACKWARD-COMPATIBLE CHANGES

Ideally, you should strive to only make backward-compatible changes. For example, as I describe in section [Section 8.2.2](#), backward-compatible changes to a REST API include adding an optional attribute to a request, and adding attributes to a response.

MAKING MAJOR, BREAKING CHANGES

Sometimes you must make major, incompatible changes to an API. Because you can't force clients to upgrade immediately, a service must simultaneously support old and new versions of an API for some period of time. The details of implementing side-by-side API depend on the IPC technology. For example, as I describe in section [Section 8.2.2](#), a REST API can prefix paths with the major version and a messaging API might specify the major version in a message header or channel name.

PREVENTING ACCIDENTAL BREAKING CHANGES

It's important to prevent unintentional breaking changes that would result in a production outage. There are a couple of ways to implement such checks.

The first option is to use a tool that compares the new candidate schema version with the previous schema version. Examples of such tools include the following:

- OpenAPI Diff
(<https://github.com/OpenAPITools/openapi-diff>) can
detect and prevent breaking changes to REST APIs by comparing the latest and previous OpenAPI specifications
- The Apollo Rover CLI for GraphQL supports checks like `rover graph check` to validate schema changes against existing clients.
- The Confluent schema registry implements compatibility checks for Avro messages

The second way to detect breaking changes is consumer-driven contract testing. As you will learn in chapter 15, client teams (consumers) provide tests - known as contracts - that verify that the service's API remains compatible with their client's expectations. The service (provider) team

incorporates these contracts into the service's automated test suite and runs them whenever the service implementation changes. In scenarios where all of a service's clients provide contract tests, a non-breaking change is simply any change that passes all of the contracts, including the removal or renaming of fields that no client depends on.

These tools and tests should be run on both the developer's laptop and the deployment pipeline.

AN ALTERNATIVE VIEW: A NON-BREAKING CHANGE IS ONE THAT DOESN'T BREAK AN ACTUAL CLIENT

Semantic versioning is oriented towards "public" APIs where there are large numbers of users who are outside your direct control. The classification of changes as breaking or non-breaking is done without considering how the API is actually used. Within an application, however, teams collaborate around APIs. They often use consumer-driven contract testing to ensure API compatibility. In such a scenario, a non-breaking change is any change that passes the contract tests, including the removal of fields.

Let's now look at the topic of message payload formats, the choice of which can impact how easy evolving an API will be.

8.1.6 Message payload formats

The essence of IPC is the exchange of messages. A *message* carries data - also known as its payload - and so an important design decision is the format of that data. The choice of message payload format can impact the efficiency of IPC, the usability of the API, and its evolvability. If you're using a messaging system or protocols such as HTTP, you get to pick your message payload format. Some IPC

mechanisms — such as gRPC, which you'll learn about shortly — might dictate the message payload format. In either case, it's essential to use a cross-language message payload format. Even if you're writing your microservices in a single language today, it's likely that you'll use other languages in the future. You shouldn't, for example, use Java serialization.

There are two main categories of message payload formats: text and binary. Let's look at each one in more detail.

TEXT-BASED MESSAGE PAYLOAD FORMATS

The first category is text-based formats such as JSON and XML. An advantage of these widely used formats is that they are human-readable. They are also self-describing, since a JSON message is a collection of named properties, and an XML message is effectively a collection of named elements and values. This format enables a consumer of a message to pick out the values of interest and ignore the rest. Consequently, many changes to the message schema can easily be backward-compatible.

The structure of XML documents is specified by an XML schema (<https://www.w3.org/XML/Schema>). Over time, the developer community has come to realize that JSON also needs a similar mechanism. One popular option is to use the JSON Schema standard (<http://json-schema.org>). A JSON schema defines the names and types of a message's properties and whether they're optional or required. As well as being useful documentation, these schemas can be used by an application to validate incoming and outgoing messages.

One downside of using a text-based message format is that the messages tend to be verbose, especially XML. Every

message has the overhead of containing the attribute names in addition to their values. Another drawback is the overhead of parsing text, especially when messages are large. Consequently, if efficiency and performance are important, you may want to consider using a binary format.

BINARY MESSAGE PAYLOAD FORMATS

The second category of message formats is binary formats such as Protocol Buffers (<https://developers.google.com/protocol-buffers/docs/overview>) and Avro (<https://avro.apache.org>). These formats are designed to be compact and efficient, which makes them a good choice for high-throughput systems. The downside is that using them is more complicated because of the additional tooling that's required.

Both formats define a typed IDL for defining the structure of your messages. An IDL compiler then generates the message classes along with code that serializes and deserializes the messages. You're forced to take an API-first approach to service design! Moreover, if you write your client in a statically typed language, the compiler checks that it uses the API correctly.

One difference between these two binary formats is that Protocol Buffers uses tagged fields, whereas an Avro consumer needs to know the schema in order to interpret messages. Each field in a Protocol Buffers serialized message is identified by a numeric tag, allowing the consumer to parse messages even if its local schema is missing some fields or has been updated—unknown tags can simply be skipped. In contrast, Avro encodes data without field identifiers, relying entirely on the writer's schema to define the order and types of fields. This design produces more compact messages but requires the consumer to have

access to the correct schema before it can deserialize a message.

To make this practical in a distributed system where producers and consumers evolve independently, Avro is typically used together with a schema registry, such as the Confluent Schema Registry (<https://github.com/confluentinc/schema-registry>). The Confluent Schema Registry is a central service for storing and retrieving Avro schemas and their versions, enforcing compatibility rules when new schemas are registered. When a producer sends a message, it includes a small schema identifier — assigned by the registry — in the message header. A consumer receiving the message uses that identifier to look up the exact writer schema from the registry and then performs schema resolution between the writer's and its own reader schema. For example, during schema resolution, the consumer can fill in any missing fields with the default values defined in its own reader schema.

The schema registry also has an API that enables build tools to enforce compatibility. This mechanism allows Avro to maintain compact binary encoding while supporting safe schema evolution and independent deployment of producers and consumers.

COMPARING TEXT AND BINARY PAYLOAD FORMATS

Table [8.3](#) summarizes the benefits and drawbacks of each option.

Table 8.3 The benefits and drawbacks of text-based vs. binary formats for message payloads

Format	Benefits	Drawbacks
Text	• Human-readable • Simple to process • Self descriptive	• Verbose • Inefficient to process large messages
Binary	• Compact • Fast to process	• Requires more tooling • Not human-readable

Now that we've looked at message payload formats, let's look at how to implement API clients.

8.1.7 Developing API clients

An API client interacts with a service through its APIs. Depending on the API's implementation technology, the client may make HTTP or gRPC requests and process responses, or send and receive asynchronous messages via a message broker. In all cases, the client is responsible for serializing outbound data into the payload format expected by the service and deserializing inbound payloads into domain objects that the application can use.

One challenge when implementing such an API client is defining the classes - Data Transfer Objects (DTOs) - that model the messages it exchanges with the service. As figure [8.3](#) shows, DTOs representing message payloads sent by the

client - such as requests, commands, events - must serialize into the format expected by the service. Similarly, DTOs for message payloads received by the client by the service — such as responses, replies, or consumed events — must deserialize incoming data. In some cases, the client may only need a **projection** of the full schema — a subset of message payload types or fields relevant to its responsibilities.

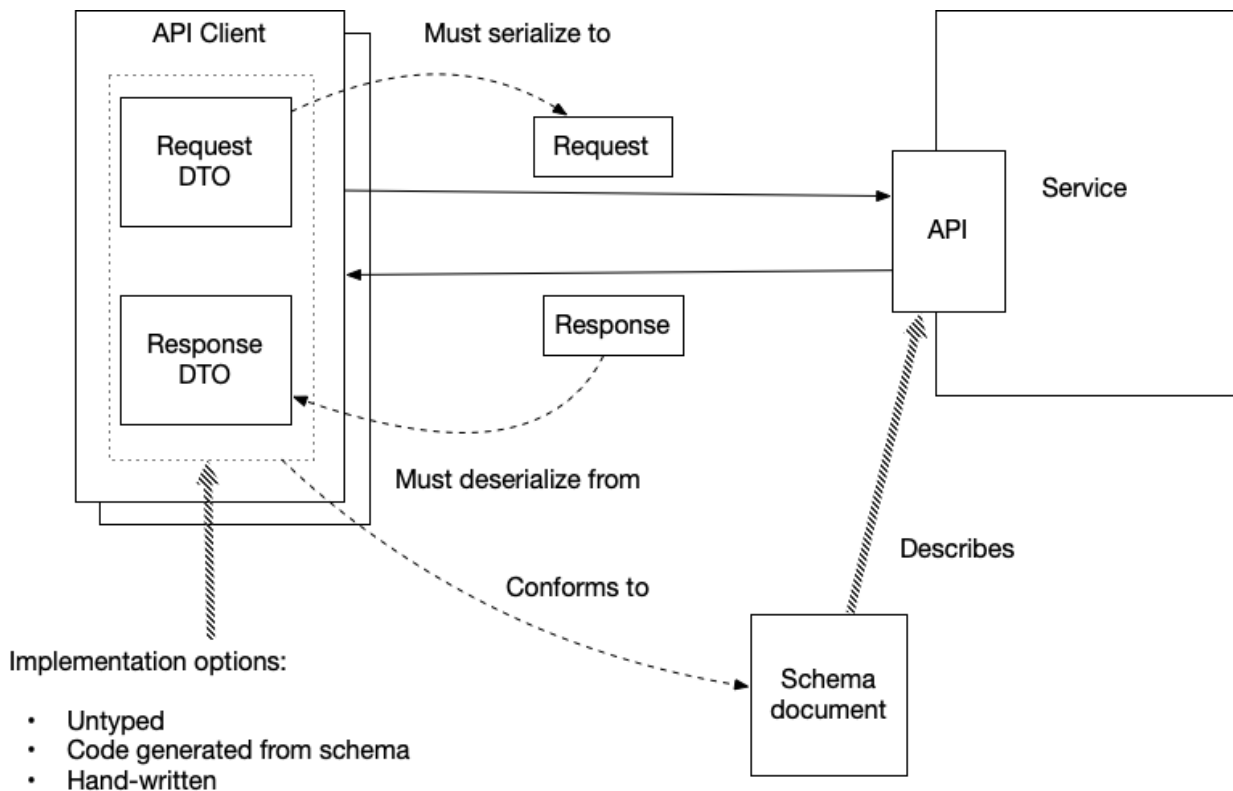


Figure 8.3 An API client must have a data model of the messages that it exchanges with the service. The data model must serialize and deserialize correctly as defined by the schema. It can either be untyped (e.g. hash maps) or consist of handwritten or code-generated classes.

Let's look at the different implementation options for the DTOs.

CHOOSING AN IMPLEMENTATION STRATEGY

There are several options for implementing client classes:

- Untyped data model
- Code-generated classes
- Handwritten classes
- Service team provided libraries

When using a binary message payload format, developers almost always use code generation tools that produce DTOs from the interface definition. However, when using text-based message payload formats, developers can choose between writing the DTOs by hand or using code generation. Alternatively, they can represent messages using an untyped structure, such as a map or dictionary.

When deciding how to implement DTOs, there are several factors to consider:

- Development effort - The time and complexity involved in creating the DTOs. This includes both the initial implementation and the ongoing maintenance required as the API evolves.
- Correctness - How reliably the DTOs conform to the API schema
- Type safety - Whether the client provides compile-time checks that catch schema mismatches or invalid field names.
- Design-time coupling - ideally, the DTOs should only model those parts of the API that the client actually uses. In particular, to reduce design-time coupling, a client should deserialize only those attributes that it cares about and ignore the rest.

Let's look at each option in more detail and examine the trade-offs.

UNTYPED DATA MODEL

A simple way to implement an API client is to represent message payloads using untyped data structures, such as maps or dictionaries, rather than typed classes. In this approach, the client serializes and deserializes message payloads dynamically, relying on field names and types at runtime instead of compile time type checking.

This approach has the following benefits:

- Minimal development effort - No code generation or class modeling is required. The client can construct and parse messages using standard language features or JSON libraries.
- Flexibility - The client can handle messages that evolve over time without recompilation, as new fields are simply ignored or added to the map.

This approach has the following drawbacks:

- Lack of type safety - Errors caused by misspelled field names or incorrect types are detected only at runtime.
- Reduced correctness - The client may drift out of sync with the schema, since there is no compile-time validation.
- Limited tooling support - It is harder to use IDE features such as autocompletion or refactoring when message structures are untyped.

GENERATE CLASSES FROM SCHEMA DOCUMENT

A common and more reliable approach is to generate DTOs directly from the service's schema document. Tools exist for most Interface Definition Languages (IDLs) that can automatically produce request, response, and message classes that conform exactly to the schema. For example, gRPC provides the protocol buffer compiler, and there's

Swagger CodeGen for OpenAPI. You can also use generative AI (GenAI) tools to create or maintain these classes.

This approach has the following benefits:

- Correctness - Generated classes are guaranteed to match the schema, reducing the risk of serialization or deserialization errors.
- Type safety - The client benefits from compile-time checks that prevent schema mismatches and invalid field references.
- Reduced maintenance - When the API changes, regenerating the client updates the affected classes automatically.

This approach has the following drawbacks:

- Build complexity - The development workflow must include code generation and regeneration whenever the schema changes.
- Potential design-time coupling - Generated clients may include types and fields that the consuming service does not use, creating unnecessary design-time coupling.
- Tooling limitations - Some code generators support only specific languages or frameworks, making this approach less portable in polyglot environments.

HAND-WRITTEN DTOS

Another option is to write DTOs by hand. There are two common approaches:

- Handwrite the DTOs from scratch
- Generate DTOs from the schema, remove unused elements and maintain the code manually thereafter

Handwriting DTOs has the following benefits:

- Reduced design-time coupling - the DTOs only have the fields the client needs, avoiding design-time coupling to unused API elements.
- Readability and ergonomics - Handwritten code can match the team's naming conventions, error handling, and coding style. The DTOs can also have methods containing logic, which avoids violating design guidelines, such as the Law of Demeter (https://en.wikipedia.org/wiki/Law_of_Demeter).
- Simpler build - No code generation step needed.

This approach has the following drawbacks:

- Higher development cost - while writing simple DTOs by hand is easy, modeling complex schemas is tedious and time-consuming.
- Schema drift risk - handwritten code can fall out of sync with the schema after API changes unless you implement consumer-driven contract tests or validate the DTOs against the schema.
- Higher maintenance - every schema change requires updating the DTOs manually.
- Inconsistent modeling - different teams may model the same messages differently, increasing integration friction.

USE A LIBRARY PROVIDED BY THE SERVICE TEAM

Another option is for the service team to publish client libraries. This library encapsulates all the API interaction logic, including serialization, deserialization, and communication details such as authentication and error

handling. The service team can develop the library using one of the previous strategies: handwritten or code-generated.

This approach has the following benefits:

- Avoids duplication of effort - Client logic and serialization code are implemented once and reused by all consumers.
- Consistency - All consumers use the same library and therefore handle authentication, errors, and message payload formats consistently.

This approach has the following drawbacks:

- Increased design-time coupling - clients are forced to depend on the entire API, not just the parts that they use
- Reduced flexibility - Consumers are constrained by the abstractions and conventions baked into the shared library, even when they need custom behavior.
- Cost of supporting multiple languages and frameworks - a service team might have to publish libraries for multiple languages and frameworks

Now that we've looked at how to implement API clients, let's look at specific IPC mechanisms used to transport the messages, starting with the Remote procedure invocation (RPI) pattern.

8.2 Communicating using the synchronous remote procedure invocation pattern

When using a remote procedure invocation-based (RPI) IPC mechanism, a client sends a request to a service, and the

service processes the request and sends back a response. Some clients may block waiting for a response, and others might have a reactive, non-blocking architecture. But unlike when using messaging, the client assumes that the response will arrive in a timely fashion.

Figure [8.4](#) shows how RPI works. The business logic in the client invokes a *proxy interface*, implemented by an *RPI proxy* adapter class. The *RPI proxy* makes a request to the service. The request is handled by an *RPI server* adapter class, which invokes the service's business logic via an interface. It then sends back a reply to the *RPI proxy*, which returns the result to the client's business logic.

PATTERN: REMOTE PROCEDURE INVOCATION

A client invokes a service using a synchronous, remote procedure invocation-based protocol, such as REST (<http://microservices.io/patterns/communication-style/messaging.html>).

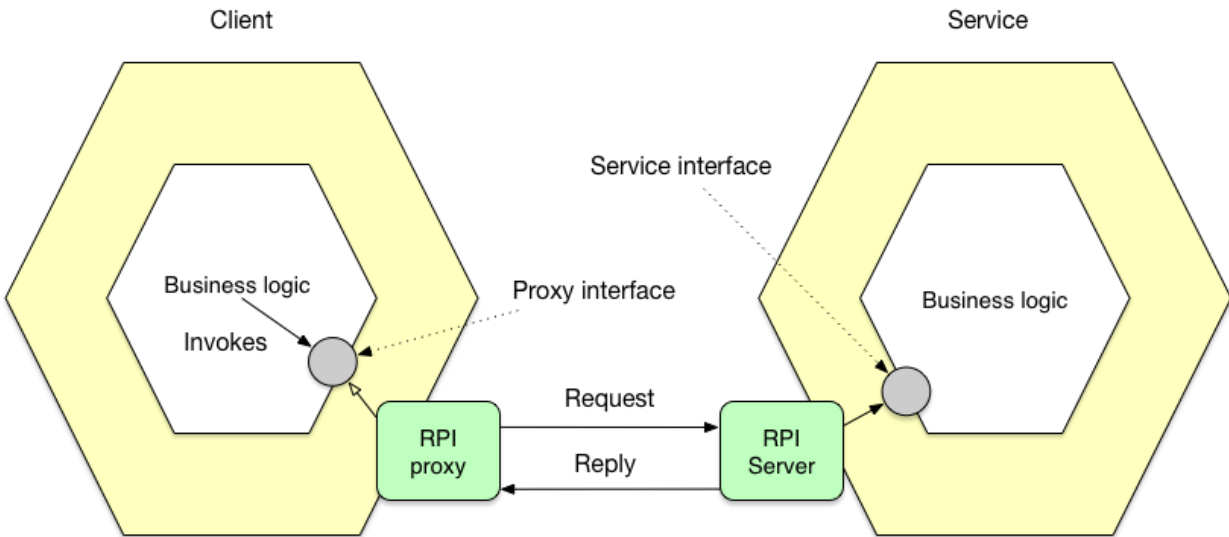


Figure 8.4 The client's business logic invokes an interface that is implemented by an *RPI proxy* adapter class. The *RPI proxy* class makes a request to the service. The *RPI server* adapter class handles the request by invoking the service's business logic.

The *proxy interface* usually encapsulates the underlying communication protocol. There are numerous protocols to choose from. Later in this section, I describe two of the most common options: REST and gRPC. I also cover how to improve the availability of your services by properly handling partial failure and explain why a microservices-based application that uses RPI must use a service discovery mechanism. But first let's look at the benefits and drawbacks of RPI.

8.2.1 Benefits and drawbacks of RPI

The Remote Procedure Invocation (RPI) pattern is widely used in distributed systems and is often the default choice for service-to-service communication. It enables a client to invoke an operation in a remote service as if it were a local method call. This simplicity, however, comes with important trade-offs that architects must understand when deciding between synchronous and asynchronous communication styles.

BENEFITS OF THE RPI PATTERN

- Simple and familiar - Most developers are already comfortable with the request/reply model from programming languages and frameworks that use function calls, HTTP APIs, or gRPC. This familiarity makes RPI easy to understand, implement, and adopt.
- Easy to implement the request/reply interaction style - RPI naturally supports request/reply interactions, where a client sends a request and waits for a response. This makes it straightforward to model query-style operations and command-style requests that require immediate confirmation.
- Simpler system architecture - Since there is no intermediate message broker, the system is easier to deploy, operate, and debug. Messages travel directly from the client to the service, which can reduce latency and operational complexity compared to a broker-based system.

DRAWBACKS OF THE RPI PATTERN

- Runtime coupling between client and service - The client and service must both be available and responsive at the same time for a request to succeed. This tight runtime coupling means that failures, or high latency in one service can quickly propagate to others. To mitigate these risks, services must use resiliency patterns, which are described later in section [Section 8.2.4](#).
- Limited interaction styles - RPI primarily supports the request/reply style of interaction. It does not naturally support other interaction styles. Supporting these patterns usually requires additional infrastructure, such as a message broker or event streaming platform.

- Requires service discovery - Clients must know the network location of the service they want to call. A service discovery mechanism provides this information, either by returning the network address of an available service instance or by giving the client a load-balanced domain name or IP address. I describe service discovery in more detail in section [Section 8.2.5](#).

Despite these drawbacks, RPI is widely used. For example, browser and mobile clients typically use RPI to invoke services. It can also work well for service-to-service communication, most notably in the API Composition pattern described in chapter 10. Let's look at the most common way to implement RPI in a microservice architecture: REST.

8.2.2 Using REST

Today it's fashionable to develop APIs in the RESTful style (https://en.wikipedia.org/wiki/Representational_state_transfer). *REST* is an IPC mechanism that (almost always) uses HTTP. Roy Fielding, the creator of REST, defines it as follows:

REST provides a set of architectural constraints that, when applied as a whole, emphasizes scalability of component interactions, generality of interfaces, independent deployment of components, and intermediary components to reduce interaction latency, enforce security, and encapsulate legacy systems.

— roy.gbiv.com/pubs/dissertation/top.htm

A key concept in REST is a *resource*, which typically represents a single business object, such as a Customer or Product, or a collection of business objects. REST uses the HTTP verbs for manipulating resources, which are referenced using a URL. For example, a GET request returns the representation of a resource, which is often in the form of an

XML document or JSON object, although other formats such as binary can be used. A POST request creates a new resource, and a PUT request updates a resource. The `Order Service`, for example, has a `POST /orders` endpoint for creating an `Order` and a `GET /orders/{orderId}` endpoint for retrieving an `Order`.

Many developers claim their HTTP-based APIs are RESTful. But as Roy Fielding describes in a blog post, not all of them actually are (<http://roy.gbiv.com/untangled/2008/rest-apis-must-be-hypertext-driven>). To understand why, let's take a look at the REST maturity model.

THE REST MATURITY MODEL

Leonard Richardson (no relation to your author) defines a very useful maturity model for REST (<http://martinfowler.com/articles/richardsonMaturityModel.html>) that consists of the following levels.

- *Level 0* — Clients of a level 0 service invoke the service by making HTTP POST requests to its sole URL endpoint. Each request specifies the action to perform, the target of the action (for example, the business object), and any parameters.
- *Level 1* — A level 1 service supports the idea of resources. To perform an action on a resource, a client makes a POST request that specifies the action to perform and any parameters.
- *Level 2* — A level 2 service uses HTTP verbs to perform actions: GET to retrieve, POST to create, and PUT to update. The request's query parameters and body, if any, specify the action's parameters. This enables services to use web infrastructure such as caching for GET requests.

- *Level 3* — The design of a level 3 service is based on the terribly named HATEOAS (Hypertext As The Engine Of Application State) principle. The basic idea is that the representation of a resource returned by a GET request contains links for performing actions on that resource. For example, a client can cancel an order using a link in the representation returned by the GET request that retrieved the order. The benefits of HATEOAS include no longer having to hard-wire URLs into client code (<https://www.baeldung.com/java-hal-vs-hateoas>).

I encourage you to review the REST APIs at your organization to see which level they correspond to.

USING OPENAPI TO DOCUMENT REST APIS

As mentioned earlier in section [Section 8.1.2](#), you must define your APIs using an interface definition language (IDL). Unlike older communication protocols like CORBA and SOAP, REST did not originally have an IDL. Fortunately, the developer community later rediscovered the value of an IDL for RESTful APIs. Today the most popular REST IDL is the OpenAPI Specification (<https://www.openapis.org>).

An OpenAPI document defines the contract of a REST API in a machine-readable format. It describes the API's endpoints, HTTP methods, parameters, request and response bodies, authentication mechanisms, and other metadata. The structure of request and response bodies is defined using a variant of JSON Schema. Tools can use an OpenAPI document to generate client SDKs, server stubs, interactive documentation, and automated tests, making it a central artifact for API design and governance.

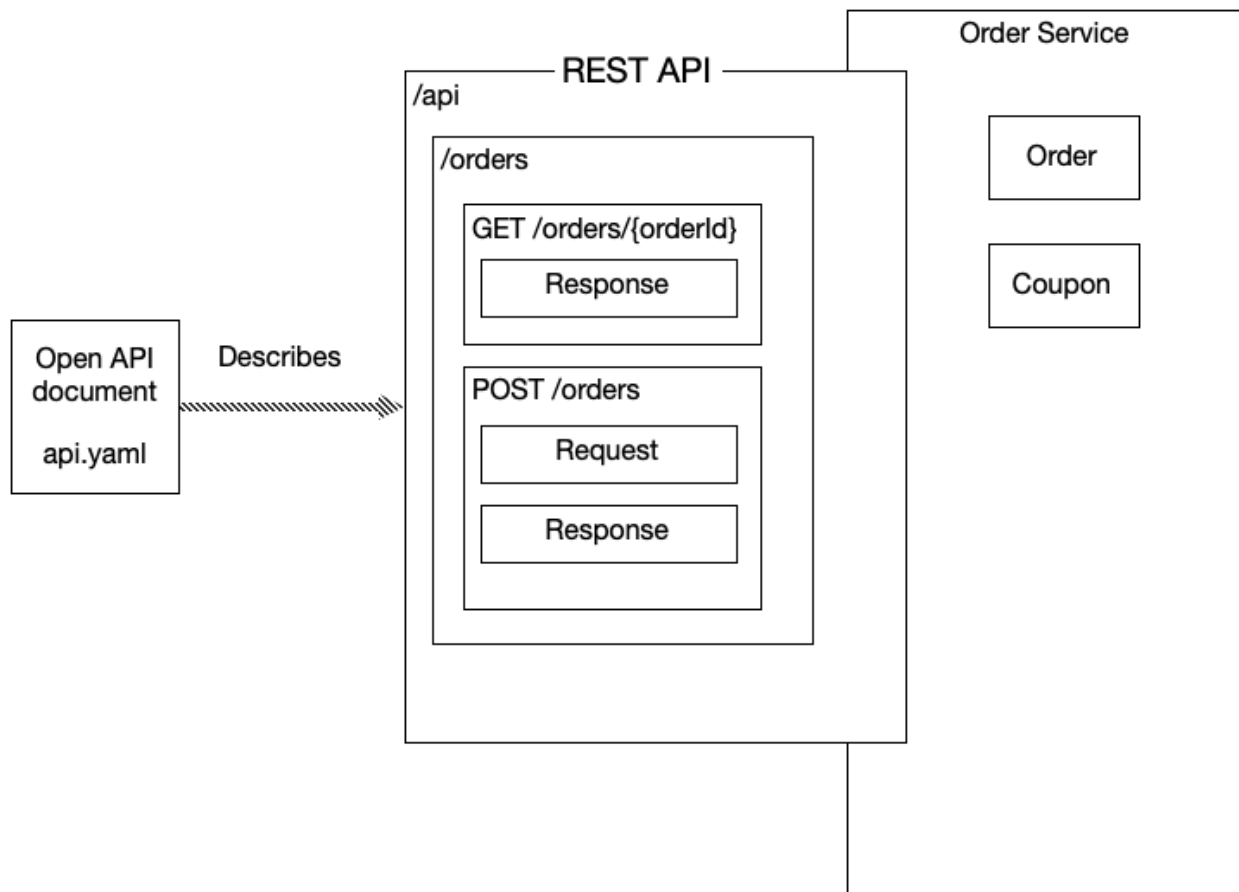


Figure 8.5 An OpenAPI document describes a REST APIs endpoints including URL paths, HTTP methods, request and response bodies, and so on.

An OpenAPI document can be handwritten, but it is often more convenient to generate it automatically from the code. Many web frameworks can produce an OpenAPI document from code annotations or metadata, ensuring that the specification accurately reflects the implemented API. For example, a Spring Boot-based service can use the SpringDoc OpenAPI library (<https://springdoc.org/>) to automatically generate an OpenAPI document from its controller classes and annotations. This approach reduces manual effort, eliminates inconsistencies between code and documentation, and helps keep the API contract up to date as the service evolves.

REST API STYLE GUIDE

As noted earlier in section [Section 8.1.2](#), every organization should define an API style guide to promote consistency across services. REST APIs, in particular, benefit from a dedicated style guide because the HTTP specification leaves many design decisions open to interpretation. Teams can make different choices about URL structure, the use of HTTP methods, pagination, error formats, and response envelopes—differences that quickly lead to inconsistency and confusion for API consumers. A REST API style guide resolves this by defining clear, organization-wide conventions for these recurring design decisions, ensuring that all REST APIs feel coherent and follow the same principles.

A REST API style guide should define consistent conventions for how APIs are designed, named, and evolved. It should cover resource modeling and URL structure, the use of HTTP methods and status codes, and conventions for query parameters, filtering, and pagination. The guide should specify how requests and responses are structured, including naming conventions, envelopes, and error formats. It should also define standards for authentication, headers, content negotiation, and versioning, along with guidance for documentation, deprecation, and security. By establishing these rules, the style guide ensures that all APIs within the organization are predictable, consistent, and easy to consume.

The [Adidas API Guidelines](#) are a good example of an organization-wide REST API style guide. They define clear conventions for URL structure, pagination, error handling, and versioning, providing practical rules and examples that promote consistency across all APIs.

A microservice chassis, which is a framework that services are built upon and is described in chapter 16, can help enforce a REST API style guide by implementing common

conventions. For example, the chassis can automatically configure the REST API to require an access token, and provide classes for pagination, and standardized responses, including error responses, such as RFC 7807 Problem Details (see <https://www.baeldung.com/spring-boot-return-errors-problem-detail>), and collection responses. By embedding these capabilities in the chassis, teams automatically follow the organization's API standards without having to reimplement them in each service.

A style guide ensures that new APIs are consistent, but real systems must also change over time. Even small changes can break clients if they are not managed carefully. Next, we look at practical techniques for evolving a REST API safely and predictably.

EVOLVING A REST API

It is relatively straightforward to make non-breaking changes to a REST/JSON API. You can add new endpoints to expose additional functionality, introduce optional request attributes, and include new response fields that existing clients can safely ignore. As long as these changes do not alter existing behavior or remove fields or endpoints that clients depend on, they are considered backward compatible.

Breaking changes are more difficult because they can disrupt existing clients. Examples include removing or renaming endpoints, changing request or response formats, or modifying field semantics. When a breaking change is necessary, the common approach is to run the old and new versions side by side and gradually migrate clients. One way to implement side-by-side versions is, as figure [8.6](#) shows, to prefix the URL path with the major version number (for example, `/v2/...`). Old clients access the old API via `/api/v1`, whereas updated clients use `/api/v2/...`.

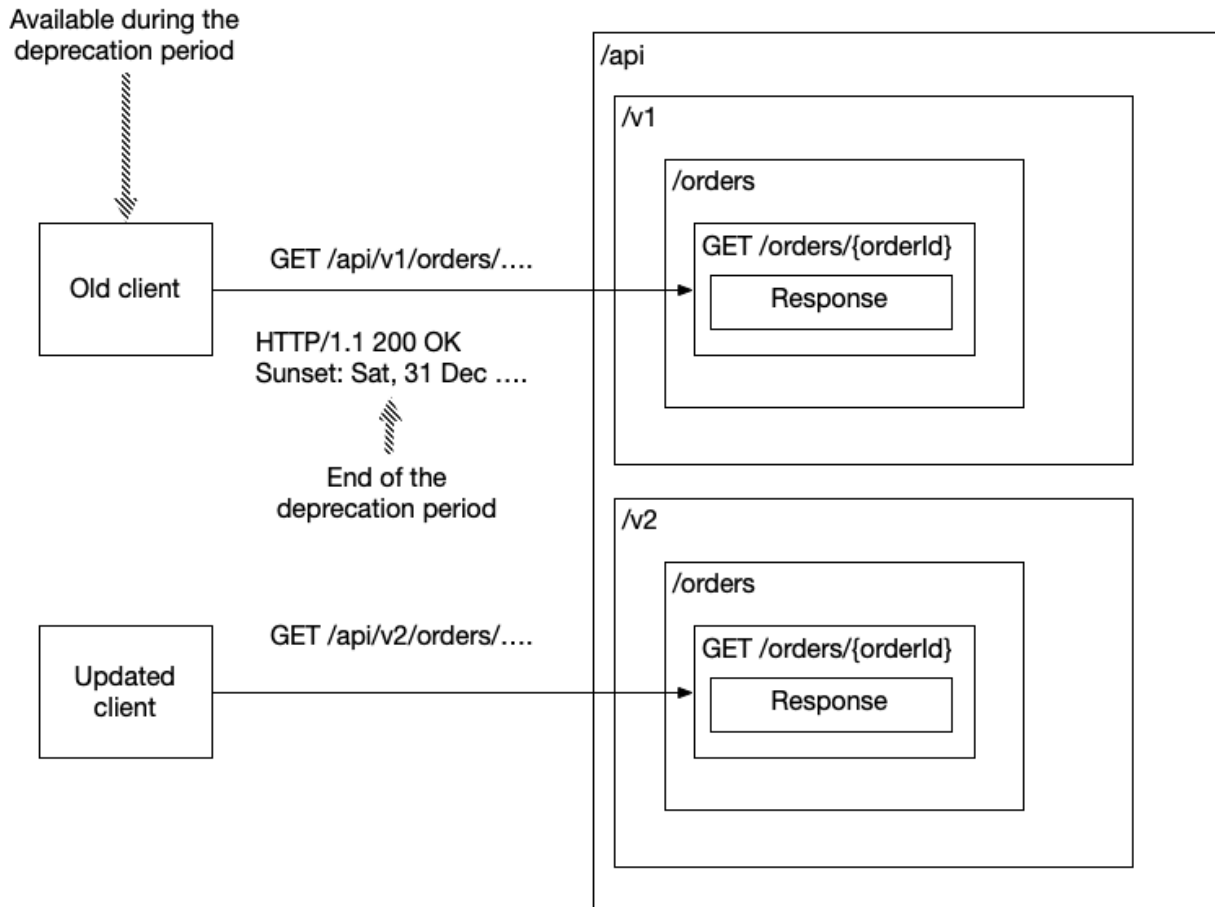


Figure 8.6 A service must implement a breaking API change by supporting both the new and old versions of the API for the deprecation period. Clients that haven't yet been updated can use the old version.

Another option is via media type content negotiation (for example, `application/vnd.example.v2+json`), where the version number represents the major version from the service's semantic version. In order to give clients visibility into when an older version will be retired, a service can communicate deprecation plans using the `Sunset` HTTP response header (<https://datatracker.ietf.org/doc/html/rfc8594>).

THE CHALLENGE OF FETCHING MULTIPLE RESOURCES IN A SINGLE REQUEST

REST resources are usually oriented around business objects, such as `Consumer` and `Order`. Consequently, a common

problem when designing a REST API is how to enable the client to retrieve multiple related objects in a single request. For example, imagine that a REST client wants to retrieve an `Order` and the `Order's Consumer`. A pure REST API would require the client to make at least two requests, one for the `Order` and another for its `Consumer`. A more complex scenario would require even more round-trips and could suffer from excessive latency.

One solution to this problem is for an API to allow the client to retrieve a resource and related resources in a single request. For example, a client could retrieve an `Order` and its `Consumer` using `GET /orders/order-id-1345?expand=consumer`. The `expand` query parameter specifies the related resources to return with the `Order`. This approach works well in many scenarios, but it's often insufficient for more complex scenarios. It's also potentially time-consuming to implement. This has led to the increasing popularity of alternative API technologies such as GraphQL (<http://graphql.org>), which is designed to enable efficient and flexible querying. I describe GraphQL in more detail in chapter 11.

THE CHALLENGE OF MAPPING OPERATIONS TO HTTP VERBS

Another common REST API design problem is how to map the operations you want to perform on a business object to an HTTP verb. A REST API should use `PUT` for updates, but there may be multiple ways to update an order, including canceling it, revising the order, and so on. Also, an update might not be idempotent, which is a requirement for using `PUT`. One solution is to define a sub-resource for updating a particular aspect of a resource. The `Order Service`, for example, has a `POST /orders/{orderId}/cancel` endpoint for canceling orders, and a `POST /orders/{orderId}/revise` endpoint

for revising orders. Another solution is to specify a verb as a URL query parameter, although that's not very RESTful.

This problem of mapping operations to HTTP verbs has led to the growing popularity of alternatives to REST, such as gRPC. GraphQL, which I mentioned earlier, also addresses this problem. I'll shortly discuss gRPC in section [Section 8.2.3](#). But first, let's look at the benefits and drawbacks of REST.

BENEFITS AND DRAWBACKS OF REST

REST offers several important benefits. It is simple and familiar, making it easy for developers to learn and use. An HTTP-based API can be tested directly from a browser or with tools such as Postman or curl, especially when it uses JSON or another text-based format. REST naturally supports request/response communication, works seamlessly through firewalls, and does not require an intermediate message broker, which simplifies the overall system architecture.

However, REST also has some notable drawbacks. It only supports the request/response interaction style and therefore introduces runtime coupling between clients and services — both must be available at the same time for a request to succeed. Clients must know the network locations of service instances, which requires a service discovery mechanism. Fetching multiple resources in a single request can be inefficient, and mapping complex update operations to HTTP verbs is often awkward.

Despite these limitations, REST is widely used, especially as the communication mechanism between browser and mobile clients and backend services. The next section introduces gRPC, a binary schema-based alternative to REST that addresses some of its limitations.

8.2.3 Using gRPC

As mentioned in the preceding section, one challenge with using REST is that because HTTP only provides a limited number of verbs, it's not always straightforward to design a REST API that supports multiple update operations. An IPC technology that avoids this issue is gRPC (<https://www.grpc.io>), a framework for writing cross-language clients and servers (see https://en.wikipedia.org/wiki/Remote_procedure_call for more).

gRPC is a binary message-based protocol, and this means — as mentioned earlier in the discussion of binary message payload formats — you're forced to take an API-first approach to service design. You define your gRPC APIs using a Protocol Buffers-based IDL, which is Google's language-neutral mechanism for serializing structured data. You use the Protocol Buffer compiler to generate client-side stubs and server-side skeletons. The compiler can generate code for a variety of languages, including Java, C#, Node.js, and GoLang. Clients and servers exchange binary messages in the Protocol Buffers format using HTTP/2.

A gRPC API consists of one or more services and request/response message definitions. A *service definition* is analogous to a Java interface and is a collection of strongly typed methods. As well as supporting simple request/response RPC, gRPC also supports streaming RPC. Clients and servers can exchange streams of messages.

Listing [8.1](#) shows an excerpt of the gRPC API for the `OrderService`. It defines several methods, including `createOrder()`. This method takes a `CreateOrderRequest` as a parameter and returns a `CreateOrderReply`.

Listing 8.1 An excerpt of the gRPC API for the Order Service

```
package net.chrichardson.ftgo.orderservice_v1;

service OrderService {
  rpc createOrder(CreateOrderRequest) returns (CreateOrderReply) {}
  rpc cancelOrder(CancelOrderRequest) returns (CancelOrderReply) {}
  rpc reviseOrder(ReviseOrderRequest) returns (ReviseOrderReply) {}
  ...
}

message CreateOrderRequest {
  int64 restaurantId = 1;
  int64 consumerId = 2;
  repeated LineItem lineItems = 3;
  ...
}

message LineItem {
  string menuItemId = 1;
  int32 quantity = 2;
}

message CreateOrderReply {
  int64 orderId = 1;
}
...
```

`CreateOrderRequest` and `CreateOrderReply` are typed messages. For example, `CreateOrderRequest` message has a `restaurantId` field of type `int64`. The field's tag value is 1.

Once you have defined the gRPC API, the `protoc` compiler generates server and client code. The generated server code provides abstract service classes that developers implement to handle incoming requests, while the generated client code includes stubs that make remote procedure calls appear as local method invocations. This approach ensures strong typing, consistency between client and server, and automatic serialization and deserialization of messages.

EVOLVING A GRPC API

It is generally straightforward to make backward-compatible changes to a gRPC API. You can add new service methods, and introduce request and response fields and include new response fields that clients can safely ignore. Because Protocol Buffers uses numeric field tags rather than field names for serialization, existing clients can continue to communicate with newer servers as long as the field numbers and data types remain unchanged. Since all fields are optional, a new server will use default values for any missing fields. Likewise, an old client will ignore unknown fields in the response.

As with REST, making a breaking change to a gRPC API requires running the old and new versions side-by-side until clients have migrated. A common way to achieve this is for the package name to include the major version number:

`net.chrichardson.ftgo.orderservice_v1`,
`net.chrichardson.ftgo.orderservice_v2`, and so on. This approach keeps existing clients compatible while allowing the service to evolve independently.

BENEFITS AND DRAWBACKS OF GRPC

gRPC has several benefits:

- It's straightforward to design an API that has a rich set of update operations.
- It is an efficient, compact IPC mechanism, especially when exchanging large messages.
- Bidirectional streaming enables both RPI and messaging styles of communication.
- Enables interoperability between clients and services written in a wide range of languages.

gRPC also has several drawbacks:

- Browser applications cannot use gRPC directly and must instead use `grpc-web`, which includes a proxy server
- It is more complicated than REST/JSON
- It has a steeper learning curve

gRPC is a compelling alternative to REST, especially for service-to-service communication. But like REST, it's a synchronous communication mechanism, so it also suffers from runtime coupling, which reduces availability and increases latency. Let's look at how to make synchronous communication more resilient.

8.2.4 Implementing resilient synchronous communication

As described in Chapter 4, when one service, such as the `API Gateway`, synchronously invokes another service, such as the `Order Service`, the two services become runtime coupled. The `API Gateway` cannot complete its request and respond to the client until the `Order Service` responds. This dependency introduces the risk of increased latency and reduced availability. If the `Order Service` has high latency or is unavailable, the `API Gateway` may fail to meet its own SLOs. To address these risks, a service that makes synchronous requests must apply resiliency patterns that limit the blast radius of downstream failures and help maintain predictable performance.

Services must use the following resiliency patterns (<https://github.com/resilience4j/resilience4j?tab=readme-ov-file#resilience-patterns>) in order to tolerate high latency and failures while meeting their SLOs:

- Bulkhead - limit the number of outstanding calls to the service and fail fast if the limit has been reached, which typically occurs when the remote service is overloaded.
- Cache - Reduce load and latency by using cached responses.
- Circuit breaker - after repeated failures, fail fast for a timeout period before attempting to invoke the remote service again
- Fallback - Provide a degraded but useful response when the primary path fails, such as default data or a previously cached value.
- Rate limiter - limit the frequency of calls to avoid overloading the remote service.
- Retry - after a failure, automatically retry the call.
- Time limiter - wait a timeout period for a response from the remote service. If the maximum rate is exceeded, the attempt to call the service fails. It ensures that the client does not violate its latency SLO or consume valuable resources for too long.

Let's discuss the Circuit breaker and Fallback patterns in more detail since they are essential for preventing cascading failures and ensuring services can still return useful responses during disruptions.

CIRCUIT BREAKER PATTERN

The Circuit breaker pattern behaves analogously to a circuit breaker in an electrical circuit that interrupts the power flow when it detects excessive current. Figure [8.7](#) shows the behavior of the circuit breaker. It tracks the number of successful and failed requests, and if the error rate exceeds some threshold, it trips the circuit breaker so that further attempts fail immediately for a timeout period. Initially, the

circuit breaker is closed which allows requests to be made. But if there are too many failed requests - indicating that the other service is temporarily unavailable and further calls are pointless or even harmful - the circuit breaker trips and transitions to the open state. For the duration of the timeout period, requests fail immediately. After the timeout period, the circuit breaker transitions to the half-open state, which allows through test requests. If those succeed, the circuit breaker transitions to the closed state. Otherwise, it opens again.

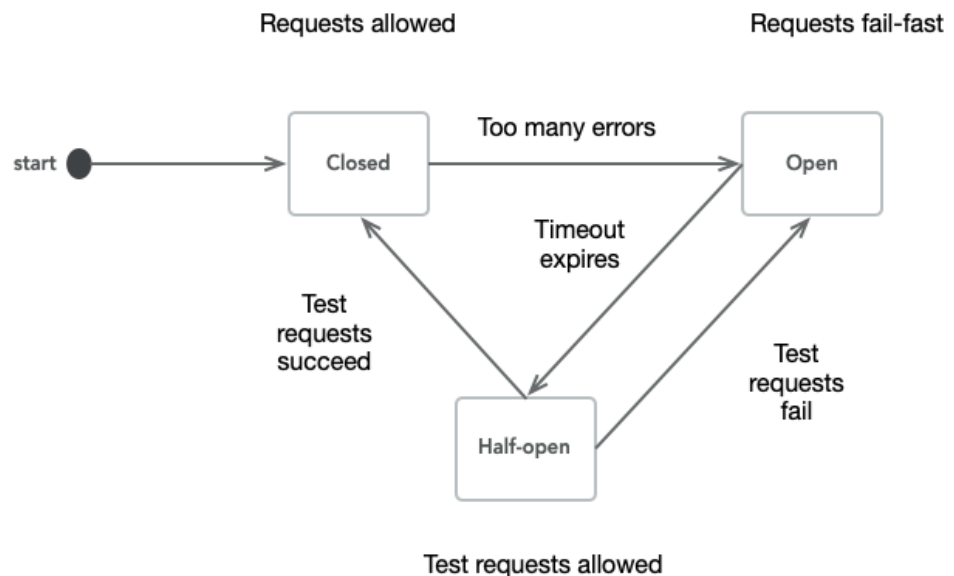


Figure 8.7 A circuit breaker implements a state machine. Initially it is in the `closed` state, which allows requests. But if there are too many errors, it transitions to the `open` state, which fails requests. After a timeout period, it then transitions to a `half-open` state, which allows a test request to be made. If the test request succeeds, the circuit breaker transitions to the `closed` state, otherwise it goes back to the `open` state.

The Circuit breaker pattern is useful because a large number of requests failing suggests that the service is unavailable. It's pointless for a client to send more requests, which will ultimately fail. What's more, those requests might even contribute to the problem. It makes sense, therefore, to temporarily stop sending requests to give the service a chance to recover.

FALLBACK PATTERN: RECOVERING FROM AN UNAVAILABLE SERVICE

The other resiliency patterns are only part of the solution. While the bulkhead, rate limiting, timeout, and circuit breaker patterns help prevent cascading failures and resource exhaustion, they do not by themselves provide a response to the client when the remote service is unavailable. You must also implement a fallback strategy, on a case-by-case basis, for handling this scenario.

Consider, for example, the design shown in figure [8.8](#) where the API gateway implements the `findOrder()` query operation using the API composition pattern, which is described in more detail in chapter 10. The implementation of the `GET /orders/{orderId}` endpoint invokes several services, including the `Order Service`, `Kitchen Service`, and `Delivery Service`, and combines the results. The API Gateway must handle the failure of any one of those services. One option is for the API Gateway to simply return an error to its client. For example, this makes sense when the `Order Service` is unavailable. This response is useless without the `Order` details.

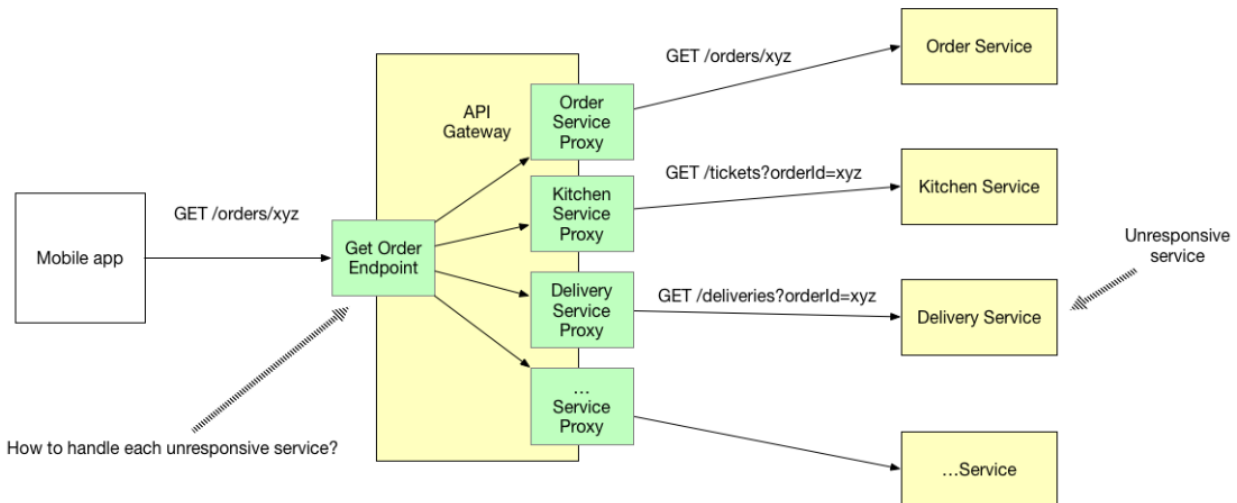


Figure 8.8 An API gateway that implements an endpoint using API composition can use the fallback pattern to return a useful response even when one or more of the services it calls are unavailable. For example, if the `Delivery Service` is unavailable, the `GET /orders/{orderId}` endpoint can return a response containing either empty or cached delivery information.

While the data from the `Order Service` is essential, it's possible that the data from other services is less important to the client. Consequently, returning a fallback value, such as either a default value or a cached response, may make sense. If the `Delivery Service` is unavailable, the API gateway could, for example, return either a cached version of its data or omit it from the response since a client could still display useful information about the `Order`.

IMPLEMENTING RESILIENCY PATTERNS

How you implement these patterns depends on the programming language and framework. For example, Resilience4j (<https://github.com/resilience4j/resilience4j>) is a popular Java library that implements the resiliency patterns listed at the start of section [Section 8.2.4](#). An RPI proxy in a Java application can call the library directly. Alternatively, in Spring Boot-based application, you can, as listing [8.2](#) shows, annotate the RPI proxy's methods. Method

calls are intercepted by Spring AOP and wrapped with resiliency logic.

Listing 8.2 In a Spring Boot application, Resilience4j can be used by annotating client proxy methods. Method calls are intercepted by Spring AOP and wrapped with resiliency logic.

```
class OsoAuthorizer {

    @CircuitBreaker(name = "osoAuthorizer")
    @TimeLimiter(name = "osoAuthorizer")
    @Retry(name = "osoAuthorizer", fallbackMethod = "isAuthorizedFallback")
    public CompletableFuture<Boolean> isAuthorized(String user, String action, String securitySystem) {
        ...
    }

    private CompletableFuture<Boolean> isAuthorizedFallback(String user, String action, String securitySystem, Exception exception) {
        return CompletableFuture.completedFuture(false);
    }
}
```

Other language ecosystems have libraries that provide similar capabilities. The Polly library is popular in the .NET community (<https://github.com/App-vNext/Polly>) that's similar to Resilience4j. For other ecosystems, libraries such as Cockatiel (<https://www.npmjs.com/package/cockatiel>) for Node.js, CircuitBreaker (<https://pypi.org/project/circuitbreaker>) for Python, and failsafe-go (<https://github.com/failsafe-go/failsafe-go>) for Go offer the same core resilience patterns, helping developers implement robust, fault-tolerant communication in their respective languages.

ABOUT NETFLIX'S HYSTRIX

The Netflix Hystrix library (<https://netflixtechblog.com/introducing-hystrix-for-resilience-engineering-13531c1ab362>) popularized the need for resilience patterns in a microservice architecture. This library provided a circuit breaker API that implemented a combination of the resiliency patterns including Circuit Breaker, and Fallback. Hystrix is now in maintenance mode, and modern Java applications should use Resilience4j.

The resiliency patterns are typically provided by the Microservice Chassis - the framework that services are built upon - which is described in more detail in chapter 16. The chassis will, for example, include a library, such as Resilience4j, as a dependency.

It's essential that you design your services to handle partial failure, but that's not the only problem you need to solve when using RPI. Another problem is that in order for one service to invoke another service using RPI, it needs to know its network location - either its DNS name or IP address. On the surface, this sounds simple, but in practice it's a challenging problem. You must use a service discovery mechanism. Let's look at how that works.

8.2.5 Using service discovery

Let's imagine you're writing an RPI proxy that invokes a service's REST API. In order to make a request, your code needs to know the network location (IP address and port) of a service instance. In a traditional application running on physical hardware, the network locations of service instances are usually static. For example, your code could read the

network locations from a configuration file that's occasionally updated. But in a modern, cloud-based microservices application, it's usually not that simple. As is shown in figure [8.9](#), a modern application is much more dynamic.

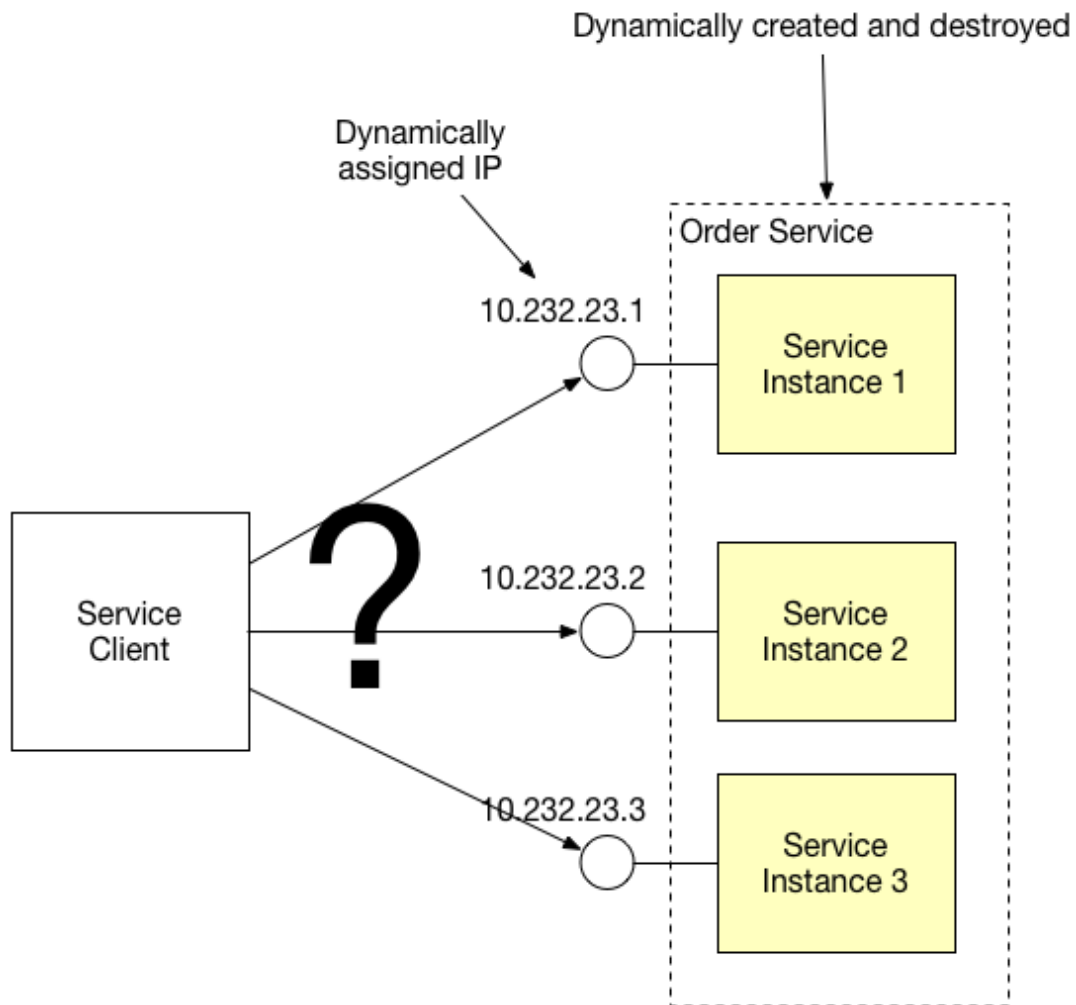


Figure 8.9 Service instances have dynamically assigned IP addresses.

Service instances have dynamically assigned network locations. Moreover, the set of service instances changes dynamically because of autoscaling, failures, and upgrades. Consequently, your client code must use a service discovery mechanism.

OVERVIEW OF SERVICE DISCOVERY

Service discovery is the mechanism in a distributed system, such as a microservice architecture, that enables services to find and communicate with each other without needing hardcoded network locations. It's conceptually quite simple: its key component is a service registry, which is a database of the network locations of an application's service instances. The service discovery mechanism updates the service registry when service instances start and stop. When a client invokes a service, the service discovery mechanism queries the service registry to obtain a list of available service instances and routes the request to one of them.

Service discovery was once a major challenge in microservice architectures, but modern platforms have made it largely automatic. Early systems used application-level service discovery, which relied on applications querying dedicated service registries such as Netflix Eureka, Consul, or Zookeeper that tracked the IP addresses of running service instances. Today, service discovery is provided by the deployment platforms, such as Kubernetes. As a result, most teams no longer need to implement service discovery at the application level. Let's briefly look at application-level service discovery before exploring platform-provided service discovery.

APPLYING THE APPLICATION-LEVEL SERVICE DISCOVERY PATTERNS

One way to implement service discovery is for the application's services and their clients to interact with the service registry. Figure [8.10](#) shows how this works. A service instance registers its network location with the service registry. A service client invokes a service by first querying the service registry to obtain a list of service instances. It then sends a request to one of those instances.

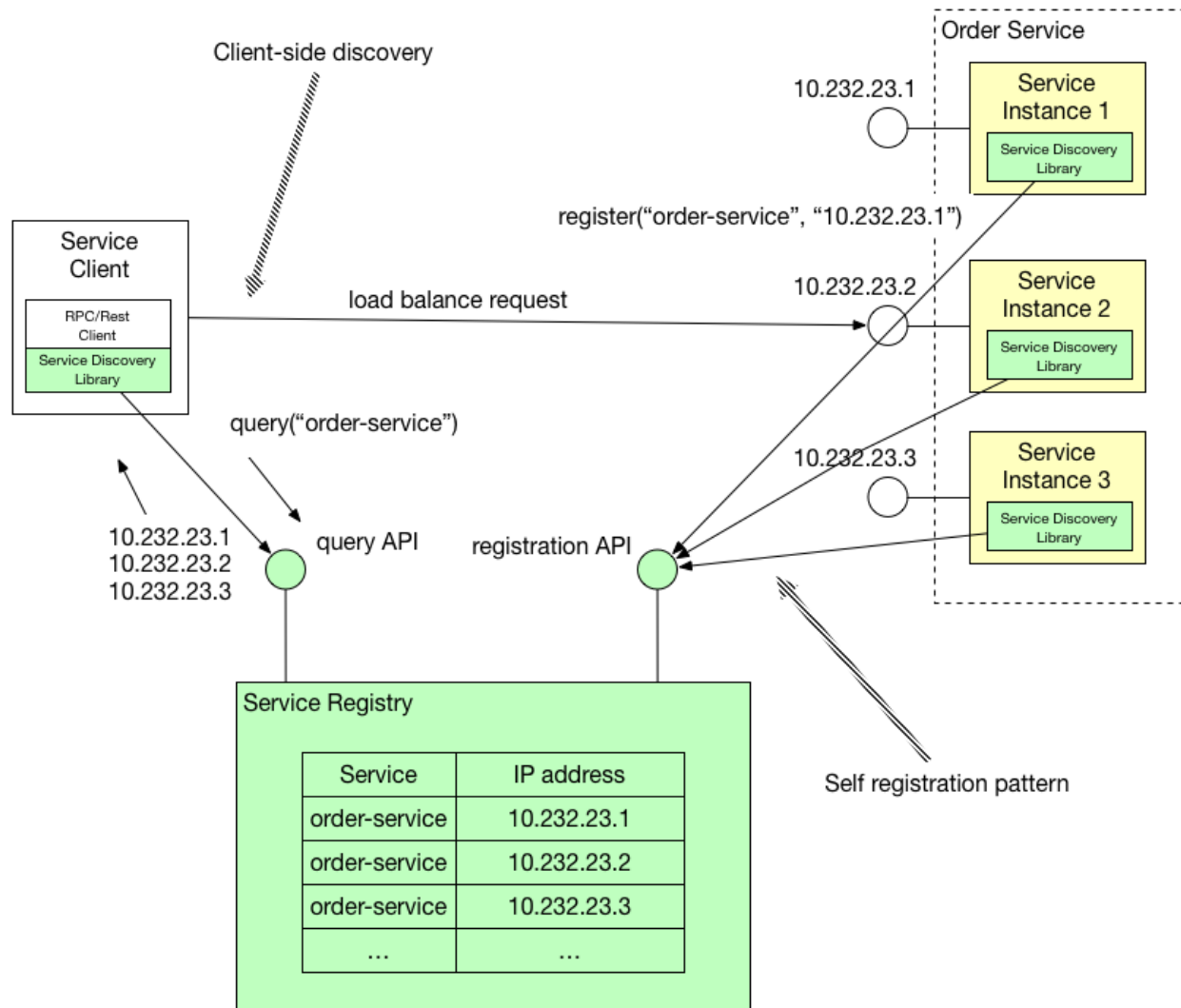


Figure 8.10 The service registry keeps track of the service instances. Clients query the service registry to find network locations of available service instances.

This approach to service discovery is a combination of two patterns. The first pattern is the Self registration pattern. A service instance invokes the service registry's registration API to register its network location. It may also supply a health check URL, described in more detail in chapter 11. The *health check* URL is an API endpoint that the service registry invokes periodically to verify that the service instance is healthy and available to handle requests. A service registry may require a service instance to

periodically invoke a "heartbeat" API in order to prevent its registration from expiring.

PATTERN: SELF REGISTRATION

A service instance registers itself with the service registry. See <http://microservices.io/patterns/self-registration.html>.

The second pattern is the Client-side discovery pattern. When a service client wants to invoke a service, it queries the service registry to obtain the list of the service's instances. To improve performance, a client might cache the service instances. The service client then uses a load-balancing algorithm, such as a round-robin or random, to select a service instance. It then makes a request to the selected service instance.

PATTERN: CLIENT-SIDE DISCOVERY

A service client retrieves the list of available service instances from the service registry and load balances across them. See <http://microservices.io/patterns/client-side-discovery.html>.

Application-level service discovery has been popularized by Netflix and Pivotal. Netflix developed and open-sourced several components: Eureka, a highly available service registry, the Eureka Java client, and Ribbon, a sophisticated HTTP client that supports the Eureka client. Pivotal developed Spring Cloud, a Spring-based framework that makes it remarkably easy to use the Netflix components. Spring Cloud-based services automatically register with

Eureka, and Spring Cloud-based clients automatically use Eureka for service discovery.

One benefit of application-level service discovery is that it handles the scenario when services are deployed on multiple deployment platforms. Imagine, for example, you've deployed only some of the services on Kubernetes, discussed in chapter 12, and the rest is running in a legacy environment. Application-level service discovery using Eureka, for example, works across both environments, whereas Kubernetes-based service discovery only works within Kubernetes.

Another benefit is that it enables intelligent client-side routing of requests. For example, a modern service registry, such as AWS Cloud Map (<https://docs.aws.amazon.com/cloud-map/latest/dg/what-is-cloud-map.html>) can return metadata, such as the service instances' location or version, that can inform the client's routing decisions. A client could, for example, route to a service instance that runs in the same availability zone in order to minimize latency and avoid cross-zone data transfer costs.

One drawback of application-level service discovery is that you need a service discovery library for every language — and possibly framework — that you use. Spring Cloud only helps Spring developers. If you're using some other Java framework or a non-JVM language such as Node.js or GoLang, you must find some other service discovery framework. Another drawback of application-level service discovery is that you're responsible for setting up and managing the service registry, which is a distraction. As a result, it's usually better to use a service discovery mechanism that's provided by the deployment infrastructure.

APPLYING THE PLATFORM-PROVIDED SERVICE DISCOVERY PATTERNS

Later in chapter 12, you'll learn that many modern deployment platforms such as Kubernetes have a built-in service registry and service discovery mechanism. The deployment platform typically gives each service a long-lived DNS name. A service client makes a request to the DNS name/VIP, and the deployment platform automatically routes the request to one of the available service instances. As a result, service registration, service discovery, and request routing are entirely handled by the deployment platform. Figure 8.11 shows how this works.

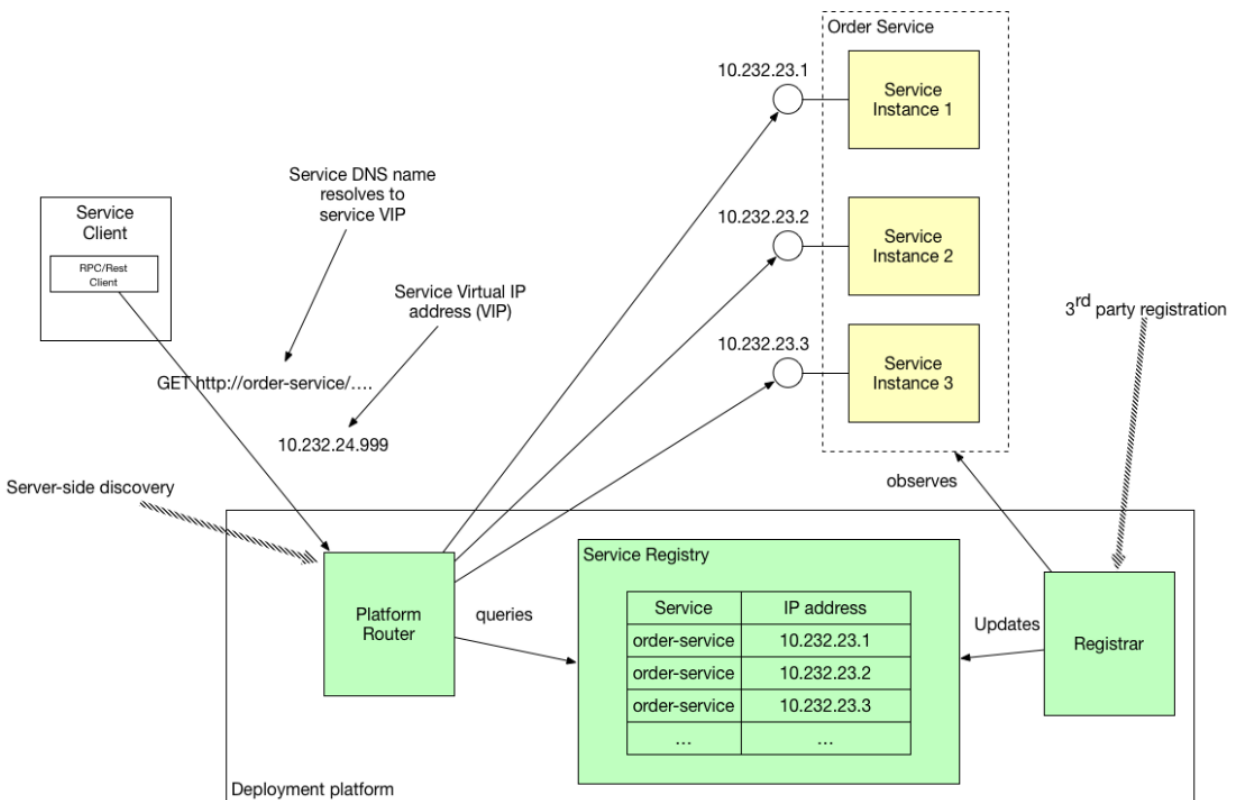


Figure 8.11 The platform is responsible for service registration, discovery, and request routing. Service instances are registered with the service registry by the *registrar*. Each service has a network location, a DNS name/virtual IP address. A client makes a request to the service's network location. The router queries the service registry and load balances requests across the available service instances.

The deployment platform includes a service registry that tracks the IP addresses of the deployed services. In this example, a client accesses the `Order Service` using the DNS name `order-service`, which resolves to a virtual IP address, such as `10.232.23.1`. The deployment platform automatically load balances requests across the three instances of the `Order Service`.

This approach is a combination of two patterns:

- *3rd party registration pattern*—Instead of a service registering itself with the service registry, a third party called the *registrar*, which is typically part of the deployment platform, handles the registration.
- *Server-side discovery pattern*—Instead of a client querying the service registry, it makes a request to a DNS name, which resolves to a request router that queries the service registry and load balances requests.

PATTERN: 3RD PARTY REGISTRATION

Service instances are automatically registered with the service registry by a third party. See <http://microservices.io/patterns/3rd-party-registration.html>.

PATTERN: SERVER-SIDE DISCOVERY

A client makes a request to a router, which is responsible for service discovery. See <http://microservices.io/patterns/server-side-discovery.html>.

The details of platform-provided service discovery depend on the deployment technology. For example, as noted earlier, Kubernetes provides each deployed service with a domain name. Services deployment on AWS EC2 can use Application Load Balancers (ALBs) for service discovery. Similarly, services deployed on AWS ECS (Elastic Container Service) can use one of the following options: ALBs, AWS Cloud Map and the Route 53 DNS service, or ECS Connect. In all these cases, service discovery is handled by the platform rather than by application-level code.

The key benefit of platform-provided service discovery is that all aspects of service discovery are entirely handled by the deployment platform. Neither the services nor the clients contain any service discovery code. Consequently, the service discovery mechanism is readily available to all services and clients regardless of which language or framework they're written in.

One drawback of platform-provided service discovery is that it only supports the discovery of services that have been deployed using the platform. For example, as mentioned earlier when describing application-level discovery, Kubernetes-based discovery only works for services running on Kubernetes. Another drawback is that the client is not involved in any routing decisions. Despite these drawbacks, I recommend using platform-provided service discovery whenever possible.

Now that we've looked at synchronous IPC using REST or gRPC, let's take a look at the alternative: asynchronous, message-based communication.

8.3 Communicating using the asynchronous messaging pattern

When using messaging, services communicate by asynchronously exchanging messages. The messages are typically exchanged using a *message broker*, which acts as an intermediary between the services, although another option is to use a brokerless architecture, where the services communicate directly with each other. A messaging client typically interacts with the service using one of the following interaction styles:

- asynchronous request/reply - the client makes a request to a service by sending it a message, which eventually sends back a reply message. The client doesn't block waiting for a reply and assumes that it won't receive the reply immediately.
- one-way notification - makes a request to a service by sending it a message without expecting or looking for a reply.
- publish/subscribe - the service publishes events that are consumed by one or more clients.

PATTERN: MESSAGING

A client invokes a service using asynchronous messaging. See <http://microservices.io/patterns/communication-style/messaging.html>.

I start this section with an overview of messaging. I show how to describe a messaging architecture independently of any specific messaging technology. Next, I briefly compare and contrast brokerless and broker-based architectures and describe the criteria for selecting a message broker. I then discuss several important topics, including scaling consumers while preserving message ordering, detecting

and discarding duplicate messages, and sending and receiving messages as part of a database transaction.

Let's begin by looking at how messaging works.

8.3.1 Overview of messaging

An extremely useful model of messaging is defined in the book *Enterprise Integration Patterns* (Addison-Wesley Professional, 2003) by Gregor Hohpe and Bobby Woolf. In this model, messages are exchanged over message channels. A sender (an application or service) writes a message to a channel, and a receiver (an application or service) reads messages from a channel.

Figure [8.12](#) shows how messaging works. Messages are exchanged over channels (www.enterpriseintegrationpatterns.com/MessageChannel.html), which are an abstraction of the messaging infrastructure. The business logic in the sender invokes a *sending port* interface, which encapsulates the underlying communication mechanism. The *sending port* is implemented by a *message sender* adapter class, which sends a message to a receiver via a message channel. A *message handler* adapter class in the receiver is invoked to handle the message. It invokes a *receiving port* interface implemented by the consumer's business logic. Any number of senders can send messages to a channel. Similarly, any number of receivers can receive messages from a channel.

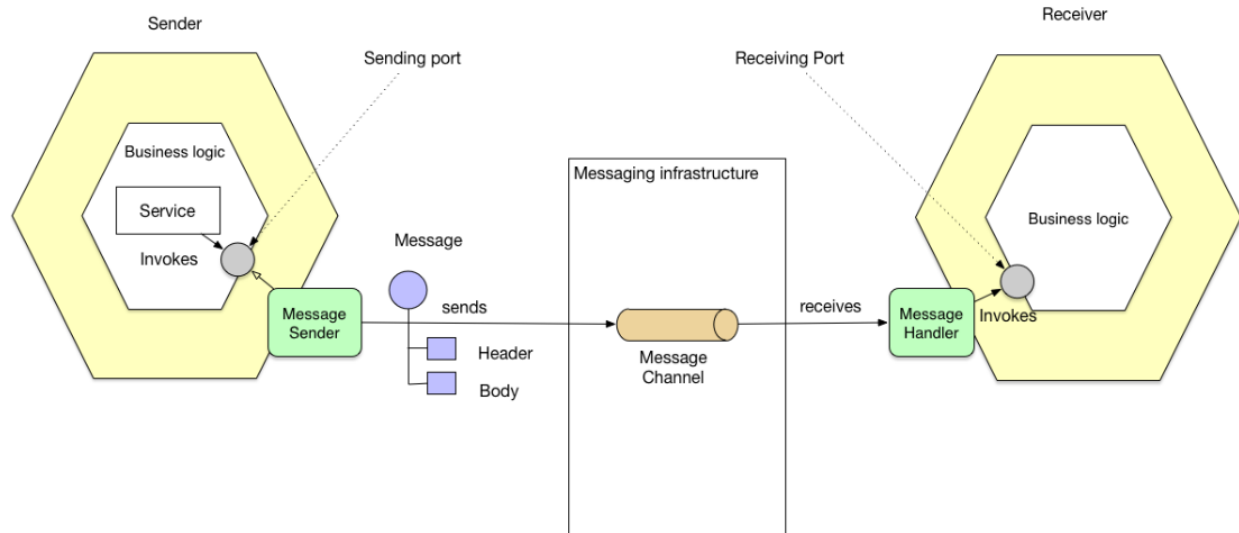


Figure 8.12 The business logic in the sender invokes a sending port interface, which is implemented by the message sender's adapter. The message sender sends a message to a receiver via a message channel. The message channel is an abstraction of messaging infrastructure. A message handler adapter in the receiver is invoked to handle the message. It invokes the receiving port interface implemented by the receiver's business logic.

Let's take a closer look at messages, and then channels.

ABOUT MESSAGES

A message consists of a header and a message body (<https://www.enterpriseintegrationpatterns.com/Message.html>). The *header* is a collection of name-value pairs - metadata that describes the data being sent. In addition to name-value pairs provided by the message's sender, the message header contains name-value pairs, such as a unique *message id* generated by either the sender or the messaging infrastructure, and an optional *return address*, which specifies the message channel that a reply should be written to. The message *body* is the data being sent, in either text or binary format.

There are four kinds of messages:

- *Document* — A generic message that contains only data. The receiver decides how to interpret it.
- *Event* — A message indicating that something notable has occurred in the sender. An event is often a domain event, which represents a state change of a domain object such as an `Order`, or a `Customer`.
- *Command* — A message that's the equivalent of an RPC request. It specifies the operation to invoke and its parameters.
- *Reply* - the message sent in response to a command

The approach to the microservice architecture described in this book uses commands and events extensively.

Let's now look at channels - the mechanism by which services communicate.

ABOUT MESSAGE CHANNELS

As mentioned earlier, a channel is an abstraction of the underlying messaging technology. As I describe soon, it's typically implemented by a message broker such as Apache Kafka or RabbitMQ.

There are two kinds of channels: point-to-point (<https://www.enterpriseintegrationpatterns.com/PointToPointChannel.html>) and publish-subscribe (<https://www.enterpriseintegrationpatterns.com/PublishSubscribeChannel.html>):

- A *point-to-point* channel delivers a message to exactly one of the consumers reading from the channel. Services use point-to-point channels for the one-to-one interaction styles described earlier. For example, a command message is often sent over a point-to-point channel.

- A *publish-subscribe* channel delivers each message to all of the consumers subscribed to the channel. Services use publish-subscribe channels for the one-to-many interaction styles described earlier. For example, an event message is usually sent over a publish-subscribe channel.

IMPLEMENTING MESSAGE CHANNELS

As I describe in section [Section 8.3.3](#), you can use a variety of messaging technologies to implement message channels. In enterprise applications, message channels are typically implemented by a message broker, which is an infrastructure service that receives messages from senders, stores them, and then eventually delivers them to receivers. You can choose from many message brokers, including RabbitMQ, Apache Kafka, and AWS SQS, each of which makes different trade-offs.

This chapter focusses on using message brokers, but it's worth remembering that message channels can also be implemented using broker-less messaging technologies. One common example is webhooks (<https://www.baeldung.com/cs/webhooks-polling-real-time-information>), where applications register callback URLs that are invoked when an event occurs. For example, a CI/CD SaaS application, such as CircleCI, can use webhooks exposed by GitHub.com (see <https://docs.github.com/en/webhooks>) to be notified when changes are pushed to a branch. Another example of brokerless messaging is Chronicle Queue (<https://github.com/OpenHFT/Chronicle-Queue>), which is a library that implements message channels using shared memory.

Now let's look at the benefits and drawbacks of asynchronous messaging.

BENEFITS AND DRAWBACKS OF ASYNCHRONOUS MESSAGING

Asynchronous messaging has many advantages:

- *No runtime coupling* - Services exchange messages without waiting for a response from the other service. Unlike with a synchronous request/response protocol such as HTTP, where both the client and service must be available for the duration of the exchange, the sender doesn't care if the receiver is temporarily unavailable.
- *Workload management and buffering* - The message broker buffers messages until they can be processed, allowing the system to absorb spikes in demand or handle slow consumers gracefully. For example, an online store can continue accepting orders from customers even when the order-fulfillment service is slow or temporarily offline.
- *Location independence* — A client makes a request by simply sending a message to the appropriate channel. The client is completely unaware of the service instances. It doesn't need to use a discovery mechanism to determine the location of a service instance.
- *Flexible communication* — Messaging supports all of the interaction styles described earlier.
- *Explicit interprocess communication* — RPC-based mechanisms attempt to make invoking a remote service look the same as calling a local service. But due to the laws of physics and the possibility of partial failure, they're in fact quite different. Messaging makes these differences very explicit so developers aren't lulled into a false sense of security.

Asynchronous messaging also has some downsides:

- *Potential performance bottleneck* — There is a risk that the message broker could be a performance bottleneck. Fortunately, many modern message brokers are designed to be highly scalable.
- *Potential single point of failure* — It's essential that the message broker is highly available — otherwise, system reliability will be impacted. Fortunately, most modern brokers have been designed to be highly available.
- *Additional operational complexity* — The messaging system is yet another infrastructure component that must be provisioned, configured, and operated.

8.3.2 Implementing the interaction styles using messaging

One of the valuable features of messaging is that it's flexible enough to support all the interaction styles described in [Section 8.1.1](#). Some interaction styles are implemented directly by messaging, while others must be implemented on top of it.

Let's look at how to implement each interaction style, starting with request/response and asynchronous request/response.

IMPLEMENTING REQUEST/RESPONSE AND ASYNCHRONOUS REQUEST/RESPONSE

When a client and service interact using either request/response or asynchronous request/response, the client sends a request and the service sends back a reply. The difference between the two interaction styles is that with request/response the client expects the service to respond

immediately, whereas with asynchronous request/response there is no such expectation. Messaging is inherently asynchronous and so only provides asynchronous request/response. But a client could block until a reply is received.

The client and service implement the asynchronous request/response style interaction by exchanging a pair of messages. As figure 8.13 shows, the client sends a command message, which specifies the operation to perform and parameters, to a point-to-point messaging channel owned by a service. The service processes the requests and sends a reply message, which contains the outcome, to a point-to-point channel owned by the client.

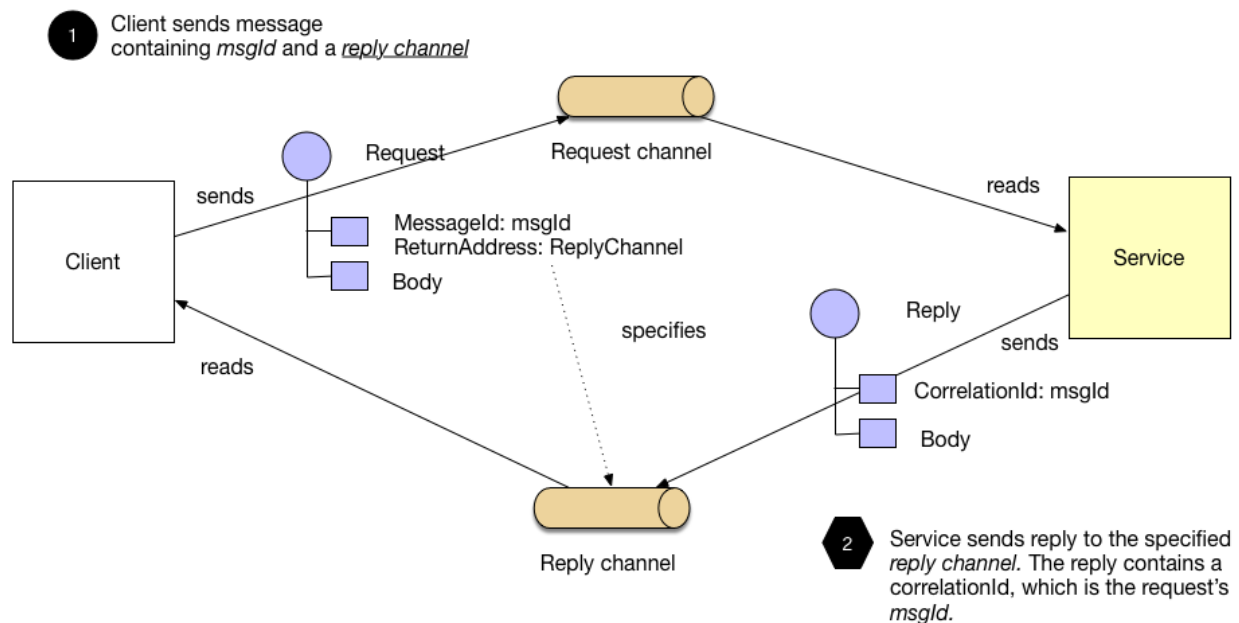


Figure 8.13 Implementing asynchronous request/response by including a reply channel and message identifier in the request message. The receiver processes the message and sends the reply to the specified reply channel.

The client must tell the service where to send a reply message and must match reply messages to requests. Fortunately, solving these two problems isn't that difficult. The client sends a command message that has a *reply*

channel header. The service writes the reply message, which contains a *correlation id* (<https://www.enterpriseintegrationpatterns.com/patterns/messaging/CorrelationIdentifier.html>) that has the same value as *message identifier*, to the reply channel. The client uses the *correlation id* to match the reply message with the request.

Because the client and service communicate using messaging, the interaction is inherently asynchronous. In theory, a messaging client could block until it receives a reply, but in practice the client will process replies asynchronously. What's more, replies are typically processed by any one of the client's instances.

IMPLEMENTING ONE-WAY NOTIFICATIONS

Implementing one-way notifications is straightforward with asynchronous messaging. The client sends a message, typically a command message, to a point-to-point channel owned by the service. The service subscribes to the channel and processes the message. It doesn't send back a reply.

IMPLEMENTING PUBLISH/SUBSCRIBE

Messaging has built-in support for the publish/subscribe style of interaction. A client publishes a message to a publish-subscribe channel that can have multiple consumers. As described in chapters 9 and 5, services use publish/subscribe to publish domain events, which represent changes to domain objects. The service that publishes the domain events owns a publish-subscribe channel, whose name is typically derived from the domain class. For example, the `Order Service` publishes `Order` events to an `Order` channel, and the `Delivery Service` publishes `Delivery` events to

a `Delivery` channel. A service that's interested in a particular domain object's events only has to subscribe to the appropriate channel.

IMPLEMENTING PUBLISH/ASYNC RESPONSES

The publish/async responses interaction style is a higher level style of interaction that's implemented by combining elements of publish/subscribe and request/response. A client publishes a message, which specifies a *reply channel* header, to a publish-subscribe channel. A consumer sends a reply message containing a *correlation id* to the reply channel. The client gathers the responses by using the *correlation id* to match the reply messages with the request.

Each service in your application that has an asynchronous API will use one or more of these implementation techniques. A service that has an asynchronous API for invoking operations will have a message channel for command messages. Similarly, a service that publishes events will publish them to an event message channel.

Let's now look at designing a message architecture using channels.

8.3.3 Designing an asynchronous messaging architecture

When designing an asynchronous architecture, it's often tempting to immediately start thinking in terms of a particular messaging technology. You could, for example, immediately start designing Apache Kafka topics. However, because there are many messaging technologies, each with different trade-offs, it's better to take a more deliberate, two-step approach:

1. Define the requirements for each channel
2. For each channel, pick the messaging technology that best supports the channel's requirements

Let's start by identifying what to consider when defining message channel requirements.

DEFINE THE REQUIREMENTS FOR EACH MESSAGE CHANNEL

Each message channel has distinct requirements that influence which messaging technology is the best fit. Key factors include the following:

- *Channel type* — what type of channel point-to-point or publish/subscribe is needed?
- *Delivery guarantees* — what kind of delivery guarantees are required, best-effort, at least once?
- *Messaging ordering* — should messages be delivered in the order they were sent?
- *Competing consumers* — does the message broker support competing consumers?
- *Persistence* — should messages survive broker crashes?
- *Durability* — if a consumer reconnects to the message broker, does it need the messages that were sent while it was disconnected?
- *Scalability* — what are the scalability requirements?
- *Latency* — what is the required end-to-end latency?

By evaluating these requirements first, you can make an informed, technology-agnostic design decision before committing to a particular broker or messaging platform.

In later chapters, you will see that the service collaboration patterns that use asynchronous messaging require message

channels that provide at-least-once delivery, ordered delivery, support for competing consumers, persistence, and durability. What's more, the required channel type — point-to-point or publish/subscribe — depends on the specific pattern.

OVERVIEW OF MESSAGE BROKERS

You can choose from many message brokers. Examples of popular open source message brokers include the following, ActiveMQ (<http://activemq.apache.org>), RabbitMQ (<https://www.rabbitmq.com>), and Apache Kafka (<http://kafka.apache.org>). You could also use cloud-based messaging services, including AWS Kinesis (<https://aws.amazon.com/kinesis/>), AWS SQS (<https://aws.amazon.com/sqs/>), and AWS SNS (<https://aws.amazon.com/sns/>). What's more, some clouds provide open-source message brokers as managed services, which avoids the operational overhead of running brokers yourself. For example, AWS offers Amazon Managed Streaming for Apache Kafka (MSK), Apache ActiveMQ, and RabbitMQ.

Let's now look at different message brokers implement message channels.

HOW MESSAGE BROKERS IMPLEMENT CHANNELS

Each message broker implements the message channel concept in a different way. As table [8.4](#) shows, JMS-based brokers such as ActiveMQ have queues and topics, AMQP-based message brokers such as RabbitMQ have exchanges and queues, Apache Kafka has topics, AWS Kinesis has streams, and AWS SQS has queues. What's more, some message brokers offer more flexible messaging models than

the simple message and channels abstraction described in this chapter.

Table 8.4 Each message broker implements the message channel concept in a different way.

	Point-to-point channel	Publish-subscribe channel
ActiveMQ	Queue	Topic
Apache Kafka	Topic	Topic
RabbitMQ	Exchange + Queue	Fanout exchange and a queue per consumer
AWS Kinesis	Stream	Stream
AWS SQS	Queue	-
AWS SNS	-	Topic

Almost all of the message brokers described in table [8.4](#) support both point-to-point and publish-subscribe channels. The exceptions are AWS SQS and AWS SNS, each one only supports one type of channel.

HOW MESSAGE BROKERS SATISFY A CHANNEL'S REQUIREMENTS

Each message broker satisfies the channel requirements described earlier in different ways:

- *Delivery guarantees* - Most brokers provide at-least-once delivery by default, ensuring that messages are not lost even if a consumer or broker fails. Apache Kafka and AWS Kinesis also support exactly-once semantics under certain configurations. AWS SQS and SNS provide at-least-once delivery, while RabbitMQ and ActiveMQ can be configured for at-most-once or at-least-once behavior depending on acknowledgment settings.
- *Message ordering* - Kafka and Kinesis preserve ordering within a partition or shard. ActiveMQ and RabbitMQ maintain order within a single queue. SQS FIFO queues guarantee ordered delivery, while SQS standard queues and SNS topics do not. In general, preserving global ordering reduces throughput and scalability.
- *Competing consumers* - Kafka and Kinesis preserve message ordering within each partition, while SQS FIFO and ActiveMQ preserve ordering within message groups, all while enabling concurrent processing across partitions or groups.
- *Persistence* - Kafka, Kinesis, SQS, and SNS always persist messages so they can survive broker restarts or failures. RabbitMQ and ActiveMQ can also persist messages, but this behavior depends on their configuration.
- *Durability* - Durable queues and topics in ActiveMQ and RabbitMQ, and Kafka's replicated logs, ensure that consumers can reconnect and continue from where they left off. SQS, SNS, and Kinesis automatically retain messages for a configurable period, allowing consumers to recover after transient failures.
- *Scalability* - Kafka, Kinesis, SQS and SNS are highly scalable, distributed systems designed for massive

throughput. RabbitMQ and ActiveMQ scale vertically or through clustering but are better suited for moderate workloads.

- *Latency* - RabbitMQ and ActiveMQ can achieve low, millisecond-level latency by using in-memory messaging and sacrificing persistence and durability. Kafka and Kinesis favor throughput and durability over latency, typically achieving end-to-end latencies in the 10ms-100ms range. SQS and SNS have variable but generally higher latencies due to their managed, multi-tenant nature.

Each broker therefore reflects a different set of trade-offs among durability, ordering, throughput, latency, and operational complexity. Choosing the right one depends on the specific requirements of the message channel and the service collaboration pattern.

8.3.4 Evolving an asynchronous API

Asynchronous messaging APIs, such as those that publish domain events or implement command/reply interactions, inevitably change as a system and its domain model evolve. Consider, for example, an `Order Service` that publishes events, such as `Order Created`, and `Order Fulfilled`. Examples of seemingly simple changes include adding attributes to events, such as adding a `discountCode` to the `Order Created` event. The event types themselves might also change. For example, to improve how the business is modelled, `Order Fulfilled` could be replaced by `Order Ready For Pickup` and `Order Picked Up`. Just as REST or gRPC APIs must evolve without breaking existing clients, asynchronous APIs must also evolve without disrupting existing consumers of messages.

Before we look in more detail at the different ways asynchronous APIs can evolve, it's important to understand one key difference between synchronous and asynchronous communication. Unlike REST or gRPC APIs, where clients and services interact in real time, messages in an asynchronous system can outlive the version of the service that produced them.

PERSISTENCE AND MESSAGE LONGEVITY

Evolving a message-based API is more complex than changing a synchronous one. In a request/response interaction, a client only exchanges messages with the current version of a service, so compatibility concerns are limited to services that are running at the same time. In contrast, messages in an asynchronous architecture are persistent. They can remain in message channels long after they were sent, especially events. A consumer might process a message hours, days, or even months after it was sent. As a result, new versions of services must be able to process messages produced by older versions.

Let's now look at a few different ways that events can evolve.

EVOLVING EVENT-BASED APIS: ADDING ATTRIBUTES TO EVENTS

One common change is to add attributes to an event. For example, while implementing coupons, the `Order Management` team would most likely add a `couponID` attribute to the `Order Created` event.

This looks like a straightforward change since an old consumer could simply ignore the newly added attribute. However, because events are often persistent, an updated

consumer must be capable of handling older events that are missing this attribute. For JSON and ProtoBuf messages, the deserialized event object's attribute would have a default value determined by the field's type. But when using AVRO, the new schema must define a default value for the attribute, otherwise the deserialization logic will throw an exception.

EVOLVING EVENT-BASED APIS: CHANGING EVENT TYPES

Another common change is evolving the event types. For example, as mentioned earlier, the `Order Management` team replace the `Order Fulfilled` event with the `Order Ready For Pickup` and `Order Picked Up` events. Since this change would break all consumers that consumed the `Order Fulfilled` event, the `Order Service` must support both versions of the schema simultaneously.

One common solution is dual publishing. During the deprecation period the `Order Service` publishes all three events. Once the consumers have been changed to consume the `Order Ready For Pickup` and `Order Picked Up` events, the `Order Service` can stop publishing the `Order Fulfilled` event. Interestingly, however, consumers might still need to handle the older events.

EVOLVING ASYNCHRONOUS REQUEST/RESPONSE APIS

Evolving command/reply-based messaging APIs is similar to evolving synchronous APIs. An updated consumer might still read an older message, although commands and replies are typically retained for a much shorter period of time. Non-breaking changes include adding optional attributes to

commands or ignorable attributes to replies. Breaking changes include renaming channels, commands, or adding mandatory fields. As always, implementing breaking changes requires side-by-side APIs, which can be implemented using versioned channels or by including schema version identifiers in message headers.

As described in section [Section 8.1.2](#), it's important to write an API specification for a service. Let's look at how to do that for an asynchronous API.

8.3.5 AsyncAPI - an IDL for messaging-based APIs

As mentioned earlier, an IDL is valuable because it provides a precise, machine-readable contract between services. It enables automated code generation, validation, and compatibility checking, and helps teams evolve APIs safely and consistently. In the context of asynchronous messaging, the IDL must describe not only the structure of the messages—defined using formats such as JSON Schema, Protocol Buffers, or Avro — but also the channels that those messages flow through. In this section, we will explore AsyncAPI specification and ecosystem, which attempts to be the equivalent of OpenAPI for messaging-based APIs.

As figure [8.14](#) shows, an AsyncAPI specification for a service's asynchronous API defines the names of the message channels, the direction of message flow (send or receive), the types of messages exchanged over each channel, and the schemas that describe their structure. This broader view of the contract captures both what messages look like and how they are communicated, enabling consistent documentation, validation, and tooling across services.

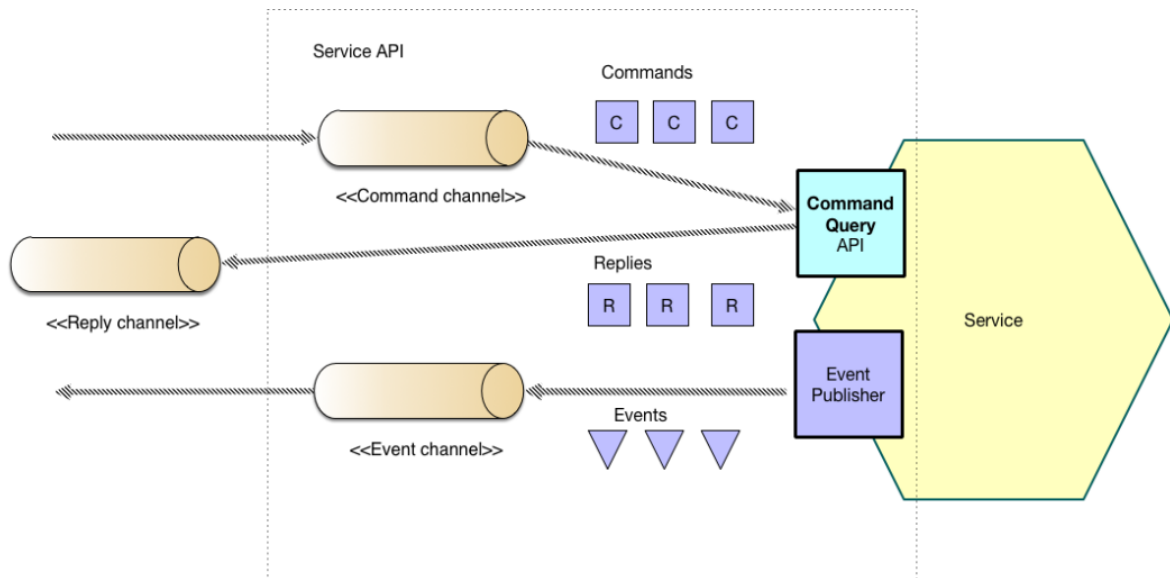


Figure 8.14 An AsyncAPI specification for a service's asynchronous API defines the names of the message channels, the direction of message flow (send or receive), the types of messages exchanged over each channel, and the schemas that describe their structure.

While not as universal as OpenAPI for REST APIs, AsyncAPI (<https://www.asyncapi.com/en>) has evolved to become a useful tool for documenting messaging APIs. Let's look at how it works.

AN OVERVIEW OF ASYNCAPI

AsyncAPI is an IDL for defining messaging APIs - channels and message types - in a machine-readable format. Despite the AsyncAPI's website emphasizing event-driven architectures, AsyncAPI is generally applicable for describing APIs that use asynchronous messaging - not just events. What's more, it does not just describe a service's API. AsyncAPI also describes how a service interacts with the messaging system. There's also an ecosystem of libraries and tools for creating, validating, and consuming AsyncAPI documents.

An AsyncAPI document is a JSON or YAML document that describes the message channels through which a service

communicates and the messages sent and received through those channels. The three key types of objects defined by the document are:

- channels - the message channels that the service uses
- messages - the structure of the messages that are exchanged over those channels
- operations - the actions that the service performs on a channel, i.e. sending or receiving messages

A channel object has a name and a list of messages that flow through the channel.

A message object has a name and defines the schemas of the message's headers and payload. The headers schema defines the names and types of the message's headers, while the payload schema defines the structure of the message payload. Both the headers and payloads schema are usually defined using a superset of JSON Schema Specification Draft 07, although other schemas, such as Avro and Protocol Buffers, are supported.

An operation object defines an action - `send` or `receive` - that the service performs on a channel. It specifies the message channel that messages are sent to or received from. The operation object also defines the channel's messages that are sent or received by this operation. A `receive` operation can optionally specify one or more reply messages that are sent in response, allowing request/reply communication patterns to be modeled.

AN EXAMPLE ASYNCAPI DOCUMENT

An AsyncAPI document that describes an event publisher consists of the channels that the events are published to, messages that define the event schemas, and operations

that send events to those channels. For example, listing [8.3](#) shows an excerpt of the AsyncAPI document that describes the `Customer Service`, which publishes various events including `CustomerCreatedEvent`. The AsyncAPI document defines the channels, the messages that each channel transports, the messages' schemas, and the operations performed on each channel.

Listing 8.3 An excerpt from an AsyncAPI document that defines a service publishing events such as CustomerCreatedEvent to the com.example.Customer channel.

```
channels:
  com.example.Customer:
    address: com.example.Customer
    messages:
      com.example.CustomerCreatedEvent:
        $ref: "#/components/messages/com.example.CustomerCreatedEvent"
      ...
messages:
  com.example.CustomerCreatedEvent:
    headers:
      $ref: "#/components/schemas/HeadersNotDocumented"
    payload:
      schemaFormat: application/vnd.aai.asyncapi+json;version=3.0.0
      schema:
        $ref: "#/components/schemas/com.example.CustomerCreatedEvent"
    name: com.example.CustomerCreatedEvent
    title: com.example.CustomerCreatedEvent
    ...
components:
  schemas:
    com.example.CustomerCreatedEvent:
      title: CustomerCreatedEvent
      type: object
      properties:
        id:
          type: string
      examples:
        - id: string
operations:
  io.eventuate.exampleapp.events.publisher.CustomerEventPublisherImpl:
    action: send
    channel:
      $ref: "#/channels/com.example.Customer"
    messages:
```



```
- $ref: "#/channels/com.example.Customer/messages/com.example.CustomerCreatedEvent"
```

This part of the AsyncAPI document describes how the `CustomerCreatedEvent` message is published. The `channels` section defines a logical messaging channel named `io.eventuate.exampleapp.events.common.Customer`, which represents the Kafka topic or message destination where customer-related events are sent. Under this channel, a single message type—`CustomerCreatedEvent`—is listed, and its full definition is referenced from the `components/messages` section.

The `messages` section specifies the structure of that message, including its optional headers and a payload that references a schema (`CustomerCreatedEvent`) defined in `components/schemas`. That schema describes the data shape of the event payload - in this case a simple object containing an `id` property of type `string`.

Finally, the `operations` section ties everything together. It defines an operation named `CustomerEventPublisherImpl` whose action is `send`, meaning this service publishes the `CustomerCreatedEvent` message to the `Customer` channel.

An AsyncAPI document describing a consumer would be similar. The main difference is that the operation is `receive` instead of `send`.

ASYNCAPI TOOLING AND LIBRARIES

The AsyncAPI ecosystem includes a variety of libraries and tools for creating and validating AsyncAPI documents, as well for generating application code and documentation. For example, Springwolf is an open-source framework that integrates AsyncAPI with Spring Boot applications. Similar to

SpringDoc OpenAPI library, Springwolf automatically generates and serves an AsyncAPI document describing your application. It uses introspection and reflection along with annotations to generate the AsyncAPI document, which is available at the `/springwolf/docs` endpoint. Springwolf also provides a simple web UI similar to Swagger UI.

Another useful tool is 'asyncapi/cli', a command line tool for working with AsyncAPI documents. It provides several commands including `validate`, which validates an AsyncAPI document, and `generate`, which generates application code and documentation. For example, `asyncapi validate springwolf.json` validates a document, and `asyncapi generate fromTemplate springwolf.json @asyncapi/html-template --output build/docs --use-new-generator` generates HTML documentation for the `springwolf.json` AsyncAPI document.

Now that we've looked at how to use AsyncAPI to document asynchronous APIs, let's look at how to solve three important design problems.

8.3.6 Pattern: Ordered competing receivers

One challenge with implementing a service that receives messages is how to increase throughput by horizontally scaling message receivers while preserving message ordering. It's common to need multiple instances of a service to process messages concurrently. Moreover, even a single service instance will probably use threads to concurrently process multiple messages. Using multiple threads and service instances to concurrently process messages increases the throughput of the application. But the challenge with processing messages concurrently is ensuring that each message is processed once and in the correct order.

For example, imagine three instances of a service reading from the same point-to-point channel, where a sender publishes `Order Created`, `Order Updated`, and `Order Cancelled` event messages sequentially. A simplistic messaging implementation could deliver each of those messages concurrently to a different receiver. Because of delays due to network issues or garbage collection, messages might be processed out of order, leading to inconsistent or invalid application state. In theory, a service instance might process the `Order Cancelled` message before another service processes the `Order Created` message!

A common solution used by modern message brokers, such as Apache Kafka and AWS Kinesis, to scale consumers while preserving ordering is to use *sharded* (partitioned) channels. Figure [8.15](#) shows how this works. The solution has three parts:

1. A sharded channel consists of two or more shards, each of which behaves like an independent channel.
2. The sender specifies a shard key in the message's header, which is typically an arbitrary string or sequence of bytes. The message broker uses the shard key to assign the message to a particular shard/partition. It might, for example, select the shard by computing a hash of the shard key modulo the number of shards.
3. The messaging broker groups together multiple instances of a receiver and treats them as the same logical receiver. Apache Kafka, for example, uses the term *consumer group*. The message broker assigns each shard to a single receiver. Shards are reassigned dynamically as receivers start up or shut down.

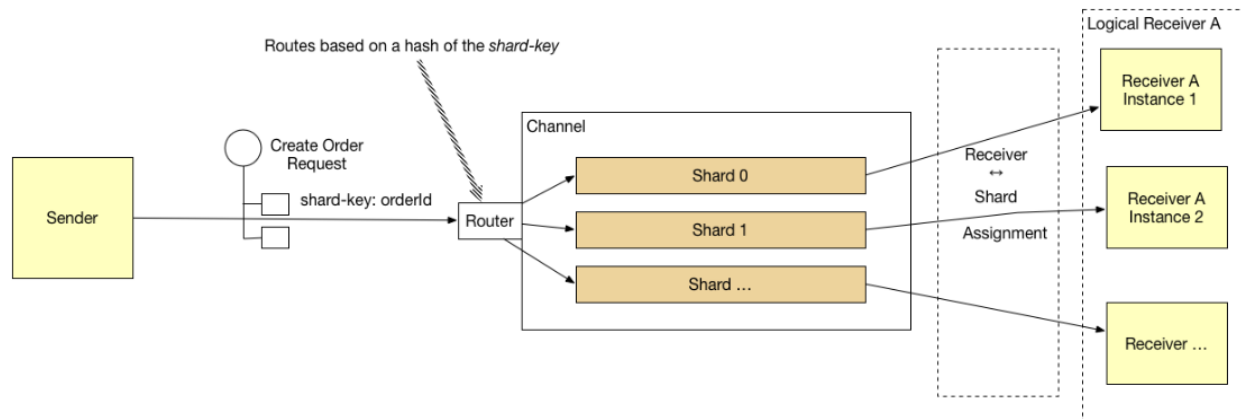


Figure 8.15 Scaling consumers while preserving message ordering by using a sharded (partitioned) message channel. The sender includes the shard key in the message. The message broker writes the message to a shard determined by the shard key. The message broker assigns each partition to an instance of the replicated receiver.

In this example, each `Order` event message uses its `orderId` as the shard key. Each event for a particular order is published to the same shard, which is read by a single consumer instance. As a result, those messages are guaranteed to be processed in order.

8.3.7 Pattern: Idempotent consumer pattern

Another challenge to address when using messaging is dealing with duplicate messages. A message broker should ideally deliver each message only once, but guaranteeing exactly - once delivery is usually too costly. What's more, doing so would require the message broker and service's database to participate in a distributed transaction. As a result, most message brokers guarantee to deliver a message *at least* once.

When the system is working normally, a message broker that guarantees at-least-once delivery will deliver each message only once. But a failure of a client, network, or message broker can result in a message being delivered multiple times. Say a client crashes after processing a

message and updating its database — but before acknowledging the message. The message broker will deliver the unacknowledged message again, either to that client when it restarts or to another instance of the same service.

Ideally, you should use a message broker that preserves ordering when redelivering messages. Imagine that the client processes an `Order Created` event followed by an `Order Cancelled` event for the same `Order`, and that somehow the `Order Created` event wasn't acknowledged. The message broker should redeliver both the `Order Created` and `Order Cancelled` events. If it only redelivers the `Order Created` event, the client may undo the canceling of the `Order`.

You can handle duplicate messages in one of two ways:

- Write idempotent message handlers
- Track messages and discard duplicates

Let's look at each option.

WRITING IDEMPOTENT MESSAGE HANDLERS

If the application logic that processes messages is idempotent, then duplicate messages are harmless. Application logic is *idempotent* if calling it multiple times with the same input values has no additional effect. For example, canceling an already cancelled order is an idempotent operation. So is creating an order with a client-supplied id. An idempotent message handler can be safely executed multiple times, provided that the message broker preserves ordering when redelivering messages.

Unfortunately, application logic is often not idempotent, or you may be using a message broker that doesn't preserve ordering when redelivering messages. In this situation, to

prevent bugs, you must write message handlers that track messages and discard duplicate messages.

TRACKING MESSAGES AND DISCARDING DUPLICATES

Consider, for example, a message handler that authorizes a consumer's credit card. It must authorize the card exactly once for each order. This is an example of application logic that has a different effect each time it's invoked. If duplicate messages caused this message handler to execute multiple times, the application would behave incorrectly. The message handler that executes this kind of application logic must achieve idempotence by detecting and discarding duplicate messages.

A simple solution is for a message consumer to track the messages that it has processed using the `message id` and discard any duplicates. It could, for example, store the `message id` of each message it consumes in a database table. Figure 8.16 shows how to do this using a dedicated table.

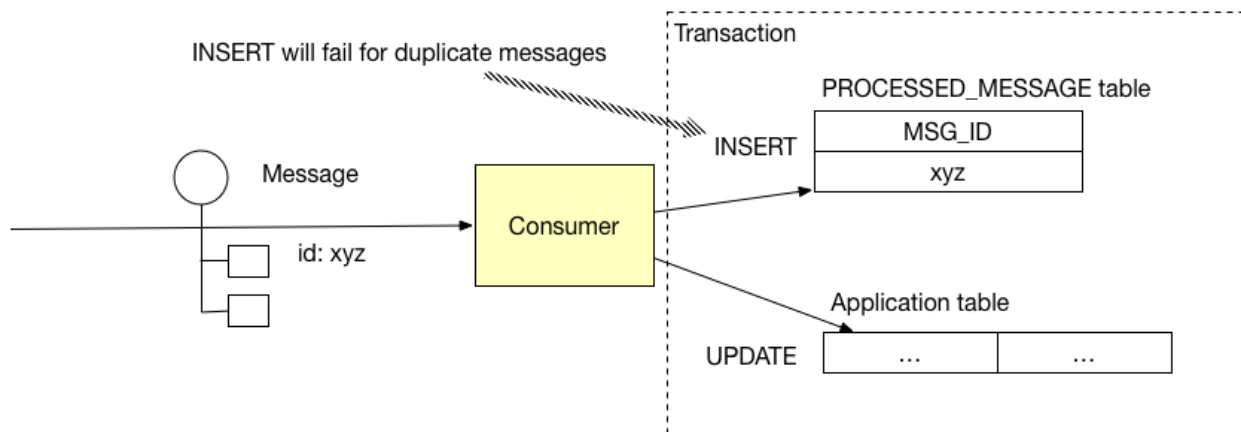


Figure 8.16 A consumer detects and discards duplicate messages by recording the IDs of processed messages in a database table. If a message has been processed before, the INSERT into the PROCESSED_MESSAGES table will fail.

When a consumer handles a message, it records the message ID in the database table as part of the transaction

that creates or updates business entities. In this example, the consumer inserts a row containing the message ID into a `PROCESSED_MESSAGES` table. If a message is a duplicate, the `INSERT` operation will fail and the consumer can discard the message.

Another option is for a message handler to record message IDs in an application table instead of a dedicated table. This approach is particularly useful when using a NoSQL database that has a limited transaction model and so doesn't support multi-table transactions. Chapter 10 shows an example of this approach.

8.3.8 Pattern: Transaction outbox

A service often needs to publish messages as part of a transaction that updates the database. For instance, throughout this book you will see examples of services that publish domain events whenever they create or update business entities. Both the database update and the sending of the message must happen within a transaction. Otherwise, a service might update the database and then crash, for example, before sending the message. If the service doesn't perform these two operations atomically, a failure could leave the system in an inconsistent state.

The traditional solution is to use a distributed transaction that spans the database and the message broker. But as you'll learn in chapter 9, distributed transactions aren't a good choice for modern applications. Moreover, many modern brokers such as Apache Kafka don't support distributed transactions.

As a result, an application must use a different mechanism to atomically update the database and publish messages. Let's look at how that works.

USING A DATABASE TABLE AS A MESSAGE QUEUE

Let's imagine that your application is using a relational database. A straightforward way to reliably publish messages is to apply the Transactional outbox pattern. This pattern uses a database table as a temporary message queue. As figure 8.17 shows, a service that sends messages has an `OUTBOX` database table. As part of the database transaction that creates, updates, or deletes business objects, the service sends messages by inserting them into the `OUTBOX` table. Atomicity is guaranteed because this is a local ACID transaction.

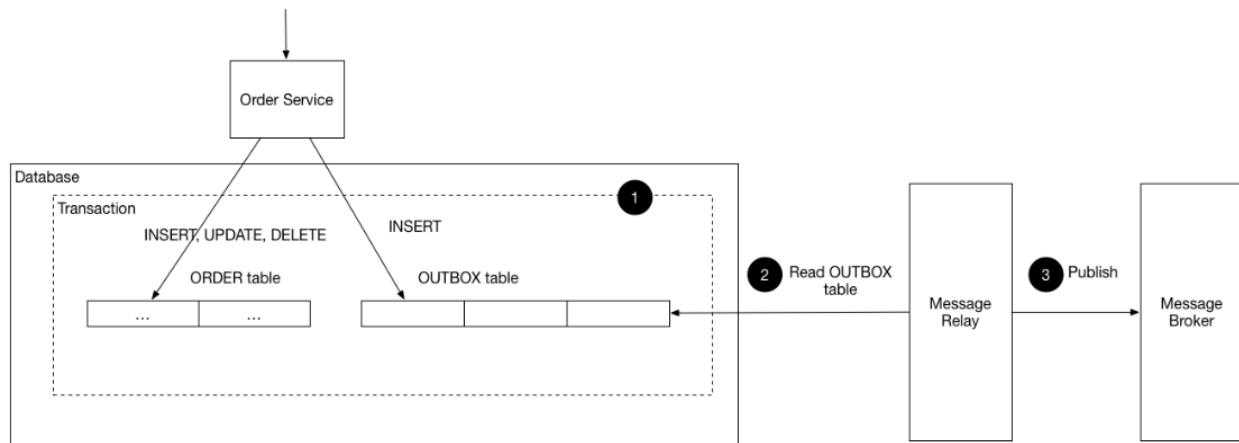


Figure 8.17 A service reliably publishes a message by inserting it into an `OUTBOX` table as part of the transaction that updates the database. The `Message Relay` reads the `OUTBOX` table and publishes the messages to a message broker.

The `OUTBOX` table acts as a temporary message queue. The `MessageRelay` is a component that reads the `OUTBOX` table and publishes messages to a message broker.

PATTERN: TRANSACTIONAL OUTBOX

Publish an event or message as part of a database transaction by saving it in an `OUTBOX` database table. See <http://microservices.io/patterns/data/transactional-outbox.html>.

You can use a similar approach with some NoSQL databases. Each business entity stored as a `record` in the database has an attribute that is a list of messages that need to be published. When a service updates an entity in the database, it appends a message to that list. This is atomic because it's done with a single database operation. The challenge, though, is efficiently finding those business entities that have pending events and publishing them.

There are a couple of different ways to move messages from the database to the message broker. We'll look at each one.

PUBLISHING EVENTS USING THE POLLING PUBLISHER PATTERN

If the application uses a relational database, one simple approach is to publish messages in the `OUTBOX` table is for the `MessageRelay` to poll the table for unpublished messages. It periodically queries the table:

```
SELECT * FROM OUTBOX ORDERED BY ... ASC
```

Next, the `MessageRelay` publishes each message to the message broker, sending each one to its destination message channel.

Finally, it deletes those messages from the `OUTBOX` table:

```
BEGIN
DELETE FROM OUTBOX WHERE ID in (....)
COMMIT
```

PATTERN: POLLING PUBLISHER

Publish messages by polling the `OUTBOX` table. See <http://microservices.io/patterns/data/polling-publisher.html>.

Polling the database is a simple approach that works reasonably well at low scale. The downside is that frequently polling the database can be expensive. Also, whether you can use this approach with a NoSQL database depends on its querying capabilities. That's because rather than querying an `OUTBOX` table, the application must query the business entities, and that may or may not be possible to implement efficiently. Because of these drawbacks and limitations, it's often better — and in some cases, necessary — to use the more sophisticated and performant approach of tailing the database transaction log.

PUBLISHING EVENTS USING THE TRANSACTION LOG TAILING PATTERN

An advanced solution is for `MessageRelay` to *tail* the database transaction log (also called the commit log). Every committed update made by an application is represented as an entry in the database's transaction log. A transaction log miner can read the transaction log and publish each change as a message to the message broker. Figure [8.18](#) shows how this approach works.

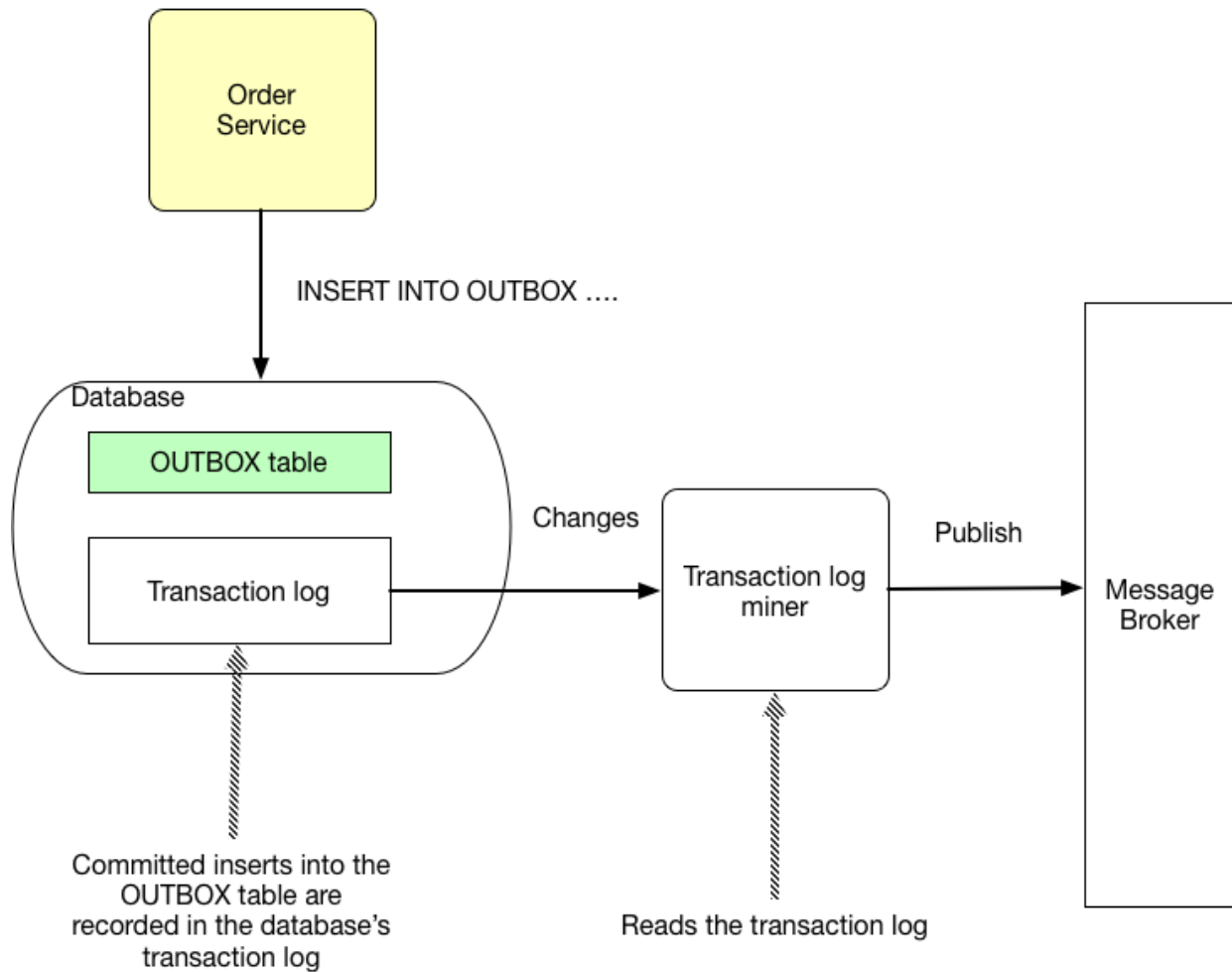


Figure 8.18 A service publishes messages inserted into the `OUTBOX` table by mining the database's transaction log.

The `Transaction Log Miner` reads the transaction log entries and converts each log entry corresponding to an inserted message into a message, which it then publishes to the message broker. This approach can be used to publish messages written to an `OUTBOX` table in an RDBMS or messages appended to records in a NoSQL database.

PATTERN: TRANSACTION LOG TAILING

Publish changes made to the database by tailing the transaction log. See <http://microservices.io/patterns/data/transaction-log-tailing.html>.

Here are a few examples of this approach used in practice:

- Debezium (<http://debezium.io>) - An open source project that publishes database changes to the Apache Kafka message broker.
- DynamoDB streams (<http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>) - DynamoDB stream contains the time-ordered sequence of changes (creates, updates, and deletes) made to the items in a DynamoDB table in the last 24 hours. An application can read those changes from the stream and, for example, publish them as events.
- Eventuate platform (<https://eventuate.io>) - Your author's very own open-source service collaboration platform that includes an Eventuate CDC service that uses MySQL binlog protocol, Postgres WAL, or polling to read changes made to an `OUTBOX` table and publish them to Apache Kafka.

Although this approach is advanced, it works remarkably well. The challenge is that implementing it requires some development effort. You could, for example, write low-level code that calls database-specific APIs. Alternatively, you could use an open source framework such as Debezium that publishes changes made by an application to MySQL, Postgres, or MongoDB to Apache Kafka. The drawback of

using Debezium is that its focus is change data capture at the database level - inserts, updates and deletes of rows - and that APIs for sending and receiving messages are outside of its scope. That's why I created the Eventuate platform, which provides both messaging APIs and built-in support for transaction tailing and polling.

8.3.9 Synchronously invoking an asynchronous implementation

One challenge when designing an asynchronous architecture is implementing system operations that are invoked synchronously but executed asynchronously. For example, let's imagine that the `Order Service` implements the `POST /orders` REST endpoint using asynchronous messaging. The service collaborates with the other services, such as the `Consumer Service`, by exchanging messages. This interaction is an example of a Saga, which is described in detail in chapter 9. The problem, however, is that unlike a message handler, which can simply exit, an HTTP controller must send back an HTTP response. The key question is when?

When implementing an HTTP request handler, you have two options for when to send an HTTP response:

- After processing completes
- Immediately after sending the first message

Let's look at each option.

SEND HTTP RESPONSE AFTER PROCESSING COMPLETES

One option is for the HTTP Controller to send the HTTP response only after the services have finished exchanging messages — for example, after order creation is complete.

Figure 8.20 shows what happens when the `createOrder()` operation is invoked. In this scenario, the `Order Service` exchanges messages with multiple services including the `Consumer Service`, and the `Restaurant Service`. Once the order has been created and approved, the service sends back the response to the `POST /orders`.

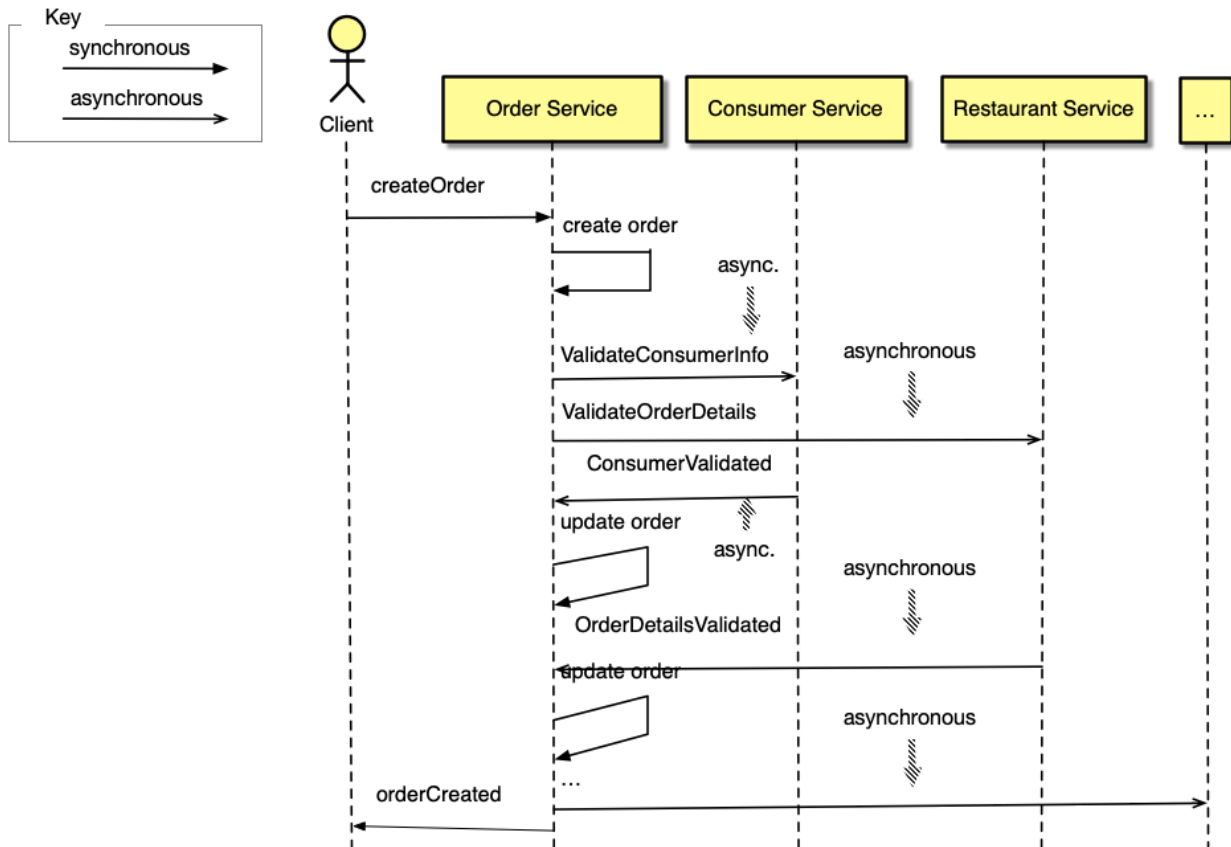


Figure 8.19 A service that receives a synchronous request can send a response after the services have finished exchanging messages. While this approach enables the response to contain the final outcome, it creates tight runtime coupling.

An important benefit of this approach is that the REST endpoint's behavior is no different than in a monolithic architecture: the response contains the final outcome of creating the order. The downside, however, is that the `Order Service` is runtime coupled to the those services. It cannot send a response if those services are unavailable. As a

result, the `POST /orders` endpoint has higher latency and lower availability.

SEND HTTP RESPONSE AFTER SENDING THE FIRST MESSAGE

The second option is for the service to handle an HTTP request as follows:

1. Validate the HTTP request using only locally available data
2. Update the business entities in its database
3. Send one or more messages, e.g. by inserting them into the `OUTBOX` table.
4. Return a response to its client

For example, if the `Order Service` uses this approach, it creates an order in a `PENDING` state and then validates the order asynchronously by exchanging messages with other services. Figure [8.20](#) shows how it handles the HTTP `POST /orders` request.

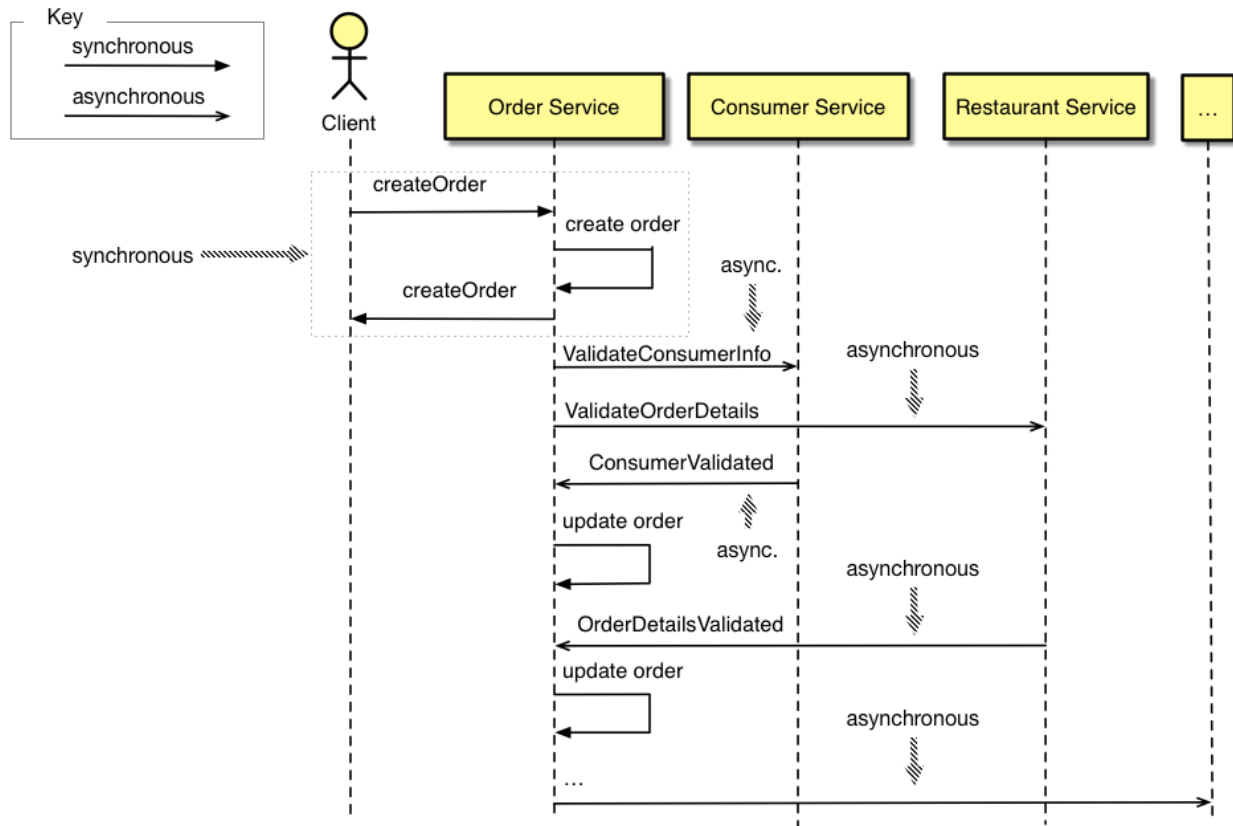


Figure 8.20 The `Order Service` creates an order without invoking any other service, which eliminates runtime coupling. The service then asynchronously validates the newly created `Order` by exchanging messages with other services, including the `Consumer Service` and the `Restaurant Service`.

The sequence of events is as follows:

1. The `Order Service` creates an `Order` in a `PENDING` state.
2. The `Order Service` returns an HTTP response to its client containing the order ID.
3. The `Order Service` sends a `ValidateConsumerInfo` command message to the `Consumer Service`.
4. The `Consumer Service` receives the `ValidateConsumerInfo` command message, verifies the consumer can place an order, and sends an `ConsumerValidated` reply message to the `Order Service`.

5. The `Order Service` receives the `ConsumerValidated` reply, updates the `Order`, and sends a `ValidateOrderDetails` command message to the `Restaurant Service`.
6. The `Restaurant Service` receives a `ValidateOrderDetails` message, verifies the menu items are valid and that the restaurant can deliver to order's delivery address, and sends an `OrderDetailsValidated` reply message to the `Order Service`.
7. The `Order Service` receives the `ConsumerValidated` and `OrderDetailsValidated` and changes the state of the order to `VALIDATED`.

An important benefit of this approach is that the `Order Service` sends an HTTP response without waiting for other services. There's no runtime coupling and so the `POST /orders` endpoint has lower latency and higher availability. If, for example, a service is temporarily unavailable, the message broker will buffer messages until they can be delivered. Eventually, the processing of the order will be completed.

One drawback of this approach is that unless the `Order Service` rejects the request, the HTTP response contains only a partial outcome. It simply tells the client that the request has been accepted and is being processed. Consequently, the client is more complicated since, to determine the final outcome, it must either periodically poll or subscribe to notifications published by the `Order Service`. Despite this downside, in many situations, this is the preferred approach because it reduces runtime coupling and improves overall system resilience.

8.3.10 Libraries and frameworks for messaging

A service needs to use a library to send and receive messages. One option is to use the message broker's client

library, although there are several drawbacks to using such a library directly:

- The client library tightly couples the business logic that publishes messages to the message broker's APIs.
- A message broker's client library is typically low-level and requires many lines of code to send or receive a message. As a developer, you don't want to repeatedly write boilerplate code. Also, as the author of this book, I don't want example code cluttered with low-level boilerplate either.
- The client library usually provides only the basic mechanism to send and receive messages and doesn't support the higher-level interaction styles described earlier.

A better approach is to use a higher-level library or framework that hides the low-level details and directly supports the higher-level interaction styles. For simplicity, the examples in this book use the Eventuate platform. It has a simple, easy-to-understand API that hides the complexity of using the message broker. In addition to an API for sending and receiving messages, Eventuate also supports higher-level interaction styles such as asynchronous request/response and domain event publishing. There's even a Saga framework, which I describe in more detail in chapter 9.

WHAT!? WHY THE EVENTUATE PLATFORM?

The code samples in this book use the open source Eventuate platform I developed for service collaboration based on transactional messaging, event sourcing, and sagas. I chose to use my platform because, unlike with, say, dependency injection and the Spring framework, there are no widely adopted frameworks for many of the features the microservice architecture requires. Without the Eventuate platform, many examples would have to use the low-level messaging APIs directly, making them much more complicated and obscuring important concepts. Or they would use a framework that isn't widely adopted, which would also provoke criticism.

Instead, the examples use the Eventuate platform, which has a simple, easy-to-understand API that hides the implementation details. You can use these frameworks in your applications. Alternatively, you can study the Eventuate Platform and reimplement the concepts yourself.

Eventuate platform also implements two important patterns:

- *Transaction outbox pattern* — atomically update business entities and send messages, as discussed in section [Section 8.3.8](#).
- *Idempotent consumer pattern* — the Eventuate message consumer detects and discards duplicate messages, which is essential for ensuring that a consumer processes messages exactly once, as discussed in section [Section 8.3.7](#).

Figure [8.21](#) shows the key parts of an Eventuate-based application.

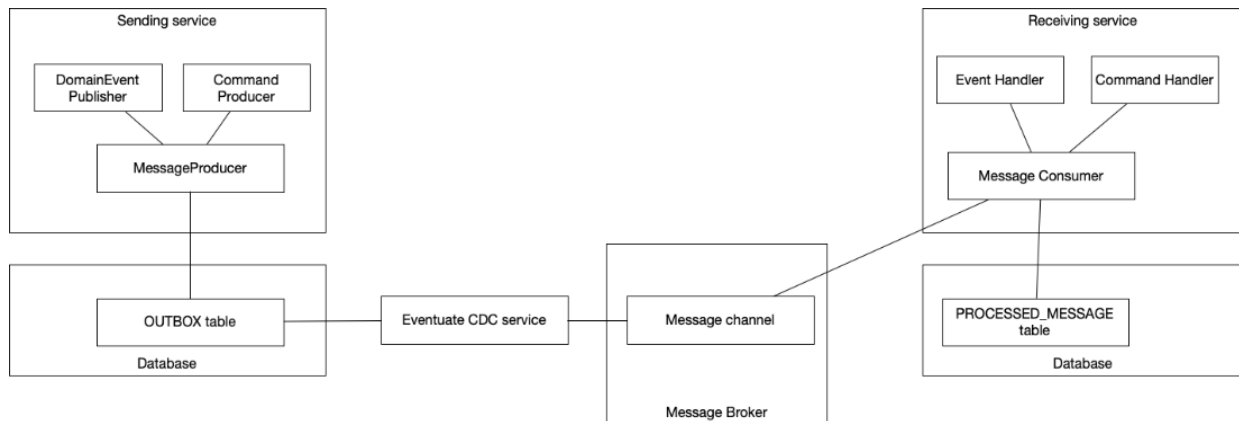


Figure 8.21 This diagram shows the architecture of an Eventuate application. The services use Eventuate APIs to send and receive messages. The message sender uses the Transaction outbox pattern. The Eventuate CDC sends messages from the OUTBOX table to the message broker. The message receiver used to the Idempotent Consumer pattern to detect and discard duplicate messages.

Let's take a look at the details of the Eventuate platform's messaging APIs.

BASIC MESSAGING

The basic messaging API consists of two Java interfaces: `MessageProducer` and `MessageConsumer`. A service sends a message using the `MessageProducer` interface. Here's an example of using this interface:

```

MessageProducer messageProducer = ...;
String channel = ...;
String payload = ...;
messageProducer.send(destination, MessageBuilder.withPayload(payload).build())
  
```

The `MessageProducer.send()` method does not directly send the message to the message broker. Instead, it uses the Transaction outbox pattern. A separate Eventuate CDC service does the actual publishing.

A service uses the `MessageConsumer` interface to subscribe to messages:

```
MessageConsumer messageConsumer;  
messageConsumer.subscribe(subscriberId, Collections.singleton(destination), message -> { ... })
```

The `MessageConsumer` implements the Idempotent consumer pattern described earlier.

`MessageProducer` and `MessageConsumer` are the foundation of the higher-level APIs for asynchronous request/response and domain event publishing. Let's look at how to publish and subscribe to events.

DOMAIN EVENT PUBLISHING AND SUBSCRIBING

The Eventuate platform has APIs for publishing and consuming domain events, which are, as chapter 5 explains, emitted by an *aggregate* (business object) when it is created, updated, or deleted. A service publishes a domain event using the `DomainEventPublisher` interface. Here is an example:

```
DomainEventPublisher domainEventPublisher;  
  
String accountId = ...;  
  
DomainEvent domainEvent = new AccountDebited(...);  
  
domainEventPublisher.publish("Account", accountId, Collections.singletonList(domainEvent));
```

The `DomainEventPublisher.publish()` method is a wrapper around `MessageProducer.send()`.

A service consumes domain events by configuring one or more event handlers:

```
public class CustomerEventConsumer {

    @EventuateDomainEventHandler(subscriberId = "accountServiceEventsConsumer", channel = "Account")
    public void handleCustomerCreatedEvent(DomainEventEnvelope<AccountDebited> domainEventEnvelope) {
        ...
    }
}
```

Events aren't the only high-level messaging pattern supported by the Eventuate platform. It also supports command/reply-based messaging.

COMMAND/REPLY-BASED MESSAGING

A service can send a command message using the `CommandProducer` interface. For example:

```
CommandProducer commandProducer = ...;

Map<String, String> extraMessageHeaders = Collections.emptyMap();

String commandId = commandProducer.send("CustomerCommandChannel",
    new DoSomethingCommand(),
    "ReplyToChannel",
    extraMessageHeaders);
```

A service consumes command messages by defining one or more command handlers:

```
public class CustomerCommandHandler {

    @EventuateCommandHandler(subscriberId="customerCommandDispatcher",
channel="customerService")
    public ReserveCreditResult reserveCredit(CommandMessage<ReserveCreditCommand> cm) {
        ...
    }
}
```

Throughout this book, you'll see code examples that use these APIs for sending and receiving messages.

As you've seen, the Eventuate platform implements transactional messaging for Java applications. It provides low-level APIs for sending and receiving messages using the Transaction outbox and Idempotent consumer patterns. It also provides higher-level APIs for publishing and consuming domain events and for sending and processing commands. In later chapters, you will see examples of how this platform is used.

8.4 Summary

- The microservice architecture is a distributed architecture, so interprocess communication plays a key role.
- There are numerous IPC technologies, each with different trade-offs. One key design decision is to choose either a synchronous remote procedure invocation pattern or the asynchronous messaging pattern. Synchronous remote procedure invocation-based protocols, such as REST, are the easiest to use. But services should ideally communicate using asynchronous messaging in order to reduce runtime coupling.
- In order to prevent failures from cascading through a system, a service client that uses a synchronous protocol must be designed to handle partial failures, which is when the invoked service is either down or exhibiting high latency. In particular, it must use resiliency patterns including the Circuit breaker pattern.
- An architecture that uses synchronous protocols must include a service discovery mechanism in order for clients to determine the network location of a service instance. The simplest approach is to use the service discovery mechanism implemented by the deployment platform. But an alternative approach is to implement service discovery at the application level. It's more work,

but it does handle the scenario where services are running on multiple deployment platforms.

- A good way to design a messaging-based architecture is to use the messages and channels model, which abstracts the details of the underlying messaging system. You can then map that design to a specific messaging infrastructure, which is typically message broker-based. Each channel's requirements - ordering requirements, delivery guarantees, and so on - determine the choice of messaging technology.
- There are three important messaging patterns:
 - Transaction outbox - ensures that the sender atomically updates the database and sends a message. It first writes the message to the database as part of the database transaction. A separate process then retrieves the message from the database using either the Polling publisher pattern or the Transaction log tailing pattern and sends it to the message broker.
 - Idempotent Consumer pattern - detects and discards duplicate messages by tracking processed messages in the database. It's important to use this pattern when a message handler is not naturally idempotent since a message broker can deliver a message multiple times.
 - Competing consumers - increases throughput by processing concurrently while still preserving ordering.
- A good way to reduce runtime coupling is for a service to respond immediately to a synchronous request, such as an HTTP POST, and complete the remaining work asynchronously by exchanging messages with other services.

- It's essential to carefully manage the evolution of a service's API. Backward-compatible changes are the easiest to make because they don't impact clients. If you make a breaking change to a service's API, it will typically need to support both the old and new versions until its clients have been upgraded.
- It's important to define a service's API using an Interface Definition Language (IDL), such as OpenAPI for REST APIs, or AsyncAPI for messaging-based APIs. An IDL defines the API in a precise, machine-readable format that supports code generation, documentation, and validation.
- The runtime behavior of a service API - latency, availability, and related properties - is described by service level objectives (SLOs) and, in some cases, a service level agreement (SLA). An SLO is a target for a service level indicator, such as availability. An SLA is a contractual commitment to one or more SLOs, often with penalties or credits for violations.
- An error budget is the allowable amount of unreliability a service can experience without breaching its SLOs. Error budgets guide whether a team should focus on feature development or ensuring that their service meets its SLOs.
- An important part of an API client is the DTOs representing the messages that are exchanged with the service. These DTOs can be written by hand - often more time-consuming but can reduce design-time coupling - or code generated from the IDL, which is convenient but can unnecessarily design-time couple the client to the entire API.
- Clients for asynchronous APIs should use a library or framework that abstracts the low-level details of the messaging infrastructure and directly supports higher-level interaction styles, such as command/reply and

domain event publishing. The open-source Eventuate platform is an example of a framework that provides these capabilities. It also implements the Transactional outbox and Idempotent consumer patterns.

9 Implementing system commands with sagas

This chapter covers

- Why system commands must be transactional
- Saga pattern fundamentals
- How to design sagas to be ACID

The previous chapter described various inter-process communication (IPC) mechanisms. Those IPC mechanisms are not ends in themselves, but are used by higher-level service collaboration patterns that implement operations spanning multiple services. This chapter explores one of those patterns: the Saga pattern. The Saga pattern implements distributed system commands, which are operations that update data in multiple services - a common and challenging design problem in a microservice architecture.

This chapter begins by explaining why system commands must be transactional and why providing transactional guarantees becomes significantly more challenging once a command spans multiple services and databases. It then introduces the Saga pattern as the primary approach for implementing distributed system commands in a microservice architecture. It discusses the implications of a saga's limited transactional guarantees and explains how application-level design techniques must replace capabilities that were previously provided by the database.

We begin by examining transaction management in a microservice architecture. Understanding why system commands must be transactional, and why traditional transaction mechanisms break down once a command spans multiple services, provides the foundation for the Saga pattern introduced later in this chapter. This discussion establishes the constraints

and trade-offs that shape the design of distributed system commands in microservices.

9.1 Transaction management in a microservice architecture

In a microservice architecture, implementing operations that modify data is significantly more complex than in a monolithic application. A system operation often spans multiple services, each with its own database and transactional boundaries. As a result, familiar database transaction mechanisms cannot be used to manage consistency across the entire operation. This section examines why system commands must be transactional and explains the challenges of managing transactions when a command spans multiple services. In particular, I explain why classic distributed transactions (XA/2PC) are not a good fit for a microservice architecture, which is why the Saga pattern is the preferred approach for implementing system commands. With this context in place, we can first look at why transactional behavior is essential for system commands, regardless of the underlying architecture.

9.1.1 System commands need to be transactional

To understand why system commands need to be transactional and what that means exactly, let's consider an incredibly simplified version of the `createOrder(consumerId, productId)` operation. This version takes two parameters:

- `consumerId` - the ID of the `Consumer` that the `Order` is for
- `productId` - the ID of a product

The operation must:

- Calculate the `orderTotal`, which is the product's price
- Verify that the `Consumer` has sufficient `availableCredit` and reduce it by the `orderTotal`

- Create an `Order`

Later in this chapter, we'll look at a more realistic version of this operation, but for now I don't want to complicate the discussion.

On the surface, this operation appears to be simple. The operation might execute the following SQL statements to retrieve the price, reserve credit and create the `Order`:

```
SELECT * FROM PRODUCTS WHERE PRODUCT_ID = ?
```

```
UPDATE CONSUMER  
  SET AVAILABLE_CREDIT = AVAILABLE_CREDIT - ?  
  WHERE CONSUMER_ID = ?  
    AND AVAILABLE_CREDIT >= ?
```

```
INSERT INTO ORDERS (...) VALUES (...)
```

It is usually straightforward in a monolithic application that uses a single relational database. In a microservice architecture, however, it is not guaranteed to be this easy. To understand why, we first need to examine what a transaction is and the problems it is intended to solve.

WHY SYSTEM OPERATIONS MUST BE TRANSACTIONAL

Implementing a system command, such as `createOrder()`, presents several fundamental correctness challenges:

- **Partial failure** - A system operation often performs multiple state changes. If the application fails after executing only some of those changes, the system can be left in an inconsistent, partially updated state.
- **Concurrency** - Multiple system operations may execute concurrently and access the same data. Without coordination, concurrent execution can lead to race conditions, lost updates, and other correctness violations.
- **Business rule and invariant enforcement** - Many system operations must preserve invariants that span multiple state

changes, such as ensuring that inventory is not oversold or that a customer's credit limit is not exceeded.

There are many different flavors of transactions, each with different trade-offs and guarantees. The most widely used is the Atomic Consistent Isolated Durable (ACID) transaction model, which we will look at shortly. But before doing so, I'll first introduce a more abstract definition of a transaction.

ABOUT TRANSACTIONS

A transaction is a conceptual grouping of operations, and often described as a unit of work, that defines a clear boundary for correctness in a system, such as a database. Within that boundary, the operations - such as database inserts, updates, and deletes - are treated as one indivisible change to system state: either the entire set of changes takes effect, or none of them do. The transaction boundary defines rules for success and failure, concurrency and conflict handling, and the visibility of changes to other transactions.

A transaction can commit only if all operations complete without error and all required invariants are satisfied. If a failure occurs, the transaction is aborted, and its effects are discarded or explicitly undone, ensuring that partial changes do not violate correctness.

These rules also govern how concurrent transactions interact with one another. They define how the system detects conflicts between overlapping operations, such as competing updates to the same data, and how those conflicts are resolved to preserve correctness. The rules further specify the level of isolation between transactions, determining whether intermediate changes are hidden or visible and when the effects of a committed transaction become observable to other transactions.

It's important to keep in mind that it's not just databases that implement transactions. For example, the Apache Kafka message

broker also implements transactions for sending and receiving messages.

ABOUT ACID TRANSACTIONS

ACID transactions are a widely used transaction model. Initially, the concept was developed to solve problems that arose in early database systems and was later formalized by database researchers, most notably Jim Gray. ACID transactions have four properties:

- Atomicity - All operations in a transaction succeed as a single unit, or the transaction has no effect at all. This defines the transaction's success and failure rules.
- Consistency - A committed transaction leaves the system in a valid state, preserving all defined invariants and constraints. These conditions must hold at the transaction boundary for a commit to be allowed. Preserving those invariants is the responsibility of the application logic and constraints executed within the transaction.
- Isolation - Concurrent transactions do not interfere with one another and execute as if they were isolated from each other, according to a defined isolation level. In practice, databases allow isolation levels to be relaxed to trade isolation guarantees for higher throughput.
- Durability - Once a transaction commits, its effects are permanent and will survive crashes or restarts. This guarantees that changes, once visible to other transactions, will not be lost.

The atomicity and isolation properties are especially important when implementing system commands. Atomicity handles partial failure by ensuring that a command's effects are either fully applied or fully rolled back. Isolation ensures that the outcome of executing concurrent system commands is equivalent to some sequential execution order, e.g., the concurrent execution of A and B is equivalent to either A followed by B or B followed by A .

As mentioned above, implementing system operations using ACID transactions is remarkably straightforward in a monolithic application that has a single database. We can simply wrap the SQL statements in a transaction:

```
BEGIN TRANSACTION

SELECT * FROM PRODUCTS WHERE PRODUCT_ID = ?

UPDATE CONSUMER
  SET AVAILABLE_CREDIT = AVAILABLE_CREDIT - ?
  WHERE CONSUMER_ID = ?
    AND AVAILABLE_CREDIT >= ?

INSERT INTO ORDERS (...) VALUES (...)
COMMIT TRANSACTION
```

The `BEGIN TRANSACTION ... COMMIT TRANSACTION` ensures the statements are executed using ACID semantics. Let's now look at ACID transactions in a microservice architecture.

9.1.2 ACID transactions in a microservice architecture

System commands that are local to a single service can be implemented using ACID transactions. In this respect, they are no different from system commands in a monolith. The challenge arises when implementing transaction management for distributed system commands. For example, consider the `createOrder()` system command described earlier spans the `Order Service` and the `Customer Service`. Both services - along with each service's database server - would need to participate in the same transaction.

Let's first look at the classic approach to implementing ACID transactions that span multiple services: distributed ACID transactions. Although this approach should not be used in a microservice architecture, it is important to understand it for two reasons. First, understanding distributed transactions clarifies the motivation for the Saga pattern and the trade-offs it makes.

Second, I have encountered teams who are tempted to use distributed transactions instead of sagas because their microservice architecture was not designed to make sagas easy to implement. Examining why distributed transactions are problematic in a microservice architecture makes clear both why sagas are necessary and why designing services to support them is essential.

USING DISTRIBUTED ACID TRANSACTIONS

The traditional approach to maintaining data consistency across multiple services, databases, or message brokers is to use distributed ACID transactions. The de facto standard for distributed transaction management is the X/Open Distributed Transaction Processing (DTP) Model, commonly referred to as XA.

An XA-based architecture relies on XA-compliant databases and message brokers, along with drivers and APIs that can participate in a global transaction. It also requires an interprocess communication mechanism that propagates a global transaction identifier to all participants. Most SQL databases are XA compliant, as are some message brokers. Java EE applications can, for example, use JTA to perform distributed transactions.

XA uses *two-phase commit* (2PC) to ensure that all participants in a transaction either commit or rollback. Figure [9.1](#) shows how this works. In addition to the services and databases, there's a transaction coordinator that's responsible for ensuring that all participants commit or rollback. The `Order Service` instance initiates the distributed transaction and is registered as the initiating participant. The `Customer Service` registers with the transaction coordinator as a participant. Both services enlist their databases in the distributed transaction.

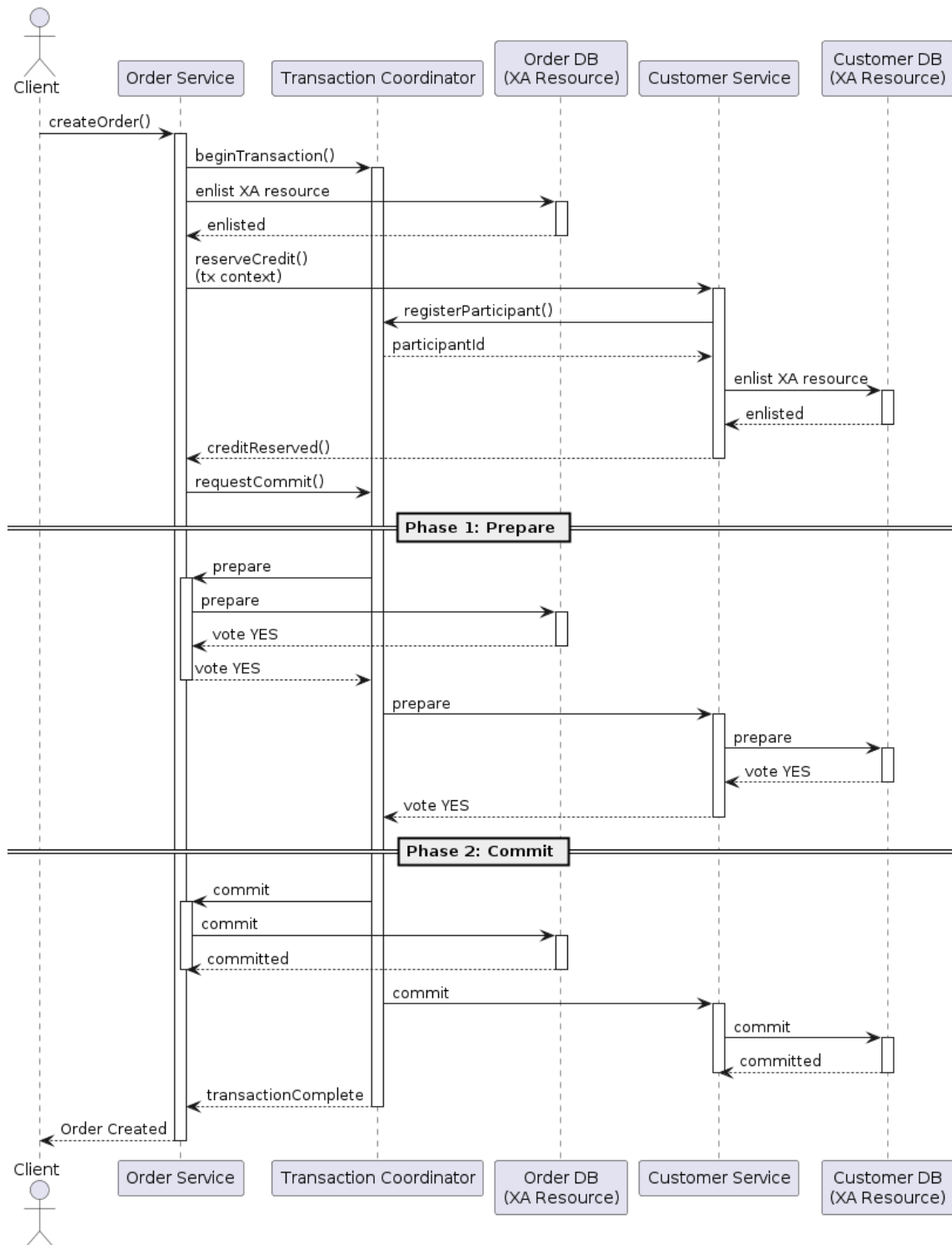


Figure 9.1 A hypothetical implementation of XA/2PC in a microservice architecture. The services register as participants with the Transaction Coordinator. They enlist their databases in the distributed transaction. The Transaction Coordinator commits the transaction in two phases. In the prepare phases, participants vote on whether they can commit. In the commit phase,

the transaction coordinator instructs all participants to either commit or roll back based on the collected votes.

As the name suggests, committing a global transaction consists of two phases, a prepare phase and a commit phase. In the prepare phase, the participants attempt to make their changes durable and then vote either yes (ready to commit) or no (must roll back), while holding any required locks. In the commit phase, the transaction coordinator instructs all participants to either commit or roll back based on the collected votes.

As you can see, although the ACID model is simple, its implementation in a distributed architecture is not. Moreover, it is typically not a good fit for modern applications. Let's look at why.

THE PROBLEM WITH DISTRIBUTED TRANSACTIONS IN A MICROSERVICE ARCHITECTURE

There are a variety of problems with 2PC/XA distributed transactions in a microservice architecture:

- They are not supported by many modern technologies
- They are not a good fit for today's dynamic deployment environments
- They rely on synchronous IPC, which reduces availability

The first problem with distributed ACID transactions is that many modern technologies do not support them. This includes NoSQL databases, such as MongoDB and Cassandra, and modern message brokers, such as RabbitMQ or Apache Kafka. As a result, adopting distributed transactions significantly restricts the set of technologies that can be used in a microservice architecture.

The second problem is that 2PC was created in a world where services were long-lived and typically ran on physical machines with stable network addresses. In contrast, modern environments - typically based on VMs or containers - are far more dynamic, ephemeral, and ideally stateless, which creates

serious challenges for 2PC. For example, a 2PC transaction can block indefinitely if a participant - or worse, the transaction coordinator - disappears, a scenario potentially requiring manual intervention to resolve the transaction and restore the system to a consistent state.

Another problem with distributed transactions is that they rely on synchronous IPC, which reduces availability. In order for a distributed transaction to commit, the transaction coordinator and all the participating services must be available at the same time. As described in chapter {chapter-ipc}, the availability of an operation is the product of the availability of all the participants. If a distributed transaction involves two services that are 99.5% available, then the overall availability is only 99%, which is significantly less. Each additional service involved in a distributed transaction further reduces availability.

There is even Eric Brewer's CAP theorem, which states that a system can only have two of the following three properties: consistency, availability, and partition tolerance (https://en.wikipedia.org/wiki/CAP_theorem). In modern architectures, availability and partition tolerance are typically prioritized over strong consistency, which is why architects favor approaches such as the Saga pattern instead of distributed ACID transactions.

For these reasons, distributed ACID transactions are rarely a good fit for a microservice architecture. Instead of relying on synchronous, strongly consistent transactions that span services, microservice architectures typically adopt the Saga pattern to manage data consistency. The next section introduces the Saga pattern and explains how it addresses these challenges.

9.2 Using the Saga pattern

As figure [9.2](#) shows, the Saga pattern implements a system command that spans multiple services as a series of local - typically ACID - transactions within those services. The saga steps are often executed sequentially with the first step being

initiated by the invocation of a system command, such as an HTTP POST request. Later in this section, you'll see how the coordination of the steps is often implemented using asynchronous messaging since it is an easy way to ensure that a saga is atomic.

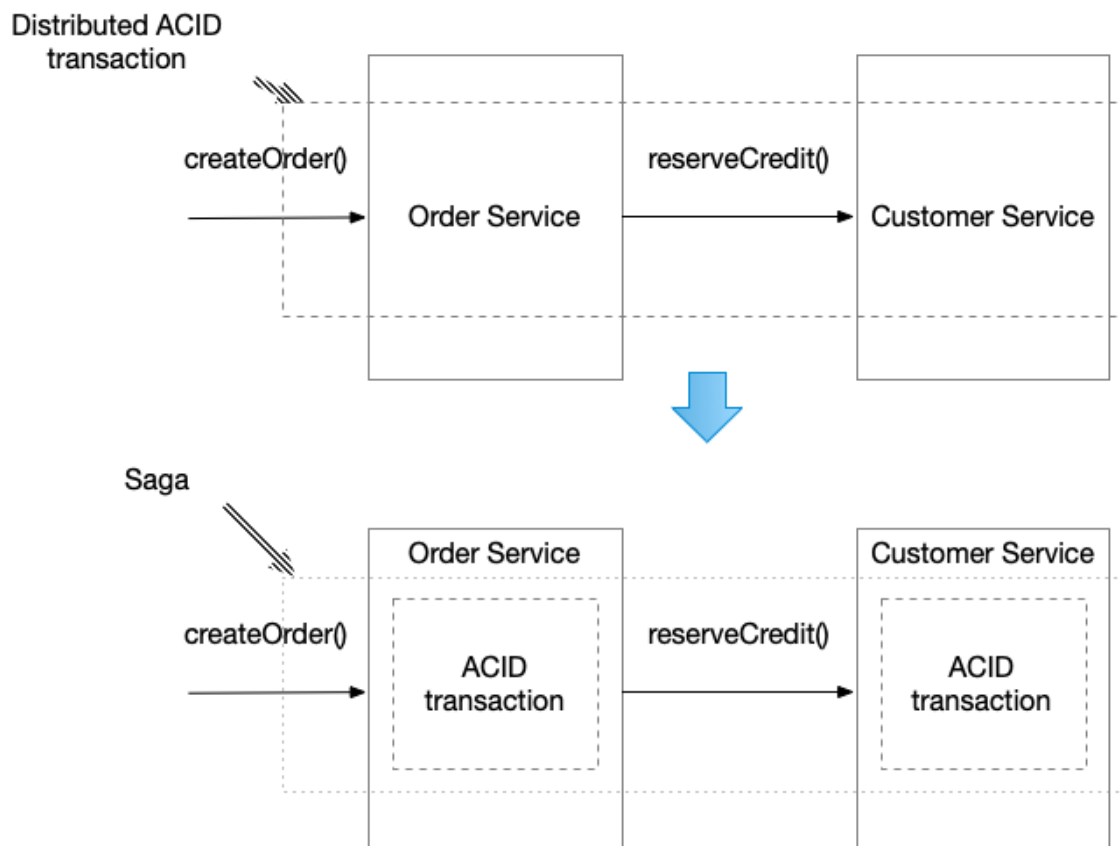


Figure 9.2 A saga replaces what would other be a distributed ACID transaction with a sequence of local ACID transactions in each of the participating services.

Each step invokes a service operation that executes business logic. The business logic typically performs validations and updates the service's persistent entities. Service operations often return data that is either the result of the system operation or is passed to another saga participant.

Sagas differ from ACID transactions in a couple of important ways. First, because local transactions commit their changes, a saga must be rolled back using compensating transactions. I talk more about compensating transactions later in this section.

Second, as I describe in detail in [Section 9.3](#), sagas lack the isolation property of ACID transactions, which means that ensuring data consistency requires application-level design techniques.

This section explores the basics of the Saga pattern. Let’s first look at an example.

9.2.1 An example saga: The Create Order saga

Let’s explore how to implement the `createOrder()` system command, which was described in [Section 9.1.1](#), as a saga. Since this operation calculates the order total and performs two updates - `UPDATE CONSUMER ...` which reserves available credit, and `INSERT INTO ORDERS ...`, which creates the `Order`. You might expect the saga to look something like the saga definition shown in table [9.1](#).

Table 9.1 First attempt at defining Create Order Saga. It creates an order in the Order Service and then reserves credit in the Customer Service

Saga canvas		
System operation:	createOrder(customerID, productId)	
Step	Participant	Transaction
1	Order Service	(orderId, orderTotal) = createOrder(...)
2	Customer Service	reserveCredit(customerId, orderTotal)

The first step of the saga invokes the `Order Service` to calculate the `Order` total and create the `Order`. The second step invokes the `Customer Service` to reserve credit. While this saga definition is a good start, it only handles the happy path. Let’s look at a failure scenario.

HANDLING FAILURES WITH COMPENSATING TRANSACTIONS

The saga shown in table [9.1](#) doesn't handle the scenario where the `reserveCredit()` operation fails because either the `customerID` is invalid, or the consumer has insufficient credit. Unlike an ACID transaction, which can simply be rolled back, the failure of a saga step requires the updates performed by the previous steps to be explicitly undone using what are called compensating transactions. This is one of the key differences between sagas and ACID transactions.

THE DEFINITION OF FAILURE

The word 'failure' is an overloaded term. When discussing sagas, I use the term failure to mean the violation of a business rule. For example, the `customerID` is invalid or the customer has insufficient available credit. These kinds of failures cause the saga to be compensated and the system command to fail.

There are, of course, also technical failures, such as service crashes. It's best to assume that these failures are temporary and only delay the execution of the saga. I discuss this further in [Section 9.2.3](#).

Each step that is followed by a step that can fail must have a compensating transaction to undo its updates. Table [9.2](#) shows a revised definition of the saga that includes a compensating transaction for the `createOrder()` step. It invokes the `Order Service`'s `rejectOrder()` operation, which marks the `Order` as rejected.

Table 9.2 The second version of the `Create Order Saga` that adds a compensating transaction to step 1 that undoes the creation of the `Order` if credit cannot be reserved in step 2.

Saga canvas			
System operation:	<code>createOrder(customerID, productSpecification, quantity)</code>		
Step	Participant	Transaction	Compensating transaction
1	Order Service	<code>(orderId, orderTotal) = createOrder(...)</code>	<code>rejectOrder(orderId)</code>
2	Customer Service	<code>reserveCredit(customerId, orderTotal)</code>	-

The compensating transaction for `createOrder()` is `rejectOrder()`, which marks the `Order` as rejected. The `reserveCredit()` step does not need a compensating transaction because it's the last step of the saga and is not followed by a step that can fail.

More generally, if the $(n + 1)^{\text{th}}$ step of a saga fails, the effects of the previous n transactions must be undone. Conceptually, each of those steps T_i has a corresponding compensating transaction C_i , which undoes the effects of the T_i . To undo the effects of those first n steps, the saga must execute each C_i in reverse order. The sequence of steps is $T_1, \dots, T_n, C_n, \dots, C_1$, as shown in figure [9.3](#). In this example, T_{n+1} fails, which requires steps $T_n, \dots, T_1$ to be undone.

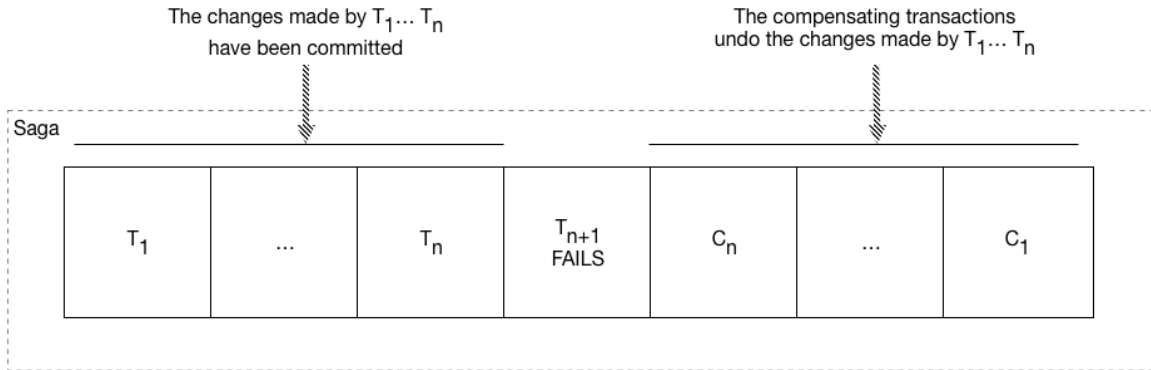


Figure 9.3 When a step of a saga fails because of a business rule violation, the saga must explicitly undo the updates made by previous steps by executing compensating transactions.

The saga executes the compensation transactions in reverse order of the forward transactions: $C_n, \dots, C_1$. The mechanics of sequencing the C_i s isn't any different from sequencing the T_i s. The completion of C_i must trigger the execution of C_{i-1} .

HANDLING CONCURRENT TRANSACTIONS

The addition of a compensating transaction handles these failure scenarios. But there's still one more problem that needs to be solved because the updates of each saga step are immediately visible. As a result, in a failure scenario, an `Order` is created and then, a short time later, it's rejected. Another transaction could access the `Order` in the brief window of time between those two steps and make incorrect decisions. As I discuss in more detail in [Section 9.3](#), this is another key difference between sagas and ACID transactions and requires careful design.

One way to prevent another transaction from accessing the `Order` before the saga completes is to implement application-level locking, which is called the semantic lock pattern in [Section 9.3](#). The `Order` is locked when it is created and unlocked when the saga completes, successfully or unsuccessfully. Table [9.3](#) shows the revised definition that includes a semantic lock.

Table 9.3 The third and final version of `Create Order Saga` implements a semantic lock that prevents other sagas or transactions from accessing the `Order` until the saga has completed.

Saga canvas			
System operation:	createOrder(customerID, productId)		
Step	Participant	Transaction	Compensating transaction
1	Order Service	(orderId, orderTotal) = createOrder(...)	rejectOrder(orderId)
2	Customer Service	reserveCredit(customerId, orderTotal)	-
3	Order Service	approveOrder(orderId)	-

The `createOrder()` step creates the `Order` in a `PENDING` state, which functions as a lock. The newly added `approveOrder()` step changes the state of the `Order` to `APPROVED`, which unlocks it, as does the `rejectOrder()` step. [Section 9.3](#) describes how this locking works in more detail.

9.2.2 The structure of a saga

As you have just seen, some but not all steps of a saga have a compensating transaction. In particular, a step only needs a compensating transaction if it's followed by a step that can fail. The paper "Semantic ACID Properties in Multidatabases Using Remote Procedure Calls and Update Propagations" by Lars Frank and Torben U. Zahle, which is discussed further in [Section 9.3](#), defines a useful model for the structure of a saga.

In this model, shown in figure 9.4, a saga's steps can be grouped into three phases:

1. Zero or more *compensatable transactions* — Transactions that precede the pivot transaction and therefore need to be rolled back
2. A *pivot transaction* — the last step of the saga that can fail
3. Zero or more *retriable transactions* — Transactions that follow the pivot transaction and are guaranteed to succeed.



Figure 9.4 A saga's steps are grouped into three phases: zero or more compensatable transactions, which can be rolled back and so have a compensating transaction, a single pivot transaction, which is saga's go/no-go point, and zero or more retriable transactions, which are transactions that are guaranteed to complete.

These terms, compensatable, pivot and retriable transaction, are not the most intuitive. Most notably, the paper uses the term retriable because, in the author's context, if a transaction that followed the pivot failed because of a transient technical failure, it simply needs to be invoked repeatedly until it succeeds.

ABOUT THE PIVOT TRANSACTION

Although the pivot transaction is the second phase of the saga, I'll describe it first because the other phases are defined by whether they precede or follow it. A saga's pivot transaction is the last step of the saga that can fail due to a violation of business rules. Since a saga is a sequence of steps, by definition, one of them will be the last step that can possibly fail.

In the simple `Create Order Saga` shown in table [9.3](#), the `reserveCredit()` step is the pivot transaction. If it succeeds, the saga will complete. But if it fails, the saga will be rolled back. It's the saga's go/no-go point.

One interesting aspect of the pivot transaction is that, since it is the last step of the saga that can fail, it doesn't have or need a compensating transaction. Consequently, the pivot transaction can implement an operation that can fail yet cannot be undone. It can, for example, invoke a legacy system that does not support undo. Or, the pivot transaction can implement an operation that is troublesome to undo, such as charging a customer's credit card. While a customer can be issued a credit, refunds are not always immediate and so should not be done as a compensating transaction.

ABOUT COMPENSATABLE TRANSACTIONS

Compensatable transactions are the steps of the saga that precede the pivot transaction. In the simple `Create Order Saga`, the `reserveCredit()` step is the pivot transaction, and so the `createOrder()` step that precedes it needs a compensating transaction.

Compensatable transactions are a potential source of concurrency problems. A different transaction will see inconsistent data if it reads that data in the time interval between the forward transaction and the compensating transaction. A saga will often need to implement one of the mechanisms that I describe in [Section 9.3](#), such as application-level locking. Consequently, an ideal saga is one where the pivot transaction is the first step, which eliminates the need for compensatable transactions.

ABOUT RETRIABLE TRANSACTIONS

Retriable transactions are the steps of the saga that follow the pivot transaction. By definition, they cannot fail. In the simple

Create Order Saga shown in table [9.3](#), the `approveOrder()` step is a retrievable transaction.

It's important to keep in mind that many operations can potentially fail when looked at in isolation. For example, `approveOrder()` will fail if the `orderId` is invalid or the `Order` is not in a `PENDING` state. But the reason such operations are guaranteed to succeed is because of the behavior of the previous steps. The `createOrder()` operation has, for example, created the `Order` in the `PENDING` state and locked it.

9.2.3 Overview of saga coordination

A saga's implementation consists of logic that coordinates the execution of its steps. When a saga is initiated by a system command, the coordination logic must select the first saga participant and invoke it. Once that transaction completes, the coordination logic selects and invokes the next saga participant. This process continues until all the saga's steps have been executed. If any local transaction fails, the saga must execute the compensating transactions in reverse order.

A saga's coordination logic is also responsible for exchanging data between its participants. A saga participant typically needs data from both the original request and previously executed saga participants. For example, in the Create Order Saga, the `reserveCredit()` operation has two parameters: `customerID`, which comes from the request, and `orderTotal`, which is calculated by the `Order Service`. The saga's coordination logic must pass such data from its origin to the participants that need it.

In the rest of this section, we'll explore the key design issues involved in coordinating sagas and how those choices affect coupling, responsiveness, and failure handling. We'll begin by examining how sagas are initiated and discuss when a saga initiated by a synchronous request should return a response. After that, we'll look at broader coordination concerns, including whether saga steps execute sequentially or in parallel, and how timeouts should be handled. Finally, I'll introduce the two main

saga coordination mechanisms, which are described in more detail in chapter {chapter-sagas-coordination}. Let's start by looking at how sagas are initiated.

HOW A SYSTEM OPERATION INITIATES A SAGA

A saga is initiated by the invocation of a system command. The invocation is typically a synchronous request, such as an HTTP POST, but it could also be an asynchronous message or a scheduler callback.

When a saga is initiated by a synchronous request, such as an HTTP POST or PUT, an important design issue is when to send the response. This is an example of a fan-out scenario that I discussed in chapter {chapter-ipc} when a synchronous request fans out to multiple services. There are two options for sending the response that make different trade-offs between the completeness of the response and minimizing runtime coupling.

The first option, which is shown in figure [9.5](#), is to send the response after the saga completes. This approach is ideal from the client's perspective. The response contains the final outcome of the system operation, for example, the `Order` was `APPROVED` or `REJECTED`. The operation's behavior is no different from if it were implemented by a monolith. However, the drawback of this option is that it increases runtime coupling and reduces availability. The response is dependent upon the availability of all the saga participants.

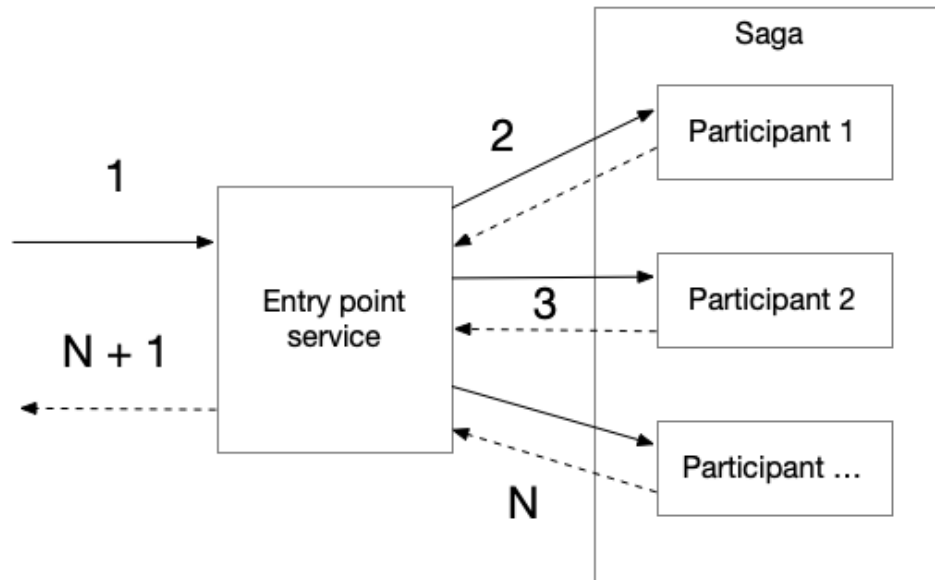


Figure 9.5 There are two options for sending the response that make different trade-offs between the completeness of the response and minimizing runtime coupling. The first option, which suffers from runtime coupling, waits until the saga has finished and sends a complete response. The second option sends back a partial response. While this option avoids runtime coupling, the client needs an additional mechanism to determine the final outcome.

The second option, which is shown in figure [9.6](#), is to send a response containing a partial outcome as soon as possible to reduce runtime coupling. A request to cancel an order could initiate the `Cancel Order Saga` and return immediately before even the first step has executed. The HTTP response is 202 (Accepted). This simply tells the client that the request has been received without any validation.

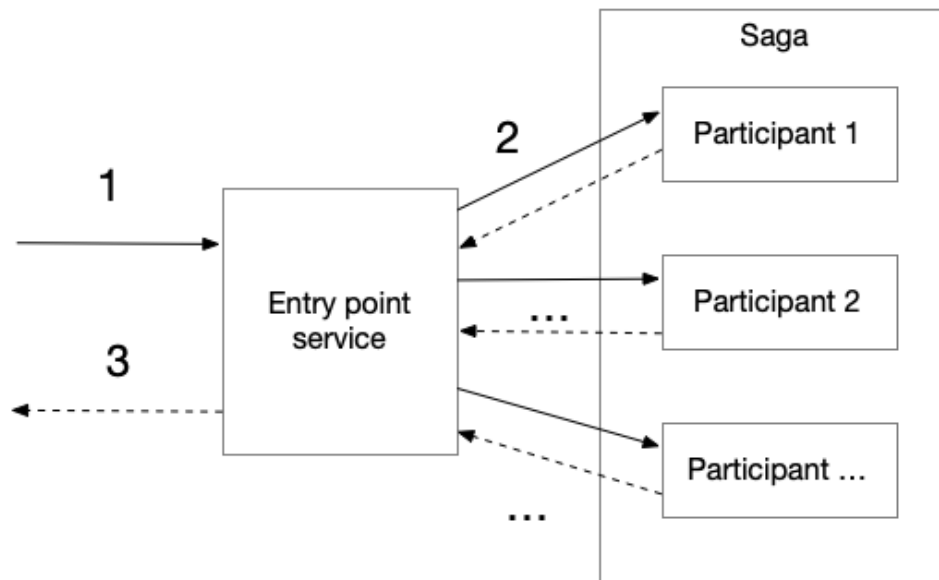


Figure 9.6 There are two options for sending the response that make different trade-offs between the completeness of the response and minimizing runtime coupling. The first option, which suffers from runtime coupling, waits until the saga has finished and sends a complete response. The second option sends back a partial response. While this option avoids runtime coupling, the client needs an additional mechanism to determine the final outcome.

Alternatively, when creating an entity, such as `Order`, it's common for the response to contain the entity's ID. In this situation, the entry point service sends the response after the execution of the step - typically the first step - that creates the entity. The HTTP status 201 (Created) reflects some initial validation and contains the `ID` of the newly created `Order`.

The downside of sending a partial response is that it complicates the client. In both cases, the client must somehow discover the system operation's final outcome. The client could periodically poll the application, e.g. invoke the endpoint for getting the `Order` status. Or, the application could notify the client using events.

SEQUENTIAL VS. PARALLEL EXECUTION

Sagas are easier to understand and reason about when the steps are executed sequentially. But in real-world sagas, there are often opportunities to execute steps in parallel, which can reduce execution time. Imagine, for example, that the `Create Order Saga`

needed to reserve both credit and inventory. Those two steps are independent and so can be executed concurrently.

The challenge, however, is that executing steps concurrently can be difficult to understand and implement. The coordination logic needs to handle the four different combinations of success and failure. If a step fails, the saga must execute compensating transactions for the steps that succeeded.

Despite parallel execution being more complicated, there are a couple of common cases for executing the steps concurrently. First, compensating transactions are typically independent and can be executed concurrently. Second, the same is often true of retrievable transactions. In neither case does the coordination logic need to deal with failures, so executing them concurrently is usually straightforward. For example, in chapter {chapter-sagas-coordination}, I described how a saga can invoke multiple participants concurrently using either publish/subscribe or one-way commands.

SHOULD SAGAS TIME OUT?

A common design issue is how to handle the scenario when a saga participant does not respond in a timely way. For example, what should the `Create Order Saga` do if the `Customer Service` fails to respond to the `reserveCredit()` request? The simplest solution is for the `Create Order Saga` to assume that any delays are temporary and that eventually the `Customer Service` will respond. Perhaps, for example, the `Customer Service` is currently being redeployed. In most cases, sagas will eventually complete. The drawback of waiting indefinitely is that the system command might not meet its execution time SLO.

Another option, which enables a system command to meet its SLO, is to implement a timeout mechanism. If the `Customer Service` does not respond within N milliseconds, treat the step as having failed and reject the `Order`. The challenge, however, is that if there's no response from the `Customer Service`, the `Create Order`

Saga cannot know whether credit was reserved. Simply rejecting the order risks leaving the system in an inconsistent state. To ensure consistency, the coordination logic must send a `cancelReserveCredit()` request to the `Customer Service`.

SAGA COORDINATION MECHANISM TYPES

Now that we have explored the various design issues, let's look at how sagas are coordinated. There are a couple of different ways to structure a saga's coordination logic:

- *Choreography* — Distribute the decision-making and sequencing among the saga participants. They communicate by exchanging events. Each saga participant executes a service operation that publishes an event that triggers the next step of the saga.
- *Orchestration* — Centralize a saga's coordination logic in a saga orchestrator. A saga *orchestrator* executes the steps of the saga by invoking saga participants. It invokes the saga participants using either request/asynchronous-response or response/response.

Chapter {chapter-sagas-coordination} describes choreography and orchestration in more detail. Let's now explore the benefits and drawbacks of sagas, starting with the benefits.

9.2.4 Benefits and drawbacks of sagas

Sagas have several benefits:

- Loose runtime coupling - unlike classic distributed transactions, a service can return a response to a system operation, and a saga can progress even when some participants are unavailable
- Deployment simplicity - unlike classic distributed transactions, sagas are well-suited to modern deployment environments with ephemeral service instances

Sagas also have several drawbacks:

- Lack of isolation - requires careful design to prevent data inconsistencies
- Lack of automatic rollback - requires explicit compensating transactions, which is more difficult to design in the absence of isolation

Now that we have looked at the benefits and drawbacks of sagas, let's deep-dive into saga coordination, including an exploration of the trade-offs of the two coordination mechanisms. Let's first look at choreography-based sagas.

9.3 Handling saga's lack of isolation

The *I* in ACID stands for *isolation*. The isolation property of ACID transactions ensures that the outcome of executing multiple transactions concurrently is the same as if they were executed in some serial order. The database provides the illusion that each ACID transaction has exclusive access to the data. Isolation makes it a lot easier to write business logic that executes concurrently. The challenge with using sagas, however, is that they lack the isolation property of ACID transactions.

The lack of isolation potentially creates numerous data consistency problems if other concurrently executing requests access the same data as the saga. For example, if multiple saga steps (e.g., a forward transaction and a compensating transaction, or two forward transactions), update the same data, a concurrently executing request might read inconsistent data in between those two transactions. It's even worse if that other request updates the data. A compensating transaction might simply overwrite its changes.

These problems occur because a saga is ACD:

- *Atomicity* — The saga implementation ensures that all transactions are executed or all changes are undone

- *Consistency* — Referential integrity within a service is handled by local databases and referential integrity across services is handled by the services
- *Durability* — Handled by local databases

This lack of isolation potentially causes what the database literature calls *anomalies*. An anomaly is when a transaction reads or writes data in a way that it wouldn't if transactions were executed one at a time. When an anomaly occurs, the outcome of executing sagas concurrently is different from if they were executed serially.

On the surface, the lack of isolation sounds unworkable. But in practice, it's common for developers to accept reduced isolation in return for higher performance. An RDBMS lets you specify the isolation level for each transaction (<https://dev.mysql.com/doc/refman/5.7/en/innodb-transaction-isolation-levels.html>). The default isolation level is usually weaker than full isolation, also known as serializable transactions. Real-world database transactions are often different from textbook definitions of ACID transactions.

And better yet, the next section discusses a set of saga design strategies that deal with the lack of isolation. These strategies are known as *countermeasures*. Some countermeasures implement isolation at the application level. Other countermeasures reduce the business risk of the lack of isolation. By using countermeasures, you can write saga-based business logic that works correctly.

I'll begin the section by describing the anomalies caused by the lack of isolation. After that, I'll talk about countermeasures that either eliminate those anomalies or reduce their business risk.

9.3.1 Overview of anomalies

Because sagas do not provide isolation across services, concurrent sagas can interleave in ways that produce

serialization anomalies. The final outcome may not be equivalent to any serial execution of the sagas. Common anomalies include:

- *Lost updates* — Two concurrent sagas read the same data, and one overwrites the other's update without detecting the conflict. This typically occurs when one step of a saga reads the data and another step updates it.
- *Dirty reads* — A transaction or saga reads data written by a saga that has not yet completed. This typically occurs when more than one of a saga's transactions updates the same data, such as a forward transaction and a compensating transaction.
- *Nonrepeatable reads* — Two different steps of a saga read the same data and get different committed values because another transaction or saga has updated that data.
- *Phantom reads* — Different steps of a saga execute the same query and obtain different results because another transaction or saga inserted or deleted rows that match the query.
- *Write skew* — Two concurrent sagas read overlapping data and update disjoint data in a way that violates a business invariant.

In a microservice architecture, even when each service correctly prevents local anomalies, concurrent sagas can still produce higher-level serialization anomalies because there is no global isolation across services.

To understand the kinds of anomalies that can arise in a microservice architecture, we will examine two example sagas and then explore two different interleaving scenarios involving those sagas. These scenarios illustrate how a saga's lack of isolation can produce outcomes that violate business invariants, even when each service correctly implements its own local transactions.

TWO EXAMPLE SAGAS

The two sagas that we'll use to illustrate anomalies are `Create Order Saga` and `Cancel Order Saga`. The `Create Order Saga` is shown in table 9.2. It consists of three steps that create the `Order`, reserve `Customer` credit and schedule a `Delivery`. Step 2 is the saga's pivot transaction, and so step 1 is a compensatable step that executes a compensating transaction to reject the `Order` when the credit cannot be reserved.

Table 9.4 This version of the `Create Order Saga` schedules a delivery for the `Order`

Saga canvas			
System operation:	createOrder(customerID, productId)		
Step	Participant	Transaction	Compensating transaction
1	Order Service	(orderId, orderTotal) = createOrder(...)	rejectOrder(orderId)
2	Customer Service	reserveCredit(customerId, orderTotal)	-
3	Delivery Service	scheduleDelivery(customerId, orderId)	-

The `Cancel Order Saga`, which is shown in table 9.4, implements the `cancelOrder()` system operation. The `Cancel Order Saga` also consists of three steps that cancel the `Order`, release the `Customer`'s credit and cancel the `Delivery`. Because a `Delivery` cannot be canceled if it has shipped, the third step is the saga's pivot transaction. The first two steps are compensatable steps that have compensating transactions to restore the `Order`'s status and the `Customer`'s available credit.

Table 9.5 The `Cancel Order Saga` cancels the `Order`, releases credit, and cancels the delivery

Saga canvas			
System operation:	cancelOrder(orderId)		
Step	Participant	Transaction	Compensating transaction
1	Order Service	(customerId, orderTotal) = cancelOrder(orderId)	undoCancelOrder(orderId)
2	Customer Service	releaseCredit(customerId, orderTotal)	undoReleaseCredit(customerId, orderTotal)
3	Delivery Service	cancelDelivery(customerId, order)	-

Now that we've looked at the two sagas, let's explore a couple of scenarios where their concurrent execution causes anomalies.

ANOMALY EXAMPLE: CANCELLING AN ORDER WHILE IT'S BEING CREATED

Let's first look at the scenario where an `Order` is canceled while the `Create Order Saga` is still executing. This is made possible by the `createOrder()` operation's entry point service returning the `orderId` immediately after the execution of the first step. The user can then potentially cancel the `Order`, which invokes the `cancelOrder()` system operation, which in turn triggers the `Cancel Order Saga`. As figure [9.7](#) shows, the `Cancel Order Saga` can execute in the middle of the `Create Order Saga`. While this scenario is unlikely with these simple sagas, order creation in a real-world application is often complicated and relatively slow, especially if external services, such as payment processors, are involved.

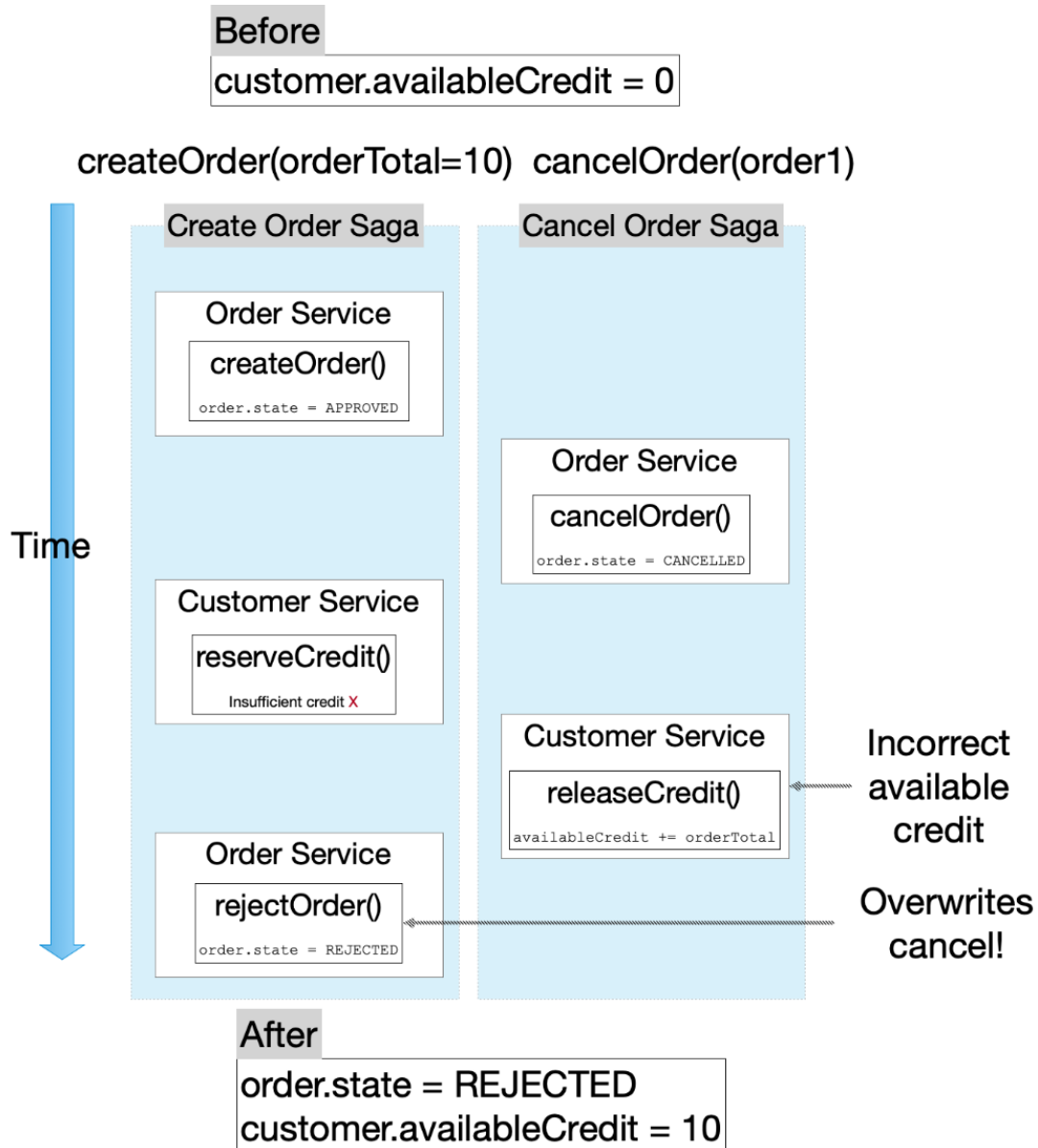


Figure 9.7 Concurrent execution of these two sagas leaves the system in an inconsistent state. The Create Order Saga overwrites the change to the Order state. The Cancel Order Saga releases credit that was never reserved.

The outcome of executing Create Order Saga and Cancel Order Saga concurrently should be equivalent to Create Order Saga followed by Cancel Order Saga - the other way round doesn't make sense since an order cannot be canceled before its created. The state of the Order should be CANCELLED and the Customer's available credit unchanged. However, because of the lack of isolation, the actual outcome is different.

The state of the `Order` is `REJECTED` due to a **lost update**. The `Order Service` incorrectly assumed that the `Order` state was `APPROVED`. And, the `Customer`'s available credit is reduced by the `orderTotal` because of a **write skew** caused by the sagas reading and updating the `Order` and the `Customer` in different transactions. As a result, the business rule defining the relationship between available credit and open `Orders` is violated.

ANOMALY EXAMPLE: CANCELLING AN ORDER WHILE CREATING ANOTHER ORDER

The previous example described how cancelling an `Order` that's still being created can result in anomalies. Let's now look at what can go wrong when cancelling one `Order` while creating a different one. In the scenario shown in figure [9.8](#), the following two sagas execute concurrently:

- a `Create Order Saga` that attempt to create a new `Order` whose `orderTotal` exceeds the `Customer`'s available credit
- a `Cancel Order Saga` that ultimately fails because the `Order` has shipped

The `Create Order Saga` should fail because the `Customer` has insufficient credit available. However, because the `Cancel Order Saga` temporarily increases the `Customer`'s available credit, the `Order` is created successfully.

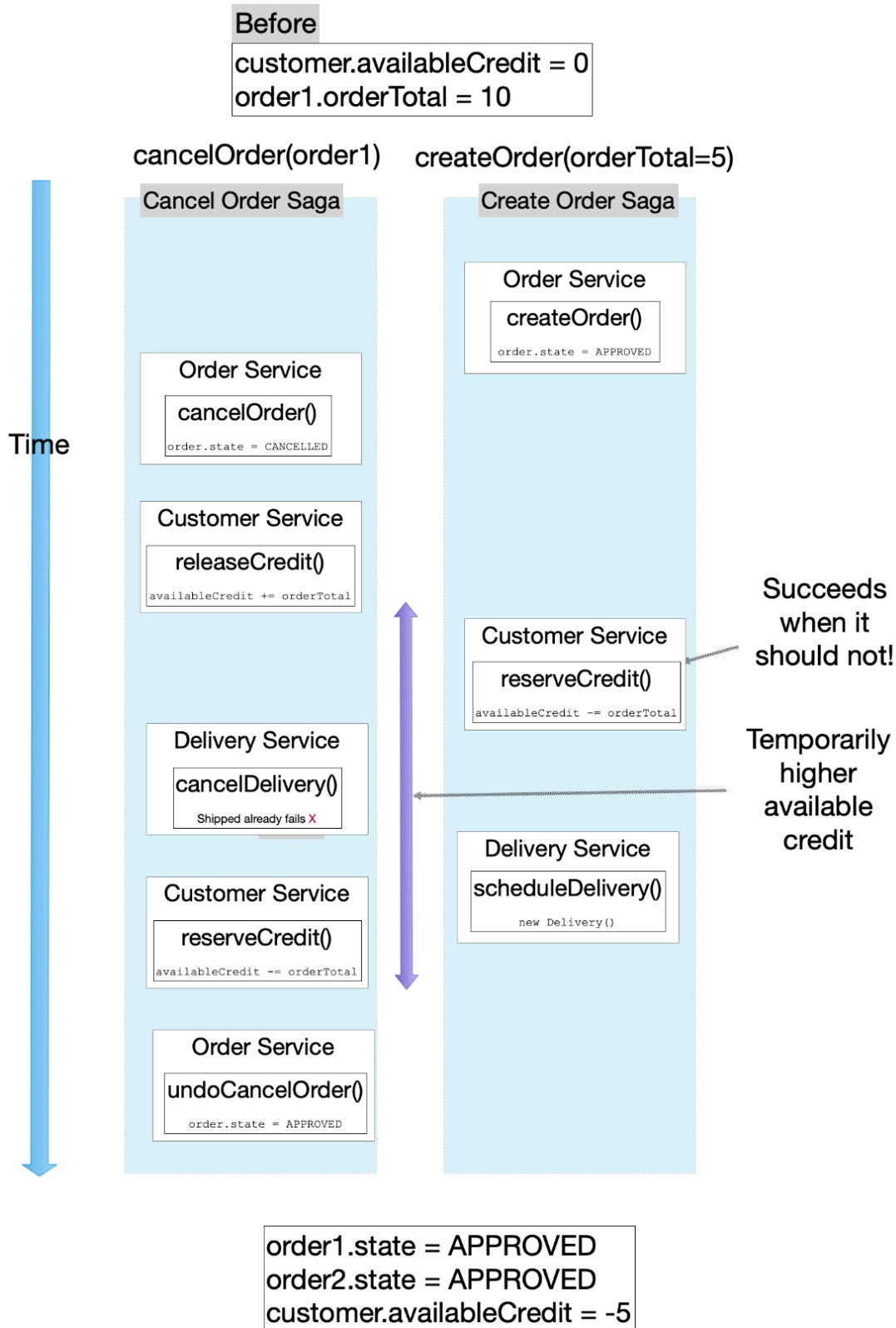


Figure 9.8 Concurrent execution of these two sagas enables the Create Order Saga to succeed even though the Customer has insufficient available credit. This occurs because the Cancel Order Saga temporarily increases the available credit.

The outcome of executing `Create Order Saga` and `Cancel Order Saga` concurrently is either `Create Order Saga → Cancel Order Saga` or `Cancel Order Saga → Create Order Saga`. In either case, both sagas should fail. However, because the `Create Order Saga` reserves credit between the `Cancel Order Saga` has releasing and then reserving credit, which is a dirty read, it's able to create the `Order`. As a result, the customer can place an order that exceeds their credit limit, which is likely to be an unacceptable risk to the business.

Let's look at how to prevent this and other kinds of anomalies from impacting an application.

9.3.2 Countermeasures for handling the lack of isolation

The saga transaction model is ACD, and its lack of isolation can result in anomalies that cause applications to misbehave. It's the responsibility of the developer to write sagas in a way that either prevents the anomalies or minimizes their impact on the business. This may sound like a daunting task, but you've already seen an example of a strategy that prevents anomalies. An `Order`'s use of `*_PENDING` states, such as `APPROVAL_PENDING`, is an example of one such strategy. Sagas that update `Orders`, such as the `Create Order Saga`, begin by setting the state of an `Order` to `*_PENDING`. The `*_PENDING` state tells other transactions that the `Order` is being updated by a saga and to act accordingly.

These pending states are an example of what the 1998 paper "Semantic ACID Properties in Multidatabases Using Remote Procedure Calls and Update Propagations" by Lars Frank and Torben U. Zahle calls a *semantic lock countermeasure* (<https://dl.acm.org/citation.cfm?id=284472.284478>). The paper describes how to deal with the lack of transaction isolation in multi-database architectures that don't use distributed transactions. Many of its ideas are useful when designing sagas. It describes a set of countermeasures for handling anomalies

caused by lack of isolation that either prevent one or more anomalies or minimize their impact on the business.

The countermeasures described by this paper are as follows:

- *Semantic lock* — An application-level lock
- *Commutative updates* — Design update operations to be executable in any order
- *Pessimistic view* — Reorder the steps of a saga to minimize business risk
- *Reread value* — Prevent dirty writes by rereading data to verify that it's unchanged before overwriting it
- *Version file* — Record the updates to a record so that they can be reordered
- *By value* — Use each request's business risk to dynamically select the concurrency mechanism

Let's now look at each countermeasure, starting with the semantic lock countermeasure.

COUNTERMEASURE: SEMANTIC LOCK

When using the semantic lock countermeasure, a saga's compensatable transaction sets a flag in any record that it creates or updates. The flag indicates that the record isn't *committed* and could potentially change. The flag can either be a lock that prevents other transactions from accessing the record or a warning that indicates that other transactions should treat that record with suspicion. It's cleared by either a retrievable transaction — the saga is completing successfully — or by a compensating transaction: the saga is rolling back.

The `Order.state` field is a great example of a semantic lock. The `*_PENDING` states, such as `APPROVAL_PENDING` and `REVISION_PENDING`, implement a semantic lock. They tell other sagas that access an `Order` that a saga is in the process of updating the `Order`. For instance, the first step of the `Create Order Saga`, which is a compensatable transaction, creates an `Order` in a `APPROVAL_PENDING`

state. The final step of the `Create Order Saga`, which is a retrievable transaction, changes the field to `APPROVED`. A compensating transaction changes the field to `REJECTED`.

Managing the lock is only half the problem. You also need to decide on a case-by-case basis how a saga should deal with a record that has been locked. Consider, for example, the `Cancel Order Saga` described earlier, which invokes the `Order Service`'s `cancelOrder()` operation. The `Create Order Saga` needs to handle an `Order` that's locked by virtue of being in the `APPROVAL_PENDING` state.

There are a few different ways to handle this scenario. One option is for the `cancelOrder()` system command to fail and tell the client to try again later. The main benefit of this approach is that it's simple to implement. The drawback, however, is that it makes the client more complicated because it has to implement retry logic.

Another option is for `cancelOrder()` to block until the lock is released. The system operation's implementation would loop and retry - after sleeping - outside of the database transaction until it can claim the lock. This approach has two main benefits. It recreates the isolation provided by ACID transactions. Sagas that update the same record are serialized, which significantly reduces the programming effort. Another benefit is that it removes the burden of retries from the client. The drawback of blocking is that the application must manage locks. This includes implementing a deadlock detection algorithm that performs a rollback of a saga to break a deadlock and re-executes it. Consequently, it's usually best for the client to retry.

COUNTERMEASURE: COMMUTATIVE UPDATES

One straightforward countermeasure is to design the update operations to be commutative. Operations are *commutative* if they can be executed in any order. For example, addition and subtraction are commutative, a `Customer`'s `reserveCredit()` and `releaseCredit()` operations are commutative. So are an `Account`'s

`debit()` and `credit()` operations are commutative (if you ignore overdraft checks). This countermeasure is useful because it eliminates lost updates.

Consider, for example, a scenario where a saga needs to be rolled back after a compensatable transaction has debited (or credited) an account. The compensating transaction can simply credit (or debit) the account to undo the update. There's no possibility of overwriting updates made by other sagas.

COUNTERMEASURE: PESSIMISTIC VIEW

Another way to deal with the lack of isolation is the *pessimistic view* countermeasure. This approach reorders the steps of a saga to minimize business risk due to a dirty read. For instance, in the second anomaly scenario describer earlier in this section, the `Create Order Saga` performed a dirty read of the available credit - caused by the `Create Order Saga` - and created an `Order` that exceeded the consumer credit limit. To reduce the risk of that happening, the steps of the `Cancel Order Saga` can be reordered as follows:

1. `Order Service` — Change the state of the `Order` to `CANCELED`
2. `Delivery Service` — Cancel the delivery
3. `Customer Service` — Increase the available credit

In this reordered version of the saga, the available credit is increased in a retrievable transaction, which eliminates the possibility of a dirty read.

COUNTERMEASURE: REREAD VALUE

The *reread value* countermeasure prevents lost updates caused by updating previously read data that had subsequently been changed by a different transaction. A saga that uses this countermeasure rereads a record before updating it, verifies that it's unchanged, and then updates the record. If the record has changed since it was first read, the saga aborts and possibly

restarts. This countermeasure is a form of the Optimistic Offline Lock pattern
(<https://martinfowler.com/eaCatalog/optimisticOfflineLock.html>)

The `Create Order Saga` could use this countermeasure to handle the scenario where the `Order` is canceled while it's in the process of being approved. The transaction that approves the `Order` verifies that the `Order` has not changed - i.e. still pending - since its creation earlier in the saga. If it's unchanged, the transaction approves the `Order`. Conversely, if the `Order` has been canceled, the transaction aborts the saga, which causes its compensating transactions to be executed. This means, of course, that the approval step becomes the saga's pivot transaction.

COUNTERMEASURE: VERSION FILE

The *version file* countermeasure is so named because it records the operations that are performed on a database record so that it can reorder them. It's a way to turn non-commutative operations into commutative operations. To see how this countermeasure works, consider a scenario where the `Create Order Saga` executes concurrently with a `Cancel Order Saga`. Unless the sagas use the semantic lock countermeasure, the `Cancel Order Saga` may cancel the authorization of the consumer's credit card before the `Create Order Saga` authorizes it.

In this instance, the `Accounting Service` would first record the `Cancel Authorization` request. Then, when the `Accounting Service` subsequently receives the `Authorize Card` request, it would notice that it had already received the `Cancel Authorization` request and skip authorizing the credit card.

COUNTERMEASURE: BY VALUE

The final countermeasure described by the paper is the *by value* countermeasure. It's a strategy that selects the concurrency mechanism based on the business risk. An application that uses

this countermeasure uses the properties of each request to decide between using sagas and distributed transactions. It executes low-risk requests using sagas, perhaps applying the countermeasures described in this section. But it executes high-risk requests involving, for example, large amounts of money using distributed transactions. This strategy enables an application to dynamically make trade-offs about business risk, availability, and scalability. However, given the problems with XA/2PC that I described in [Section 9.1.2](#), this countermeasure is not a good fit for a modern microservice architecture.

Now that we've looked at various saga design issues and some simple examples, let's explore how to use the two saga coordination mechanisms - choreography and orchestration.

9.4 Summary

- System commands must be transactional to ensure atomicity, consistency, isolation, and durability. Ideally, they should use ACID transactions, which are a simple yet powerful transaction model.
- System operations that are local to a service can use ACID transactions.
- Although classic distributed transactions (XA/2PC) implement the ACID transaction model, they are not a good fit for a modern microservice architecture due to their complexity and runtime coupling.
- Distributed system operations, which span multiple services, must use the Saga pattern, which is a series of local ACID transactions in the participants.
- Sagas do not have the runtime coupling of classic distributed transactions. They are well-suited to modern deployment environments where service instances are ephemeral. However, even though a saga can be composed of ACID transactions, the saga itself is not. Consequently, the application must implement certain capabilities that are no longer provided by the database.

- A saga consists of a series of steps, each of which consists of a forward transaction that invokes a saga participant. A step can also have a compensating transaction, which undoes the changes committed by the forward transaction, if the step's forward transaction invokes a service command and the step is followed by a step that can fail due to business rule violations.
- It's easier to reason about and implement a saga if the steps are executed sequentially. However, executing steps in parallel improves performance.
- Because sagas consist of multiple ACID transactions, sagas are ACD - they lack the isolation property. Consequently, the concurrent execution of multiple sagas can result in data anomalies, such as lost updates, dirty reads, and fuzzy reads.
- To prevent these data anomalies, sagas often use countermeasures, which are design techniques that implement isolation at the application-level. The five countermeasures that are most useful when implementing sagas in a microservice architecture are:
 - *Semantic lock* — An application-level lock
 - *Commutative updates* — Design update operations to be executable in any order
 - *Pessimistic view* — Reorder the steps of a saga to minimize business risk
 - *Reread value* — Prevent dirty writes by rereading data to verify that it's unchanged before overwriting it
 - *Version file* — Record the updates to a record so that they can be reordered
- Sagas are often initiated by a synchronous request, such as an HTTP POST. A key design issue is when to send the response. There are two options:
 - Send a full response after the saga completes - the response contains the outcome of executing the saga. While this simplifies the client, the drawback is that it increases runtime coupling and reduces availability.

- Send a partial response as soon as possible - send the response either before executing the saga or after executing a subset of the steps, such as just the first one. This approach reduces runtime coupling and increases availability. The downside is that the client is more complicated. It must somehow discover the saga's final outcome by either polling or subscribing to events.